

RTL Design Sherpa

RAPIDS Beats Micro-Architecture Specification 0.9

July 2, 2026

Table of Contents

1 Rapids Beats Mas Index....39

2 Document Information....40

2.1 References....40

2.1.1 Related Documents....40

2.2 Terminology....40

2.3 “Beats” Architecture Concept....42

2.4 Revision History....43

3 Architecture Overview....47

3.1 Overview....47

3.1.1 Key Features....47

3.1.2 Block Diagram....48

3.1.3 Figure 1.1.1: RAPIDS Beats Architecture Block Diagram....48

3.2 Data Flow Overview....48

3.2.1 Sink Path (Network to Memory)....48

3.2.2 Figure 1.1.2: Sink Path Data Flow....48

3.2.3 Source Path (Memory to Network)....49

3.2.4 Figure 1.1.3: Source Path Data Flow....49

3.3 Key Architectural Decisions....49

3.3.1 Beat-Level Tracking....49

3.3.2 Concurrent Read/Write....50

3.3.3 Virtual FIFOs (Alloc/Drain)....50

3.4 Module Hierarchy....50

3.5 Timing Diagram: Basic Transfer....52
3.5.1 Figure 1.1.4: Basic Sink Path Transfer Timing....52
3.6 Related Documentation....52
4 Top-Level Port List....54
4.1 Parameters....54
4.2 Port List (324 ports)....55
4.2.1 Clock and Reset (shared - single clock/reset for the whole wrapper)....56
4.3 SOURCE HALF (u_src) - shared-infrastructure ports (src_ prefixed)....56
4.3.1 APB Programming Interface....56
4.3.2 Per-channel configuration....56
4.3.3 Scheduler Configuration (global)....56
4.3.4 Descriptor Engine Configuration (global)....57
4.3.5 Control Engine Configuration (Phase 2, global)....57
4.3.6 Descriptor AXI Monitor Configuration....58
4.3.7 Network-plane monitor input (rapids TASK-024)....59
4.3.8 Status....59
4.3.9 Descriptor AXI Monitor Status....59
4.3.10 Descriptor AXI Monitor perf window (feeds SRC_.MON.DAXMON_PERF_*)....60
4.3.11 Descriptor Fetch AXI Master....61
4.3.12 Control Read AXI Master (32-bit) [Phase 2]....62
4.3.13 Control Write AXI Master (32-bit) [Phase 2]....63
4.4 SOURCE HALF (u_src) - direction-unique ports (no prefix)....64
4.4.1 AXI Transfer Configuration (source-only)....64

4.4.2 Source Path - AXIS Master Interface (SRAM -> Network); tid = channel id 64
4.4.3 AXI4 Master - Data Read (Memory -> Source SRAM) 65
4.5 SINK HALF (u_snk) - shared-infrastructure ports (snk_ prefixed) 66
4.5.1 APB Programming Interface 66
4.5.2 Per-channel configuration 66
4.5.3 Scheduler Configuration (global) 66
4.5.4 Descriptor Engine Configuration (global) 67
4.5.5 Control Engine Configuration (Phase 2, global) 67
4.5.6 Descriptor AXI Monitor Configuration 67
4.5.7 Network-plane monitor input (rapids TASK-024) 68
4.5.8 Status 69
4.5.9 Descriptor AXI Monitor Status 69
4.5.10 Descriptor AXI Monitor perf window (feeds SNK_.MON.DAXMON_PERF_*) 70
4.5.11 Descriptor Fetch AXI Master 70
4.5.12 Control Read AXI Master (32-bit) [Phase 2] 71
4.5.13 Control Write AXI Master (32-bit) [Phase 2] 72
4.6 Monitor Bus (SINGLE aggregated stream for the whole core) 74
4.6.1 Monitor Bus 74
4.7 SINK HALF (u_snk) - direction-unique ports (no prefix) 74
4.7.1 AXI Transfer Configuration (sink-only) 74
4.7.2 Sink Path - AXIS Slave Interface (Network -> SRAM); tid = channel id 74
4.7.3 AXI4 Master - Data Write (Sink SRAM -> Memory) 75

4.8 Debug Interface (per half, src_dbg_ / snk_dbg_ prefixed)....76
4.8.1 Source half debug....76
4.8.2 Sink half debug....77
4.8.3 Active-channel sideband for per-channel bus instrumentation....77
5 Clocks and Reset....78
5.1 Clock Domain....78
5.1.1 Clock Specifications....78
5.1.2 Clock Distribution....78
5.2 Reset Specification....78
5.2.1 Reset Signal....79
5.2.2 Reset Timing Diagram....79
5.2.3 Figure 1.3.1: Reset Timing....79
5.3 Reset Behavior....80
5.3.1 On Reset Assert....80
5.3.2 Reset Sequence....80
5.4 Configuration During Reset....80
5.5 Reset Macros....81
5.6 Power-On Reset Considerations....81
6 Scheduler Specification....83
6.1 Overview....83
6.1.1 Key Features....83
6.1.2 Block Diagram....84
6.1.3 Figure 2.1.1: Scheduler Block Diagram....84
6.2 Parameters....84

6.3 Port List....85
6.3.1 Clock and Reset....85
6.3.2 Configuration Interface....85
6.3.3 Descriptor Engine Interface....85
6.3.4 Data Read Interface....86
6.3.5 Data Write Interface....86
6.3.6 Control Interface....87
6.4 FSM States....88
6.4.1 Figure 2.1.2: Scheduler FSM State Diagram....88
6.4.2 State Encoding....90
6.5 Operation....90
6.5.1 Descriptor Format....90
6.5.2 Transfer Sequence....91
6.5.3 Figure 2.1.3: Basic Transfer Timing....91
6.6 Error Handling....91
6.6.1 Recoverable Write-Progress Timeout....92
6.7 MonBus Events....92
7 Descriptor Engine Specification....94
7.1 Overview....94
7.1.1 Key Features....94
7.1.2 Block Diagram....94
7.1.3 Figure 2.2.1: Descriptor Engine Block Diagram....94
7.2 Parameters....95
7.3 Port List....95

7.3.1 Clock and Reset....95
7.3.2 APB Programming Interface....95
7.3.3 Configuration Interface....96
7.3.4 AXI4 Read Master Interface....96
7.3.5 Scheduler Interface....97
7.4 FSM States....98
7.4.1 Figure 2.2.2: Descriptor Engine FSM....98
7.5 Operation....99
7.5.1 Descriptor Chain Processing....99
7.5.2 Figure 2.2.3: Descriptor Chain Timing....99
7.6 Address Range Checking....99
7.7 Prefetch and Autonomous Chaining....100
7.7.1 Chaining Decision....100
7.7.2 Deferred (Throttle-Blocked) Chain Fetch....100
8 AXI Read Engine Specification....102
8.1 Overview....102
8.1.1 Key Features....102
8.1.2 Block Diagram....103
8.1.3 Figure 2.3.1: AXI Read Engine Block Diagram....103
8.2 Parameters....103
8.3 Port List....104
8.3.1 Clock and Reset....104
8.3.2 Scheduler Interface....104
8.3.3 AXI4 Read Master Interface....104

8.3.4 SRAM Allocation and Fill Interface....105
8.4 Operation....106
8.4.1 Burst Length Cap (rapids BUG-009)....106
8.4.2 Streaming Pipeline Architecture....106
8.4.3 Timing Diagram....107
8.4.4 Figure 2.3.2: AXI Read Burst Timing....107
8.5 Burst Segmentation....107
8.6 Error Handling....108
9 AXI Write Engine Specification....109
9.1 Overview....109
9.1.1 Key Features....109
9.1.2 Block Diagram....110
9.1.3 Figure 2.4.1: AXI Write Engine Block Diagram....110
9.2 Parameters....110
9.3 Port List....111
9.3.1 Clock and Reset....111
9.3.2 Scheduler Interface....111
9.3.3 AXI4 Write Master Interface....112
9.3.4 SRAM Reservation and Drain Interface....113
9.4 Operation....114
9.4.1 Burst Length Cap (rapids BUG-009)....114
9.4.2 Write Transaction Phases....114
9.4.3 Timing Diagram....115
9.4.4 Figure 2.4.2: AXI Write Burst Timing....115

9.5 Error Handling....116
10 Beats Alloc Control Specification....117
10.1 Overview....117
10.1.1 Key Features....117
10.1.2 Block Diagram....118
10.1.3 Figure 2.5.1: Beats Alloc Control Block Diagram....118
10.2 Concept: Virtual FIFO....118
10.2.1 Use Case: Pre-Allocation for AXI Reads....118
10.3 Parameters....119
10.4 Port List....119
10.4.1 Clock and Reset....119
10.4.2 Write Interface (Allocation Requests)....119
10.4.3 Read Interface (Data Written)....120
10.4.4 Status Outputs....120
10.5 Operation....120
10.5.1 Pointer Arithmetic....120
10.5.2 Timing Diagram....121
10.5.3 Figure 2.5.2: Allocation and Release Timing....121
10.6 Integration Example....121
11 Beats Drain Control Specification....123
11.1 Overview....123
11.1.1 Key Features....123
11.1.2 Block Diagram....124
11.1.3 Figure 2.6.1: Beats Drain Control Block Diagram....124

11.2 Concept: Data Availability Tracking....124
11.2.1 Complementary Roles....124
11.3 Parameters....125
11.4 Port List....125
11.4.1 Clock and Reset....125
11.4.2 Write Interface (Data Written to SRAM)....125
11.4.3 Read Interface (Drain Requests)....125
11.4.4 Status Outputs....126
11.5 Operation....126
11.5.1 Pointer Arithmetic....126
11.5.2 Timing Diagram....127
11.5.3 Figure 2.6.2: Data Arrival and Drain Timing....127
11.6 Integration Example....127
11.7 Relationship: alloc_ctrl vs drain_ctrl....128
12 Latency Bridge (beats) Specification....129
12.1 Overview....129
12.1.1 Key Features....129
12.1.2 Block Diagram....130
12.1.3 Figure 2.7.1: Latency Bridge Block Diagram....130
12.2 Concept: bridging a registered read....130
12.3 Parameters....131
12.4 Port List....131
12.4.1 Clock and Reset....131
12.4.2 Upstream Interface (from the registered FIFO)....131

12.4.3 Downstream Interface (to the consumer)....131
12.4.4 Status and Debug....132
12.5 Operation....133
12.5.1 Figure 2.7.2: Latency Bridge Timing (consumer always ready)....133
12.6 Integration Context....134
12.7 Design Considerations....134
13 Control-Read Engine Specification....136
13.1 Overview....136
13.1.1 Key Features....136
13.1.2 Block Diagram....137
13.1.3 Figure 2.8.1: Control-Read Engine Block Diagram....137
13.2 Parameters....137
13.3 Port List....138
13.3.1 Clock and Reset....138
13.3.2 Scheduler Interface....138
13.3.3 Configuration Interface....139
13.3.4 AXI4 Read Master Interface....139
13.3.5 MonBus Interface....140
13.4 FSM States....141
13.4.1 Figure 2.8.2: Control-Read Engine FSM....141
13.5 Operation....142
13.5.1 Masked Compare....142
13.5.2 Retry Budget and Pacing....142
13.5.3 Null Address Fast-Path....142

13.5.4 AXI Read Transaction....142
13.5.5 Channel Reset....143
13.6 MonBus Reporting....143
13.7 Descriptor Field Mapping....144
14 Control-Write Engine Specification....145
14.1 Overview....145
14.1.1 Key Features....145
14.1.2 Block Diagram....146
14.1.3 Figure 2.9.1: Control-Write Engine Block Diagram....146
14.2 Parameters....146
14.3 Port List....147
14.3.1 Clock and Reset....147
14.3.2 Configuration Interface....147
14.3.3 Scheduler Interface....147
14.3.4 AXI4 Write Master Interface....148
14.3.5 MonBus Interface....149
14.4 FSM States....149
14.4.1 Figure 2.9.2: Control-Write Engine FSM....149
14.5 Operation....151
14.5.1 Doorbell Write Sequence....151
14.5.2 Figure 2.9.3: Control-Write Doorbell Timing....151
14.5.3 Request Handshake and Skid Buffer....152
14.5.4 Address Validation....152
14.5.5 Error Reporting....152

14.5.6 Channel Reset....152
14.6 Contrast with the Control-Read Engine....153
15 Beats Scheduler Group Specification....155
15.1 Overview....155
15.1.1 Key Features....155
15.1.2 Block Diagram....155
15.1.3 Figure 3.1.1: Beats Scheduler Group Block Diagram....155
15.2 Parameters....156
15.3 Port List....156
15.3.1 Clock and Reset....156
15.3.2 APB Programming Interface....156
15.3.3 Configuration Interface....157
15.3.4 Descriptor AXI Master Interface....158
15.3.5 Scheduler Data Interfaces....159
15.3.6 Status....160
15.3.7 MonBus Interface....160
15.4 Internal Architecture....160
16 Beats Scheduler Group Array Specification....162
16.1 Overview....162
16.1.1 Key Features....162
16.1.2 Block Diagram....162
16.1.3 Figure 3.2.1: Beats Scheduler Group Array Block Diagram....162
16.2 Parameters....163
16.3 Port List....163

16.3.1 Clock and Reset....163
16.3.2 Per-Channel APB Programming....163
16.3.3 Per-Channel Configuration....164
16.3.4 Shared Descriptor AXI Master Interface....164
16.3.5 Per-Channel Scheduler Data Interfaces....165
16.3.6 Aggregate Status....166
16.3.7 Unified MonBus Interface....166
16.3.8 Descriptor-AXI Perf Window....167
16.4 AXI Arbitration....168
16.4.1 Figure 3.2.2: Descriptor AXI Arbitration Sequence....168
16.5 MonBus Aggregation....168
16.6 Integration Context....169
17 Sink Data Path Specification....171
17.1 Overview....171
17.1.1 Key Features....171
17.1.2 Block Diagram....171
17.1.3 Figure 3.3.1: Sink Data Path Block Diagram....171
17.2 Parameters....172
17.3 Data Flow....172
17.3.1 Figure 3.3.2: Sink Path Data Flow Sequence....172
17.4 Timing Diagram....173
17.4.1 Figure 3.3.3: Sink Path Transfer Timing....173
17.5 Interface Summary....175
17.5.1 Fill Interface (Input)....175

17.5.2 Scheduler Interface....175
18 Sink Data Path AXIS Wrapper Specification....177
18.1 Overview....177
18.1.1 Key Features....177
18.1.2 Block Diagram....178
18.1.3 Figure 3.4.1: Sink Data Path AXIS Block Diagram....178
18.2 Parameters....178
18.3 Port List....179
18.3.1 Clock and Reset....179
18.3.2 AXI-Stream Slave Interface....179
18.3.3 Allocation Configuration....179
18.3.4 Scheduler Interface....180
18.3.5 AXI Write Master Interface....180
18.4 Signal Mapping....181
18.4.1 Figure 3.4.2: AXIS to Fill Interface Mapping....181
18.5 Timing Diagram....182
18.5.1 Figure 3.4.3: AXIS Ingress Timing....182
18.6 Usage Notes....182
19 Sink SRAM Controller Specification....184
19.1 Overview....184
19.1.1 Key Features....184
19.1.2 Block Diagram....185
19.1.3 Figure 3.5.1: Sink SRAM Controller Block Diagram....185
19.2 Parameters....185

19.3 Port List....186
19.3.1 Clock and Reset....186
19.3.2 Fill Interface (ID-Selected)....186
19.3.3 Arbitrated Drain Interface (to AXI Write Engine)....186
19.3.4 Debug Interface....187
19.4 Drain Selection....187
19.4.1 Figure 3.5.2: Drain Selection by the Consumer....187
19.5 Integration with AXI Write Engine....188
20 Source Data Path Specification....190
20.1 Overview....190
20.1.1 Key Features....190
20.1.2 Block Diagram....191
20.1.3 Figure 3.6.1: Source Data Path Block Diagram....191
20.2 Parameters....191
20.3 Data Flow....192
20.3.1 Figure 3.6.2: Source Path Data Flow Sequence....192
20.4 Port List....193
20.4.1 Clock and Reset....193
20.4.2 Configuration....193
20.4.3 Scheduler Interface....193
20.4.4 AXI Read Master Interface....193
20.4.5 Drain Interface (Output to Network)....194
20.5 Timing Diagram....195
20.5.1 Figure 3.6.3: Source Path Transfer Timing....195

20.6 Integration Example....195
21 Source Data Path AXIS Wrapper Specification....197
21.1 Overview....197
21.1.1 Key Features....197
21.1.2 Block Diagram....197
21.1.3 Figure 3.7.1: Source Data Path AXIS Block Diagram....197
21.2 Parameters....198
21.3 Port List....198
21.3.1 Clock and Reset....198
21.3.2 Configuration....198
21.3.3 Scheduler Interface....198
21.3.4 AXI Read Master Interface....199
21.3.5 AXI-Stream Master Interface....200
21.4 Signal Mapping....200
21.4.1 Figure 3.7.2: Drain to AXIS Interface Mapping....200
21.5 Timing Diagram....201
21.5.1 Figure 3.7.3: AXIS Egress Timing....201
21.6 Usage Notes....201
21.7 Integration Example....202
22 Source SRAM Controller Specification....203
22.1 Overview....203
22.1.1 Key Features....203
22.1.2 Block Diagram....204
22.1.3 Figure 3.8.1: Source SRAM Controller Block Diagram....204

22.2 Parameters....204
22.3 Port List....205
22.3.1 Clock and Reset....205
22.3.2 Fill Interface (ID-Selected, from AXI Read Engine)....205
22.3.3 Arbitrated Drain Interface (to Network)....205
22.3.4 Debug Interface....206
22.4 Drain Selection....206
22.4.1 Figure 3.8.2: Drain Selection by the Consumer....206
22.5 Comparison: Sink vs Source SRAM Controller....207
22.6 Integration Example....207
23 RAPIDS Core Beats Specification....209
23.1 Overview....209
23.1.1 Key Features....209
23.1.2 Block Diagram....209
23.1.3 Figure 3.9.1: RAPIDS Core Beats Block Diagram....209
23.2 Parameters....210
23.3 Port List....210
23.3.1 Clock and Reset....210
23.3.2 Per-Channel APB Programming....210
23.3.3 Per-Channel Configuration....211
23.3.4 Descriptor AXI Master Interfaces....212
23.3.5 Sink AXI Write Master Interface....214
23.3.6 Source AXI Read Master Interface....215
23.3.7 Sink Ingress – AXIS Slave....216

23.3.8 Source Egress – AXIS Master....216
23.3.9 Unified MonBus Interface....217
23.3.10 Aggregate Status....217
23.4 Data Flow....219
23.4.1 Figure 3.9.2: RAPIDS Core Complete Data Flow....219
23.5 MonBus Aggregation....219
23.6 Integration Example....220
24 RAPIDS Registers Specification....223
24.1 Overview....223
24.2 Base Register Map (0x100-0x3FF)....223
24.2.1 Key Register Fields....225
24.3 Monitor Register Map (rapids_mon_regs @ 0x1000)....226
25 RAPIDS Config Block Specification....228
25.1 Overview....228
25.2 Mapping Groups....228
25.2.1 Base Configuration....228
25.2.2 Monitor Configuration (hwif_out.MON.*)....229
25.2.3 Performance and Observation....229
26 RAPIDS Beats Top Specification....231
26.1 Overview....231
26.2 Integration Datapath....232
26.2.1 Figure 3.12.1: RAPIDS Beats Top Integration....232
26.2.2 Address Decode....232
26.3 Write-engine pipelining (PIPELINE)....233

26.4 AXI Monitors (USE_AXI_MONITORS)....233
26.4.1 Monitor registers on a monitors-off build (USE_MON_REGS)....234
26.5 Parameters....234
26.6 Top-Level Interfaces....235
27 Network Planes Specification....237
27.1 Overview....237
27.2 Generated Tops....238
27.3 CXS Planes....239
27.3.1 Figure 3.13.1: CXS Network Planes....239
27.3.2 Receive (sink) plane....239
27.3.3 Transmit (source) plane....239
27.3.4 Per-stream framing....240
27.3.5 Parameters....240
27.3.6 Monitoring....240
27.4 Planned Changes....241
27.5 Verification....241
28 Interface Overview....242
28.1 External Interfaces....242
28.1.1 AXI4 Master Interfaces....242
28.1.2 Streaming Interfaces....242
28.1.3 Control/Monitor Interfaces....243
28.2 Interface Documentation....243
28.3 Interface Signal Naming Conventions....243
29 AXI4 Interface Specification....245

29.1 Overview....245
29.2 Descriptor AXI Master Interface....245
29.2.1 Purpose....245
29.2.2 Signal Table....245
29.2.3 Transaction Characteristics....246
29.2.4 Timing Diagram....247
29.2.5 Figure 4.1.1: Descriptor Fetch Timing....247
29.3 Sink AXI Write Master Interface....248
29.3.1 Purpose....248
29.3.2 Signal Table....248
29.3.3 Transaction Characteristics....249
29.3.4 Timing Diagram....249
29.3.5 Figure 4.1.2: Sink AXI Write Burst Timing....249
29.4 Source AXI Read Master Interface....250
29.4.1 Purpose....250
29.4.2 Signal Table....250
29.4.3 Transaction Characteristics....251
29.4.4 Timing Diagram....251
29.4.5 Figure 4.1.3: Source AXI Read Burst Timing....251
29.5 AXI Protocol Compliance....252
29.5.1 Supported Features....252
29.5.2 Error Handling....252
30 AXI-Stream Interface Specification....254
30.1 Overview....254

30.2 AXIS Sink Slave Interface....254
30.2.1 Purpose....254
30.2.2 Signal Table....254
30.2.3 Signal Details....255
30.2.4 Timing Diagram....256
30.2.5 Figure 4.2.1: AXIS Sink Slave Timing....256
30.2.6 Backpressure....256
30.3 AXIS Source Master Interface....256
30.3.1 Purpose....256
30.3.2 Signal Table....257
30.3.3 Timing Diagram....257
30.3.4 Figure 4.2.2: AXIS Source Master Timing....257
30.3.5 Flow Control....258
30.4 AXIS vs Custom Fill/Drain Interfaces....258
30.4.1 Comparison....258
30.4.2 Selection Guide....258
30.5 Configuration Parameters....258
31 MonBus Interface Specification....260
31.1 Overview....260
31.2 MonBus Signal Interface....260
31.2.1 Signal Table....260
31.2.2 Handshaking Protocol....260
31.2.3 Timing Diagram....261
31.2.4 Figure 4.3.1: MonBus Packet Transfer Timing....261

31.3 MonBus Packet Format....261
31.3.1 64-bit Packet Structure....261
31.3.2 Figure 4.3.2: MonBus Packet Format....261
31.4 Packet Types....262
31.4.1 Type Code Definitions....262
31.4.2 Payload Formats by Type....263
31.5 MonBus Source Aggregation....264
31.5.1 RAPIDS Core MonBus Sources....264
31.5.2 Aggregation Hierarchy....265
31.5.3 Figure 4.3.3: MonBus Aggregation Tree....265
31.6 Integration Guidelines....265
31.6.1 Consumer Requirements....265
31.6.2 Recommended Consumer Patterns....265
31.7 Timing Requirements....266
32 CXS Interface Specification....267
32.1 Overview....267
32.2 Link Profile....267
32.3 Signal Table....268
32.4 Credit Flow....269
32.5 Link Activation....269
32.6 Backpressure....270
32.7 Planned Changes....270
33 Product Requirements Document (PRD)....271
33.1 RAPIDS - Rapid AXI Programmable In-band Descriptor System....271

33.2 1. Executive Summary....271
33.2.1 1.1 Quick Stats....271
33.2.2 1.2 Subsystem Goals....272
33.3 2. Documentation Structure....272
33.3.1 Complete RAPIDS Specification....272
33.3.2 Known Issues....273
33.3.3 Other Documentation....273
33.4 2.4 Organizational Standards - RAPIDS Code Location....273
33.4.1 Code Organization Principle....273
33.4.2 RAPIDS Directory Structure....273
33.4.3 What Goes Where?....274
33.4.4 Import Pattern for RAPIDS Tests....274
33.4.5 Benefits of This Organization....275
33.4.6 Compliance Status....275
33.5 3. Architecture Overview....276
33.5.1 3.1 Top-Level Block Diagram....276
33.5.2 3.2 Data Flow....276
33.6 4. Key Features....277
33.6.1 4.1 Descriptor-Based Operation....277
33.6.2 4.2 Data Path Features....277
33.6.3 4.3 Scheduler Features....277
33.6.4 4.4 Monitoring Integration....278
33.7 5. Interfaces....278
33.7.1 5.1 External Interfaces....278

33.7.2 5.2 Configuration Parameters....279
33.8 6. Use Cases....279
33.8.1 6.1 DMA-Style Transfers....279
33.8.2 6.2 Network Packet Processing....279
33.8.3 6.3 Custom Data Path Acceleration....279
33.9 7. Test Coverage....280
33.9.1 7.1 Current Status....280
33.9.2 7.2 Test Strategy....281
33.10 8. Known Issues Summary....281
33.10.1 8.1 Critical Issues....281
33.10.2 8.2 Medium Priority Issues....282
33.11 9. Integration Guidelines....282
33.11.1 9.1 Quick Start....282
33.11.2 9.2 Configuration Steps....282
33.12 10. Development Status....283
33.12.1 10.1 Current Phase....283
33.12.2 10.2 Roadmap....283
33.13 11. Performance Characteristics....284
33.13.1 11.1 Throughput....284
33.13.2 11.2 Latency....284
33.13.3 11.3 Resource Utilization....284
33.14 12. Verification Infrastructure....284
33.14.1 12.1 Test Organization....284
33.14.2 12.2 CocoTB Framework....284

33.14.3 12.2.1 MANDATORY: BFM Usage for FUB Tests....285
33.14.4 12.3 Test File Structure (Standard Pattern)....286
33.15 13. Quick Reference....291
33.15.1 13.1 Key Files....291
33.15.2 13.2 Commands....291
33.16 14. Success Criteria....292
33.16.1 14.1 Functional....292
33.16.2 14.2 Quality....292
33.16.3 14.3 Documentation....292
33.17 15. Educational Value....292
33.18 16. Documentation Generation....293
33.18.1 16.1 Generating PDF/DOCX from Specification....293
33.19 16.2 PDF Generation Location....294
33.20 17. References....294
33.20.1 16.1 Internal Documentation....294
33.20.2 16.2 Related Subsystems....295
33.20.3 16.3 External References....295
33.21 Navigation....295
34 Claude Code Guide: RAPIDS Subsystem....296
34.1 Quick Context....296
34.2 Global Requirements Reference....298
34.3 Critical Rules for This Subsystem....298
34.3.1 Rule #0.1: Testbench Location and Test Structure (MANDATORY)....298
34.3.2 Rule #0.5: Config Before Reset (RAPIDS-specific)....298

34.3.3 Rule #0.75: Audit Signal Naming BEFORE Writing Testbenches.....299
34.3.4 Rule #1: MANDATORY BFM Usage for FUB-Level Tests.....299
34.3.5 Rule #2: Know the Known Issues.....300
34.3.6 Rule #3: RAPIDS is Complex - Understand Block Interactions.....300
34.3.7 Rule #4: Test Strategy is Multi-Layered.....300
34.4 Architecture Quick Reference.....301
34.4.1 Block Organization.....301
34.4.2 Module Quick Reference.....302
34.4.3 Interface Summary.....303
34.5 Common User Questions and Responses.....304
34.5.1 Q: “How does the scheduler work?”.....304
34.5.2 Q: “How do I configure RAPIDS?”.....304
34.5.3 Q: “What’s the data flow for network to memory transfer?”.....305
34.5.4 Q: “What’s the credit counter bug and how does exponential encoding work?”.....306
34.5.5 Q: “How do I run RAPIDS tests?”.....307
34.6 Integration Patterns.....307
34.6.1 Pattern 1: Basic RAPIDS Instantiation.....307
34.6.2 Pattern 2: Configuration Sequence.....309
34.6.3 Pattern 3: MonBus Integration.....309
34.7 Anti-Patterns to Catch.....310
34.7.1 Anti-Pattern 1: Not Understanding Exponential Credit Encoding.....310
34.7.2 Anti-Pattern 2: Insufficient SRAM Depth.....310
34.7.3 Anti-Pattern 3: No MonBus Downstream Handling.....310

34.7.4 Anti-Pattern 4: Testing Individual Blocks in Isolation....311
34.8 Debugging Workflow....311
34.8.1 Issue: Scheduler Not Processing Descriptors....311
34.8.2 Issue: Data Path Stalls....311
34.8.3 Issue: MonBus Packets Not Generated....312
34.9 Testing Guidance....312
34.9.1 Running Tests....312
34.9.2 Test Coverage Status....312
34.10 Key Documentation Links....312
34.11 Verification Patterns....313
34.12 Documentation Generation....313
35 RAPIDS Beats Architecture HAS....315
35.1 Document Organization....315
35.1.1 Front Matter....315
35.1.2 Chapter 1: Overview....315
35.1.3 Chapter 2: Architecture....315
35.1.4 Chapter 3: Interfaces....315
35.1.5 Chapter 4: Use Cases....315
35.1.6 Chapter 5: Programming Model....316
35.1.7 Chapter 6: Performance....316
35.2 Quick Reference....316
35.2.1 RAPIDS Beats at a Glance....316
35.2.2 Interface Summary....316
35.3 Related Documentation....317

35.4 Specification Conventions.....317
35.4.1 Signal Naming.....317
35.4.2 Diagram Conventions.....317

List of Figures

Figure 1.1.1: RAPIDS Beats Architecture Block Diagram....48
Figure 1.1.2: Sink Path Data Flow....48
Figure 1.1.3: Source Path Data Flow....49
Figure 1.1.4: Basic Sink Path Transfer Timing....52
Figure 1.3.1: Reset Timing....79
Figure 2.1.1: Scheduler Block Diagram....83
Figure 2.1.2: Scheduler FSM State Diagram....87
Figure 2.1.3: Basic Transfer Timing....90
Figure 2.2.1: Descriptor Engine Block Diagram....93
Figure 2.2.2: Descriptor Engine FSM....97
Figure 2.2.3: Descriptor Chain Timing....98
Figure 2.3.1: AXI Read Engine Block Diagram....102
Figure 2.3.2: AXI Read Burst Timing....106
Figure 2.4.1: AXI Write Engine Block Diagram....109
Figure 2.4.2: AXI Write Burst Timing....114
Figure 2.5.1: Beats Alloc Control Block Diagram....117
Figure 2.5.2: Allocation and Release Timing....120
Figure 2.6.1: Beats Drain Control Block Diagram....123
Figure 2.6.2: Data Arrival and Drain Timing....126
Figure 2.7.1: Latency Bridge Block Diagram....129
Figure 2.7.2: Latency Bridge Timing (consumer always ready)....132
Figure 2.8.1: Control-Read Engine Block Diagram....136
Figure 2.8.2: Control-Read Engine FSM....140
Figure 2.9.1: Control-Write Engine Block Diagram....145
Figure 2.9.2: Control-Write Engine FSM....148
Figure 2.9.3: Control-Write Doorbell Timing....150
Figure 3.1.1: Beats Scheduler Group Block Diagram....154
Figure 3.2.1: Beats Scheduler Group Array Block Diagram....161
Figure 3.2.2: Descriptor AXI Arbitration Sequence....167
Figure 3.3.1: Sink Data Path Block Diagram....170
Figure 3.3.2: Sink Path Data Flow Sequence....171
Figure 3.3.3: Sink Path Transfer Timing....172
Figure 3.4.1: Sink Data Path AXIS Block Diagram....176
Figure 3.4.2: AXIS to Fill Interface Mapping....180
Figure 3.4.3: AXIS Ingress Timing....180
Figure 3.5.1: Sink SRAM Controller Block Diagram....184

Figure 3.5.2: Drain Selection by the Consumer....186
Figure 3.6.1: Source Data Path Block Diagram....190
Figure 3.6.2: Source Path Data Flow Sequence....191
Figure 3.6.3: Source Path Transfer Timing....194
Figure 3.7.1: Source Data Path AXIS Block Diagram....196
Figure 3.7.2: Drain to AXIS Interface Mapping....199
Figure 3.7.3: AXIS Egress Timing....200
Figure 3.8.1: Source SRAM Controller Block Diagram....203
Figure 3.8.2: Drain Selection by the Consumer....205
Figure 3.9.1: RAPIDS Core Beats Block Diagram....208
Figure 3.9.2: RAPIDS Core Complete Data Flow....218
Figure 3.12.1: RAPIDS Beats Top Integration....230
Figure 3.13.1: CXS Network Planes....237
Figure 4.1.1: Descriptor Fetch Timing....245
Figure 4.1.2: Sink AXI Write Burst Timing....247
Figure 4.1.3: Source AXI Read Burst Timing....249
Figure 4.2.1: AXIS Sink Slave Timing....254
Figure 4.2.2: AXIS Source Master Timing....255
Figure 4.3.1: MonBus Packet Transfer Timing....259
Figure 4.3.2: MonBus Packet Format....259
Figure 4.3.3: MonBus Aggregation Tree....263

List of Tables

Table 1: Table 0.1: Related Documents and Specifications....40
Table 2: Table 0.2: RAPIDS Beats MAS Document Revision History....43
Table 3: Table 1.1.1: Beat Granularity Examples....49
Table 4: Table 1.2.1: Parameters (from rapids_core_beats.sv)....54
Table 5: Table 1.2.2: Clock and Reset (shared - single clock/reset for the whole wrapper)....56
Table 6: Table 1.2.3: APB Programming Interface....56
Table 7: Table 1.2.4: Per-channel configuration....56
Table 8: Table 1.2.5: Scheduler Configuration (global)....56
Table 9: Table 1.2.6: Descriptor Engine Configuration (global)....57
Table 10: Table 1.2.7: Control Engine Configuration (Phase 2, global)....57
Table 11: Table 1.2.8: Descriptor AXI Monitor Configuration....58
Table 12: Table 1.2.9: Network-plane monitor input (rapids TASK-024)....59
Table 13: Table 1.2.10: Status....59
Table 14: Table 1.2.11: Descriptor AXI Monitor Status....59
Table 15: Table 1.2.12: Descriptor AXI Monitor perf window (feeds SRC_.MON.DAXMON_PERF_*)....60
Table 16: Table 1.2.13: Descriptor Fetch AXI Master....61
Table 17: Table 1.2.14: Control Read AXI Master (32-bit) [Phase 2]....62
Table 18: Table 1.2.15: Control Write AXI Master (32-bit) [Phase 2]....63
Table 19: Table 1.2.16: AXI Transfer Configuration (source-only)....64
Table 20: Table 1.2.17: Source Path - AXIS Master Interface (SRAM -> Network); tid = channel id....64
Table 21: Table 1.2.18: AXI4 Master - Data Read (Memory -> Source SRAM)....65
Table 22: Table 1.2.19: APB Programming Interface....66
Table 23: Table 1.2.20: Per-channel configuration....66
Table 24: Table 1.2.21: Scheduler Configuration (global)....66
Table 25: Table 1.2.22: Descriptor Engine Configuration (global)....67
Table 26: Table 1.2.23: Control Engine Configuration (Phase 2, global)....67
Table 27: Table 1.2.24: Descriptor AXI Monitor Configuration....67
Table 28: Table 1.2.25: Network-plane monitor input (rapids TASK-024)....69
Table 29: Table 1.2.26: Status....69
Table 30: Table 1.2.27: Descriptor AXI Monitor Status....69
Table 31: Table 1.2.28: Descriptor AXI Monitor perf window (feeds SNK_.MON.DAXMON_PERF_*)....70
Table 32: Table 1.2.29: Descriptor Fetch AXI Master....70

Table 33: Table 1.2.30: Control Read AXI Master (32-bit) [Phase 2]....71
Table 34: Table 1.2.31: Control Write AXI Master (32-bit) [Phase 2]....72
Table 35: Table 1.2.32: Monitor Bus....74
Table 36: Table 1.2.33: AXI Transfer Configuration (sink-only)....74
Table 37: Table 1.2.34: Sink Path - AXIS Slave Interface (Network -> SRAM); tid = channel id....74
Table 38: Table 1.2.35: AXI4 Master - Data Write (Sink SRAM -> Memory)....75
Table 39: Table 1.2.36: Source half debug....76
Table 40: Table 1.2.37: Sink half debug....77
Table 41: Table 1.2.38: Active-channel sideband for per-channel bus instrumentation....77
Table 42: Table 1.3.1: Clock Specifications....78
Table 43: Table 1.3.2: Reset Specifications....78
Table 44: Table 1.3.3: Reset States....80
Table 45: Table 1.3.4: Configuration Timing Requirements....80
Table 46: Table 2.1.2: Clock and Reset....84
Table 47: Table 2.1.3: Configuration Interface....84
Table 48: Table 2.1.4: Descriptor Engine Interface....84
Table 49: Table 2.1.5: Data Read Interface....85
Table 50: Table 2.1.6: Data Write Interface....85
Table 51: Table 2.1.7: Control Interface (CTRL_READ / CTRL_WRITE)....86
Table 52: Table 2.1.7: FSM State Encoding....89
Table 53: Table 2.1.8: Error Handling....90
Table 54: Table 2.1.9: MonBus Event Codes....91
Table 55: Table 2.2.2: Clock and Reset....94
Table 56: Table 2.2.3: APB Programming Interface....94
Table 57: Table 2.2.4: Configuration Interface....95
Table 58: Table 2.2.5: AXI4 Read Master Interface....95
Table 59: Table 2.2.6: Scheduler Interface....96
Table 60: Table 2.3.2: Clock and Reset....103
Table 61: Table 2.3.3: Scheduler Interface....103
Table 62: Table 2.3.4: AXI4 Read Master Interface....103
Table 63: Table 2.3.5: SRAM Write Interface....104
Table 64: Table 2.3.6: Error Handling....107
Table 65: Table 2.4.2: Clock and Reset....110
Table 66: Table 2.4.3: Scheduler Interface....110
Table 67: Table 2.4.4: AXI4 Write Master Interface....111
Table 68: Table 2.4.5: SRAM Read Interface....112

Table 69: Table 2.4.6: Error Handling....115
Table 70: Table 2.5.2: Clock and Reset....118
Table 71: Table 2.5.3: Write Interface....118
Table 72: Table 2.5.4: Read Interface....119
Table 73: Table 2.5.5: Status Outputs....119
Table 74: Table 2.6.2: Clock and Reset....124
Table 75: Table 2.6.3: Write Interface....124
Table 76: Table 2.6.4: Read Interface....124
Table 77: Table 2.6.5: Status Outputs....125
Table 78: Table 2.6.6: alloc_ctrl vs drain_ctrl Comparison....127
Table 79: Table 2.7.2: Clock and Reset....130
Table 80: Table 2.7.3: Upstream Interface....130
Table 81: Table 2.7.4: Downstream Interface....130
Table 82: Table 2.7.5: Status and Debug....131
Table 83: Table 2.7.6: Skid Depth Selection....133
Table 84: Table 2.8.2: Clock and Reset....137
Table 85: Table 2.8.3: Scheduler Interface....137
Table 86: Table 2.8.4: Configuration Interface....138
Table 87: Table 2.8.5: AXI4 Read Master Interface....138
Table 88: Table 2.8.6: MonBus Interface....139
Table 89: Table 2.8.7: MonBus Events....142
Table 90: Table 2.8.8: CTRL_READ Descriptor Field Mapping....143
Table 91: Table 2.9.2: Clock and Reset....146
Table 92: Table 2.9.3: Configuration Interface....146
Table 93: Table 2.9.4: Scheduler Interface....146
Table 94: Table 2.9.5: AXI4 Write Master Interface....147
Table 95: Table 2.9.6: MonBus Interface....148
Table 96: Table 3.1.2: Clock and Reset....155
Table 97: Table 3.1.3: APB Programming Interface....155
Table 98: Table 3.1.4: Configuration Interface....156
Table 99: Table 3.1.5: Descriptor AXI Master Interface....157
Table 100: Table 3.1.6: Scheduler Data Interfaces....158
Table 101: Table 3.1.7: Status Interface....159
Table 102: Table 3.1.8: MonBus Interface....159
Table 103: Table 3.2.2: Clock and Reset....162
Table 104: Table 3.2.3: APB Programming Interface....162
Table 105: Table 3.2.4: Per-Channel Configuration....163
Table 106: Table 3.2.5: Shared Descriptor AXI Master Interface....163

Table 107: Table 3.2.6: Per-Channel Scheduler Data Interfaces....164
Table 108: Table 3.2.7: Aggregate Status....165
Table 109: Table 3.2.8: Unified MonBus Interface....165
Table 110: Table 3.2.9: Descriptor-AXI Perf Window....166
Table 111: Table 3.2.10: MonBus Source Assignment....167
Table 112: Table 3.3.2: Fill Interface....174
Table 113: Table 3.3.3: Scheduler Interface....174
Table 114: Table 3.4.2: Clock and Reset....177
Table 115: Table 3.4.3: AXI-Stream Slave Interface....177
Table 116: Table 3.4.4: Allocation Configuration....178
Table 117: Table 3.4.5: Scheduler Interface....178
Table 118: Table 3.4.6: AXI Write Master Interface....179
Table 119: Table 3.5.2: Clock and Reset....185
Table 120: Table 3.5.3: Fill Interface (ID-Selected)....185
Table 121: Table 3.5.4: Arbitrated Drain Interface....185
Table 122: Table 3.5.5: Debug Interface....186
Table 123: Table 3.6.2: Clock and Reset....192
Table 124: Table 3.6.3: Configuration....192
Table 125: Table 3.6.4: Scheduler Interface....192
Table 126: Table 3.6.5: AXI Read Master Interface....192
Table 127: Table 3.6.6: Drain Interface....193
Table 128: Table 3.7.2: Clock and Reset....197
Table 129: Table 3.7.3: Configuration....197
Table 130: Table 3.7.4: Scheduler Interface....197
Table 131: Table 3.7.5: AXI Read Master Interface....198
Table 132: Table 3.7.6: AXI-Stream Master Interface....199
Table 133: Table 3.8.2: Clock and Reset....204
Table 134: Table 3.8.3: Fill Interface (ID-Selected)....204
Table 135: Table 3.8.4: Arbitrated Drain Interface....204
Table 136: Table 3.8.5: Debug Interface....205
Table 137: Table 3.8.6: Sink vs Source Controller Comparison....206
Table 138: Table 3.9.2: Clock and Reset....209
Table 139: Table 3.9.3: APB Programming Interface....210
Table 140: Table 3.9.4: Per-Channel Configuration....210
Table 141: Table 3.9.5: Descriptor AXI Master Interfaces (src and snk)....211
Table 142: Table 3.9.6: Sink AXI Write Master Interface....213
Table 143: Table 3.9.7: Source AXI Read Master Interface....214
Table 144: Table 3.9.8: Sink Ingress AXIS Slave....215

Table 145: Table 3.9.9: Source Egress AXIS Master....216
Table 146: Table 3.9.10: Unified MonBus Interface....216
Table 147: Table 3.9.11: Aggregate Status....217
Table 148: Table 3.9.12: MonBus Source Assignment....218
Table 149: Table 3.10.1: Base Register Map....222
Table 150: Table 3.10.2: Monitor Register Map (base 0x1000)....225
Table 151: Table 3.11.1: Monitor Register-to-Config Mapping....228
Table 152: Table 3.12.1: Top-Level Address Decode....231
Table 153: Table 3.12.3: Top-Level Interfaces....233
Table 154: Table 3.13.1: RAPIDS Network Planes....235
Table 155: Table 3.13.2: Generated Tops....236
Table 156: Table 3.13.3: CXS Plane Parameters....238
Table 157: Table 3.13.4: CXS Top Test Cells....239
Table 158: Table 4.0.1: AXI4 Master Interfaces....240
Table 159: Table 4.0.2: Streaming Interfaces....240
Table 160: Table 4.0.3: Control and Monitor Interfaces....241
Table 161: Table 4.0.4: Signal Naming Conventions....241
Table 162: Table 4.1.1: Descriptor AXI Master Interface Signals....243
Table 163: Table 4.1.2: Descriptor AXI Transaction Characteristics....244
Table 164: Table 4.1.3: Sink AXI Write Master Interface Signals....246
Table 165: Table 4.1.4: Sink AXI Write Transaction Characteristics....247
Table 166: Table 4.1.5: Source AXI Read Master Interface Signals....248
Table 167: Table 4.1.6: Source AXI Read Transaction Characteristics....249
Table 168: Table 4.1.7: AXI Feature Support....250
Table 169: Table 4.2.1: AXIS Sink Slave Interface Signals....252
Table 170: Table 4.2.2: TID Channel Mapping....253
Table 171: Table 4.2.3: TSTRB Configuration....253
Table 172: Table 4.2.4: AXIS Source Master Interface Signals....255
Table 173: Table 4.2.5: Interface Comparison....256
Table 174: Table 4.2.6: Interface Selection Guide....256
Table 175: Table 4.3.1: MonBus Interface Signals....258
Table 176: Table 4.3.2: MonBus Packet Fields....259
Table 177: Table 4.3.3: MonBus Packet Types....260
Table 178: Table 4.3.4: Descriptor Packet Payload....261
Table 179: Table 4.3.5: Scheduler Packet Payload....261
Table 180: Table 4.3.6: AXI Event Packet Payload....261
Table 181: Table 4.3.7: Error Packet Payload....261
Table 182: Table 4.3.8: MonBus Source Assignment....262

Table 183: Table 4.3.9: MonBus Timing Parameters....264
Table 184: Table 4.4.1: CXS Link Profile....265
Table 185: Table 4.4.2: CXS Interface Signals....266
Table 186: Table 4.4.3: Link Activation Sequence....267
Table 187: RAPIDS Beats Feature Summary....314
Table 188: External Interface Summary....314

List of Waveforms

No waveforms in this document.

1 Rapids Beats Mas Index

Generated: 2026-10-16

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

This document describes the RAPIDS “Beats” architecture, a Phase 1 implementation of the Rapid AXI Programmable In-band Descriptor System. The beats architecture provides network-to-memory and memory-to-network data transfer capabilities using descriptor-based DMA with 8-channel support. It emphasizes “beat-level” tracking for precise flow control and latency management.

2.1 References

2.1.1 Related Documents

Table 0.1: Related Documents and Specifications

Source	Title	Version
RTL Design Sherpa	RAPIDS Product Requirements Document	1.0
RTL Design Sherpa	RAPIDS Original Specification	0.25
RTL Design Sherpa	STREAM MAS (Reference Architecture)	0.90
ARM	AMBA AXI and ACE Protocol Specification	IHI0022H
ARM	AMBA AXI-Stream Protocol Specification	IHI0051A
ARM	AMBA CXS Protocol Specification	IHI0079D

2.2 Terminology

AXIS

AXI-Stream. ARM’s streaming protocol for high-bandwidth, low-latency data transfer without addressing.

AXI4

Version 4 of the AXI protocol, supporting burst transfers up to 256 beats.

Beat

A single data transfer within an AXI burst. For RAPIDS with 512-bit data width, one beat = 64 bytes.

Beats Architecture

The Phase 1 RAPIDS implementation that tracks transfers at beat granularity for precise flow control.

Burst

A group of consecutive data transfers (beats) initiated by a single address phase.

Channel

One of 8 independent DMA channels in RAPIDS. Each channel has its own descriptor chain and SRAM buffer allocation.

Descriptor

A 256-bit data structure containing transfer parameters: address, length, channel ID, and control flags.

Descriptor Chain

A linked list of descriptors where each descriptor points to the next, enabling scatter-gather operations.

DMA

Direct Memory Access. Hardware-controlled data movement between memory locations without CPU intervention.

FUB

Functional Unit Block. A self-contained RTL module with defined interfaces and testbench.

Latency Bridge

A module that provides buffering to compensate for pipeline latency between alloc/drain controllers.

MAC

Macro. An integration-level block that instantiates and connects multiple FUBs.

MonBus

Monitor Bus. A 64-bit internal bus for performance monitoring and debug event reporting.

RAPIDS

Rapid AXI Programmable In-band Descriptor System. A DMA accelerator for network-to-memory transfers.

Sink Path

The data path from network (AXIS slave) to system memory (AXI write master).

Source Path

The data path from system memory (AXI read master) to network (AXIS master).

SRAM

Static Random Access Memory. Used for internal data buffering between network and memory interfaces.

Virtual FIFO

A tracking mechanism (alloc_ctrl/drain_ctrl) that manages SRAM space without moving actual data.

2.3 "Beats" Architecture Concept

The "beats" architecture name reflects the key design decision to track all transfers at **beat granularity** (data-width units) rather than byte or chunk granularity:

1. **Beat-level Allocation:** Space is allocated in beats (e.g., 64-byte units for 512-bit datapath)
2. **Beat-level Drain:** Data availability is tracked in beats
3. **Beat-level Latency:** The latency bridge compensates for pipeline delays in beat counts
4. **Beat-level Completion:** AXI engines report completion in beats

This simplifies the math and aligns naturally with AXI burst semantics where each beat represents one data-width transfer.

2.4 Revision History

Revisions follow the convention x.y, where x is the major version and y is the minor version.

Table 0.2: RAPIDS Beats MAS Document Revision History

Rev	Date	Author	Notes
0.25	2025-01-10	seang	Initial RAPIDS Beats MAS structure
0.6	2026-07-02	seang	Resynced to RTL: scheduler recoverable write-progress timeout (`cfg_sched_timeout_limit`, strike escalation, `scheduler_idle` excludes CH_ERROR) and B-response commit gating; functional descriptor prefetch (`cfg_prefetch_enable`/`cfg_fifo_threshold`, deferred chaining); AXI write engine COMMIT strobes; commit/timeout-limit plumbing through `rapids_core_beats`; added `rapids_regs` register block (monitors @ 0x1000), `rapids_config_block`, and `rapids_beats_top` integration with the MonBus AXI-Lite

Rev	Date	Author	Notes
			group
0.7	2026-07-13	seang	Documented the control-read / control-write engine subsystem that was in the RTL but undocumented: new fub-block specs for `ctrlrd_engine` (Section 2.8, consumer gate: poll-until-`(rd & mask)==expected`, retry budget) and `ctrlwr_engine` (Section 2.9, producer doorbell: single-beat write); scheduler Control Interface + three-opcode (`DATA`/`CTRL_READ`/`CTRL_WRITE`) operation (Section 2.1); `CTRL_CONFIG` @ 0x240 (`CTRLRD_MAX_TRY`) added to the register map (Section 3.12); `scheduler_group_beats` control-engine instantiation. Naming cleanup: the per-instance register file is

Rev	Date	Author	Notes
			`rapids_engine_regs` (was `rapids_half_regs`).
0.8	2026-09-21	seang	Documentation hygiene: removed 30 `**Source:**` links pointing at `assets/mermaid/*.mmd` files that were never created (authored dead in `a1760aaf6`, 2026-07-17). None had an image above it, so no figure was lost; the 3 source links that resolve are unchanged.
0.9	2026-10-16	seang	Network planes (rapids TASK-024/025): new Section 3.13 Network Planes (the four `rapids_plane_*` modules, generated tops, CXS plane internals and parameters, per-stream framing, monitoring, test cells) and Section 4.4 CXS Interface Specification (link profile, signals, credit flow, activation,

Rev	Date	Author	Notes
			backpressure); `rapids_beats_top` page covers the base / generated split, the plane monitors on client 1 and the AXIS / CXS network ports, and its parameter and interface tables are resynced (the fill / drain rows were never top ports); core hierarchy and port list drop the monitor-lite and its three parameters, which moved to the planes. All block diagrams updated for the planes. The CXS ID mapping is marked provisional: routing and payload-type fields are planned.

Last Updated: 2026-09-21

3 Architecture Overview

Module: `rapids_core_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

3.1 Overview

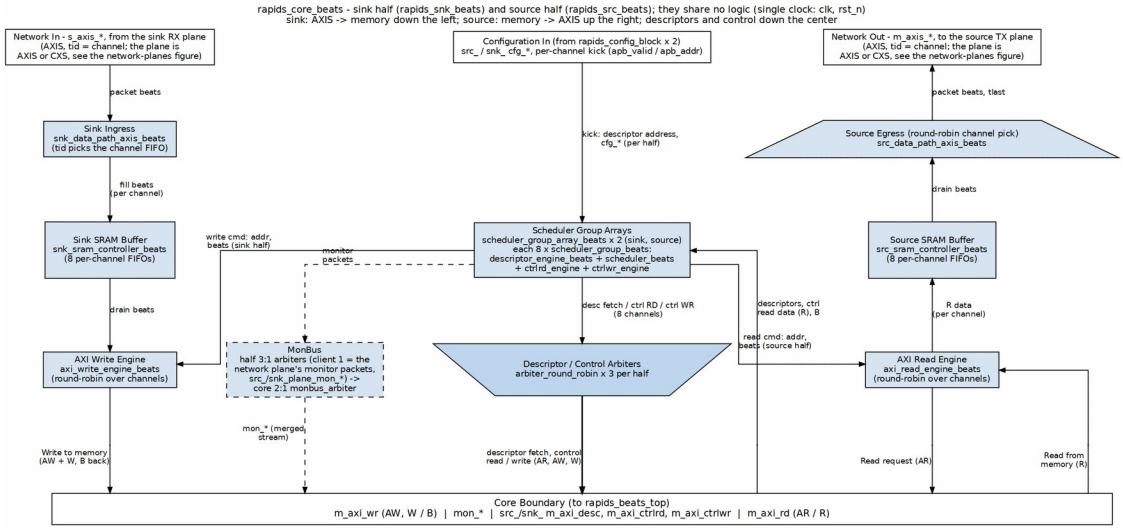
The RAPIDS "Beats" architecture is a Phase 1 implementation providing network-to-memory and memory-to-network data transfer capabilities. The name "beats" reflects the design decision to track all transfers at beat granularity (data-width units) for simplified flow control.

3.1.1 Key Features

- **8 Independent Channels:** Each channel has its own descriptor chain and SRAM allocation
- **Separate Data Paths:** Sink (network-to-memory) and source (memory-to-network) paths
- **Beat-Level Tracking:** All flow control operates at beat granularity
- **SRAM Buffering:** Separate sink and source SRAM buffers for data staging
- **MonBus Integration:** Comprehensive monitoring for all subsystems
- **Streaming Pipelines:** No FSM in data engines - pure streaming for maximum throughput

3.1.2 Block Diagram

3.1.3 Figure 1.1.1: RAPIDS Beats Architecture Block Diagram



rapids_core_beats block diagram

Source: 19_rapids_core.dot

3.2 Data Flow Overview

3.2.1 Sink Path (Network to Memory)

The sink path receives data from an external source (via Fill interface) and writes it to system memory:

3.2.2 Figure 1.1.2: Sink Path Data Flow

```
1. External Fill Valid
        |
        v
2. sram_controller (STREAM): stream_alloc_ctrl reserves space
        |
        v
3. sram_controller: per-channel gaxi_fifo_sync takes the beats
        |
        v
4. sram_controller: stream_drain_ctrl reports data available
        |
        v
5. axi_write_engine_beats (AXI Burst Write)
```

```
        |
        v
6. System Memory
```

3.2.3 Source Path (Memory to Network)

The source path reads data from system memory and sends it to an external destination (via Drain interface):

3.2.4 Figure 1.1.3: Source Path Data Flow

```
1. Scheduler Request
        |
        v
2. sram_controller (STREAM): stream_alloc_ctrl reserves space
        |
        v
3. axi_read_engine_beats (AXI Burst Read)
        |
        v
4. sram_controller: per-channel gaxi_fifo_sync takes the beats
        |
        v
5. sram_controller: stream_drain_ctrl reports data available
        |
        v
6. External Drain Ready
```

3.3 Key Architectural Decisions

3.3.1 Beat-Level Tracking

All flow control uses beat granularity:

Table 1.1.1: Beat Granularity Examples

Operation	Granularity	Example (512-bit DW)
Space allocation	Beats	Request 8 beats = 512 bytes
Data availability	Beats	16 beats ready = 1024 bytes
AXI burst length	Beats	ARLEN=15 = 16 beats
Latency compensation	Beats	2-beat pipeline delay

3.3.2 Concurrent Read/Write

To prevent deadlock with large transfers:

```
Example: 100MB transfer with 2KB SRAM buffer

Sequential operation (WRONG):
1. Read 100MB -> DEADLOCK at 2KB (SRAM full, can't complete read)

Concurrent operation (CORRECT):
1. Read starts filling SRAM -> SRAM becomes full (2KB)
2. Read pauses (natural backpressure)
3. Write drains SRAM -> SRAM has free space
4. Read resumes -> Both continue until 100MB complete
```

3.3.3 Virtual FIFOs (Alloc/Drain)

The alloc_ctrl and drain_ctrl modules are “virtual FIFOs” that track space/data without storing actual data:

- **stream_alloc_ctrl:** Tracks allocated space (write pointer advances on allocation, read pointer on actual write)
- **stream_drain_ctrl:** Tracks available data (write pointer advances on data arrival, read pointer on drain reservation). Its virtual depth is 2 x SRAM_DEPTH so the latency bridge’s parked beats never make it refuse a real write (stream BUG-011)
- Both live inside STREAM’s `sram_controller_unit`, one per channel, reached through the `snk_/src_sram_controller_beats` naming wrappers since `bdf4e0dff`. RAPIDS’ own `alloc_ctrl_beats` / `drain_ctrl_beats` / `latency_bridge_beats` are kept and tested but no longer in this path.

3.4 Module Hierarchy

The core is wrapped by a generated top: `rapids_beats_top` (AXIS network planes) or `rapids_beats_cxs_top` (CXS network planes), each `rapids_beats_top_base` plus an RX plane in front of the sink half and a TX plane behind the source half. See Network Planes.

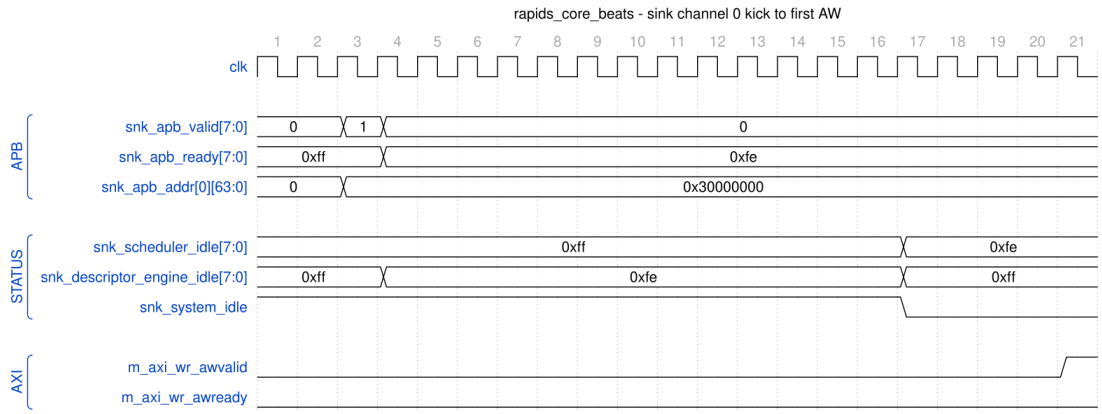
```
rapids_beats_top | rapids_beats_cxs_top  (generated,
bin/gen_plane_tops.py)
├── u_base: rapids_beats_top_base        (regs, config blocks, monbus
group)
│   └── u_core: rapids_core_beats        (below)
├── u_snk_plane: rapids_plane_axis_rx | rapids_plane_cxs_rx
└── u_src_plane: rapids_plane_axis_tx | rapids_plane_cxs_tx
```

```
rapids_core_beats
├── monbus_arbiter                        (merges the two halves'
monbus)
├── rapids_src_beats                      (SOURCE half)
│   ├── scheduler_group_array_beats
│   │   ├── scheduler_group_beats [0..7]
│   │   │   ├── scheduler_beats
│   │   │   ├── descriptor_engine_beats
│   │   │   ├── ctrlrd_engine
│   │   │   └── ctrlwr_engine
│   │   ├── arbiter_round_robin [x3]     (desc / ctrlrd / ctrlwr AXI
masters)
│   │   ├── axi4_master_rd_monlite       (descriptor-fetch monitor)
│   │   └── axi_bus_meter
│   ├── monbus_arbiter (3:1)             (client 1 = src_plane_mon_*)
│   └── src_data_path_axis_beats
│       └── src_data_path_beats
│           ├── axi_read_engine_beats
│           └── src_sram_controller_beats    (naming wrapper)
│               └── sram_controller          (STREAM, shared)
│                   └── sram_controller_unit [0..7]
│                       ├── stream_alloc_ctrl
│                       ├── gaxi_fifo_sync   (the per-channel SRAM)
│                       ├── stream_drain_ctrl
│                       └── stream_latency_bridge
└── rapids_snk_beats                      (SINK half, same shape)
    ├── scheduler_group_array_beats
    │   └── ...
    ├── monbus_arbiter (3:1)             (client 1 = snk_plane_mon_*)
    └── snk_data_path_axis_beats
        └── snk_data_path_beats
            ├── axi_write_engine_beats
            └── snk_sram_controller_beats
                └── sram_controller (STREAM, shared)
                    └── sram_controller_unit [0..7]
```

`alloc_ctrl_beats`, `drain_ctrl_beats` and `latency_bridge_beats` (fub_beats) are not in this tree: since `bdf4e0dff` the SRAM path is STREAM's controller. They keep their tests and filelists as standalone FUBs.

3.5 Timing Diagram: Basic Transfer

3.5.1 Figure 1.1.4: Basic Sink Path Transfer Timing



rapids_core_beats - sink channel 0 kick to first AW

Source: rapids_core_beats_sink_kick.json, captured from `dv/tests/top_beats/test_rapids_core_beats.py` (sink path, channel 0, 32 beats, 512-bit data, `TEST_LEVEL=full`) with `WAVES=1`.

Reading it: the AXIS packet is already sitting in the sink SRAM when software kicks channel 0 (`snk_apb_valid[0]` for one cycle with the descriptor address 0x30000000). `snk_apb_ready[0]` drops the same cycle and stays low while the channel is busy. The descriptor engine leaves idle immediately to fetch the descriptor; twelve cycles later it hands the descriptor to the scheduler, its own idle returns, and `snk_scheduler_idle[0]` and `snk_system_idle` fall together. The first AW follows four cycles after that. `snk_system_idle` returns after the last B response of the transfer (not in this window); the busy-then-idle sequence is what the top-level tests wait on.

3.6 Related Documentation

- **Top-Level Port List** - Complete port specification
- **Clocks and Reset** - Timing requirements
- **Beats Scheduler Group** - Scheduler integration
- **Sink Data Path** - Sink path details

Last Updated: 2025-01-10

4 Top-Level Port List

Module: `rapids_core_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2026-10-16

4.1 Parameters

Every parameter of `rapids_core_beats`, as declared. The short aliases at the end are what the port widths below are written in (`NC`, `AW`, `DW`, `IW`, `SCW`, `CIW`, `SW`).

Table 1.2.1: Parameters (from `rapids_core_beats.sv`)

Parameter	Type	Default	Notes
`NUM_CHANNELS`	int	8	-
`CHAN_WIDTH`	int	`$clog2(NUM_CHANNELS)`	-
`ADDR_WIDTH`	int	64	-
`DATA_WIDTH`	int	512	-
`AXI_ID_WIDTH`	int	8	-
`SRAM_DEPTH`	int	512	-
`SEG_COUNT_WIDTH`	int	`$clog2(SRAM_DEPTH) + 1`	-
`PIPELINE`	int	1	write-engine pipelining; 1 is the perf design point (see rapids_beats_top)
`AR_MAX_OUTSTANDING`	int	8	-
`AW_MAX_OUTSTANDING`	int	8	-
`W_PHASE_FIFO_DEPTH`	int	64	-
`B_PHASE_FIFO_DEPTH`	int	16	-
`AXIS_ID_WIDTH`	int	8	-
`AXIS_DEST_WIDTH`	int	4	-

Parameter	Type	Default	Notes
AXIS_USER_WIDTH	int	1	-
DESC_MON_BASE_AGENT_ID	int	16	0x10 - Descriptor Engines (16-23)
SCHED_MON_BASE_AGENT_ID	int	48	0x30 - Schedulers (48-55)
DESC_AXI_MON_AGENT_ID	int	8	0x08 - Descriptor AXI Master Monitor
MON_UNIT_ID	int	1	0x1
MON_MAX_TRANSACTIONS	int	16	-
USE_AXI_MONITORS	int	1	-
USE_ROW_COL_MAJOR_ADDRESSING	int	1	-
GEN_MON	bit	1'b1	-
NC	int	NUM_CHANNELS	-
AW	int	ADDR_WIDTH	-
DW	int	DATA_WIDTH	-
IW	int	AXI_ID_WIDTH	-
SD	int	SRAM_DEPTH	-
SCW	int	SEG_COUNT_WIDTH	-
CIW	int	(NC > 1) ? $clog2(NC) : 1	-
SW	int	DW / 8	-

4.2 Port List (324 ports)

The module is two independent halves. Ports both halves carry are prefixed `src_` / `snk_`; the direction-unique ports (the source's AXI read master and AXIS egress, the sink's AXI write master and AXIS ingress) carry no prefix. Sections follow the declaration order in the RTL, so a reader can walk this page and the module side by side. Widths use the parameter aliases.

4.2.1 Clock and Reset (shared - single clock/reset for the whole wrapper)

Table 1.2.2: Clock and Reset (shared - single clock/reset for the whole wrapper)

Signal	Direction	Width	Description
`clk`	input	1	-
`rst_n`	input	1	-

4.3 SOURCE HALF (u_src) - shared-infrastructure ports (src_ prefixed)

4.3.1 APB Programming Interface

Table 1.2.3: APB Programming Interface

Signal	Direction	Width	Description
`src_apb_valid`	input	NC	-
`src_apb_ready`	output	NC	-
`src_apb_addr`	input	NC x AW	-

4.3.2 Per-channel configuration

Table 1.2.4: Per-channel configuration

Signal	Direction	Width	Description
`src_cfg_channel_enable`	input	NC	-
`src_cfg_channel_reset`	input	NC	-

4.3.3 Scheduler Configuration (global)

Table 1.2.5: Scheduler Configuration (global)

Signal	Direction	Width	Description
`src_cfg_sched_enable`	input	1	-
`src_cfg_sched_timeout_cycles`	input	32	-
`src_cfg_sched_timeout_limit`	input	8	-
`src_cfg_sched_timeout_enab`	input	1	-

Signal	Direction	Width	Description
le			
src_cfg_sched_err_enable	input	1	-
src_cfg_sched_compl_enable	input	1	-
src_cfg_sched_perf_enable	input	1	-

4.3.4 Descriptor Engine Configuration (global)

Table 1.2.6: Descriptor Engine Configuration (global)

Signal	Direction	Width	Description
src_cfg_desceng_enable	input	1	-
src_cfg_desceng_prefetch	input	1	-
src_cfg_desceng_fifo_thresh	input	4	-
src_cfg_desceng_addr0_base	input	AW	-
src_cfg_desceng_addr0_limit	input	AW	-
src_cfg_desceng_addr1_base	input	AW	-
src_cfg_desceng_addr1_limit	input	AW	-

4.3.5 Control Engine Configuration (Phase 2, global)

Table 1.2.7: Control Engine Configuration (Phase 2, global)

Signal	Direction	Width	Description
src_cfg_ctrlrd_max_try	input	9	-
src_tick_1us	input	1	-

4.3.6 Descriptor AXI Monitor Configuration

Table 1.2.8: Descriptor AXI Monitor Configuration

Signal	Direction	Width	Description
src_cfg_desc_mon_enable	input	1	-
src_cfg_desc_mon_err_enable	input	1	-
src_cfg_desc_mon_perf_enable	input	1	-
src_cfg_desc_mon_perf_run	input	1	-
src_cfg_desc_mon_timeout_enable	input	1	-
src_cfg_desc_mon_timeout_cycles	input	32	-
src_cfg_desc_mon_latency_thresh	input	32	-
src_cfg_desc_mon_pkt_mask	input	16	-
src_cfg_desc_mon_err_select	input	4	-
src_cfg_desc_mon_err_mask	input	8	-
src_cfg_desc_mon_timeout_mask	input	8	-
src_cfg_desc_mon_compl_mask	input	8	-
src_cfg_desc_mon_thresh_mask	input	8	-
src_cfg_desc_mon_perf_mask	input	8	-
src_cfg_desc_mon_addr_mask	input	8	-

Signal	Direction	Width	Description
src_cfg_desc_mon_debug_mask	input	8	-

4.3.7 Network-plane monitor input (rapids TASK-024)

The network endpoint and its monitor-lite live in the plane (rtl/plane/) in the top. Its monbus packets enter here as client 1 of this half's monbus arbiter; the plane's monitor configuration and status come from this half's register block in the top.

Table 1.2.9: Network-plane monitor input (rapids TASK-024)

Signal	Direction	Width	Description
src_plane_mon_valid	input	1	-
src_plane_mon_ready	output	1	-
src_plane_mon_packet	input	128	monitor_packet_t
src_plane_mon_timestamp	input	64	monbus_timestamp_t

4.3.8 Status

Table 1.2.10: Status

Signal	Direction	Width	Description
src_system_idle	output	1	-
src_descriptor_engine_idle	output	NC	-
src_scheduler_idle	output	NC	-
src_scheduler_state	output	NC x 7	-
src_sched_error	output	NC	-

4.3.9 Descriptor AXI Monitor Status

Table 1.2.11: Descriptor AXI Monitor Status

Signal	Direction	Width	Description
src_cfg_sts_desc_mon_busy	output	1	-

Signal	Direction	Width	Description
src_cfg_sts_desc_mon_active_txns	output	8	-
src_cfg_sts_desc_mon_error_count	output	16	-
src_cfg_sts_desc_mon_txn_count	output	32	-
src_cfg_sts_desc_mon_conflict_error	output	1	-

4.3.10 Descriptor AXI Monitor perf window (feeds SRC_.MON.DAXMON_PERF_*).

Table 1.2.12: Descriptor AXI Monitor perf window (feeds SRC_.MON.DAXMON_PERF_*).

Signal	Direction	Width	Description
src_sts_desc_mon_win_active	output	1	-
src_sts_desc_mon_win_cycles	output	32	-
src_sts_desc_mon_prod_cycles	output	32	-
src_sts_desc_mon_bp_cycles	output	32	-
src_sts_desc_mon_starv_cycles	output	32	-
src_sts_desc_mon_idle_cycles	output	32	-
src_sts_desc_mon_beat_count	output	32	-
src_sts_desc_mon_byte_count	output	64	-

Signal	Direction	Width	Description
src_sts_desc_mon_burst_count	output	32	-

4.3.11 Descriptor Fetch AXI Master

Table 1.2.13: Descriptor Fetch AXI Master

Signal	Direction	Width	Description
src_m_axi_desc_arvalid	output	1	-
src_m_axi_desc_arready	input	1	-
src_m_axi_desc_araddr	output	AW	-
src_m_axi_desc_arlen	output	8	-
src_m_axi_desc_arsize	output	3	-
src_m_axi_desc_arburst	output	2	-
src_m_axi_desc_arid	output	IW	-
src_m_axi_desc_arlock	output	1	-
src_m_axi_desc_arcache	output	4	-
src_m_axi_desc_arprot	output	3	-
src_m_axi_desc_arqos	output	4	-
src_m_axi_desc_arregion	output	4	-
src_m_axi_desc_rvalid	input	1	-
src_m_axi_desc_rready	output	1	-
src_m_axi_desc_rdata	input	256	-
src_m_axi_desc_rresp	input	2	-

Signal	Direction	Width	Description
src_m_axi_desc_rlast	input	1	-
src_m_axi_desc_rid	input	IW	-

4.3.12 Control Read AXI Master (32-bit) [Phase 2]

Table 1.2.14: Control Read AXI Master (32-bit) [Phase 2]

Signal	Direction	Width	Description
src_m_axi_ctrlrd_arvalid	output	1	-
src_m_axi_ctrlrd_arready	input	1	-
src_m_axi_ctrlrd_araddr	output	AW	-
src_m_axi_ctrlrd_arlen	output	8	-
src_m_axi_ctrlrd_arsize	output	3	-
src_m_axi_ctrlrd_arburst	output	2	-
src_m_axi_ctrlrd_arid	output	IW	-
src_m_axi_ctrlrd_arlock	output	1	-
src_m_axi_ctrlrd_arcache	output	4	-
src_m_axi_ctrlrd_arprot	output	3	-
src_m_axi_ctrlrd_arqos	output	4	-
src_m_axi_ctrlrd_arregion	output	4	-
src_m_axi_ctrlrd_rvalid	input	1	-
src_m_axi_ctrlrd_rready	output	1	-
src_m_axi_ctrlrd_rdata	input	32	-
src_m_axi_ctrlrd_rresp	input	2	-

Signal	Direction	Width	Description
src_m_axi_ctrlrd_rlast	input	1	-
src_m_axi_ctrlrd_rid	input	IW	-

4.3.13 Control Write AXI Master (32-bit) [Phase 2]

Table 1.2.15: Control Write AXI Master (32-bit) [Phase 2]

Signal	Direction	Width	Description
src_m_axi_ctrlwr_awvalid	output	1	-
src_m_axi_ctrlwr_awready	input	1	-
src_m_axi_ctrlwr_awaddr	output	AW	-
src_m_axi_ctrlwr_awlen	output	8	-
src_m_axi_ctrlwr_awsize	output	3	-
src_m_axi_ctrlwr_awburst	output	2	-
src_m_axi_ctrlwr_awid	output	IW	-
src_m_axi_ctrlwr_awlock	output	1	-
src_m_axi_ctrlwr_awcache	output	4	-
src_m_axi_ctrlwr_awprot	output	3	-
src_m_axi_ctrlwr_awqos	output	4	-
src_m_axi_ctrlwr_awregion	output	4	-
src_m_axi_ctrlwr_wvalid	output	1	-
src_m_axi_ctrlwr_wready	input	1	-
src_m_axi_ctrlwr_wdata	output	32	-
src_m_axi_ctrlwr_wstrb	output	4	-

Signal	Direction	Width	Description
src_m_axi_ctrlwr_wlast	output	1	-
src_m_axi_ctrlwr_bvalid	input	1	-
src_m_axi_ctrlwr_bready	output	1	-
src_m_axi_ctrlwr_bid	input	IW	-
src_m_axi_ctrlwr_bresp	input	2	-

4.4 SOURCE HALF (u_src) - direction-unique ports (no prefix)

4.4.1 AXI Transfer Configuration (source-only)

Table 1.2.16: AXI Transfer Configuration (source-only)

Signal	Direction	Width	Description
cfg_axi_rd_xfer_beats	input	8	-
cfg_drain_size	input	8	source: beats drained per AXIS packet

4.4.2 Source Path - AXIS Master Interface (SRAM -> Network); tid = channel id

Table 1.2.17: Source Path - AXIS Master Interface (SRAM -> Network); tid = channel id

Signal	Direction	Width	Description
m_axis_tdata	output	DW	-
m_axis_tstrb	output	SW	-
m_axis_tlast	output	1	-
m_axis_tid	output	AXIS_ID_WIDTH	-
m_axis_tdest	output	AXIS_DEST_WIDTH	-
m_axis_tuser	output	AXIS_USER_WIDTH	-
m_axis_tvalid	output	1	-

Signal	Direction	Width	Description
m_axis_tready	input	1	-

4.4.3 AXI4 Master - Data Read (Memory -> Source SRAM)

Table 1.2.18: AXI4 Master - Data Read (Memory -> Source SRAM)

Signal	Direction	Width	Description
m_axi_rd_arid	output	IW	-
m_axi_rd_araddr	output	AW	-
m_axi_rd_arlen	output	8	-
m_axi_rd_arsize	output	3	-
m_axi_rd_arburst	output	2	-
m_axi_rd_arvalid	output	1	-
m_axi_rd_arready	input	1	-
m_axi_rd_rid	input	IW	-
m_axi_rd_rdata	input	DW	-
m_axi_rd_rresp	input	2	-
m_axi_rd_rlast	input	1	-
m_axi_rd_rvalid	input	1	-
m_axi_rd_rready	output	1	-

4.5 SINK HALF (u_snk) - shared-infrastructure ports (snk_ prefixed)

4.5.1 APB Programming Interface

Table 1.2.19: APB Programming Interface

Signal	Direction	Width	Description
snk_apb_valid	input	NC	-
snk_apb_ready	output	NC	-
snk_apb_addr	input	NC x AW	-

4.5.2 Per-channel configuration

Table 1.2.20: Per-channel configuration

Signal	Direction	Width	Description
snk_cfg_channel_enable	input	NC	-
snk_cfg_channel_reset	input	NC	-

4.5.3 Scheduler Configuration (global)

Table 1.2.21: Scheduler Configuration (global)

Signal	Direction	Width	Description
snk_cfg_sched_enable	input	1	-
snk_cfg_sched_timeout_cycles	input	32	-
snk_cfg_sched_timeout_limit	input	8	-
snk_cfg_sched_timeout_enable	input	1	-
snk_cfg_sched_err_enable	input	1	-
snk_cfg_sched_compl_enable	input	1	-
snk_cfg_sched_perf_enable	input	1	-

4.5.4 Descriptor Engine Configuration (global)

Table 1.2.22: Descriptor Engine Configuration (global)

Signal	Direction	Width	Description
snk_cfg_desceng_enable	input	1	-
snk_cfg_desceng_prefetch	input	1	-
snk_cfg_desceng_fifo_thresh	input	4	-
snk_cfg_desceng_addr0_base	input	AW	-
snk_cfg_desceng_addr0_limit	input	AW	-
snk_cfg_desceng_addr1_base	input	AW	-
snk_cfg_desceng_addr1_limit	input	AW	-

4.5.5 Control Engine Configuration (Phase 2, global)

Table 1.2.23: Control Engine Configuration (Phase 2, global)

Signal	Direction	Width	Description
snk_cfg_ctrlrd_max_try	input	9	-
snk_tick_1us	input	1	-

4.5.6 Descriptor AXI Monitor Configuration

Table 1.2.24: Descriptor AXI Monitor Configuration

Signal	Direction	Width	Description
snk_cfg_desc_mon_enable	input	1	-
snk_cfg_desc_mon_err_enable	input	1	-
snk_cfg_desc_mon_perf_enable	input	1	-
snk_cfg_desc_	input	1	-

Signal	Direction	Width	Description
mon_perf_run			
snk_cfg_desc_mon_timeout_enable	input	1	-
snk_cfg_desc_mon_timeout_cycles	input	32	-
snk_cfg_desc_mon_latency_thresh	input	32	-
snk_cfg_desc_mon_pkt_mask	input	16	-
snk_cfg_desc_mon_err_select	input	4	-
snk_cfg_desc_mon_err_mask	input	8	-
snk_cfg_desc_mon_timeout_mask	input	8	-
snk_cfg_desc_mon_compl_mask	input	8	-
snk_cfg_desc_mon_thresh_mask	input	8	-
snk_cfg_desc_mon_perf_mask	input	8	-
snk_cfg_desc_mon_addr_mask	input	8	-
snk_cfg_desc_mon_debug_mask	input	8	-

4.5.7 Network-plane monitor input (rapids TASK-024)

The network endpoint and its monitor-lite live in the plane (`rtl/plane/`) in the top. Its monbus packets enter here as client 1 of this half's monbus arbiter; the plane's monitor configuration and status come from this half's register block in the top.

Table 1.2.25: Network-plane monitor input (rapids TASK-024)

Signal	Direction	Width	Description
snk_plane_mon_valid	input	1	-
snk_plane_mon_ready	output	1	-
snk_plane_mon_packet	input	128	monitor_packet_t
snk_plane_mon_timestamp	input	64	monbus_timestamp_t

4.5.8 Status

Table 1.2.26: Status

Signal	Direction	Width	Description
snk_system_idle	output	1	-
snk_descriptor_engine_idle	output	NC	-
snk_scheduler_idle	output	NC	-
snk_scheduler_state	output	NC x 7	-
snk_sched_error	output	NC	-

4.5.9 Descriptor AXI Monitor Status

Table 1.2.27: Descriptor AXI Monitor Status

Signal	Direction	Width	Description
snk_cfg_sts_desc_mon_busy	output	1	-
snk_cfg_sts_desc_mon_active_txns	output	8	-
snk_cfg_sts_desc_mon_error_count	output	16	-
snk_cfg_sts_desc_mon_txn_count	output	32	-
snk_cfg_sts_d	output	1	-

Signal	Direction	Width	Description
esc_mon_conflict_error			

4.5.10 Descriptor AXI Monitor perf window (feeds SNK_.MON.DAXMON_PERF_*).

Table 1.2.28: Descriptor AXI Monitor perf window (feeds SNK_.MON.DAXMON_PERF_*).

Signal	Direction	Width	Description
snk_sts_desc_mon_win_active	output	1	-
snk_sts_desc_mon_win_cycles	output	32	-
snk_sts_desc_mon_prod_cycles	output	32	-
snk_sts_desc_mon_bp_cycles	output	32	-
snk_sts_desc_mon_starv_cycles	output	32	-
snk_sts_desc_mon_idle_cycles	output	32	-
snk_sts_desc_mon_beat_count	output	32	-
snk_sts_desc_mon_byte_count	output	64	-
snk_sts_desc_mon_burst_count	output	32	-

4.5.11 Descriptor Fetch AXI Master

Table 1.2.29: Descriptor Fetch AXI Master

Signal	Direction	Width	Description
snk_m_axi_desc_arvalid	output	1	-

Signal	Direction	Width	Description
snk_m_axi_desc_arready	input	1	-
snk_m_axi_desc_araddr	output	AW	-
snk_m_axi_desc_arlen	output	8	-
snk_m_axi_desc_arsize	output	3	-
snk_m_axi_desc_arburst	output	2	-
snk_m_axi_desc_arid	output	IW	-
snk_m_axi_desc_arlock	output	1	-
snk_m_axi_desc_arcache	output	4	-
snk_m_axi_desc_arprot	output	3	-
snk_m_axi_desc_arqos	output	4	-
snk_m_axi_desc_arregion	output	4	-
snk_m_axi_desc_rvalid	input	1	-
snk_m_axi_desc_rready	output	1	-
snk_m_axi_desc_rdata	input	256	-
snk_m_axi_desc_rresp	input	2	-
snk_m_axi_desc_rlast	input	1	-
snk_m_axi_desc_rid	input	IW	-

4.5.12 Control Read AXI Master (32-bit) [Phase 2]

Table 1.2.30: Control Read AXI Master (32-bit) [Phase 2]

Signal	Direction	Width	Description
snk_m_axi_ctrlrd_arvalid	output	1	-

Signal	Direction	Width	Description
snk_m_axi_ctrlrd_arready	input	1	-
snk_m_axi_ctrlrd_araddr	output	AW	-
snk_m_axi_ctrlrd_arlen	output	8	-
snk_m_axi_ctrlrd_arsize	output	3	-
snk_m_axi_ctrlrd_arburst	output	2	-
snk_m_axi_ctrlrd_arid	output	IW	-
snk_m_axi_ctrlrd_arlock	output	1	-
snk_m_axi_ctrlrd_arcache	output	4	-
snk_m_axi_ctrlrd_arprot	output	3	-
snk_m_axi_ctrlrd_arqos	output	4	-
snk_m_axi_ctrlrd_arregion	output	4	-
snk_m_axi_ctrlrd_rvalid	input	1	-
snk_m_axi_ctrlrd_rready	output	1	-
snk_m_axi_ctrlrd_rdata	input	32	-
snk_m_axi_ctrlrd_rresp	input	2	-
snk_m_axi_ctrlrd_rlast	input	1	-
snk_m_axi_ctrlrd_rid	input	IW	-

4.5.13 Control Write AXI Master (32-bit) [Phase 2]

Table 1.2.31: Control Write AXI Master (32-bit) [Phase 2]

Signal	Direction	Width	Description
snk_m_axi_ctrlwr_awvalid	output	1	-

Signal	Direction	Width	Description
snk_m_axi_ctrlwr_awready	input	1	-
snk_m_axi_ctrlwr_awaddr	output	AW	-
snk_m_axi_ctrlwr_awlen	output	8	-
snk_m_axi_ctrlwr_awsize	output	3	-
snk_m_axi_ctrlwr_awburst	output	2	-
snk_m_axi_ctrlwr_awid	output	IW	-
snk_m_axi_ctrlwr_awlock	output	1	-
snk_m_axi_ctrlwr_awcache	output	4	-
snk_m_axi_ctrlwr_awprot	output	3	-
snk_m_axi_ctrlwr_awqos	output	4	-
snk_m_axi_ctrlwr_awregion	output	4	-
snk_m_axi_ctrlwr_wvalid	output	1	-
snk_m_axi_ctrlwr_wready	input	1	-
snk_m_axi_ctrlwr_wdata	output	32	-
snk_m_axi_ctrlwr_wstrb	output	4	-
snk_m_axi_ctrlwr_wlast	output	1	-
snk_m_axi_ctrlwr_bvalid	input	1	-
snk_m_axi_ctrlwr_bready	output	1	-
snk_m_axi_ctrlwr_bid	input	IW	-
snk_m_axi_ctrlwr_bresp	input	2	-

4.6 Monitor Bus (SINGLE aggregated stream for the whole core)

The two halves' monitor outputs are merged through a top-level monbus_arbiter, so the core exposes exactly one monitor stream.

4.6.1 Monitor Bus

Table 1.2.32: Monitor Bus

Signal	Direction	Width	Description
mon_valid	output	1	-
mon_ready	input	1	-
mon_packet	output	monitor_common_pkg::monitor_packet_t	-
mon_timestamp	output	monitor_common_pkg::monbus_timestamp_t	-

4.7 SINK HALF (u_snk) - direction-unique ports (no prefix)

4.7.1 AXI Transfer Configuration (sink-only)

Table 1.2.33: AXI Transfer Configuration (sink-only)

Signal	Direction	Width	Description
cfg_axi_wr_xfer_beats	input	8	-
cfg_alloc_size	input	8	sink: SRAM alloc size per AXIS fill

4.7.2 Sink Path - AXIS Slave Interface (Network -> SRAM); tid = channel id

Table 1.2.34: Sink Path - AXIS Slave Interface (Network -> SRAM); tid = channel id

Signal	Direction	Width	Description
s_axis_tdata	input	DW	-
s_axis_tstrb	input	SW	-
s_axis_tlast	input	1	-
s_axis_tid	input	AXIS_ID_WIDT	-

Signal	Direction	Width	Description
		H	
s_axis_tdest	input	AXIS_DEST_WIDTH	-
s_axis_tuser	input	AXIS_USER_WIDTH	-
s_axis_tvalid	input	1	-
s_axis_tready	output	1	-

4.7.3 AXI4 Master - Data Write (Sink SRAM -> Memory)

Table 1.2.35: AXI4 Master - Data Write (Sink SRAM -> Memory)

Signal	Direction	Width	Description
m_axi_wr_awid	output	IW	-
m_axi_wr_awaddr	output	AW	-
m_axi_wr_awlen	output	8	-
m_axi_wr_awsize	output	3	-
m_axi_wr_awburst	output	2	-
m_axi_wr_awlock	output	1	-
m_axi_wr_awcache	output	4	-
m_axi_wr_awprot	output	3	-
m_axi_wr_awqos	output	4	-
m_axi_wr_awregion	output	4	-
m_axi_wr_awvalid	output	1	-
m_axi_wr_awready	input	1	-
m_axi_wr_wdata	output	DW	-
m_axi_wr_wstrb	output	(DW/8)	-

Signal	Direction	Width	Description
m_axi_wr_wlast	output	1	-
m_axi_wr_wvalid	output	1	-
m_axi_wr_wready	input	1	-
m_axi_wr_bid	input	IW	-
m_axi_wr_bresp	input	2	-
m_axi_wr_bvalid	input	1	-
m_axi_wr_bready	output	1	-

4.8 Debug Interface (per half, src_dbg_ / snk_dbg_ prefixed)

4.8.1 Source half debug

Table 1.2.36: Source half debug

Signal	Direction	Width	Description
src_dbg_rd_all_complete	output	NC	-
src_dbg_r_beats_rcvd	output	32	-
src_dbg_sram_writes	output	32	-
src_dbg_arb_request	output	NC	-
src_dbg_src_sram_bridge_pending	output	NC	-
src_dbg_src_sram_bridge_out_valid	output	NC	-
src_dbg_axis_beats_sent	output	32	-
src_dbg_axis_packets_sent	output	32	-

4.8.2 Sink half debug

Table 1.2.37: Sink half debug

Signal	Direction	Width	Description
snk_dbg_snk_sram_bridge_pending	output	NC	-
snk_dbg_snk_sram_bridge_out_valid	output	NC	-
snk_dbg_axis_beats_received	output	32	-
snk_dbg_axis_packets_received	output	32	-

4.8.3 Active-channel sideband for per-channel bus instrumentation

(axi_bus_meter). The W bus carries no wid, so the write engine's channel index must travel out of band to reach the meter at the top.

Table 1.2.38: Active-channel sideband for per-channel bus instrumentation

Signal	Direction	Width	Description
snk_active_channel_id	output	CIW	-
snk_active_channel_valid	output	1	-

Verified: every row above is a declared port of `rapids_core_beats` and every declared port has a row (324 of 324), generated from the module declaration on 2026-09-28. Regenerate rather than hand-edit when the interface changes.

5 Clocks and Reset

Module: `rapids_core_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

5.1 Clock Domain

RAPIDS Beats operates in a single clock domain for simplified design and maximum throughput.

5.1.1 Clock Specifications

Table 1.3.1: Clock Specifications

Parameter	Specification
Clock Signal	`clk` (also `axi_aclk` in some submodules)
Frequency Range	100 - 500 MHz
Duty Cycle	45% - 55%
Clock Jitter	< 100 ps peak-to-peak

5.1.2 Clock Distribution

```
clk (one clock domain; rst_n active-low, async assert)
├── scheduler_group_array_beats (one per half)
│   └── descriptor_engine_beats, scheduler_beats, ctrlrd_engine,
ctrlwr_engine (x8)
├── snk_data_path_axis_beats
│   └── snk_sram_controller_beats, axi_write_engine_beats
└── src_data_path_axis_beats
    └── axi_read_engine_beats, src_sram_controller_beats
```

5.2 Reset Specification

RAPIDS uses asynchronous assert, synchronous deassert reset methodology.

5.2.1 Reset Signal

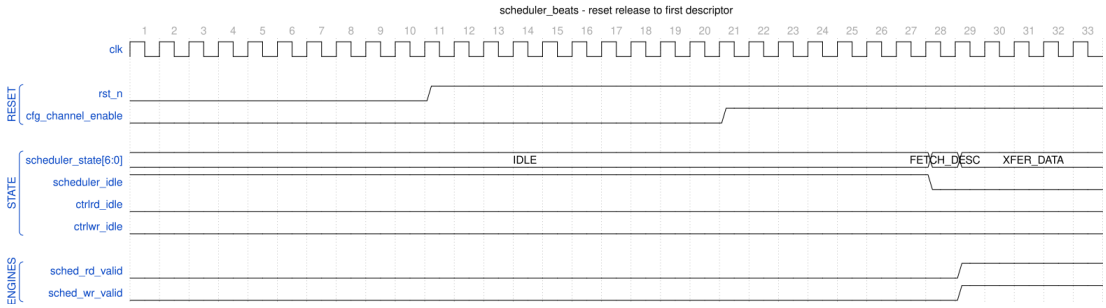
Table 1.3.2: Reset Specifications

Parameter	Specification

Reset Signal	`rst_n` (also `axi_aresetn` in some submodules)
Polarity	Active-low
Assert	Asynchronous (immediate)
Deassert	Synchronous (on rising clock edge)
Minimum Duration	10 clock cycles

5.2.2 Reset Timing Diagram

5.2.3 Figure 1.3.1: Reset Timing



scheduler_beats - reset release to first descriptor

Source: scheduler_reset.json, captured from `dv/tests/fub_beats/test_scheduler_beats.py` (basic transfer, the same run as Figure 2.1.3) with `WAVES=1`.

Reading it: `rst_n` is held low for ten cycles and released on a clock edge. Every state register is already at its reset value while reset is asserted – the FSM sits in `IDLE`, `scheduler_idle` is high, both control engines report idle and neither engine request is raised. Nothing changes at the release edge itself; the scheduler stays in `IDLE` until the testbench raises `cfg_channel_enable` ten cycles later and a descriptor becomes available, at which point it steps through `FETCH_DESC` to `XFER_DATA` and raises the read and write requests together.

5.3 Reset Behavior

5.3.1 On Reset Assert

All modules initialize to known states:

Table 1.3.3: Reset States

Component	Reset State
Scheduler FSM	IDLE
Descriptor Engine	IDLE, FIFOs empty
AXI Read Engine	Idle, no outstanding transactions
AXI Write Engine	Idle, no outstanding transactions
SRAM Pointers	All zeros
Alloc/Drain Controllers	Empty, full space available
MonBus	No packets pending

5.3.2 Reset Sequence

```
1. External reset asserted (rst_n = 0)
   - All flip-flops async reset
   - All outputs driven to safe states
   - No AXI transactions

2. System stabilization (minimum 10 cycles)
   - Clock running and stable
   - Configuration registers programmed (optional)

3. Reset deasserted (rst_n = 1, sync to clk rising edge)
   - FSMs begin operation
   - Ready to accept descriptors/data
```

5.4 Configuration During Reset

Some configuration signals MUST be stable before reset deassertion:

Table 1.3.4: Configuration Timing Requirements

Signal	Requirement
`src_cfg_channel_enable` / `snk_cfg_channel_enable`	Stable before rst_n=1 (one per half)
`src_cfg_sched_timeout_cycles` / `snk_cfg_sched_timeout_cyc`	Stable before rst_n=1 (one per half)

Signal	Requirement
les	
cfg_axi_rd_xfer_beats	Stable before rst_n=1
cfg_axi_wr_xfer_beats	Stable before rst_n=1

5.5 Reset Macros

RAPIDS uses standardized reset macros from `reset_defs.svh`:

```
`include "reset_defs.svh"

// Standard reset pattern
`ALWAYS_FF_RST(clk, rst_n,
    if (`RST_ASSERTED(rst_n)) begin
        r_state <= IDLE;
        r_counter <= '0;
    end else begin
        r_state <= w_next_state;
        r_counter <= w_next_counter;
    end
)
```

5.6 Power-On Reset Considerations

- External reset should be held for minimum 10 clock cycles after clock is stable
- Configuration registers should be programmed before or during reset
- No AXI transactions until reset is deasserted and channels are enabled

---

Last Updated: 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 Scheduler Specification

Module: `scheduler_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

6.1 Overview

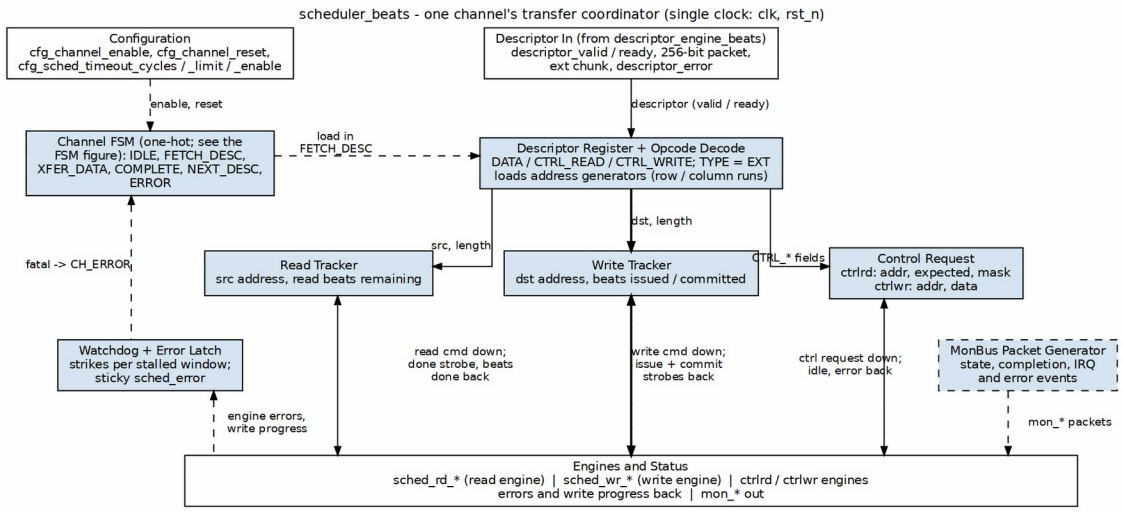
The Scheduler coordinates descriptor-based data transfers for a single RAPIDS channel. It receives descriptors from the descriptor engine, parses transfer parameters, and coordinates read/write operations through the data paths.

6.1.1 Key Features

- **Descriptor Processing:** Parses 256-bit descriptors for transfer parameters
- **Beat-Based Tracking:** All lengths tracked in beats (data-width units)
- **Concurrent Read/Write:** Simultaneous read and write to prevent deadlock
- **Three descriptor opcodes:** `DATA` (concurrent read/write), `CTRL_READ` (consumer gate via the control-read engine), `CTRL_WRITE` (producer doorbell via the control-write engine) – see Control Interface below
- **Timeout Detection:** Configurable watchdog for stalled transfers
- **MonBus Integration:** State transition and error event reporting
- **Error Aggregation:** Combines errors from descriptor engine and data paths

6.1.2 Block Diagram

6.1.3 Figure 2.1.1: Scheduler Block Diagram



Scheduler block diagram

Source: 01_scheduler.dot

6.2 Parameters

```
parameter int CHANNEL_ID = 0;                    // Channel identifier
parameter int NUM_CHANNELS = 8;                  // Total channels in
system
parameter int CHAN_WIDTH = $clog2(NUM_CHANNELS); // Channel ID width
parameter int ADDR_WIDTH = 64;                   // Address bus width
parameter int DATA_WIDTH = 512;                  // Data bus width
(beats)

// Monitor Bus Parameters
parameter logic [7:0] MON_AGENT_ID = 8'h30;      // RAPIDS Scheduler
Agent ID
parameter logic [3:0] MON_UNIT_ID = 4'h1;        // Unit identifier
parameter logic [5:0] MON_CHANNEL_ID = 6'h0;     // Base channel ID

// Descriptor Width
parameter int DESC_WIDTH = 256;                  // RAPIDS descriptor
width
```

: Table 2.1.1: Scheduler Parameters

6.3 Port List

6.3.1 Clock and Reset

Table 2.1.2: Clock and Reset

Signal	Direction	Width	Description
`clk`	input	1	System clock
`rst_n`	input	1	Active-low asynchronous reset

6.3.2 Configuration Interface

Table 2.1.3: Configuration Interface

Signal	Direction	Width	Description
`cfg_channel_enable`	input	1	Enable this channel
`cfg_channel_reset`	input	1	Channel soft reset
`cfg_sched_timeout_cycles`	input	32	Write-progress timeout window (cycles)
`cfg_sched_timeout_limit`	input	8	Consecutive-timeout windows before fatal escalation (0 = never)
`cfg_sched_timeout_enable`	input	1	Enable timeout detection

6.3.3 Descriptor Engine Interface

Table 2.1.4: Descriptor Engine Interface

Signal	Direction	Width	Description
`descriptor_valid`	input	1	Descriptor valid
`descriptor_ready`	output	1	Ready to accept descriptor
`descriptor_pa`	input	256	256-bit

Signal	Direction	Width	Description
cket			descriptor
descriptor_error	input	1	Error from descriptor engine

6.3.4 Data Read Interface

Table 2.1.5: Data Read Interface

Signal	Direction	Width	Description
sched_rd_valid	output	1	Read request valid
sched_rd_addr	output	AW	Source address
sched_rd_beats	output	32	Beats to read
sched_rd_done_strobe	input	1	Read complete strobe
sched_rd_beats_done	input	32	Beats completed
sched_rd_error	input	1	Read error

6.3.5 Data Write Interface

Table 2.1.6: Data Write Interface

Signal	Direction	Width	Description
sched_wr_valid	output	1	Write request valid (gated on remaining ISSUE beats)
sched_wr_addr	output	AW	Destination address
sched_wr_beats	output	32	Beats to write
sched_wr_done_strobe	input	1	Write AW-issue strobe (burst accepted on AW)
sched_wr_beats_done	input	32	Beats issued this strobe
sched_wr_commit_strobe	input	1	Write COMMIT strobe (B

Signal	Direction	Width	Description
			response received)
sched_wr_commit_beats	input	32	Beats committed this strobe
sched_wr_error	input	1	Write error

6.3.6 Control Interface

For CTRL_READ / CTRL_WRITE descriptors the scheduler drives two per-channel control engines instead of the data path. Both are request/response handshakes whose completion is signalled by the engine's *_idle going high after issue.

Table 2.1.7: Control Interface (CTRL_READ / CTRL_WRITE)

Signal	Direction	Width	Description
ctrlrd_valid	output	1	Control-read request valid (to ctrlrd_engine)
ctrlrd_ready	input	1	Engine accepted the poll request
ctrlrd_addr	output	AW	Gate poll address (descriptor poll_addr)
ctrlrd_data	output	32	Expected value (descriptor expected)
ctrlrd_mask	output	32	Compare mask (descriptor mask)
ctrlrd_error	input	1	Engine error (poll retries exhausted / AXI error)
ctrlrd_idle	input	1	ctrlrd_engine_idle – gate satisfied (completion)
ctrlwr_valid	output	1	Control-write request valid (to ctrlwr_engine)
ctrlwr_ready	input	1	Engine accepted the doorbell request
ctrlwr_addr	output	AW	Doorbell address (descriptor wr_addr)
ctrlwr_data	output	32	Doorbell value

Signal	Direction	Width	Description
			(descriptor `wr_data`)
`ctrlwr_error`	input	1	Engine error (AXI error)
`ctrlwr_idle`	input	1	`ctrlwr_engine_idle` – doorbell written (completion)

A `CTRL_READ` descriptor is a **consumer gate**: the scheduler asserts `ctrlrd_valid` and holds off the descriptor's completion (and the chain) until the control-read engine reports the masked poll matched (`ctrlrd_idle`). The poll retry budget is bounded by `CTRL_CONFIG.CTRLRD_MAX_TRY` (Section 3.12) so a never-matching gate cannot hang the channel – exhaustion raises `ctrlrd_error`. A `CTRL_WRITE` descriptor is a **producer doorbell**: a single-beat write of `ctrlwr_data` to `ctrlwr_addr`, after which the chain continues. The scheduler decodes the opcode from the descriptor (`rapids_pkg` `DESC_OP_DATA` / `DESC_OP_CTRL_READ` / `DESC_OP_CTRL_WRITE`); `DATA` descriptors ignore this interface. The engines are instantiated per channel in `scheduler_group_beats` (`u_ctrlrd_engine` / `u_ctrlwr_engine`); see Sections 2.8 and 2.9.

Completion is gated on write-data COMMIT, not AW issue. The scheduler maintains two independent beat counters for the write side:

- `r_write_beats_remaining` decrements on `sched_wr_done_strobe` (AW handshake) and drives the destination address and `sched_wr_valid` (writes are requested only while ISSUE beats remain).
- `r_write_beats_to_commit` decrements on `sched_wr_commit_strobe` (B response). A transfer is considered write-complete only when this counter reaches zero (`w_write_complete`), so a channel is never reported done until every issued burst has been acknowledged on the B channel.

6.4 FSM States

6.4.1 Figure 2.1.2: Scheduler FSM State Diagram

States and transitions of `scheduler_beats` (one-hot `channel_state_t` in `rapids_pkg`).

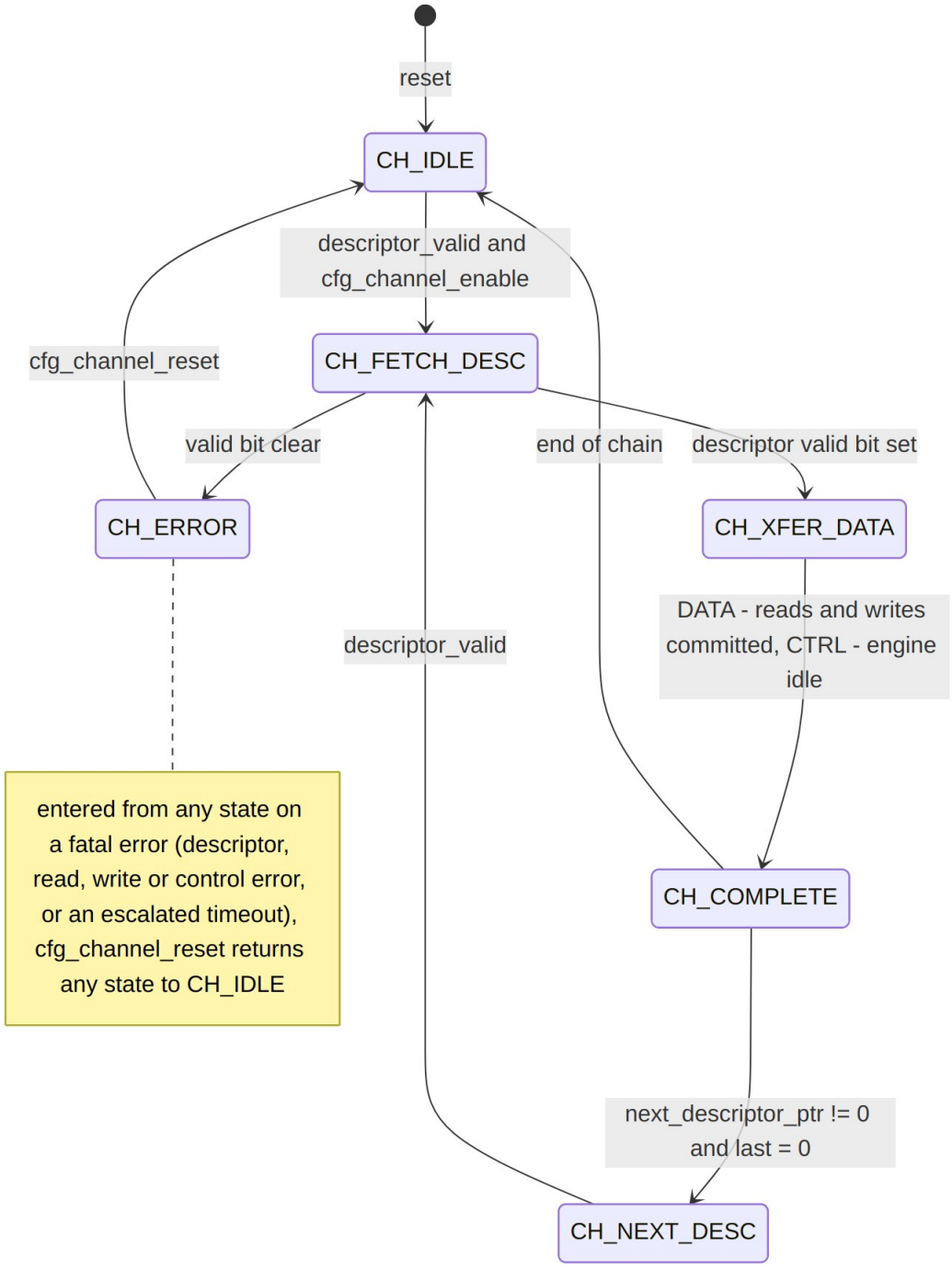

reset
CH_IDLE
descriptor_valid and
cfg_channel_enable
cfg_channel_reset
CH_FETCH_DESC
valid bit clear
end of chain
descriptor valid bit set
CH_ERROR
CH_XFER_DATA
DATA - reads and writes
committed, CTRL - engine
idle
descriptor_valid
entered from any state on
a fatal error (descriptor,
read, write or control error,
or an escalated timeout),
cfg_channel_reset returns
any state to CH_IDLE
CH_COMPLETE
next_descriptor_ptr != 0
and last = 0
CH_NEXT_DESC

Diagram 1

6.4.2 State Encoding

Table 2.1.7: FSM State Encoding

State	Encoding	Description
CH_IDLE	7'b0000001	Waiting for a descriptor
CH_FETCH_DESC	7'b0000010	Descriptor latched and decoded
CH_XFER_DATA	7'b0000100	Concurrent read/write transfer, or the control operation
CH_COMPLETE	7'b0001000	Transfer complete; chain check
CH_NEXT_DESC	7'b0010000	Waiting for the next chained descriptor
CH_ERROR	7'b0100000	Sticky error; left only by channel reset
CH_RESERVED	7'b1000000	Reserved

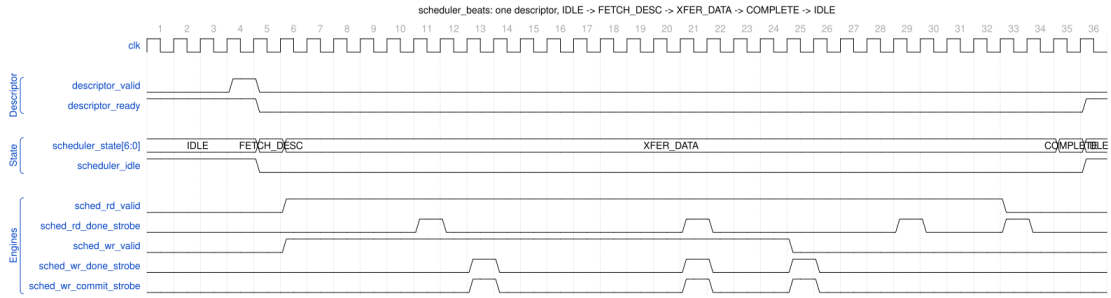
6.5 Operation

6.5.1 Descriptor Format

```
Bits [63:0]    - src_addr: Source memory address
Bits [127:64]  - dst_addr: Destination memory address
Bits [159:128] - length: Transfer length in beats
Bits [191:160] - next_ptr: Next descriptor pointer (0 = last)
Bits [195:192] - channel_id: Channel identifier
Bits [196]     - last: Last descriptor flag
Bits [197]     - gen_irq: Generate interrupt
Bits [255:198] - reserved
```

6.5.2 Transfer Sequence

6.5.3 Figure 2.1.3: Basic Transfer Timing



Scheduler - one descriptor through the FSM

Source: scheduler_basic_transfer.json, captured from `dv/tests/fub_beats/test_scheduler_beats.py` (`basic_flow`, channel 0 of 8, 512-bit, seed 7) with `WAVES=1`.

Reading it: `descriptor_valid` is accepted in IDLE; the FSM steps through FETCH_DESC into XFER_DATA and `scheduler_idle` drops. In XFER_DATA both engines are requested at once (`sched_rd_valid` and `sched_wr_valid` high together): the read engine reports each burst issued on `sched_rd_done_strobe`, the write engine reports issue on `sched_wr_done_strobe` and the B response on `sched_wr_commit_strobe` (here the model answers B immediately, so the two coincide). The write side finishes its beats first and drops `sched_wr_valid`; when the read side has issued its last burst the FSM moves to COMPLETE and back to IDLE, and `descriptor_ready` re-asserts for the next descriptor.

6.6 Error Handling

Table 2.1.8: Error Handling

Error Source	Detection	Response
Descriptor engine	`descriptor_error`	Transition to CH_ERROR state
Read engine	`sched_rd_error`	Set sticky error flag
Write engine	`sched_wr_error`	Set sticky error flag
Write-progress timeout	Recoverable, escalates after limit	See below

6.6.1 Recoverable Write-Progress Timeout

The write-progress watchdog is recoverable and only escalates to a fatal fault after a configurable number of consecutive stalled windows:

1. **Window counting:** `r_timeout_counter` increments while `sched_wr_valid && !sched_wr_ready` (a write request is stalled). The window expires when it reaches `cfg_sched_timeout_cycles`.
2. **Re-arm:** On expiry the counter re-arms (resets to 0) and the scheduler keeps waiting; a single expired window is not fatal.
3. **Strike counting:** `r_timeout_strikes` (8-bit) increments once per expired window. Any write progress – `sched_wr_done_strobe` (AW issue) or `sched_wr_commit_strobe` (B response) – clears both the window counter and the strike counter.
4. **Escalation:** When `cfg_sched_timeout_limit != 0` and `r_timeout_strikes >= cfg_sched_timeout_limit`, the channel escalates to a sticky `CH_ERROR` (surfaced on `sched_error`). Total time to escalate is approximately `cfg_sched_timeout_limit * cfg_sched_timeout_cycles`.
5. **Never-escalate mode:** `cfg_sched_timeout_limit == 0` is a pure soft timeout – the event may be reported, but the channel never faults.

`scheduler_idle` excludes `CH_ERROR`. Only `CH_IDLE` (with no active channel reset) is reported idle; a faulted channel in `CH_ERROR` is deliberately NOT reported idle so a wedged channel cannot read back as idle. The fault is surfaced via `sched_error` instead.

6.7 MonBus Events

Table 2.1.9: MonBus Event Codes

Event Code	Description
0x01	Descriptor received
0x02	Transfer started
0x03	Transfer complete
0x04	Error occurred
0x05	Timeout

Last Updated: 2026-07-02

7 Descriptor Engine Specification

Module: `descriptor_engine_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

7.1 Overview

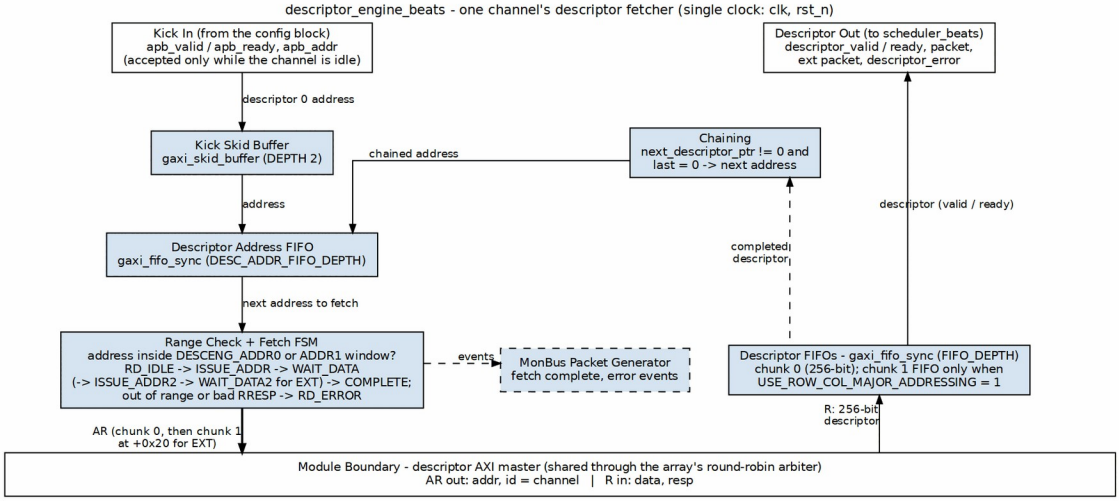
The Descriptor Engine fetches and manages 256-bit descriptors from memory via AXI4. It maintains a prefetch FIFO to hide memory latency and provides parsed descriptors to the scheduler.

7.1.1 Key Features

- **AXI4 Descriptor Fetch:** Single-beat 256-bit reads
- **Prefetch FIFO:** Configurable depth for latency hiding
- **Address Range Checking:** Validates descriptor addresses
- **Error Detection:** Address violations, AXI errors
- **MonBus Integration:** Fetch events and error reporting

7.1.2 Block Diagram

7.1.3 Figure 2.2.1: Descriptor Engine Block Diagram



Descriptor engine block diagram

Source: 02_descriptor_engine.dot

7.2 Parameters

```
parameter int CHANNEL_ID = 0;                    // Channel identifier
parameter int NUM_CHANNELS = 8;                  // Total channels
parameter int ADDR_WIDTH = 64;                   // Address bus width
parameter int FIFO_DEPTH = 4;                    // Prefetch FIFO
depth

// Monitor Bus Parameters
parameter logic [7:0] MON_AGENT_ID = 8'h10;      // Descriptor Engine
Agent ID
parameter logic [3:0] MON_UNIT_ID = 4'h1;        // Unit identifier
```

: Table 2.2.1: Descriptor Engine Parameters

7.3 Port List

7.3.1 Clock and Reset

Table 2.2.2: Clock and Reset

Signal	Direction	Width	Description
`clk`	input	1	System clock
`rst_n`	input	1	Active-low asynchronous reset

7.3.2 APB Programming Interface

Table 2.2.3: APB Programming Interface

Signal	Direction	Width	Description
`apb_valid`	input	1	Kick-off valid (start descriptor chain)
`apb_ready`	output	1	Ready to accept kick-off
`apb_addr`	input	AW	First descriptor address

7.3.3 Configuration Interface

Table 2.2.4: Configuration Interface

Signal	Direction	Width	Description
cfg_prefetch_enable	input	1	Enable prefetch chaining (0 = on-demand, ~1 ahead)
cfg_fifo_threshold	input	4	Prefetch depth (descriptors buffered ahead when enabled)
cfg_addr0_base	input	AW	Valid address range 0 base
cfg_addr0_limit	input	AW	Valid address range 0 limit
cfg_addr1_base	input	AW	Valid address range 1 base
cfg_addr1_limit	input	AW	Valid address range 1 limit

7.3.4 AXI4 Read Master Interface

Table 2.2.5: AXI4 Read Master Interface

Signal	Direction	Width	Description
ar_valid	output	1	Address valid
ar_ready	input	1	Address ready
ar_addr	output	ADDR_WIDTH	Descriptor fetch address
ar_len	output	8	Burst length - 1
ar_size	output	3	Burst size (log2 bytes)
ar_burst	output	2	Burst type (INCR)
ar_id	output	AXI_ID_WIDTH	Transaction ID
ar_lock	output	1	Lock type
ar_cache	output	4	Cache

Signal	Direction	Width	Description
			attributes
ar_prot	output	3	Protection attributes
ar_qos	output	4	Quality of service
ar_region	output	4	Region identifier
r_valid	input	1	Data valid
r_ready	output	1	Data ready
r_data	input	256	Descriptor payload (fixed 256-bit)
r_resp	input	2	Read response
r_last	input	1	Last beat
r_id	input	AXI_ID_WIDTH	Response ID

7.3.5 Scheduler Interface

Table 2.2.6: Scheduler Interface

Signal	Direction	Width	Description
descriptor_valid	output	1	Descriptor valid
descriptor_ready	input	1	Scheduler ready
descriptor_packet	output	256	Parsed descriptor (fixed 256-bit)
descriptor_ext_packet	output	256	Extended chunk 1 (0 when unused)
descriptor_error	output	1	Error flag
descriptor_eos	output	1	End of Stream
descriptor_eol	output	1	End of Line
descriptor_eod	output	1	End of Data

Signal	Direction	Width	Description
`descriptor_type`	output	2	Packet type
`channel_idle`	input	1	Scheduler idle (enables APB kick-off)
`descriptor_engine_idle`	output	1	Engine idle

7.4 FSM States

7.4.1 Figure 2.2.2: Descriptor Engine FSM

States and transitions of the `descriptor_engine_beats` fetch FSM.

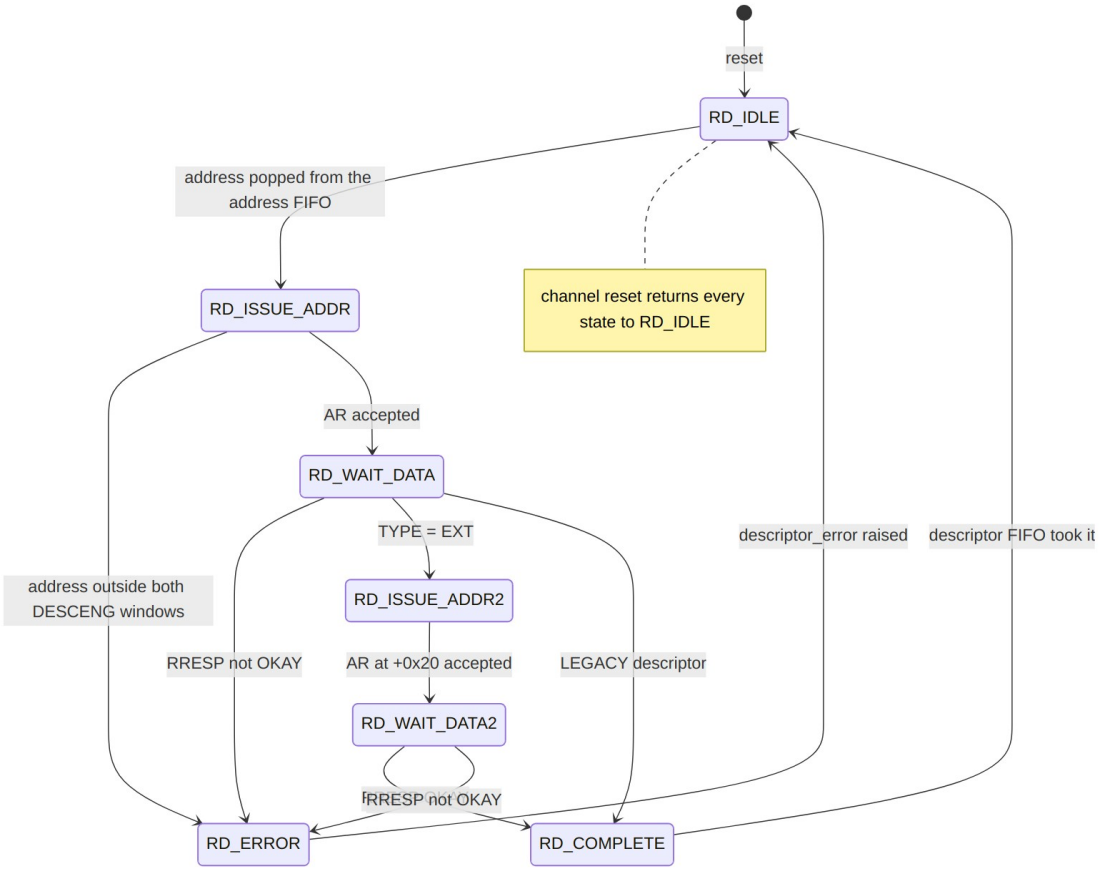
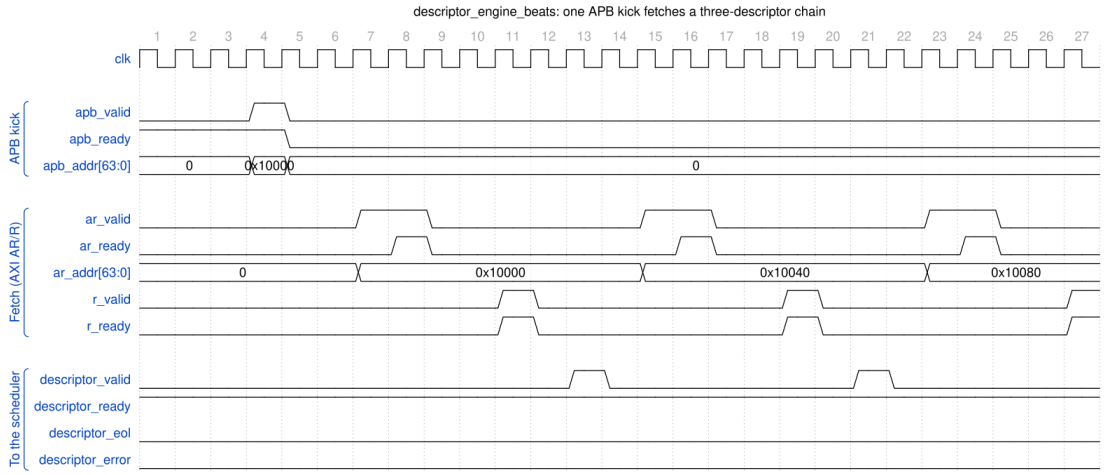


Diagram 2

7.5 Operation

7.5.1 Descriptor Chain Processing

7.5.2 Figure 2.2.3: Descriptor Chain Timing



Descriptor Engine - three-descriptor chain

Source: descriptor_engine_chain.json, captured from `dv/tests/fub_beats/test_descriptor_engine_beats.py` (`descriptor_chaining`, 32 channels, seed 7) with `WAVES=1`.

Reading it: one `apb_valid` kick carries the head address (0x10000). The engine issues an AXI read for it (`ar_valid`, `ar_addr`), the data returns on `r_valid`, and the parsed descriptor is presented on `descriptor_valid` to a ready scheduler. Because the descriptor's `next_ptr` points at 0x10040 and `last` is clear, the engine fetches the next one on its own – and the one after that at 0x10080 – with no further APB traffic; `descriptor_eol` marks the last of the chain. Each fetch costs the same AR -> R -> present latency, so a chain streams at one descriptor per fetch round trip.

7.6 Address Range Checking

Descriptors must fall within configured valid address ranges:

```
Valid if: (addr >= cfg_addr0_base && addr <= cfg_addr0_limit)
       || (addr >= cfg_addr1_base && addr <= cfg_addr1_limit)
```

Invalid addresses generate an error and halt the descriptor chain.

7.7 Prefetch and Autonomous Chaining

`cfg_prefetch_enable` and `cfg_fifo_threshold` are functional prefetch controls (previously declared-but-unused, which left chaining permanently greedy). They throttle how far ahead the engine fetches chained descriptors by comparing the output descriptor FIFO occupancy (`w_desc_fifo_count`) against a computed `w_prefetch_limit`:

```
if      (!cfg_prefetch_enable)      w_prefetch_limit = 1;   // on-
demand
else if (cfg_fifo_threshold == 0)   w_prefetch_limit = 1;   // guard
(0 would stall)
else                                w_prefetch_limit =
cfg_fifo_threshold;

w_prefetch_allows = (w_desc_fifo_count < w_prefetch_limit);
```

- **Prefetch OFF (`cfg_prefetch_enable = 0`):** limit forced to 1, so a chained fetch is only allowed when the output FIFO has drained – the engine stays roughly one descriptor ahead of the scheduler (on-demand).
- **Prefetch ON:** the engine buffers up to `cfg_fifo_threshold` descriptors ahead.

7.7.1 Chaining Decision

A chain fetch issues immediately when all of the following hold:

```
w_should_chain = w_chain_eligible     // valid non-last descriptor, in
range, no error
               && w_desc_committed      // current descriptor accepted
into output FIFO
               && w_prefetch_allows     // occupancy below prefetch
limit
               && w_desc_addr_fifo_wr_ready;  // address FIFO has a
slot
```

7.7.2 Deferred (Throttle-Blocked) Chain Fetch

If a descriptor commits and is chain-eligible but the fetch cannot issue this cycle (throttled by `w_prefetch_allows`, or the address FIFO is full), the owed fetch is recorded rather than dropped:

- `r_chain_pending` is set and the next-descriptor pointer is held in `r_pending_chain_addr`.
- The deferred fetch is reissued (`w_pending_push_fire`) once the output FIFO drains enough that `w_prefetch_allows` is true again and the address FIFO has a slot, clearing `r_chain_pending`.

Last Updated: 2026-07-02

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

8 AXI Read Engine Specification

Module: `axi_read_engine_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub_beats/` **Status:** Implemented **Last Updated:** 2026-09-29

8.1 Overview

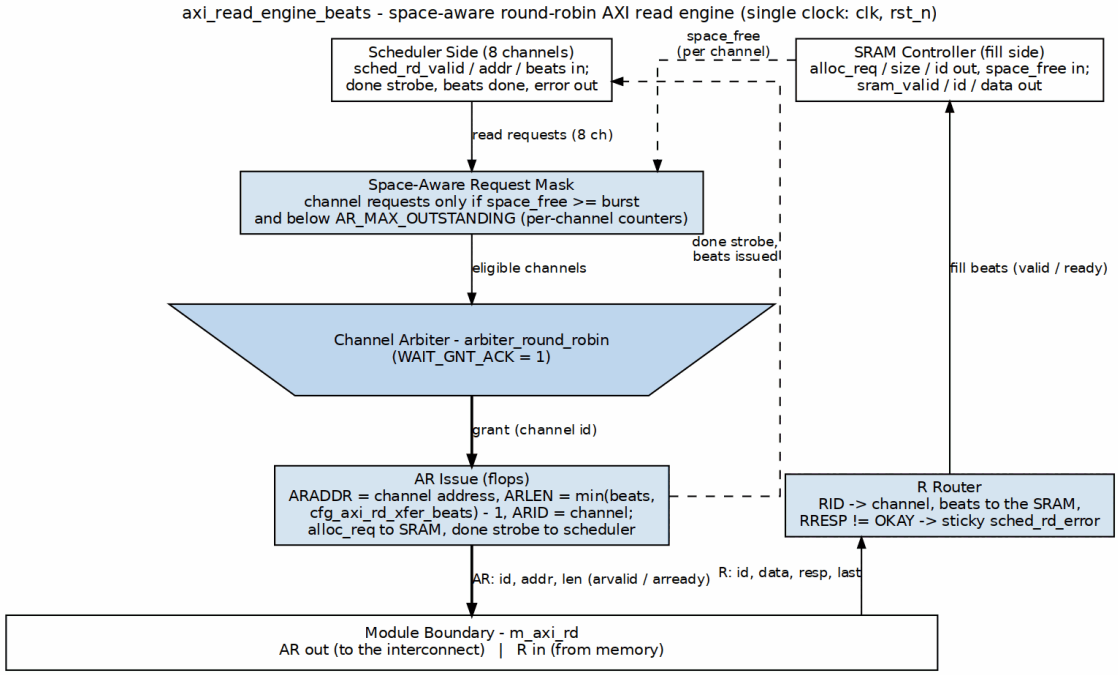
The AXI Read Engine performs burst reads from system memory and writes data to the SRAM buffer. It operates as a streaming pipeline with no FSM, maximizing throughput through continuous data flow.

8.1.1 Key Features

- **Streaming Pipeline:** No FSM - pure streaming architecture
- **Multi-Channel Support:** Channel ID passed through AXI ID field
- **Configurable Burst Length:** Up to 256 beats per burst
- **SRAM fill only:** writes R-channel beats straight into the SRAM controller's `fill_*` port (the latency bridge lives in STREAM's `sram_controller_unit`, not here)
- **Error Handling:** AXI RRESP error detection and reporting

8.1.2 Block Diagram

8.1.3 Figure 2.3.1: AXI Read Engine Block Diagram



AXI read engine block diagram

Source: 03_axi_read_engine.dot

8.2 Parameters

```
parameter int NUM_CHANNELS = 8;                  // Number of channels
parameter int ADDR_WIDTH = 64;                   // Address bus width
parameter int DATA_WIDTH = 512;                  // Data bus width
parameter int AXI_ID_WIDTH = 8;                  // AXI ID width
parameter int MAX_OUTSTANDING = 8;               // Max outstanding AR
transactions
parameter int PIPELINE = 1;                      // Pipeline stages

// Derived
parameter int CHAN_WIDTH = $clog2(NUM_CHANNELS);
```

: Table 2.3.1: AXI Read Engine Parameters

8.3 Port List

8.3.1 Clock and Reset

Table 2.3.2: Clock and Reset

Signal	Direction	Width	Description
`clk`	input	1	System clock
`rst_n`	input	1	Active-low reset

8.3.2 Scheduler Interface

Table 2.3.3: Scheduler Interface

Signal	Direction	Width	Description
`sched_rd_valid`	input	NC	Channel requests read
`sched_rd_addr`	input	NC x AW	Source addresses
`sched_rd_beats`	input	NC x 32	Beats remaining to read
`sched_rd_done_strobe`	output	NC	Burst completed (pulsed 1 cycle)
`sched_rd_beats_done`	output	NC x 32	Beats completed in burst
`sched_rd_error`	output	NC	Sticky error flag per channel (bad R response)

8.3.3 AXI4 Read Master Interface

Table 2.3.4: AXI4 Read Master Interface

Signal	Direction	Width	Description
`m_axi_arid`	output	IW	Read address ID
`m_axi_araddr`	output	AW	Read address
`m_axi_arlen`	output	8	Burst length (beats - 1)
`m_axi_arsize`	output	3	Burst size
`m_axi_arburst`	output	2	Burst type

Signal	Direction	Width	Description
			(INCR)
m_axi_arvalid	output	1	Address valid
m_axi_arready	input	1	Address ready
m_axi_rid	input	IW	Read data ID
m_axi_rdata	input	DW	Read data
m_axi_rresp	input	2	Read response
m_axi_rlast	input	1	Last beat
m_axi_rvalid	input	1	Data valid
m_axi_rready	output	1	Data ready

8.3.4 SRAM Allocation and Fill Interface

Table 2.3.5: SRAM Write Interface

Signal	Direction	Width	Description
axi_rd_alloc_req	output	1	Channel requests space
axi_rd_alloc_size	output	8	Beats to reserve
axi_rd_alloc_id	output	IW	Transaction ID selects the channel
axi_rd_alloc_space_free	input	NC x SCW	Free space (beats available) per channel
axi_rd_sram_valid	output	1	Read data valid
axi_rd_sram_ready	input	1	SRAM ready to accept data
axi_rd_sram_id	output	IW	Transaction ID selects the channel
axi_rd_sram_data	output	DW	Read data payload

8.4 Operation

8.4.1 Burst Length Cap (rapids BUG-009)

`cfg_axi_rd_xfer_beats` is an ARLEN (0..255), but a burst can never be larger than the SRAM it lands in: `w_space_ok` pre-allocates the whole burst before the AR, and a burst the buffer cannot hold would never be allocatable. The engine therefore clamps the configured value to `XFER_MAX = min(2^(SCW-1) - 1, 254)` – the buffer depth `SEG_COUNT_WIDTH` encodes, minus one, and 254 beyond that so every beat count fits the 8-bit size ports shared with `sram_controller`. On the Genesys 2 build (128-deep) ARLEN 255 runs as 128-beat bursts.

Before the clamp (2026-09-29), ARLEN 255 was taken literally and the 8-bit `ARLEN + 1` in the gate wrapped to 0: `w_space_ok` passed on any free count, the AR for 256 beats allocated 0, and the engine over-fetched past the buffer (4103 R beats for a 4096-beat descriptor on the board, golden mismatch).

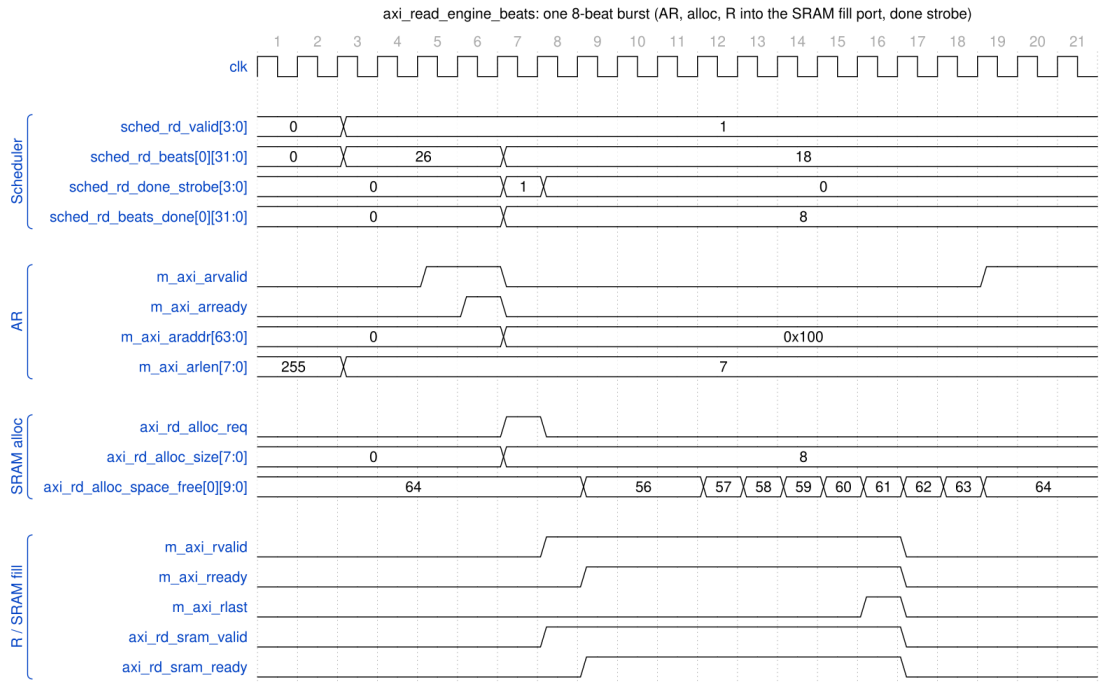
8.4.2 Streaming Pipeline Architecture

The AXI read engine uses a **streaming pipeline** with no FSM:

1. Scheduler request: per-channel beat count, masked by SRAM space.
2. AR issue: round-robin grant, burst address and length, space allocation.
3. R receive: data routed to the channel by RID.
4. SRAM write: one fill beat per R beat.

8.4.3 Timing Diagram

8.4.4 Figure 2.3.2: AXI Read Burst Timing



AXI Read Engine - one 8-beat burst

Source: axi_read_engine_burst.json, captured from `dv/tests/fub_beats/test_axi_read_engine_beats.py` (`single`, 4 ch, 256-bit, `cfg_axi_rd_xfer_beats = 7`, seed 7) with `WAVES=1`.

Reading it: the scheduler holds `sched_rd_valid` with 26 beats remaining; the arbiter grants channel 0 and the AR (`arlen = 7`) handshakes two cycles later. The cycle after the handshake the engine pulses `sched_rd_done_strobe` with `beats_done = 8` (the scheduler advances address and remaining on it – note `sched_rd_beats` drops 26 to 18 and `araddr` for the next AR is already 0x100) and `axi_rd_alloc_req` with `alloc_size = 8`; the modelled `space_free` debits two cycles later, as the real controller does. R beats pass straight through to `axi_rd_sram_valid`; `rlast` closes the burst. The engine never waits for the data before issuing the next AR of another channel, but at `PIPELINE = 0` this channel's next AR waits for `rlast`.

8.5 Burst Segmentation

Large transfers are segmented into AXI-compliant bursts:

```
Total beats = 256
cfg_xfer_beats = 64
```

```
Bursts issued:
  AR[0]: addr=0x1000, len=63 (64 beats)
  AR[1]: addr=0x2000, len=63 (64 beats)
  AR[2]: addr=0x3000, len=63 (64 beats)
  AR[3]: addr=0x4000, len=63 (64 beats)
```

8.6 Error Handling

Table 2.3.6: Error Handling

Error	Detection	Response
AXI SLVERR	`m_axi_rresp == 2'b10`	Set `sched_rd_error`, continue
AXI DECERR	`m_axi_rresp == 2'b11`	Set `sched_rd_error`, continue

Last Updated: 2026-09-29

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

9 AXI Write Engine Specification

Module: `axi_write_engine_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub_beats/` **Status:** Implemented **Last Updated:** 2026-09-29

9.1 Overview

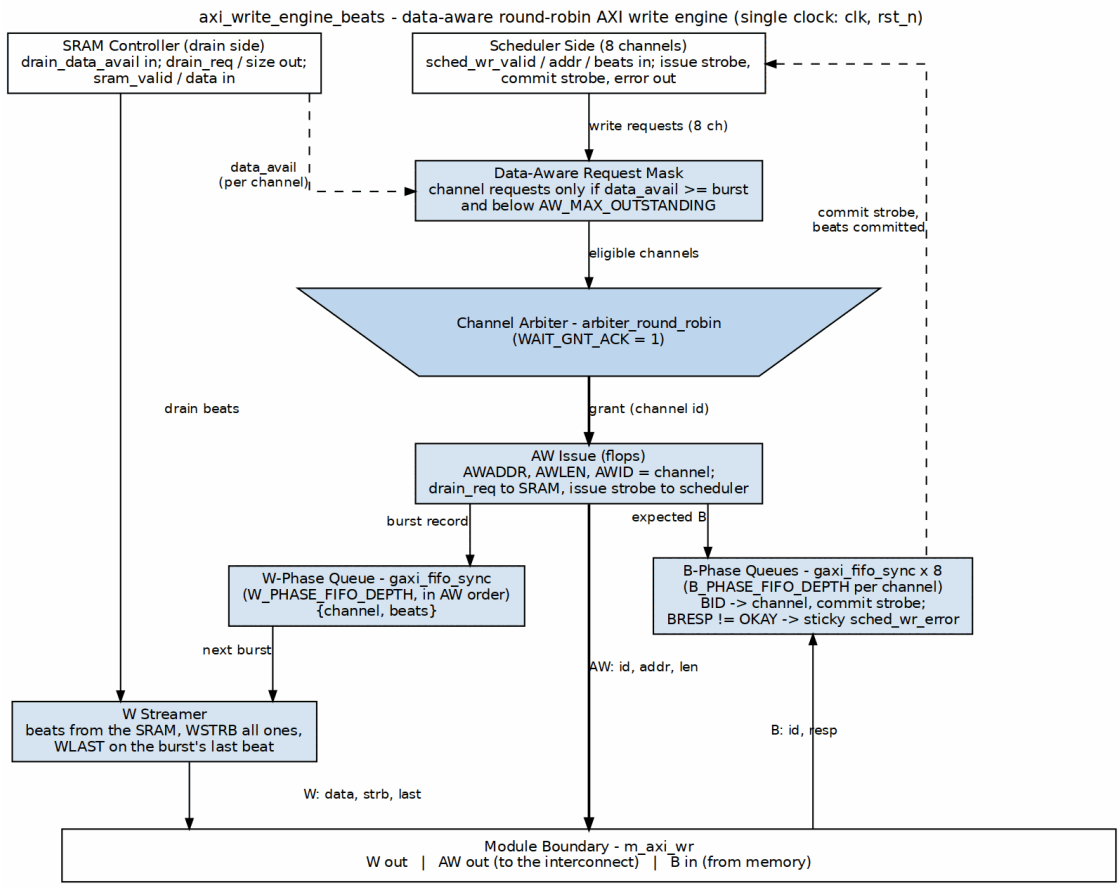
The AXI Write Engine reads data from SRAM and performs burst writes to system memory. It operates as a streaming pipeline and includes write response handling for completion tracking.

9.1.1 Key Features

- **Streaming Pipeline:** No FSM - pure streaming architecture
- **Multi-Channel Support:** Channel ID in AXI ID field
- **Write Response Tracking:** B channel completion handling
- **Configurable Burst Length:** Up to 256 beats per burst
- **Registered SRAM read:** consumes the SRAM controller's registered `drain_*` stream (the latency bridge lives in STREAM's `sram_controller_unit`, not here)

9.1.2 Block Diagram

9.1.3 Figure 2.4.1: AXI Write Engine Block Diagram



AXI write engine block diagram

Source: 04_axi_write_engine.dot

9.2 Parameters

```
parameter int NUM_CHANNELS = 8;                 // Number of channels
parameter int ADDR_WIDTH = 64;                  // Address bus width
parameter int DATA_WIDTH = 512;                 // Data bus width
parameter int AXI_ID_WIDTH = 8;                 // AXI ID width
parameter int MAX_OUTSTANDING = 8;              // Max outstanding AW
transactions
parameter int W_FIFO_DEPTH = 64;                // Write data FIFO
depth
parameter int B_FIFO_DEPTH = 16;                // Write response
FIFO depth
```

```
parameter int PIPELINE = 1;                         // Pipeline stages

// Derived
parameter int CHAN_WIDTH = $clog2(NUM_CHANNELS);
```

: Table 2.4.1: AXI Write Engine Parameters

9.3 Port List

9.3.1 Clock and Reset

Table 2.4.2: Clock and Reset

Signal	Direction	Width	Description
`clk`	input	1	System clock
`rst_n`	input	1	Active-low reset

9.3.2 Scheduler Interface

Table 2.4.3: Scheduler Interface

Signal	Direction	Width	Description
`sched_wr_valid`	input	NC	Channel requests write
`sched_wr_ready`	output	NC	Engine ready for channel
`sched_wr_addr`	input	NC x AW	Destination addresses
`sched_wr_beats`	input	NC x 32	Beats remaining to write
`sched_wr_burst_len`	input	NC x 8	Requested burst length
`sched_wr_done_strobe`	output	NC	Burst ISSUED on AW handshake (pulsed 1 cycle)
`sched_wr_beats_done`	output	NC x 32	Beats issued in burst (`awlen + 1`)
`sched_wr_commit_strobe`	output	NC	Burst COMMITTED on B response (pulsed 1 cycle)
`sched_wr_com`	output	NC x 32	Beats committed in burst

Signal	Direction	Width	Description
mit_beats			
sched_wr_error	output	NC	Sticky error flag per channel (bad B response)

Two completion strobes per channel. The engine reports write progress to the scheduler at two points, letting the scheduler gate completion on data actually landing in memory:

- sched_wr_done_strobe / sched_wr_beats_done pulse when the **AW command handshakes** (m_axi_awvalid && m_axi_awready); the beat count is taken directly from m_axi_awlen + 1. This advances the scheduler's destination address (ISSUE tracking).
- sched_wr_commit_strobe / sched_wr_commit_beats pulse when the matching **B response** arrives (m_axi_bvalid && m_axi_bready for the channel); the beat count is recovered from the per-channel B-phase transaction FIFO. This is the COMMIT the scheduler uses to declare the transfer complete.

9.3.3 AXI4 Write Master Interface

Table 2.4.4: AXI4 Write Master Interface

Signal	Direction	Width	Description
m_axi_awid	output	IW	Write address ID
m_axi_awaddr	output	AW	Write address
m_axi_awlen	output	8	Burst length (beats - 1)
m_axi_awsize	output	3	Burst size
m_axi_awburst	output	2	Burst type (INCR)
m_axi_awvalid	output	1	Address valid
m_axi_awready	input	1	Address ready
m_axi_wdata	output	DW	Write data
m_axi_wstrb	output	DW/8	Write strobes
m_axi_wlast	output	1	Last beat
m_axi_wuser	output	UW	Channel ID for transaction

Signal	Direction	Width	Description
			tracking
m_axi_wvalid	output	1	Data valid
m_axi_wready	input	1	Data ready
m_axi_bid	input	IW	Response ID
m_axi_bresp	input	2	Write response
m_axi_bvalid	input	1	Response valid
m_axi_bready	output	1	Response ready

9.3.4 SRAM Reservation and Drain Interface

Table 2.4.5: SRAM Read Interface

Signal	Direction	Width	Description
axi_wr_drain_req	output	NC	Channel requests to reserve data
axi_wr_drain_size	output	NC x 8	Beats to reserve
axi_wr_drain_data_avail	input	NC x SCW	Data available after reservations
axi_wr_sram_valid	input	NC	Per-channel valid (registered, for arbitration)
axi_wr_sram_valid_comb	input	NC	Per-channel valid (combinational, gates m_axi_wvalid)
axi_wr_sram_drain	output	1	Drain request (consumer ready)
axi_wr_sram_id	output	CIW	Channel ID select for drain
axi_wr_sram_data	input	DW	Data from the selected channel (muxed)

9.4 Operation

9.4.1 Burst Length Cap (rapids BUG-009)

`cfg_axi_wr_xfer_beats` is an AWLEN (0..255), but a burst can never be larger than the SRAM that stages it: `w_has_data` waits for the whole burst to be in the buffer, and a burst the buffer cannot hold would wait forever. The engine therefore clamps the configured value to `XFER_MAX = min(2^(SCW-1) - 1, 254)` – the buffer depth `SEG_COUNT_WIDTH` encodes, minus one, and 254 beyond that so every beat count fits the 8-bit size ports shared with `sram_controller`. On the Genesys 2 build (128-deep) AWLEN 255 runs as 128-beat bursts.

Before the clamp (2026-09-29), AWLEN 255 was taken literally and the 8-bit `AWLEN + 1` in the gate wrapped to 0: `w_has_data` passed on an empty buffer, the AW went out, `drain_ctrl` reserved 0 beats and the W phase pulled 256 beats that were never reserved. The channel's `drain_data_available` then read 0 for good (`CHANNEL_RESET` does not reach `sram_controller`), and every later run on the channel accepted one buffer of ingress and never issued an AW.

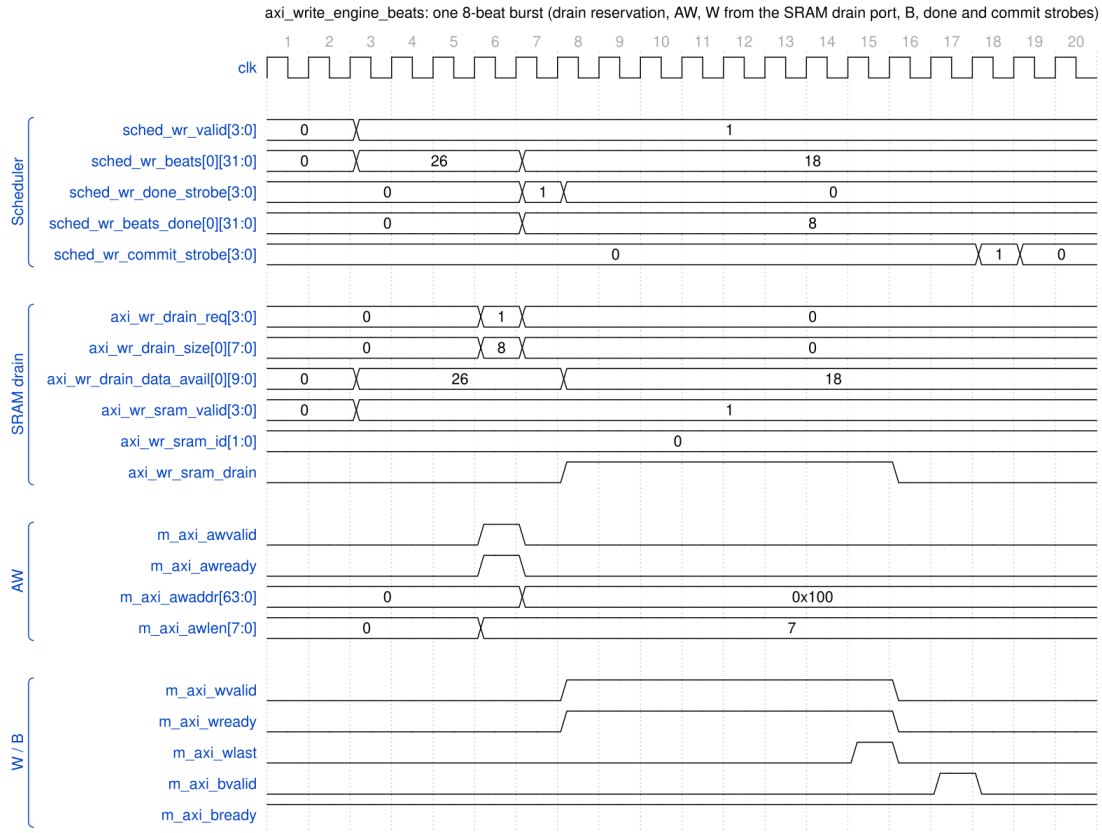
9.4.2 Write Transaction Phases

AXI writes have three phases that can overlap:

```
Phase 1: AW (Address)    Phase 2: W (Data)    Phase 3: B (Response)
    |                        |                    |
    +-- Issue address        +-- Stream data      +-- Receive
response
        before data             from SRAM            track
completion
```

9.4.3 Timing Diagram

9.4.4 Figure 2.4.2: AXI Write Burst Timing



AXI Write Engine - one 8-beat burst

Source: axi_write_engine_burst.json, captured from `dv/tests/fub_beats/test_axi_write_engine_beats.py` (`single`, 4 ch, 256-bit, `cfg_axi_wr_xfer_beats = 7`, seed 7) with `WAVES=1`.

Reading it: the SRAM model reports 26 beats available for channel 0 and the scheduler holds `sched_wr_valid`. The arbiter grants the channel, the AW (`awlen = 7`) is registered and handshakes; in that same cycle `axi_wr_drain_req` reserves 8 beats (`data_avail` drops to 18 two cycles later, the controller's reporting latency), and the cycle after, `sched_wr_done_strobe` reports 8 beats issued so the scheduler advances (`awaddr` for the next AW is already 0x100). W beats stream from the ID-selected drain port (`axi_wr_sram_drain` is exactly `wvalid && wready`), `wlast` closes the burst, and the B response two cycles later raises `sched_wr_commit_strobe`. At `PIPELINE = 0` the channel's next AW waits for that B.

That wait is the throughput cost of `PIPELINE = 0`: with 8-beat bursts a channel spends the AW-to-B round trip idle, and on the Genesys 2 the 8-channel sink write path measured 49.9 % engaged utilization at 4096 beats per channel once BUG-005 made the engine honour one-in-flight (the pre-fix engine had run two by accident and read 100 %). The top and core therefore default `PIPELINE`

= 1, which restores line rate, and since 2026-09-28 so does every module that declares the parameter, this engine included: 0 is selectable, never the default.

9.5 Error Handling

Table 2.4.6: Error Handling

Error	Detection	Response
AXI SLVERR	`m_axi_bresp == 2'b10`	Set `sched_wr_error`
AXI DECERR	`m_axi_bresp == 2'b11`	Set `sched_wr_error`

Last Updated: 2026-09-29

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 Beats Alloc Control Specification

Module: `alloc_ctrl_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

> **No longer in the SRAM datapath (2026-09-26).** RAPIDS' SRAM path now uses STREAM's `stream_alloc_ctrl` / `stream_drain_ctrl` / `stream_latency_bridge`, reached through the `snk_/src_sram_controller_beats` naming wrappers (`bdf4e0dff`). This FUB is still built and still verified – it keeps its own test, its own filelist and its `rapids_all.f` entry – it simply is not instantiated by the SRAM controllers any more. Treat this page as the FUB's own specification, not as a description of the current SRAM path.

10.1 Overview

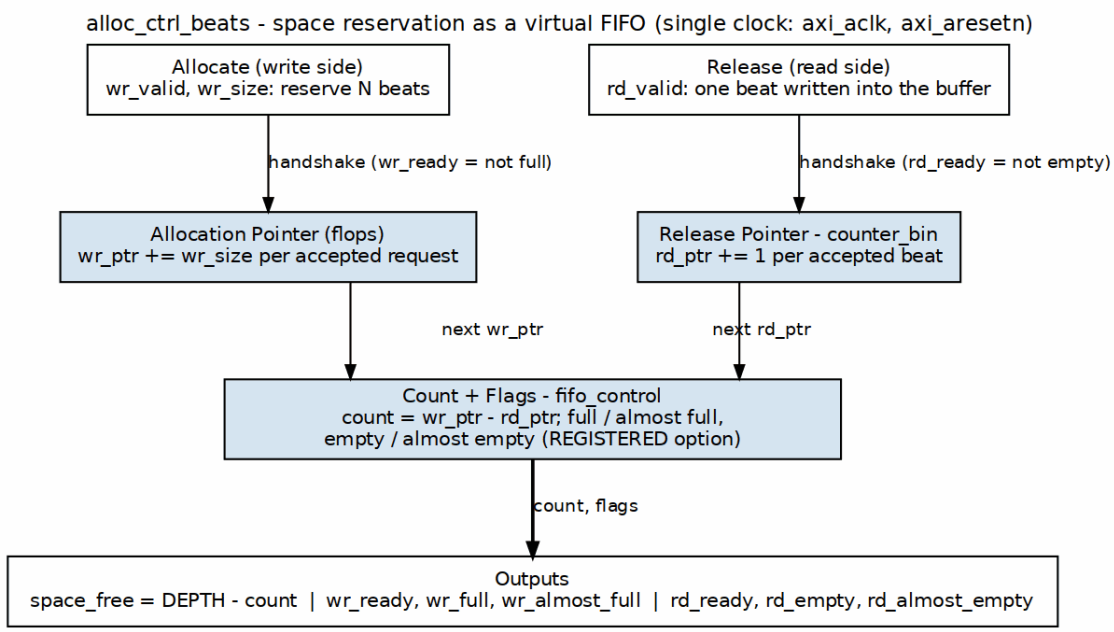
The Beats Alloc Control is a "virtual FIFO" that tracks space allocation without storing actual data. It uses FIFO pointer logic to manage pre-allocation for AXI engines, preventing race conditions between allocation and actual data arrival.

10.1.1 Key Features

- **Virtual FIFO:** Pointer tracking without data storage
- **Variable-Size Allocation:** Supports multi-beat allocation requests
- **Single-Beat Release:** Data writes release one beat at a time
- **Status Flags:** Full, almost full, empty, almost empty
- **Space Tracking:** Reports available space for flow control

10.1.2 Block Diagram

10.1.3 Figure 2.5.1: Beats Alloc Control Block Diagram



Alloc control block diagram

Source: 05_alloc_ctrl.dot

10.2 Concept: Virtual FIFO

Unlike a traditional FIFO that stores data, alloc_ctrl_beats only tracks **space reservations**:

	Traditional FIFO	Virtual FIFO (alloc_ctrl_beats)
Write side	`wr_data` into storage	`wr_size` advances a pointer
Read side	`rd_data` out of storage	one beat advances a pointer
Output	data	`space_free`

10.2.1 Use Case: Pre-Allocation for AXI Reads

```
1. AXI engine requests N beats of space
   - alloc_ctrl.wr_valid = 1
   - alloc_ctrl.wr_size = N
```

```
   - wr_ptr advances by N

2. As AXI read data arrives (1 beat at a time)
   - alloc_ctrl.rd_valid = 1 (each beat)
   - rd_ptr advances by 1

3. Space becomes available again
   - space_free reflects available capacity
```

10.3 Parameters

```
parameter int DEPTH = 512;                    // Virtual FIFO depth
parameter int ALMOST_WR_MARGIN = 1;           // Almost full margin
parameter int ALMOST_RD_MARGIN = 1;           // Almost empty margin
parameter int REGISTERED = 1;                 // Registered outputs

// Derived
parameter int AW = $clog2(DEPTH);             // Address width
```

: Table 2.5.1: Beats Alloc Control Parameters

10.4 Port List

10.4.1 Clock and Reset

Table 2.5.2: Clock and Reset

Signal	Direction	Width	Description
axi_aclk	input	1	System clock
axi_aresetn	input	1	Active-low reset

10.4.2 Write Interface (Allocation Requests)

Table 2.5.3: Write Interface

Signal	Direction	Width	Description
wr_valid	input	1	Allocate space
wr_size	input	8	Number of beats to allocate

Signal	Direction	Width	Description
wr_ready	output	1	Space available for allocation

10.4.3 Read Interface (Data Written)

Table 2.5.4: Read Interface

Signal	Direction	Width	Description
rd_valid	input	1	One beat of data written
rd_ready	output	1	Not full (can accept)

10.4.4 Status Outputs

Table 2.5.5: Status Outputs

Signal	Direction	Width	Description
space_free	output	AW+1	Available space in beats
wr_full	output	1	No space available
wr_almost_full	output	1	Near full
rd_empty	output	1	No allocations pending
rd_almost_empty	output	1	Near empty

10.5 Operation

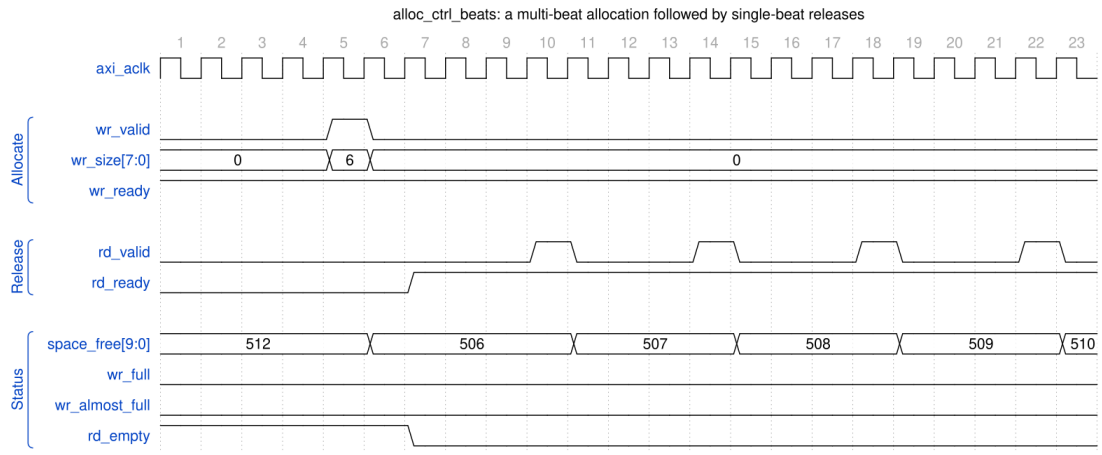
10.5.1 Pointer Arithmetic

```
space_free = DEPTH - (wr_ptr - rd_ptr)

Full condition:    wr_ptr - rd_ptr >= DEPTH
Empty condition:   wr_ptr == rd_ptr
```

10.5.2 Timing Diagram

10.5.3 Figure 2.5.2: Allocation and Release Timing



Allocation control - allocate then release

Source: alloc_ctrl_beats_alloc_release.json, captured from `dv/tests/fub_beats/test_alloc_ctrl_beats.py` (`basic_alloc_drain`, depth 512, seed 7) with `WAVES=1`.

Reading it: one `wr_valid` handshake with `wr_size = 6` reserves six beats and `space_free` drops 512 to 506 the next cycle; `rd_empty` falls because reserved space is now outstanding. Each later `rd_valid` handshake releases one beat and `space_free` climbs back one at a time (507, 508, ...). `wr_full` and `wr_almost_full` stay low: the virtual FIFO is nowhere near its depth.

10.6 Integration Example

```
// Source path: Pre-allocate SRAM space before AXI read
alloc_ctrl_beats #(
    .DEPTH(SRAM_DEPTH),
    .ALMOST_WR_MARGIN(8)
) u_src_alloc (
    .axi_aclk       (clk),
    .axi_aresetn    (rst_n),

    // Scheduler allocates space before issuing AXI read
    .wr_valid       (sched_rd_valid),
    .wr_size        (sched_rd_beats[7:0]),
    .wr_ready       (alloc_ready),

    // AXI read data arrival releases allocation
```

```
    .rd_valid       (m_axi_rvalid && m_axi_rready),
    .rd_ready       (),  // Not used for single-beat

    // Status
    .space_free     (sram_space_available),
    .wr_full        (sram_full),
    .wr_almost_full (sram_almost_full)
);
```

Last Updated: 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

11 Beats Drain Control Specification

Module: `drain_ctrl_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

> **No longer in the SRAM datapath (2026-09-26).** RAPIDS' SRAM path now uses STREAM's `stream_alloc_ctrl` / `stream_drain_ctrl` / `stream_latency_bridge`, reached through the `snk_/src_sram_controller_beats` naming wrappers (`bdf4e0dff`). This FUB is still built and still verified – it keeps its own test, its own filelist and its `rapids_all.f` entry – it simply is not instantiated by the SRAM controllers any more. Treat this page as the FUB's own specification, not as a description of the current SRAM path.

11.1 Overview

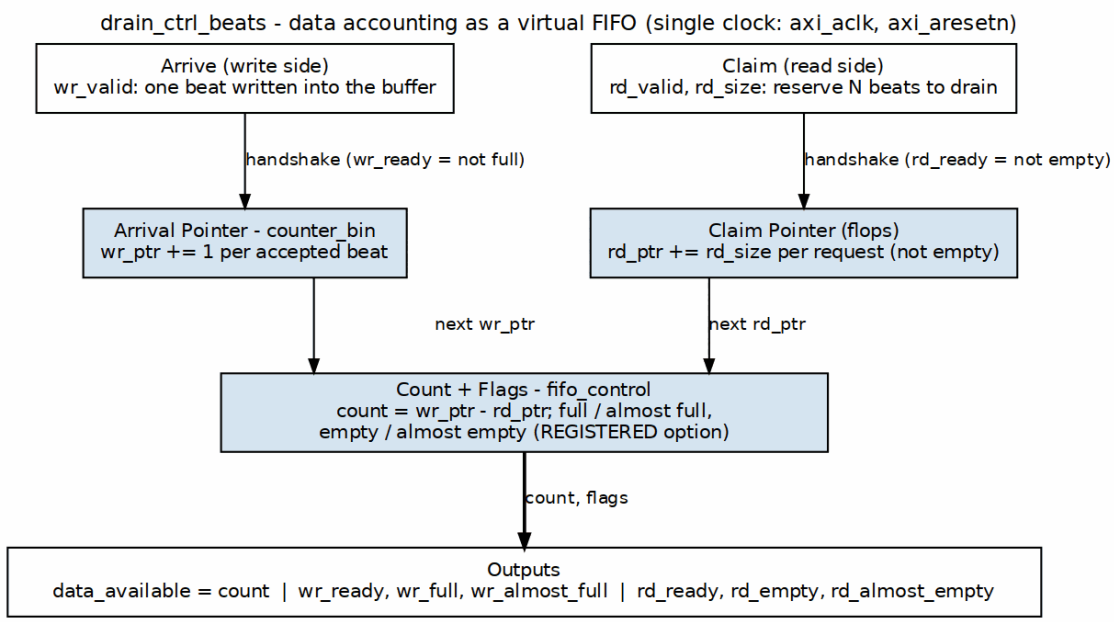
The Beats Drain Control is a "virtual FIFO" that tracks data availability without storing actual data. It complements alloc_ctrl_beats by tracking when data has been written to SRAM and is available for draining.

11.1.1 Key Features

- **Virtual FIFO:** Pointer tracking without data storage
- **Single-Beat Write:** Data arrivals tracked one beat at a time
- **Variable-Size Drain:** Supports multi-beat drain requests
- **Status Flags:** Full, almost full, empty, almost empty
- **Data Availability:** Reports beats available for drain

11.1.2 Block Diagram

11.1.3 Figure 2.6.1: Beats Drain Control Block Diagram



Drain control block diagram

Source: 06_drain_ctrl.dot

11.2 Concept: Data Availability Tracking

The drain_ctrl tracks when data has been written to SRAM:

```
alloc_ctrl:  Tracks SPACE reservations (allocation -> actual write)
drain_ctrl:  Tracks DATA availability (data write -> drain request)
```

11.2.1 Complementary Roles

```
Source Path:
  1. Scheduler allocates space (alloc_ctrl.wr)
  2. AXI reads data into SRAM
  3. Data written to SRAM (drain_ctrl.wr)
  4. Drain consumer reads (drain_ctrl.rd)

Sink Path:
  1. Fill allocates space (alloc_ctrl.wr)
  2. Fill writes to SRAM (alloc_ctrl.rd)
  3. Data available in SRAM (drain_ctrl.wr)
  4. AXI write drains SRAM (drain_ctrl.rd)
```

11.3 Parameters

```
parameter int DEPTH = 512;                    // Virtual FIFO depth
parameter int ALMOST_WR_MARGIN = 1;           // Almost full margin
parameter int ALMOST_RD_MARGIN = 1;           // Almost empty margin
parameter int REGISTERED = 1;                 // Registered outputs

// Derived
parameter int AW = $clog2(DEPTH);             // Address width
```

: Table 2.6.1: Beats Drain Control Parameters

11.4 Port List

11.4.1 Clock and Reset

Table 2.6.2: Clock and Reset

Signal	Direction	Width	Description
axi_aclk	input	1	System clock
axi_aresetn	input	1	Active-low reset

11.4.2 Write Interface (Data Written to SRAM)

Table 2.6.3: Write Interface

Signal	Direction	Width	Description
wr_valid	input	1	One beat written to SRAM
wr_ready	output	1	Can track more data

11.4.3 Read Interface (Drain Requests)

Table 2.6.4: Read Interface

Signal	Direction	Width	Description
rd_valid	input	1	Drain request

Signal	Direction	Width	Description
rd_size	input	8	Number of beats to drain
rd_ready	output	1	Data available for drain

11.4.4 Status Outputs

Table 2.6.5: Status Outputs

Signal	Direction	Width	Description
data_available	output	AW+1	Beats available to drain
wr_full	output	1	Tracking full
wr_almost_full	output	1	Near full
rd_empty	output	1	No data to drain
rd_almost_empty	output	1	Near empty

11.5 Operation

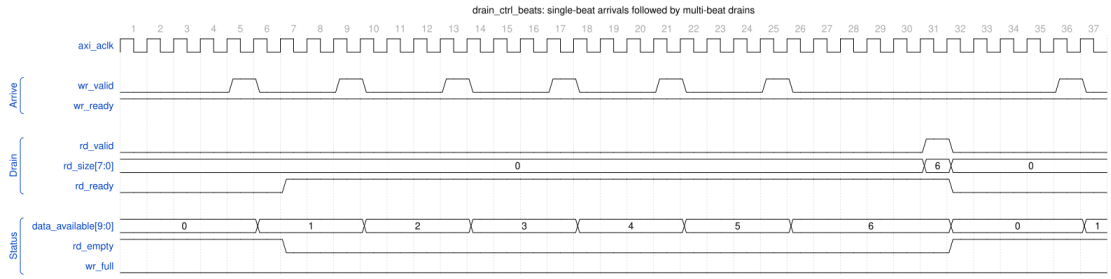
11.5.1 Pointer Arithmetic

```
data_avail = wr_ptr - rd_ptr

Full condition:    wr_ptr - rd_ptr >= DEPTH
Empty condition:   wr_ptr == rd_ptr
```

11.5.2 Timing Diagram

11.5.3 Figure 2.6.2: Data Arrival and Drain Timing



Drain control - single-beat arrivals then a multi-beat drain

Source: drain_ctrl_beats_arrive_drain.json, captured from `dv/tests/fub_beats/test_drain_ctrl_beats.py` (`basic_write_drain`, depth 512, seed 7) with `WAVES=1`.

Reading it: six `wr_valid` handshakes, one beat each, raise `data_available` 1, 2, … 6 and drop `rd_empty` on the first arrival. A single `rd_valid` handshake with `rd_size = 6` then reserves all six at once: `data_available` returns to 0 and `rd_empty` rises the next cycle. The counter tracks beats, not bytes, and never touches the data itself.

11.6 Integration Example

```
// Sink path: Track data availability for AXI write engine
drain_ctrl_beats #(
    .DEPTH(SRAM_DEPTH),
    .ALMOST_RD_MARGIN(8)
) u_snk_drain (
    .axi_aclk       (clk),
    .axi_aresetn    (rst_n),

    // Fill writes data to SRAM
    .wr_valid       (fill_valid && fill_ready),
    .wr_ready       (),  // Single-beat, always ready

    // AXI write engine drains data
    .rd_valid       (axi_wr_drain_req),
    .rd_size        (axi_wr_burst_len),
    .rd_ready       (data_ready_for_drain),

    // Status
    .data_available (beats_to_write),
```

```
    .rd_empty       (no_data_to_write)
);
```

11.7 Relationship: alloc_ctrl vs drain_ctrl

Table 2.6.6: alloc_ctrl vs drain_ctrl Comparison

Aspect	alloc_ctrl_beats	drain_ctrl_beats
Tracks	Space reservations	Data availability
Write port	Multi-beat allocation	Single-beat arrival
Read port	Single-beat release	Multi-beat drain
Output	`space_free`	`data_available`
Use case	Pre-allocate before fill	Know when to drain

Last Updated: 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

12 Latency Bridge (beats) Specification

Module: `latency_bridge_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub_beats/` **Status:** Implemented, tested, not in the SRAM path **Last Updated:** 2026-09-27

> **Not in the SRAM datapath since 2026-09-26.** RAPIDS' SRAM path is STREAM's `sram_controller`, which carries its own `stream_latency_bridge` (the same design), reached through the `snk_/src_sram_controller_beats` naming wrappers (`bdf4e0dff`). This FUB is still built and verified – it keeps its test, its filelist and its `rapids_all.f` entry – but nothing in `rapids_core_beats` instantiates it. This page is the FUB's own specification.

12.1 Overview

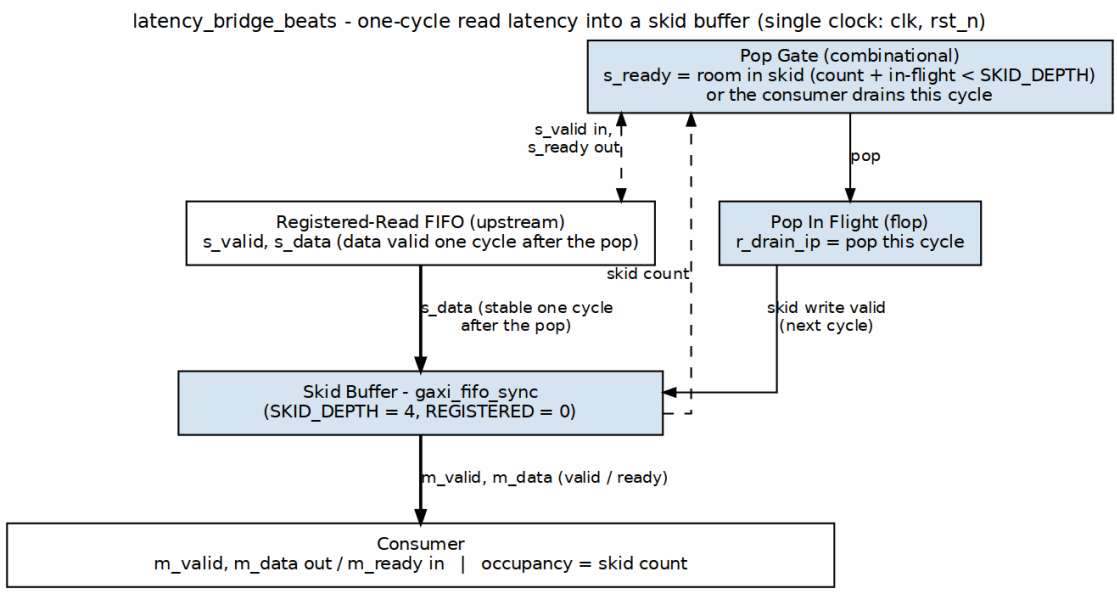
`latency_bridge_beats` turns a **registered-read FIFO** – one whose data appears the cycle *after* the read handshake – into a plain **valid/ready stream** the consumer can stall at will. It is a data-width bridge: it moves `DATA_WIDTH` bits per beat and carries nothing else. There is no beat count, no channel ID and no programmable delay in it; an earlier version of this page described such an interface, and it never existed in this module.

12.1.1 Key Features

- **One-cycle glue, one skid buffer:** a single flop (`r_drain_ip`) remembers that a FIFO read was accepted, so the data that lands the next cycle is pushed into a `gaxi_fifo_sync` skid of depth `SKID_DEPTH`
- **Full throughput when the consumer is ready:** back-to-back reads are accepted as long as the skid keeps draining
- **Backpressure absorbed in the skid:** `s_ready` drops only when a skid write is actually stalled, so the FIFO is never read into a slot that does not exist
- **Occupancy reported:** `occupancy` counts the beats the bridge holds (the one in flight plus the skid contents), which is what an occupancy-based data-available counter needs
- **Debug taps:** `dbg_r_pending` and `dbg_r_out_valid` expose the in-flight flop and the skid output valid, for catching stuck data from a test

12.1.2 Block Diagram

12.1.3 Figure 2.7.1: Latency Bridge Block Diagram



Latency bridge block diagram

Source: 07_latency_bridge.dot

12.2 Concept: bridging a registered read

A FIFO with `REGISTERED=1` read (the SRAM style this repo uses so BRAM infers cleanly) answers a read handshake with data **one cycle later**. A consumer speaking valid/ready expects data *with* valid, and may deassert ready at any time. Without a bridge the FIFO has already popped a beat the consumer did not take.

```
cycle 0   s_valid && s_ready        FIFO read accepted
r_drain_ip <= 1
cycle 1   data arrives on s_data    skid_wr_valid = r_drain_ip, data
pushed
cycle N   m_valid && m_ready        consumer drains the skid at its
own pace
```

The bridge never guesses about downstream readiness: `s_ready` is derived from the skid's room, counting only a write that is *stalled* this cycle as pending (a write that completes this cycle frees its slot at the same edge). That is why full throughput holds while writes complete immediately, and why the FIFO is never over-read when they do not.

12.3 Parameters

```
parameter int DATA_WIDTH = 64;    // Beat width
parameter int SKID_DEPTH = 4;     // Skid buffer depth (2-4
recommended)
parameter int DW = DATA_WIDTH;    // Short alias
```

: Table 2.7.1: Latency Bridge Parameters

`occupancy` is 3 bits wide and counts 0..5 at the default depth (one beat in flight plus four in the skid); a larger `SKID_DEPTH` needs the port widened.

12.4 Port List

12.4.1 Clock and Reset

Table 2.7.2: Clock and Reset

Signal	Direction	Width	Description
`clk`	input	1	System clock
`rst_n`	input	1	Active-low reset

12.4.2 Upstream Interface (from the registered FIFO)

Table 2.7.3: Upstream Interface

Signal	Direction	Width	Description
`s_valid`	input	1	FIFO has a beat to read (not empty)
`s_ready`	output	1	Bridge accepts a read this cycle; the data lands NEXT cycle
`s_data`	input	DW	Read data, valid one cycle after the `s_valid && s_ready` handshake

12.4.3 Downstream Interface (to the consumer)

Table 2.7.4: Downstream Interface

Signal	Direction	Width	Description
`m_valid`	output	1	Beat available

Signal	Direction	Width	Description
m_ready	input	1	Consumer takes the beat
m_data	output	DW	Beat data, valid with m_valid

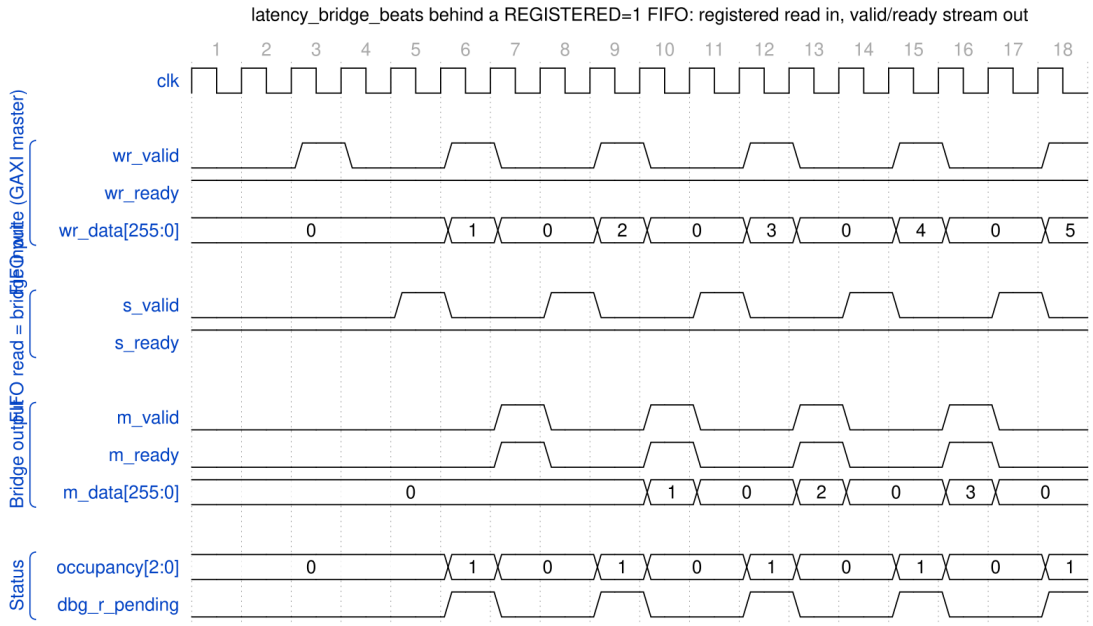
12.4.4 Status and Debug

Table 2.7.5: Status and Debug

Signal	Direction	Width	Description
occupancy	output	3	Beats held by the bridge: the in-flight read plus the skid contents (0..5)
dbg_r_pending	output	1	r_drain_ip: a read was accepted and its data is due next cycle
dbg_r_out_valid	output	1	Skid output valid (mirrors m_valid)

12.5 Operation

12.5.1 Figure 2.7.2: Latency Bridge Timing (consumer always ready)



Latency bridge behind a registered-read FIFO

Source: latency_bridge_beats_streaming.json, captured from `dv/tests/fub_beats/test_latency_bridge_beats.py` (`streaming`, 256-bit, seed 7) with `WAVES=1`. The test drives a `REGISTERED = 1` `gaxi_fifo_sync` in front of the bridge (`dv/tb/latency_bridge_beats_tb_top.sv`), because that is the only way to produce the bridge's upstream contract: a valid/ready master presents data *with* valid, a registered FIFO presents it one cycle *after* the read handshake.

Reading it: each `wr_valid` beat lands in the FIFO and `s_valid` (FIFO not empty) rises a cycle later; the bridge reads it at once (`s_ready` stays high) and `dbg_r_pending` marks the cycle the data is in flight. Two cycles after the write the beat is on `m_valid`/`m_data` with its payload intact (1, 2, 3, ...), and `occupancy` shows exactly one beat inside the bridge per transfer. With a consumer that is always ready the bridge adds the FIFO's read latency and no more.

An earlier revision of this test drove `s_*` directly from the BFM and never compared `m_data`; every beat came out 0 and the test passed (rapids TASK-003). (`dv/tests/fub_beats/test_latency_bridge_beats.py`).

12.6 Integration Context

The bridge sits between a `gaxi_fifo_sync` with `REGISTERED=1` and whatever consumes the stream. The same arrangement, with STREAM's twin `stream_latency_bridge`, is what `sram_controller_unit` uses inside the shared SRAM controller today; `occupancy` there is what the drain accounting must allow for (stream BUG-011).

```
latency_bridge_beats #(
    .DATA_WIDTH (512),
    .SKID_DEPTH (4)
) u_bridge (
    .clk             (clk),
    .rst_n           (rst_n),
    // registered-read FIFO side
    .s_valid         (fifo_rd_valid),
    .s_ready         (fifo_rd_ready),
    .s_data          (fifo_rd_data),
    // consumer side
    .m_valid         (drain_valid),
    .m_ready         (drain_read),
    .m_data          (drain_data),
    // status / debug
    .occupancy       (bridge_occupancy),
    .dbg_r_pending   (dbg_bridge_pending),
    .dbg_r_out_valid (dbg_bridge_out_valid)
);
```

12.7 Design Considerations

Table 2.7.6: Skid Depth Selection

SKID_DEPTH	Behaviour
2	Minimum: one stall of the consumer costs a bubble on the next read
4	Default: absorbs a short consumer stall with no upstream bubble
8	Deeper stall absorption; `occupancy` must be widened past 3 bits

The skid does not need to cover the FIFO's read latency (that is the one `r_drain_ip` flop's job); it needs to cover how long the *consumer* stalls without the bridge stopping upstream reads.

Last Updated: 2026-09-27

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

13 Control-Read Engine Specification

Module: ctrlrd_engine.sv **Location:** projects/components/dma-ip/rapids/rtl/fub/
Status: Implemented **Last Updated:** 2026-07-12

13.1 Overview

The Control-Read Engine implements the CTRL_READ descriptor opcode (DESC_OP_CTRL_READ = 2'b01) as a **consumer gate**. When the scheduler decodes a CTRL_READ descriptor, it hands the engine a poll address, an expected value, and a compare mask. The engine issues an AXI4 read to the poll address, applies the mask to the returned data, and compares (read_data & mask) against the expected value. If they match, the gate is satisfied and the descriptor chain proceeds; if not, the engine retries. Retries are bounded so a gate that never matches cannot hang the channel.

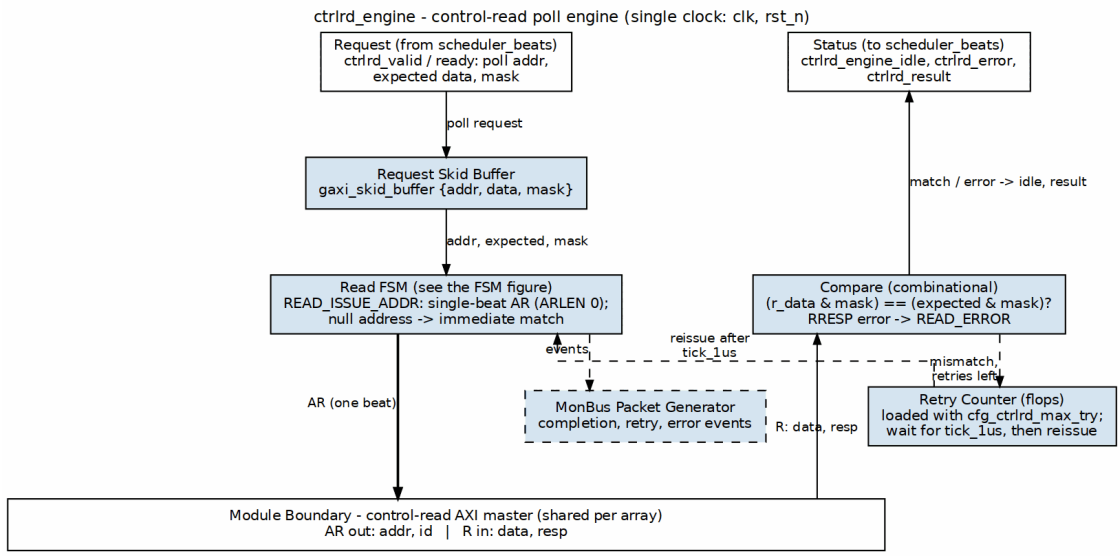
One Control-Read Engine is instantiated per channel in scheduler_group_beats.sv (u_ctrlrd_engine), configured for 32-bit AXI reads.

13.1.1 Key Features

- **Consumer Gate Semantics:** Polls memory until (read_data & mask) == expected
- **Masked Comparison:** Only the masked bits of the polled word participate in the match
- **Bounded Retry:** cfg_ctrlrd_max_try (0-511) caps the number of poll attempts
- **1 microsecond Retry Pacing:** Retries wait for tick_1us from the scheduler group
- **Error on Exhaustion:** Asserts ctrlrd_error if the gate never matches within budget
- **Null Address Fast-Path:** A poll address of zero completes immediately as a match
- **Channel Reset Support:** cfg_channel_reset returns the FSM to idle and drains any read still on the fabric
- **MonBus Integration:** Completion, retry, and error events

13.1.2 Block Diagram

13.1.3 Figure 2.8.1: Control-Read Engine Block Diagram



Control-read engine block diagram

Source: 08_ctrlrd_engine.dot

13.2 Parameters

```
parameter int CHANNEL_ID = 0;                    // Channel identifier
parameter int NUM_CHANNELS = 32;                 // Total channels
parameter int CHAN_WIDTH = $clog2(NUM_CHANNELS); // Channel index
width (derived)
parameter int ADDR_WIDTH = 64;                   // Address bus width
parameter int AXI_DATA_WIDTH = 64;               // AXI read data
width (>= 32)
parameter int AXI_ID_WIDTH = 8;                  // AXI ID width (>=
CHAN_WIDTH)

// Monitor Bus Parameters
parameter logic [15:0] MON_AGENT_ID  = 16'h0030; // Control-Read
Engine Agent ID
parameter logic [7:0]  MON_UNIT_ID   = 8'h02;    // Unit identifier
parameter logic [8:0]  MON_CHANNEL_ID = 9'h000;  // Base channel ID
```

: Table 2.8.1: Control-Read Engine Parameters

Parameter validation enforces `AXI_ID_WIDTH >= CHAN_WIDTH` and `AXI_DATA_WIDTH >= 32` (the engine consumes only the lower 32 bits of the read data). The per-channel instance in `scheduler_group_beats.sv` overrides `AXI_DATA_WIDTH` to 32.

13.3 Port List

13.3.1 Clock and Reset

Table 2.8.2: Clock and Reset

Signal	Direction	Width	Description
`clk`	input	1	System clock
`rst_n`	input	1	Active-low asynchronous reset

13.3.2 Scheduler Interface

Table 2.8.3: Scheduler Interface

Signal	Direction	Width	Description
`ctrlrd_valid`	input	1	Gate request valid from scheduler
`ctrlrd_ready`	output	1	Engine ready to accept a request
`ctrlrd_pkt_addr`	input	AW	Poll address (descriptor `poll_addr[63:0]`)
`ctrlrd_pkt_data`	input	32	Expected value (descriptor `expected[95:64]`)
`ctrlrd_pkt_mask`	input	32	Compare mask (descriptor `mask[127:96]`)
`ctrlrd_error`	output	1	Gate failed (retries exhausted or AXI error)
`ctrlrd_result`	output	32	Last read value (lower 32 bits)
`ctrlrd_engin`	output	1	Engine idle (post-issue

Signal	Direction	Width	Description
e_idle			completion indicator)

13.3.3 Configuration Interface

Table 2.8.4: Configuration Interface

Signal	Direction	Width	Description
cfg_ctrlrd_max_try	input	9	Poll retry budget, 0-511 (from CTRL_CONFIG.CTRLRD_MAX_TRY)
cfg_channel_reset	input	1	Per-channel reset request
tick_1us	input	1	1 microsecond tick that paces retries

13.3.4 AXI4 Read Master Interface

Table 2.8.5: AXI4 Read Master Interface

Signal	Direction	Width	Description
ar_valid	output	1	Read address valid
ar_ready	input	1	Read address ready
ar_addr	output	AW	Read address (poll address)
ar_len	output	8	Burst length (8'h00, single beat)
ar_size	output	3	Burst size (3'b010, 4 bytes)
ar_burst	output	2	Burst type (2'b01, INCR)
ar_id	output	ID	Transaction ID (channel-derived)
ar_lock	output	1	Lock (1'b0, normal access)
ar_cache	output	4	Cache (4'b0010, non-cacheable bufferable)
ar_prot	output	3	Protection (3'b000)

Signal	Direction	Width	Description
ar_qos	output	4	Quality of service (4'h0)
ar_region	output	4	Region (4'h0)
r_valid	input	1	Read data valid (shared channel)
r_ready	output	1	Read data ready
r_data	input	ADW	Read data (lower 32 bits used)
r_id	input	ID	Read response ID (matched to our request)
r_resp	input	2	Read response
r_last	input	1	Read last beat

13.3.5 MonBus Interface

Table 2.8.6: MonBus Interface

Signal	Direction	Width	Description
i_mon_time	input	64	Side-band monitor timestamp
mon_valid	output	1	Monitor packet valid
mon_ready	input	1	Monitor consumer ready
mon_packet	output	128	Monitor packet (monitor_packet_t)
mon_timestamp	output	64	Monitor packet timestamp

13.4 FSM States

13.4.1 Figure 2.8.2: Control-Read Engine FSM

States and transitions of the `ctrlrd_engine` poll FSM.

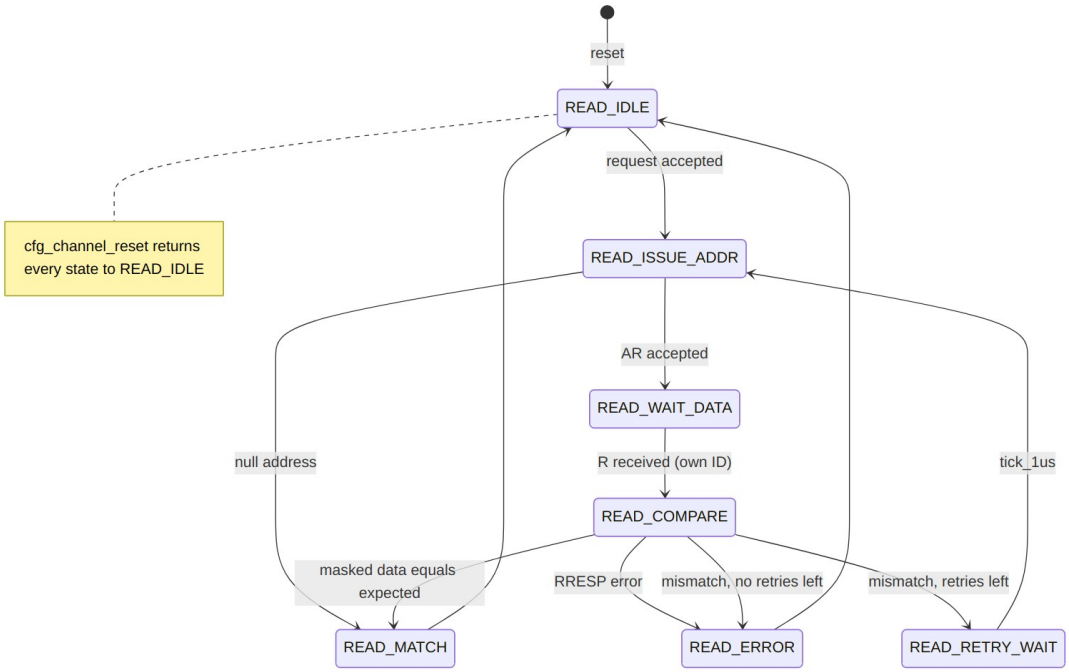


Diagram 3

State summary:

- **READ_IDLE:** Waits for a request from the scheduler (via a two-deep skid buffer). Latches poll address, expected value, mask, and the AXI ID; initializes the retry counter from `cfg_ctrlrd_max_try`.
- **READ_ISSUE_ADDR:** Drives `ar_valid`. A null address (`64'h0`) shortcuts to `READ_MATCH`; otherwise, once `ar_ready` handshakes, advances to `READ_WAIT_DATA`.
- **READ_WAIT_DATA:** Accepts the read response whose `r_id` matches the channel ID, capturing the lower 32 bits and the response code.
- **READ_COMPARE:** On an AXI error response, goes to `READ_ERROR`. On a masked match, goes to `READ_MATCH`. Otherwise, if retries remain, decrements the counter and goes to `READ_RETRY_WAIT`; if the budget is exhausted, goes to `READ_ERROR`.
- **READ_RETRY_WAIT:** Waits for `tick_1us`, then reissues the read.

- **READ_MATCH:** Gate satisfied. Clears the error flag and returns to READ_IDLE.
- **READ_ERROR:** Sets ctrlrd_error and returns to READ_IDLE.

13.5 Operation

13.5.1 Masked Compare

The engine gates on a masked equality check:

```
w_masked_expected = ctrlrd_pkt_data & ctrlrd_pkt_mask
w_masked_actual   = r_data[31:0]    & ctrlrd_pkt_mask
w_data_match      = (w_masked_expected == w_masked_actual)
```

Only the bits set in mask participate. A mask of 32'hFFFFFFFF compares the full word; a mask of a single bit turns the gate into a flag-poll.

13.5.2 Retry Budget and Pacing

The retry counter is loaded from cfg_ctrlrd_max_try when the request is accepted. On each non-matching, non-error compare with retries remaining, the counter decrements and the engine parks in READ_RETRY_WAIT until the next tick_1us, which paces successive polls at roughly 1 microsecond intervals. When the counter reaches zero without a match, the engine transitions to READ_ERROR and asserts ctrlrd_error, so a never-matching gate cannot stall the channel indefinitely.

cfg_ctrlrd_max_try is driven by the CTRL_CONFIG register at offset 0x240, field CTRLRD_MAX_TRY[8:0] (0-511, reset 16).

13.5.3 Null Address Fast-Path

If the latched poll address is 64'h0, the engine treats the gate as already satisfied: READ_ISSUE_ADDR transitions directly to READ_MATCH without issuing an AXI read. This lets software emit an unconditional (no-op) gate.

13.5.4 AXI Read Transaction

Polls are single-beat, 4-byte INCR reads (ar_len = 0, ar_size = 3'b010, ar_burst = 2'b01). The ar_id is derived from CHANNEL_ID, and only responses whose r_id matches are consumed on the shared read data channel. ctrlrd_result exposes the last captured 32-bit value; in the per-channel instantiation the scheduler gates on match/error rather than the raw value, so ctrlrd_result is left unconnected there.

13.5.5 Channel Reset

`cfg_channel_reset` is registered into `r_channel_reset_active`. While asserted, the engine refuses new requests (`ctrlrd_ready` is forced low), the retry and error state is cleared, and the FSM is driven back to `READ_IDLE`.

What the reset cannot do is cancel a read already on the fabric, because AXI has no cancel. The engine therefore drains it (rapids TASK-014, 2026-09-27):

- an AR that was raised and not yet accepted stays raised until the slave takes it (`r_drain_ar`), then
- the R beat that the accepted AR is owed is taken and discarded (`r_drain_pending`, which alone lifts `r_ready` outside `READ_WAIT_DATA`).

Until the beat is gone `ctrlrd_engine_idle` stays low and the request path stays closed, so the next read on this ID cannot be answered by the stale beat. Before this the reset dropped the FSM to idle with `r_ready` low, the beat sat on the bus, and a re-kicked channel took it as its own answer.

`ctrlrd_engine_idle` asserts only in `READ_IDLE` with no pending request, no active channel reset and nothing left to drain.

13.6 MonBus Reporting

The engine emits monitor packets on notable events:

Table 2.8.7: MonBus Events

Event	Packet Type	Code	Payload
Gate satisfied (non-null)	`PktTypeCompletion`	`CORE_COMPL_CTRLRD_COMPLETED`	Poll address
Retry attempt	`PktTypePerf`	`CORE_PERF_CTRLRD_RETRY`	Remaining retry count
Retries exhausted	`PktTypeError`	`CORE_ERR_CTRLRD_MAX_RETRIES`	Poll address
AXI error response	`PktTypeError`	`AXI_ERR_RESP_SLVERR` / `AXI_ERR_RESP_DECERR`	Response code

All packets carry MON_AGENT_ID, MON_UNIT_ID, and MON_CHANNEL_ID, and are timestamped from i_mon_time.

13.7 Descriptor Field Mapping

The CTRL_READ opcode reinterprets the shared 256-bit descriptor (the descriptor engine's existing field extraction is reused):

Table 2.8.8: CTRL_READ Descriptor Field Mapping

Descriptor Field	Bits	Engine Port
poll_addr	[63:0]	ctrlrd_pkt_addr
expected	[95:64]	ctrlrd_pkt_data
mask	[127:96]	ctrlrd_pkt_mask
max_try	[143:128]	Per-descriptor budget (0 = use cfg_ctrlrd_max_try default)

Last Updated: 2026-07-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

14 Control-Write Engine Specification

Module: ctrlwr_engine.sv **Location:** projects/components/dma-ip/rapids/rtl/fub/
Status: Implemented **Last Updated:** 2025-01-10

14.1 Overview

The Control-Write Engine executes a CTRL_WRITE control descriptor (DESC_OP_CTRL_WRITE = 2'b10). A CTRL_WRITE is a producer doorbell: the engine performs a single-beat AXI4 write of the descriptor's 32-bit wr_data value to its 64-bit wr_addr target, then completes so the descriptor chain continues. One instance is dedicated per channel.

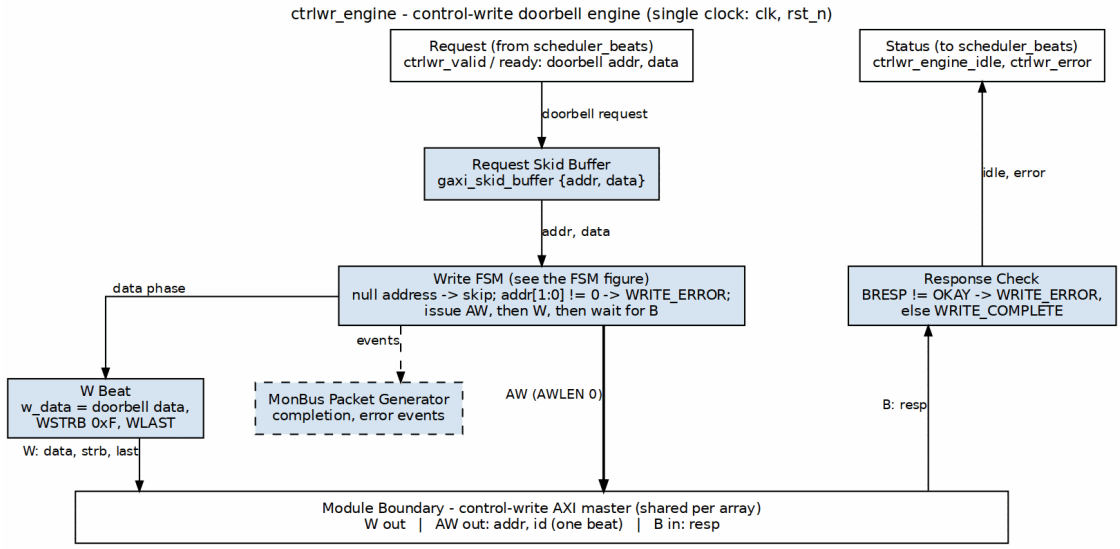
The write target address must be 4-byte aligned because the transfer is a 32-bit (aw_size = 3'b010) single-beat write. A null address (wr_addr == 0) is treated as a no-op and the request completes without issuing an AXI transaction.

14.1.1 Key Features

- **Single-Beat AXI4 Write:** One 32-bit doorbell write per request (aw_len = 0, w_last = 1)
- **Producer Doorbell Semantics:** Write value to target, then release the chain (no poll, no retry)
- **Address Alignment Check:** Requires 4-byte-aligned wr_addr; misalignment raises an error
- **Null-Address Skip:** wr_addr == 0 completes as a no-op
- **Channel Reset Support:** cfg_channel_reset returns the FSM to idle and drains any write still on the fabric
- **MonBus Integration:** Completion and error event reporting

14.1.2 Block Diagram

14.1.3 Figure 2.9.1: Control-Write Engine Block Diagram



Control-write engine block diagram

Source: 09_ctrlwr_engine.dot

14.2 Parameters

```
parameter int CHANNEL_ID = 0;                    // Channel identifier
parameter int NUM_CHANNELS = 32;                 // Total channels
parameter int CHAN_WIDTH = $clog2(NUM_CHANNELS); // Derived channel
index width
parameter int ADDR_WIDTH = 64;                   // Address bus width
parameter int AXI_ID_WIDTH = 8;                  // AXI transaction ID
width

// Monitor Bus Parameters
parameter logic [15:0] MON_AGENT_ID  = 16'h0020; // Ctrlwr Engine
Agent ID
parameter logic [7:0]  MON_UNIT_ID   = 8'h01;    // Unit identifier
parameter logic [8:0]  MON_CHANNEL_ID = 9'h000;  // Base channel ID
```

: Table 2.9.1: Control-Write Engine Parameters

Note: `AXI_ID_WIDTH` must be `>= CHAN_WIDTH` (checked by an elaboration-time `$fatal`). When instantiated in `scheduler_group_beats.sv` as `u_ctrlwr_engine`, `MON_AGENT_ID` is overridden to `16'h0021` (`CTRLWR_MON_AGENT_ID`) to distinguish it from the Control-Read Engine's `16'h0020`.

14.3 Port List

14.3.1 Clock and Reset

Table 2.9.2: Clock and Reset

Signal	Direction	Width	Description
`clk`	input	1	System clock
`rst_n`	input	1	Active-low asynchronous reset

14.3.2 Configuration Interface

Table 2.9.3: Configuration Interface

Signal	Direction	Width	Description
`cfg_channel_reset`	input	1	Per-channel reset: blocks new requests, drains transaction, forces idle

14.3.3 Scheduler Interface

Table 2.9.4: Scheduler Interface

Signal	Direction	Width	Description
`ctrlwr_valid`	input	1	Request valid (doorbell write requested)
`ctrlwr_ready`	output	1	Ready to accept request
`ctrlwr_pkt_addr`	input	AW	Doorbell target address (`wr_addr`, 4-byte aligned)
`ctrlwr_pkt_data`	input	32	Doorbell write value (`wr_data`)
`ctrlwr_error`	output	1	Error flag (alignment or AXI response error)
`ctrlwr_engine_idle`	output	1	Engine idle (in `WRITE_IDLE`, no active transaction, not in channel reset)

14.3.4 AXI4 Write Master Interface

Table 2.9.5: AXI4 Write Master Interface

Signal	Direction	Width	Description
`aw_valid`	output	1	Write address valid
`aw_ready`	input	1	Write address ready
`aw_addr`	output	AW	Write address (= `wr_addr`)
`aw_len`	output	8	Burst length (`0` = single beat)
`aw_size`	output	3	Transfer size (`3'b010` = 4 bytes / 32-bit)
`aw_burst`	output	2	Burst type (`2'b01` = INCR)
`aw_id`	output	AXI_ID_WIDTH	Transaction ID (derived from `CHANNEL_ID`)
`aw_lock`	output	1	Lock type (`0` = normal)
`aw_cache`	output	4	Cache type (`4'b0010` = non-cacheable bufferable)
`aw_prot`	output	3	Protection type (`3'b000`)
`aw_qos`	output	4	Quality of service (`0`)
`aw_region`	output	4	Region identifier (`0`)
`w_valid`	output	1	Write data valid
`w_ready`	input	1	Write data ready
`w_data`	output	32	Write data (= `wr_data`)
`w_strb`	output	4	Write strobes (`4'b1111` = all bytes)
`w_last`	output	1	Last beat (`1` = single beat)
`b_valid`	input	1	Write response valid
`b_ready`	output	1	Write response ready
`b_id`	input	AXI_ID_WIDTH	Response ID (matched against expected AXI ID)
`b_resp`	input	2	Write response code

14.3.5 MonBus Interface

Table 2.9.6: MonBus Interface

Signal	Direction	Width	Description
`i_mon_time`	input	64	Monitor timestamp side-band input
`mon_valid`	output	1	Monitor packet valid
`mon_ready`	input	1	Monitor packet ready
`mon_packet`	output	128	Monitor packet (`monitor_packet_t`)
`mon_timestamp`	output	64	Monitor packet timestamp

14.4 FSM States

14.4.1 Figure 2.9.2: Control-Write Engine FSM

States and transitions of the `ctrlwr_engine` doorbell FSM.

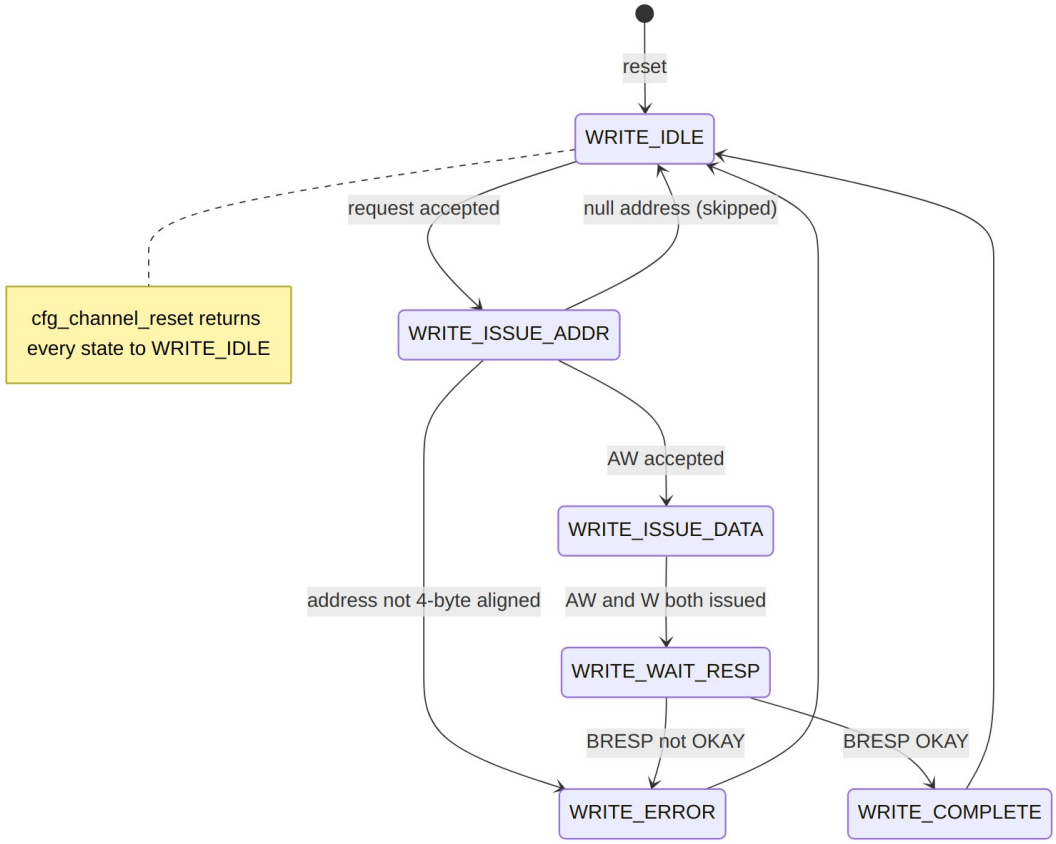


Diagram 4

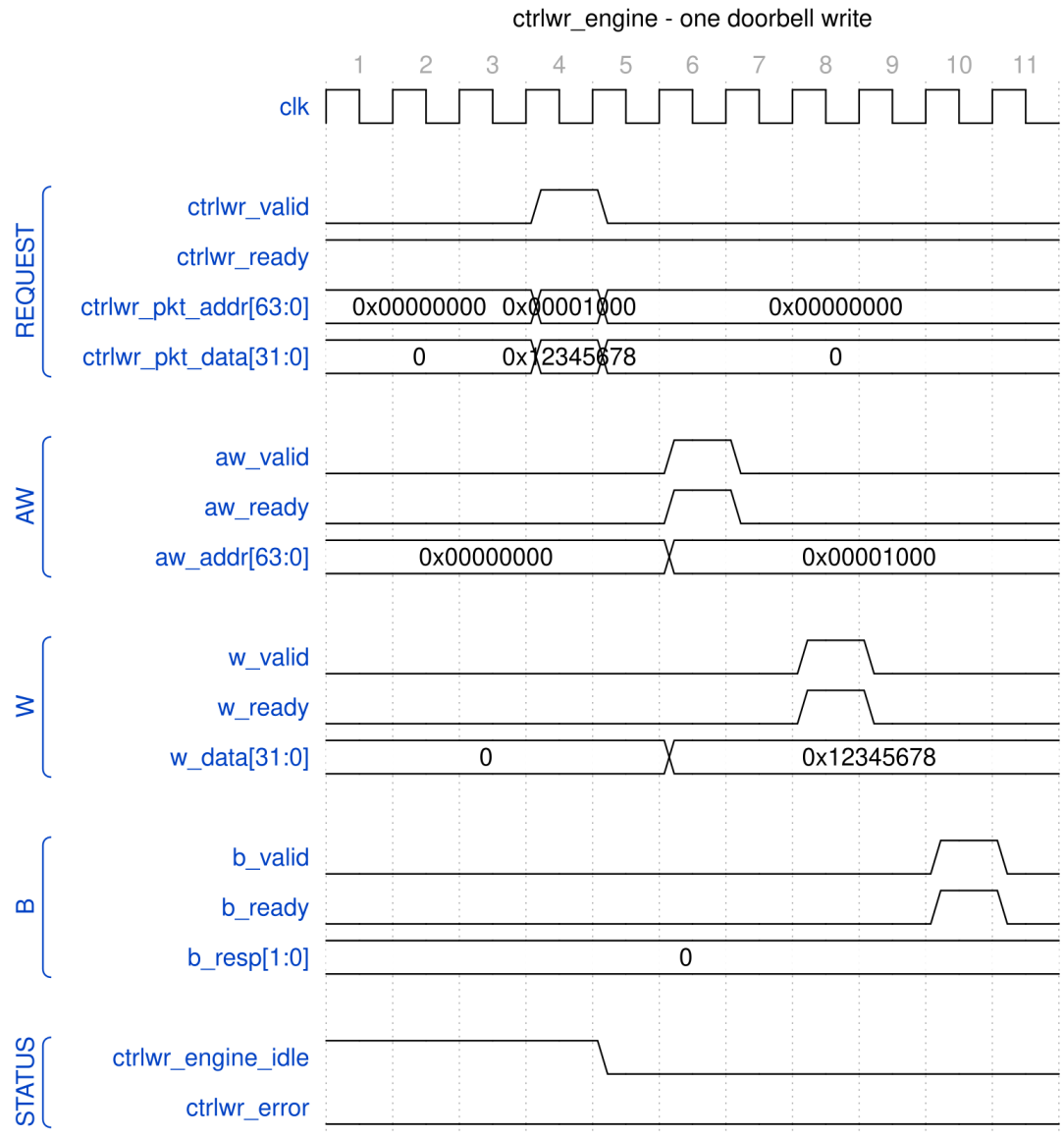
State summary:

- **WRITE_IDLE:** Wait for a request from the request skid buffer; latch `wr_addr`/`wr_data` and the expected AXI ID.
- **WRITE_ISSUE_ADDR:** Drive `aw_valid` until `aw_ready`. A misaligned address routes to `WRITE_ERROR`; a null address returns to `WRITE_IDLE` (no-op skip).
- **WRITE_ISSUE_DATA:** Drive `w_valid` until `w_ready`; advance once both address and data phases have been issued.
- **WRITE_WAIT_RESP:** Accept the matching `b`-channel response; a non-OKAY `b_resp` routes to `WRITE_ERROR`, otherwise to `WRITE_COMPLETE`.
- **WRITE_COMPLETE:** Emit a completion MonBus packet (unless null address) and return to `WRITE_IDLE`.
- **WRITE_ERROR:** Latch `ctrlwr_error`, emit an error MonBus packet, and return to `WRITE_IDLE`. The error flag persists until reset or `cfg_channel_reset`.

14.5 Operation

14.5.1 Doorbell Write Sequence

14.5.2 Figure 2.9.3: Control-Write Doorbell Timing



ctrlwr_engine - one doorbell write

Source: ctrlwr_engine_doorbell.json, captured from `dv/tests/fub/test_ctrlwr_engine.py` (`basic_write`, channel 0, 64-bit address, `REG_LEVEL=GATE`) with `WAVES=1`; the AXI side is the framework write slave.

Reading it: the request handshakes in one cycle (`ctrlwr_ready` is held high by the skid buffer) with the doorbell address 0x1000 and data 0x12345678, and the engine leaves idle the next cycle. AW goes out two cycles after the request, W two cycles after AW, and the slave answers B (OKAY) two cycles after W; `ctrlwr_engine_idle` returns the cycle after the B handshake, eight cycles after the request. Nothing is pipelined across doorbells: the engine takes one request at a time.

14.5.3 Request Handshake and Skid Buffer

Incoming requests are captured through a depth-2 `gaxi_skid_buffer`. The engine accepts a request (`ctrlwr_ready`) only when not in an active channel reset. The FSM pops the buffered request in `WRITE_IDLE`, latching `{wr_addr, wr_data}` and computing the expected AXI ID from `CHANNEL_ID`:

```
aw_id = {{(AXI_ID_WIDTH-CHAN_WIDTH){1'b0}}, CHANNEL_ID[CHAN_WIDTH-
1:0]};
```

The b-channel response is matched against this expected ID (`b_id == r_expected_axi_id`) so a shared write master can be demultiplexed per channel.

14.5.4 Address Validation

```
Null (skip)  : wr_addr == 64'h0
Align error  : wr_addr[1:0] != 2'b00  (and not null)
```

A null address completes immediately as a no-op with no AXI activity and no completion packet. A misaligned address never issues on AXI; it transitions to `WRITE_ERROR`, asserts `ctrlwr_error`, and emits an error MonBus packet.

14.5.5 Error Reporting

`ctrlwr_error` is a registered, sticky flag. It is set on an alignment error or on a non-OKAY AXI write response (`b_resp != 2'b00`), and it persists until `rst_n` or `cfg_channel_reset`. The MonBus error packet carries either the offending address (alignment error) or the captured `b_resp` code (AXI response error).

14.5.6 Channel Reset

Asserting `cfg_channel_reset` registers `r_channel_reset_active`, which blocks new requests (`ctrlwr_ready` low), clears the error flag and the phase-issued state, and drives the FSM back to `WRITE_IDLE`.

A write already on the fabric cannot be cancelled, so the engine drains it (rapids TASK-014, 2026-09-27): a raised AW stays up until accepted (`r_drain_aw`), an accepted AW is followed by its W beat with the latched data (`r_drain_w`), and the B response is taken and discarded (`r_drain_b`, which alone lifts `b_ready` outside `WRITE_WAIT_RESP`). The write that was already issued

therefore lands. Until B is taken `ctrlwr_engine_idle` stays low and the request path stays closed. Before this the reset abandoned whatever phase was in flight and left a dangling response on the channel's ID.

`ctrlwr_engine_idle` asserts only in `WRITE_IDLE` with no pending request, no active channel reset and nothing left to drain.

14.6 Contrast with the Control-Read Engine

The Control-Write Engine is the producer counterpart to the Control-Read Engine (Section 2.8). Both are per-channel control-descriptor executors, but they implement opposite half of the producer/consumer handshake:

Aspect	Control-Read Engine (2.8)	Control-Write Engine (2.9)
Opcode	`DESC_OP_CTRL_READ` (`2'b01`)	`DESC_OP_CTRL_WRITE` (`2'b10`)
Role	Consumer gate	Producer doorbell
AXI direction	Read (poll)	Single-beat write
Semantics	Poll until `(rd & mask) == expected`	Write `wr_data` to `wr_addr`, then continue
Retry budget	`max_try` retry budget (config default)	None (single unconditional write)
Chain effect	Holds off the chain until the gate passes	Releases the chain after the write completes
Descriptor fields	`poll_addr` / `expected` / `mask` / `max_try`	`wr_addr` / `wr_data`

A `CTRL_WRITE` therefore has no maximum-try configuration: it is one unconditional 32-bit write, whereas `CTRL_READ` polls with a bounded retry budget before either passing the gate or timing out.

Last Updated: 2026-07-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

15 Beats Scheduler Group Specification

Module: `scheduler_group_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

15.1 Overview

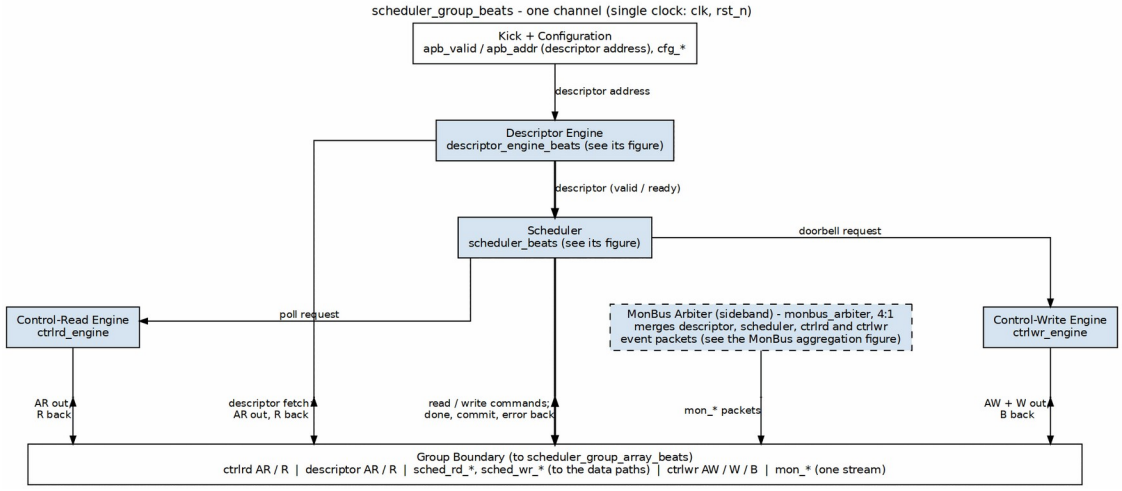
The Beats Scheduler Group wraps a single channel's scheduler and descriptor engine together with MonBus aggregation. It provides a clean integration point for the scheduler_group_array.

15.1.1 Key Features

- **Single Channel Integration:** One scheduler + one descriptor engine
- **MonBus Aggregation:** Combines scheduler and descriptor engine MonBus outputs
- **Configuration Pass-Through:** Routes configuration to both sub-modules
- **Status Aggregation:** Combined idle and error status

15.1.2 Block Diagram

15.1.3 Figure 3.1.1: Beats Scheduler Group Block Diagram



Scheduler group block diagram

Source: 10_scheduler_group.dot

15.2 Parameters

```
parameter int CHANNEL_ID = 0;                    // Channel identifier
parameter int NUM_CHANNELS = 8;                  // Total channels
parameter int ADDR_WIDTH = 64;                   // Address bus width
parameter int DATA_WIDTH = 512;                  // Data bus width

// Monitor Bus Parameters
parameter int DESC_MON_AGENT_ID = 16 + CHANNEL_ID;  // Descriptor
engine agent
parameter int SCHED_MON_AGENT_ID = 48 + CHANNEL_ID; // Scheduler agent
parameter int MON_UNIT_ID = 1;
```

: Table 3.1.1: Beats Scheduler Group Parameters

15.3 Port List

15.3.1 Clock and Reset

Table 3.1.2: Clock and Reset

Signal	Direction	Width	Description
`clk`	input	1	System clock
`rst_n`	input	1	Active-low reset

15.3.2 APB Programming Interface

Table 3.1.3: APB Programming Interface

Signal	Direction	Width	Description
`apb_valid`	input	1	Channel kick-off
`apb_ready`	output	1	Ready for kick-off
`apb_addr`	input	AW	First descriptor address

15.3.3 Configuration Interface

Table 3.1.4: Configuration Interface

Signal	Direction	Width	Description
cfg_channel_enable	input	1	Enable this channel
cfg_channel_reset	input	1	Per-channel soft reset
cfg_sched_timeout_cycles	input	32	Write-progress timeout window (cycles)
cfg_sched_timeout_limit	input	8	Consecutive-timeout windows before fatal escalation (0 = never)
cfg_sched_timeout_enable	input	1	Enable timeout detection
cfg_sched_err_enable	input	1	Enable error reporting
cfg_sched_compl_enable	input	1	SCHED_CONFIG.COMPL_EN: 1 = the scheduler's and descriptor engine's CORE Completion packets reach the group's monitor bus; 0 = the group drops them at its arbiter inputs and acknowledges the emitters, so nothing stalls (rapids ISSUE-005). Errors are never gated.
cfg_sched_perf_enable	input	1	Enable performance monitoring
cfg_desceng_prefetch	input	1	Enable descriptor prefetch chaining
cfg_desceng_fifo_thresh	input	4	Prefetch threshold (descriptors buffered ahead)
cfg_desceng_addr0_base	input	ADDR_WIDT	Valid address range 0

Signal	Direction	Width	Description
		H	base
cfg_desceng_addr0_limit	input	ADDR_WIDTH	Valid address range 0 limit
cfg_desceng_addr1_base	input	ADDR_WIDTH	Valid address range 1 base
cfg_desceng_addr1_limit	input	ADDR_WIDTH	Valid address range 1 limit
cfg_ctrlrd_max_try	input	9	ctrlrd poll retry budget (0-511)
tick_1us	input	1	1 us tick for ctrlrd retry spacing

15.3.4 Descriptor AXI Master Interface

Table 3.1.5: Descriptor AXI Master Interface

Signal	Direction	Width	Description
desc_ar_valid	output	1	AR valid
desc_ar_ready	input	1	AR ready
desc_ar_addr	output	ADDR_WIDTH	Descriptor fetch address
desc_ar_len	output	8	Burst length - 1
desc_ar_size	output	3	Burst size (log2 bytes)
desc_ar_burst	output	2	Burst type
desc_ar_id	output	AXI_ID_WIDTH	Transaction ID
desc_ar_lock	output	1	Lock type
desc_ar_cache	output	4	Cache attributes
desc_ar_prot	output	3	Protection attributes
desc_ar_qos	output	4	Quality of service
desc_ar_region	output	4	Region

Signal	Direction	Width	Description
			identifier
desc_r_valid	input	1	R valid
desc_r_ready	output	1	R ready
desc_r_data	input	256	Descriptor payload (fixed 256-bit)
desc_r_resp	input	2	Read response
desc_r_last	input	1	Last beat
desc_r_id	input	AXI_ID_WIDTH	Response ID

15.3.5 Scheduler Data Interfaces

Table 3.1.6: Scheduler Data Interfaces

Signal	Direction	Width	Description
sched_rd_valid	output	1	Read request
sched_rd_addr	output	AW	Read address
sched_rd_beats	output	32	Read beats
sched_rd_done_strobe	input	1	Read complete
sched_wr_valid	output	1	Write request
sched_wr_addr	output	AW	Write address
sched_wr_beats	output	32	Write beats
sched_wr_done_strobe	input	1	Write AW-issue strobe
sched_wr_commit_strobe	input	1	Write COMMIT strobe (B response)
sched_wr_commit_beats	input	32	Beats committed this strobe

15.3.6 Status

Table 3.1.7: Status Interface

Signal	Direction	Width	Description
`scheduler_idle`	output	1	Scheduler idle
`descriptor_engine_idle`	output	1	Descriptor engine idle
`scheduler_state`	output	7	FSM state
`sched_error`	output	1	Error flag

15.3.7 MonBus Interface

Table 3.1.8: MonBus Interface

Signal	Direction	Width	Description
`i_mon_time`	input	monbus_timestamp_t	Shared monitor timebase
`mon_valid`	output	1	Monitor packet valid
`mon_ready`	input	1	Consumer ready
`mon_packet`	output	monitor_packet_t	Monitor packet
`mon_timestamp`	output	monbus_timestamp_t	Packet timestamp

15.4 Internal Architecture

The descriptor engine hands each fetched descriptor to the scheduler over a valid/ready pair, and the four MonBus emitters (descriptor engine, scheduler, ctrlrd engine, ctrlwr engine) merge in the group's 4:1 `monbus_arbiter`; both are drawn in Figure 3.1.1.

Last Updated: 2026-07-02

16 Beats Scheduler Group Array Specification

Module: `scheduler_group_array_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

16.1 Overview

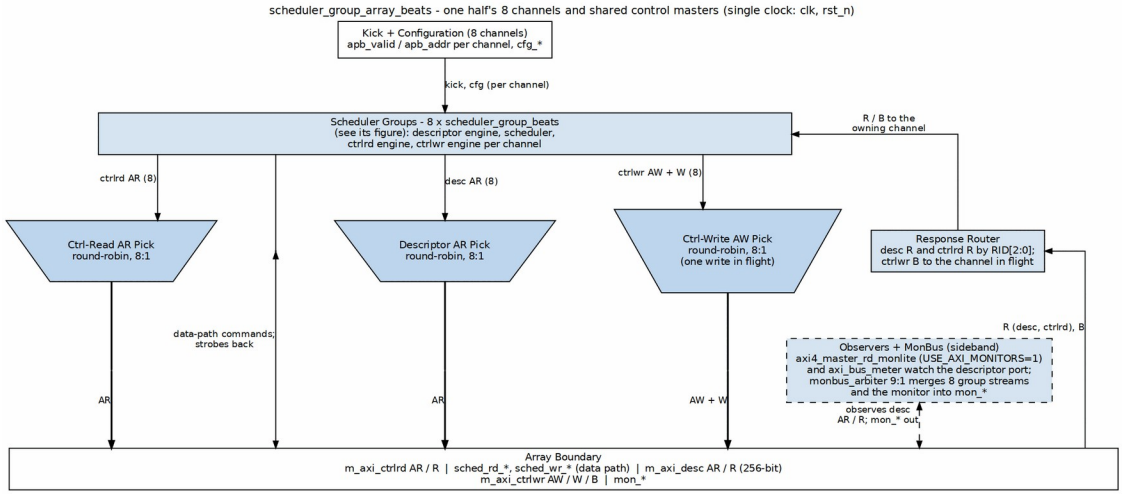
The Beats Scheduler Group Array instantiates 8 scheduler groups with a shared descriptor AXI master and unified MonBus output. It provides the complete scheduler infrastructure for all RAPIDS channels.

16.1.1 Key Features

- **8-Channel Array:** One scheduler_group per channel
- **Shared Descriptor AXI:** Round-robin arbitration for descriptor fetches
- **Unified MonBus:** Aggregates 9 MonBus sources (8 scheduler + 1 arbiter)
- **Per-Channel Configuration:** Individual enable/reset per channel
- **Aggregate Status:** Combined idle and error flags

16.1.2 Block Diagram

16.1.3 Figure 3.2.1: Beats Scheduler Group Array Block Diagram



Scheduler group array block diagram

Source: 11_scheduler_group_array.dot

16.2 Parameters

```
parameter int NUM_CHANNELS = 8;                 // Number of channels
parameter int ADDR_WIDTH = 64;                  // Address bus width
parameter int DATA_WIDTH = 512;                 // Data bus width
parameter int DESC_DATA_WIDTH = 256;            // Descriptor width

// AXI ID Management
parameter int AXI_ID_WIDTH = 8;                 // ID field width
parameter int MAX_OUTSTANDING = 8;              // Outstanding
descriptors

// Monitor Bus Parameters
parameter int MON_UNIT_ID = 1;                  // Unit ID for MonBus
```

: Table 3.2.1: Beats Scheduler Group Array Parameters

16.3 Port List

16.3.1 Clock and Reset

Table 3.2.2: Clock and Reset

Signal	Direction	Width	Description
`clk`	input	1	System clock
`rst_n`	input	1	Active-low reset

16.3.2 Per-Channel APB Programming

Table 3.2.3: APB Programming Interface

Signal	Direction	Width	Description
`apb_valid`	input	NC	Channel kick-off (per-channel)
`apb_ready`	output	NC	Ready for kick-off
`apb_addr`	input	NC*AW	First descriptor address

16.3.3 Per-Channel Configuration

Table 3.2.4: Per-Channel Configuration

Signal	Direction	Width	Description
`cfg_channel_enable`	input	NC	Enable per channel
`cfg_channel_reset`	input	NC	Soft reset per channel
`cfg_sched_timeout_cycles`	input	NC*16	Timeout threshold
`cfg_sched_timeout_enable`	input	NC	Enable timeout
`cfg_desceng_enable`	input	NC	Enable descriptor engine
`cfg_desceng_prefetch`	input	NC	Enable prefetching
`cfg_desceng_fifo_thresh`	input	NC*4	Prefetch threshold
`cfg_desceng_addr0_base`	input	NC*AW	Address range 0 base
`cfg_desceng_addr0_limit`	input	NC*AW	Address range 0 limit

16.3.4 Shared Descriptor AXI Master Interface

Table 3.2.5: Shared Descriptor AXI Master Interface

Signal	Direction	Width	Description
`desc_axi_arvalid`	output	1	AR valid
`desc_axi_arready`	input	1	AR ready
`desc_axi_araddr`	output	ADDR_WIDTH	Descriptor fetch address
`desc_axi_arlen`	output	8	Burst length - 1
`desc_axi_arsize`	output	3	Burst size

Signal	Direction	Width	Description
desc_axi_arburst	output	2	Burst type
desc_axi_arid	output	AXI_ID_WIDTH	Transaction ID
desc_axi_arlock	output	1	Lock type
desc_axi_arcache	output	4	Cache attributes
desc_axi_arprot	output	3	Protection attributes
desc_axi_arqos	output	4	Quality of service
desc_axi_arregion	output	4	Region identifier
desc_axi_rvalid	input	1	R valid
desc_axi_rready	output	1	R ready
desc_axi_rdata	input	256	Descriptor payload (fixed 256-bit)
desc_axi_rresp	input	2	Read response
desc_axi_rlast	input	1	Last beat
desc_axi_rid	input	AXI_ID_WIDTH	Response ID

16.3.5 Per-Channel Scheduler Data Interfaces

Table 3.2.6: Per-Channel Scheduler Data Interfaces

Signal	Direction	Width	Description
sched_rd_valid	output	NC	Read request per channel
sched_rd_addr	output	NC*AW	Read address
sched_rd_beats	output	NC*32	Read beats
sched_rd_done_strobe	input	NC	Read complete

Signal	Direction	Width	Description
sched_wr_valid	output	NC	Write request per channel
sched_wr_addr	output	NC*AW	Write address
sched_wr_beats	output	NC*32	Write beats
sched_wr_done_strobe	input	NC	Write complete

16.3.6 Aggregate Status

Table 3.2.7: Aggregate Status

Signal	Direction	Width	Description
descriptor_engine_idle	output	NC	Per-channel descriptor engine idle
scheduler_idle	output	NC	Per-channel scheduler idle
scheduler_state	output	NC x 7	Per-channel FSM state (one-hot)
sched_error	output	NC	Per-channel scheduler error (sticky)

16.3.7 Unified MonBus Interface

Table 3.2.8: Unified MonBus Interface

Signal	Direction	Width	Description
i_mon_time	input	monbus_timestamp_t	Shared monitor timebase
mon_valid	output	1	Monitor packet valid
mon_ready	input	1	Consumer ready
mon_packet	output	monitor_packet_t	Monitor packet
mon_timestamp	output	monbus_timestamp_t	Packet timestamp

16.3.8 Descriptor-AXI Perf Window

The descriptor monitor is `axi_monitor_lite`, which has no performance cone. The DAXMON_PERF_* CSRs are fed instead by an always-on `axi_bus_meter` on the descriptor R channel plus window/beat/byte/burst accumulators beside it, the same meter `rapids_beats_top` uses for RDMON/WRMON. `cfg_desc_mon_perf_run` (DAXMON_PERF_CTRL.RUN) opens the window: counters clear on its rising edge, count while it is high and hold while it is low. The meter is not gated by `USE_AXI_MONITORS`.

Table 3.2.9: Descriptor-AXI Perf Window

Signal	Direction	Width	Description
`cfg_desc_mon_perf_run`	input	1	Window control (DAXMON_PERF_CTRL.RUN)
`sts_desc_mon_win_active`	output	1	Window open (equals RUN)
`sts_desc_mon_win_cycles`	output	32	Live cycle count; reads 0 while the window is closed
`sts_desc_mon_prod_cycles`	output	32	R valid and ready
`sts_desc_mon_bp_cycles`	output	32	R valid, not ready (backpressure)
`sts_desc_mon_starv_cycles`	output	32	R ready, not valid (starvation)
`sts_desc_mon_idle_cycles`	output	32	R neither valid nor ready
`sts_desc_mon_beat_count`	output	32	R handshakes in the window
`sts_desc_mon_byte_count`	output	64	32 bytes per 256-bit descriptor beat
`sts_desc_mon_burst_count`	output	32	AR handshakes in the window

16.4 AXI Arbitration

16.4.1 Figure 3.2.2: Descriptor AXI Arbitration Sequence

Two channels contending for the descriptor master; responses return by RID.

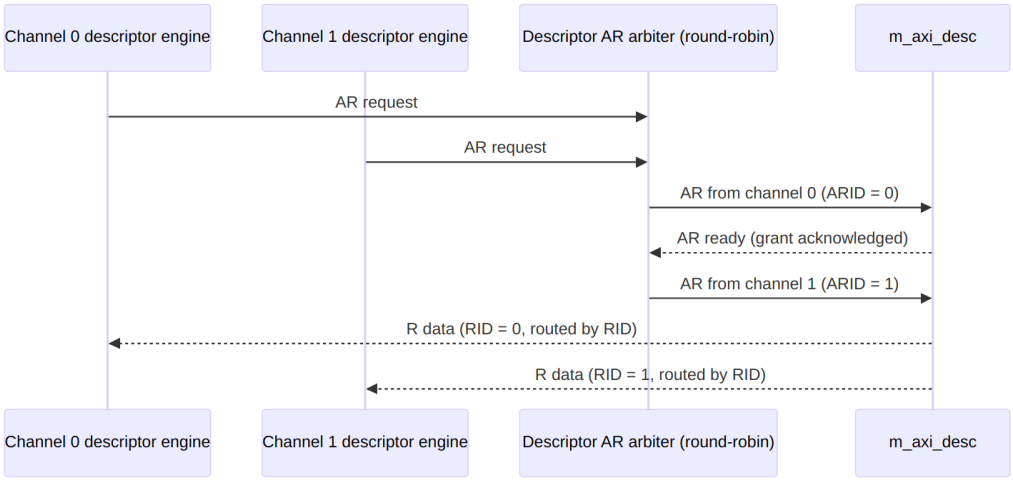


Diagram 5

16.5 MonBus Aggregation

The unified MonBus output aggregates 9 sources using round-robin arbitration:

Table 3.2.10: MonBus Source Assignment

Source ID	Origin	Description
0-7	scheduler_group[0:7]	Per-channel aggregated MonBus
8	Descriptor AXI monitor	`axi4_master_rd_monlite` packets (USE_AXI_MONITORS = 1)

Each scheduler_group provides a single MonBus output that combines its descriptor engine, scheduler, ctrlrd engine and ctrlwr engine packets (4:1 internal arbitration).

16.6 Integration Context

```
scheduler_group_array_beats #(
    .NUM_CHANNELS(8),
    .ADDR_WIDTH(64),
    .DATA_WIDTH(512)
) u_scheduler_array (
    .clk                    (clk),
    .rst_n                  (rst_n),

    // APB programming (per-channel)
    .apb_valid              (apb_kick_valid),
    .apb_ready              (apb_kick_ready),
    .apb_addr               (apb_kick_addr),

    // Configuration (per-channel)
    .cfg_channel_enable     (cfg_ch_enable),
    .cfg_sched_timeout_cycles(cfg_timeout_cycles),
    // ... additional config ...

    // Shared descriptor AXI
    .desc_axi_arvalid       (desc_axi_arvalid),
    .desc_axi_arready       (desc_axi_arready),
    .desc_axi_araddr        (desc_axi_araddr),
    .desc_axi_rvalid        (desc_axi_rvalid),
    .desc_axi_rready        (desc_axi_rready),
    .desc_axi_rdata         (desc_axi_rdata),

    // Per-channel scheduler outputs
    .sched_rd_valid         (sched_rd_valid),
    .sched_rd_addr          (sched_rd_addr),
    .sched_rd_beats         (sched_rd_beats),
    .sched_rd_done_strobe   (sched_rd_done_strobe),
    .sched_wr_valid         (sched_wr_valid),
    .sched_wr_addr          (sched_wr_addr),
    .sched_wr_beats         (sched_wr_beats),
    .sched_wr_done_strobe   (sched_wr_done_strobe),

    // Status
    .scheduler_idle         (all_schedulers_idle),

    // Unified MonBus
    .mon_valid              (sched_array_monbus_valid),
    .mon_ready              (sched_array_monbus_ready),
    .mon_packet             (sched_array_monbus_data)
);
```

Last Updated: 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

17 Sink Data Path Specification

Module: `snk_data_path_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

17.1 Overview

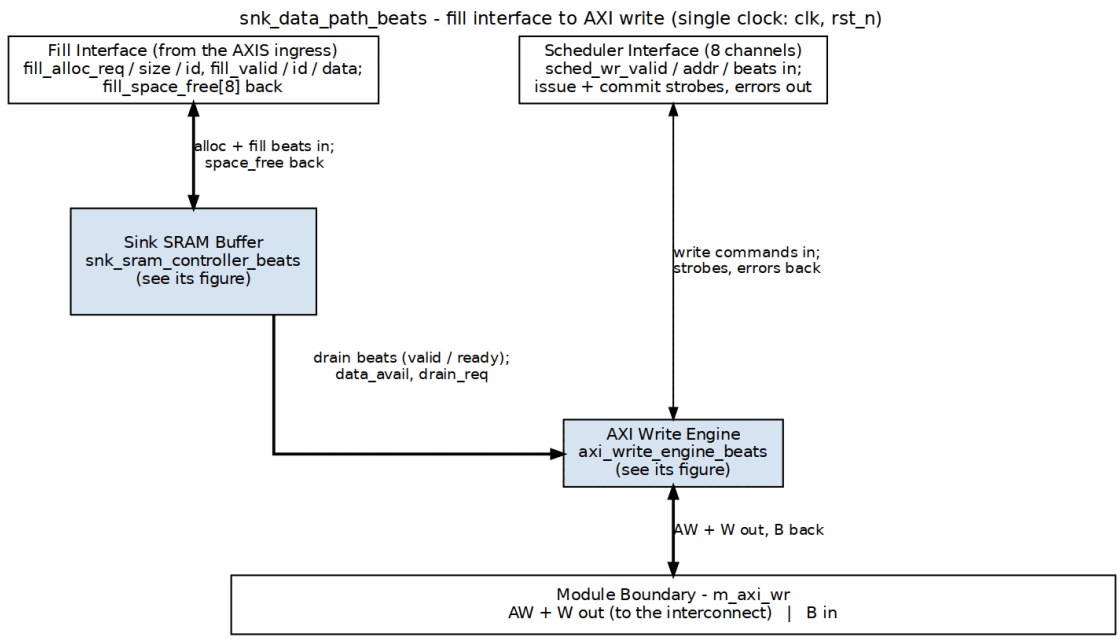
The Sink Data Path integrates the SRAM controller and AXI write engine for network-to-memory transfers. Data enters through the fill interface, is buffered in SRAM, and is written to system memory via AXI4.

17.1.1 Key Features

- **8-Channel SRAM Controller:** Per-channel buffering with flow control
- **AXI Write Engine:** Streaming burst writes to system memory
- **Beat-Level Tracking:** Precise flow control at beat granularity
- **Scheduler Integration:** Receives transfer requests from scheduler

17.1.2 Block Diagram

17.1.3 Figure 3.3.1: Sink Data Path Block Diagram



Sink data path block diagram

Source: 13_sink_data_path.dot

17.2 Parameters

```
parameter int NUM_CHANNELS = 8;
parameter int ADDR_WIDTH = 64;
parameter int DATA_WIDTH = 512;
parameter int AXI_ID_WIDTH = 8;
parameter int SRAM_DEPTH = 512;
parameter int AW_MAX_OUTSTANDING = 8;
parameter int W_PHASE_FIFO_DEPTH = 64;
parameter int B_PHASE_FIFO_DEPTH = 16;
```

: Table 3.3.1: Sink Data Path Parameters

17.3 Data Flow

17.3.1 Figure 3.3.2: Sink Path Data Flow Sequence

```
1. Fill Request
   - snk_fill_alloc_req = 1
   - snk_fill_alloc_size = N (beats to allocate)
   - snk_fill_alloc_id = channel
        |
        v
2. Space Check (stream_alloc_ctrl inside STREAM's sram_controller)
   - Check snk_fill_space_free[channel] >= N
   - Allocate N beats of space
        |
        v
3. Fill Data Transfer
   - snk_fill_valid = 1
   - snk_fill_data = data beats
   - snk_fill_id = channel
        |
        v
4. SRAM Write
   - Data written to channel's SRAM partition
   - stream_drain_ctrl notified of data availability
        |
        v
5. AXI Write Request
   - Scheduler requests write: sched_wr_valid
   - Address and beat count provided
```

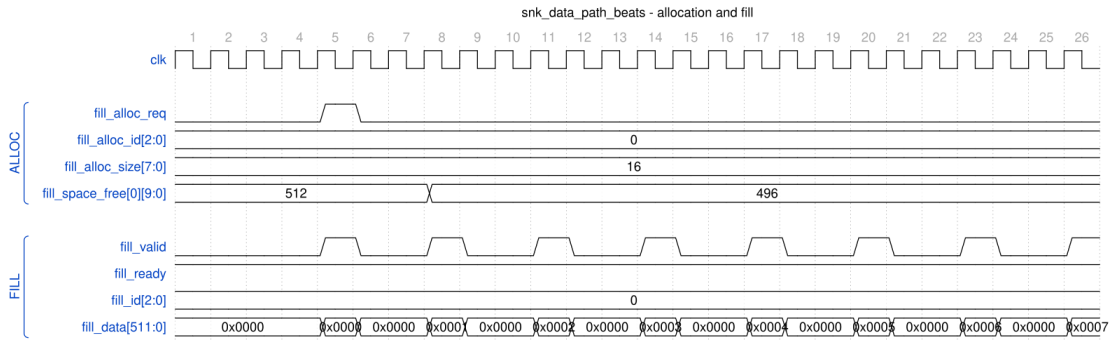
```
        |
        v
6. SRAM Read + AXI Write
   - axi_write_engine reads from SRAM
   - Issues AXI bursts to memory
        |
        v
7. Completion
   - B channel response received
   - sched_wr_done_strobe = 1
```

17.4 Timing Diagram

17.4.1 Figure 3.3.3: Sink Path Transfer Timing

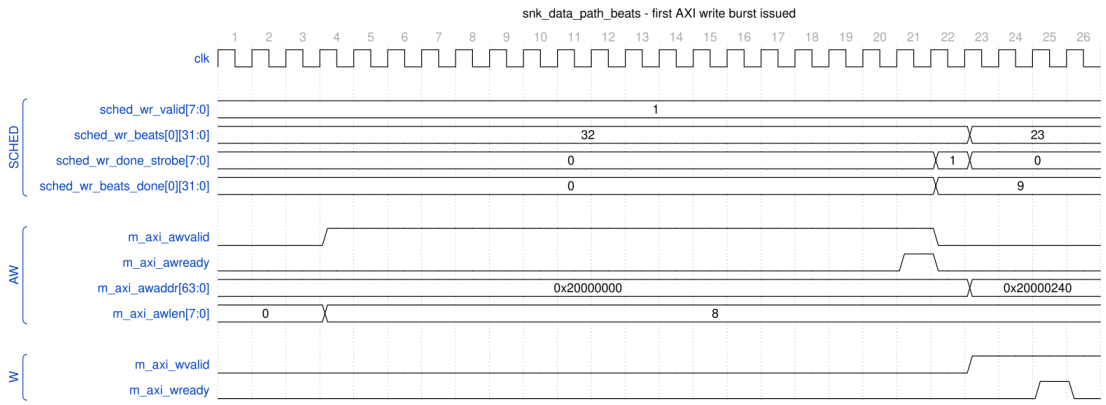
Three windows from one transfer: the allocation and fill, the first AXI write burst being issued, and the end of that burst with its response.

(a) Allocation and fill



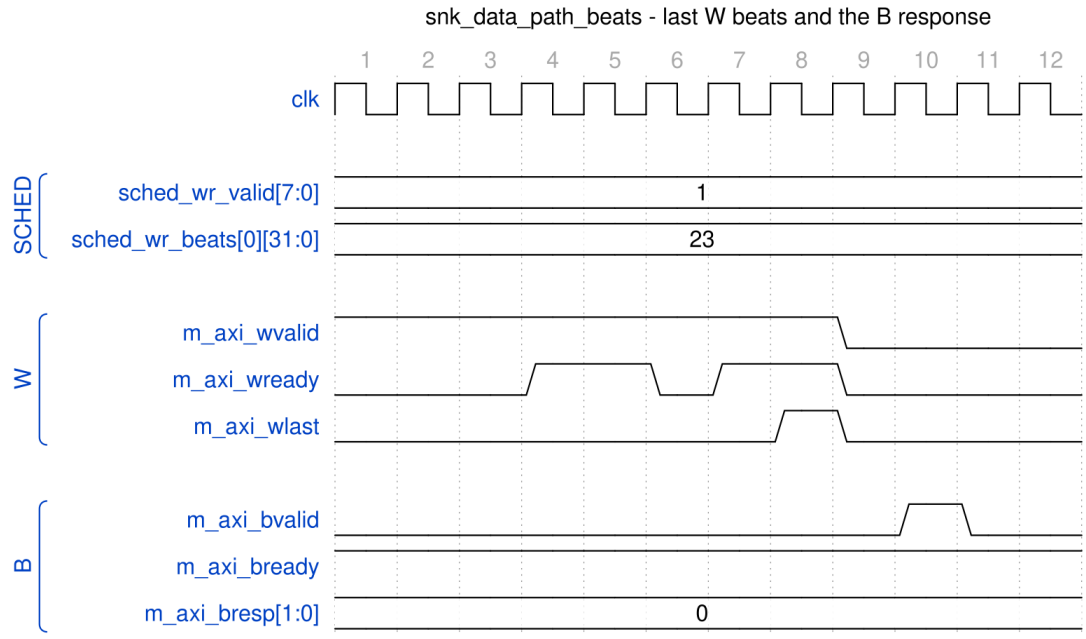
snk_data_path_beats - allocation and fill

(b) First AXI write burst issued



snk_data_path_beats - first AXI write burst issued

(c) Last W beats and the B response



snk_data_path_beats - last W beats and the B response

Source: snk_data_path_fill_alloc.json, snk_data_path_axi_write_aw.json, snk_data_path_axi_write_b.json, captured inside `rapids_core_beats.u_snk.u_sink_data_path` from `dv/tests/top_beats/test_rapids_core_beats.py` (sink path, channel 0, 32 beats, 512-bit data, `cfg_alloc_size = 16`, `TEST_LEVEL=full`) with `WAVES=1`.

Reading (a): the first beat for channel 0 arrives with no space allocated, so the path raises `fill_alloc_req` for 16 beats; `fill_space_free[0]` drops 512 to 496 three cycles later and the beat is accepted on the same cycle as the request. The remaining beats are accepted one per

handshake (`fill_ready` stays high; the gaps are the AXIS producer's pacing). Reading (b): once the descriptor arrives the scheduler holds `sched_wr_valid` with 32 beats remaining; the engine issues the AW (`awlen = 8`, 9 beats) and, the cycle after the AW handshake, pulses `sched_wr_done_strobe` with `beats_done = 9`, on which the scheduler advances to 23 remaining and the next AW address is already 0x20000240. Reading (c): W beats drain at the memory's pace (`wready` from the AXI slave BFM), `wlast` marks the ninth beat, and the B response arrives two cycles later with `bresp = OKAY`. The done strobe is reported at issue, not at B; completion of the whole transfer is what `snk_system_idle` reflects.

17.5 Interface Summary

17.5.1 Fill Interface (Input)

Table 3.3.2: Fill Interface

Signal	Direction	Description
`fill_alloc_req`	input	Allocation request (single, ID-selected)
`fill_alloc_size`	input	Beats to allocate
`fill_alloc_id`	input	Transaction ID selects the channel
`fill_space_free`	output	Available space per channel
`fill_valid`	input	Fill data valid
`fill_ready`	output	Ready for fill data
`fill_id`	input	Transaction ID selects the channel
`fill_data`	input	Fill data

17.5.2 Scheduler Interface

Table 3.3.3: Scheduler Interface

Signal	Direction	Description
`sched_wr_valid`	input	Channel requests write
`sched_wr_ready`	output	Path ready for channel

Signal	Direction	Description
sched_wr_addr	input	Destination addresses
sched_wr_beats	input	Beats remaining to write
sched_wr_burst_len	input	Requested burst length
sched_wr_done_strobe	output	Burst ISSUED on AW handshake
sched_wr_beats_done	output	Beats issued in burst
sched_wr_commit_strobe	output	Burst COMMITTED on B response
sched_wr_commit_beats	output	Beats committed in burst
sched_wr_error	output	Sticky error flag per channel

Last Updated: 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

18 Sink Data Path AXIS Wrapper Specification

Module: `snk_data_path_axis_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2026-09-29

18.1 Overview

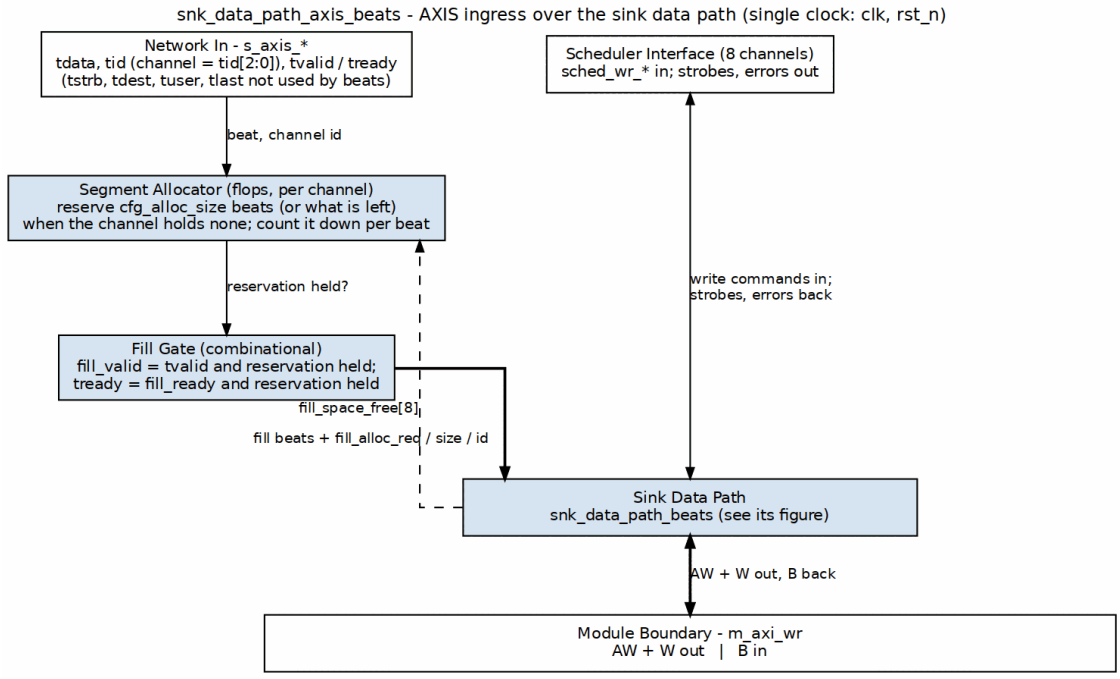
The Sink Data Path AXIS wrapper adds an AXI-Stream slave interface to the sink data path, enabling standard AXIS ingress for network-to-memory transfers.

18.1.1 Key Features

- **AXI-Stream Slave Interface:** Standard AXIS for data ingress
- **Per-Channel TID Mapping:** AXIS TID maps to fill channel ID
- **TLAST to Last Mapping:** Packet boundary tracking
- **Core Sink Integration:** Wraps sink_data_path module

18.1.2 Block Diagram

18.1.3 Figure 3.4.1: Sink Data Path AXIS Block Diagram



Sink data path AXIS block diagram

Source: 14_sink_data_path_axis.dot

18.2 Parameters

```
parameter int NUM_CHANNELS = 8;
parameter int ADDR_WIDTH = 64;
parameter int DATA_WIDTH = 512;
parameter int AXI_ID_WIDTH = 8;
parameter int SRAM_DEPTH = 512;
parameter int TID_WIDTH = 3;                    // log2(NUM_CHANNELS)
parameter int TDEST_WIDTH = 1;
parameter int TUSER_WIDTH = 1;
```

: Table 3.4.1: Sink Data Path AXIS Parameters

18.3 Port List

18.3.1 Clock and Reset

Table 3.4.2: Clock and Reset

Signal	Direction	Width	Description
`clk`	input	1	System clock
`rst_n`	input	1	Active-low reset

18.3.2 AXI-Stream Slave Interface

Table 3.4.3: AXI-Stream Slave Interface

Signal	Direction	Width	Description
`s_axis_tvalid`	input	1	Data valid
`s_axis_tready`	output	1	Ready for data
`s_axis_tdata`	input	DW	Data payload
`s_axis_tstrb`	input	SW	Byte strobes
`s_axis_tlast`	input	1	Last beat of packet
`s_axis_tid`	input	AXIS_ID_WIDTH	Stream ID (channel)

Signal	Direction	Width	Description
s_axis_tdest	input	AXIS_DEST_WIDTH	Destination
s_axis_tuser	input	AXIS_USER_WIDTH	User sideband

18.3.3 Allocation Configuration

Table 3.4.4: Allocation Configuration

Signal	Direction	Width	Description
cfg_alloc_size	input	8	Default allocation size per request (beats)

18.3.4 Scheduler Interface

Table 3.4.5: Scheduler Interface

Signal	Direction	Width	Description
sched_wr_valid	input	NC	Channel requests write
sched_wr_ready	output	NC	Path ready for channel
sched_wr_addr	input	NC x AW	Destination addresses
sched_wr_beats	input	NC x 32	Beats remaining to write
sched_wr_burst_len	input	NC x 8	Requested burst length
sched_wr_done_strobe	output	NC	Burst ISSUED on AW handshake
sched_wr_beats_done	output	NC x 32	Beats issued in burst
sched_wr_commit_strobe	output	NC	Burst COMMITTED on B response
sched_wr_commit_beats	output	NC x 32	Beats committed in burst
sched_wr_error	output	NC	Sticky error flag per channel

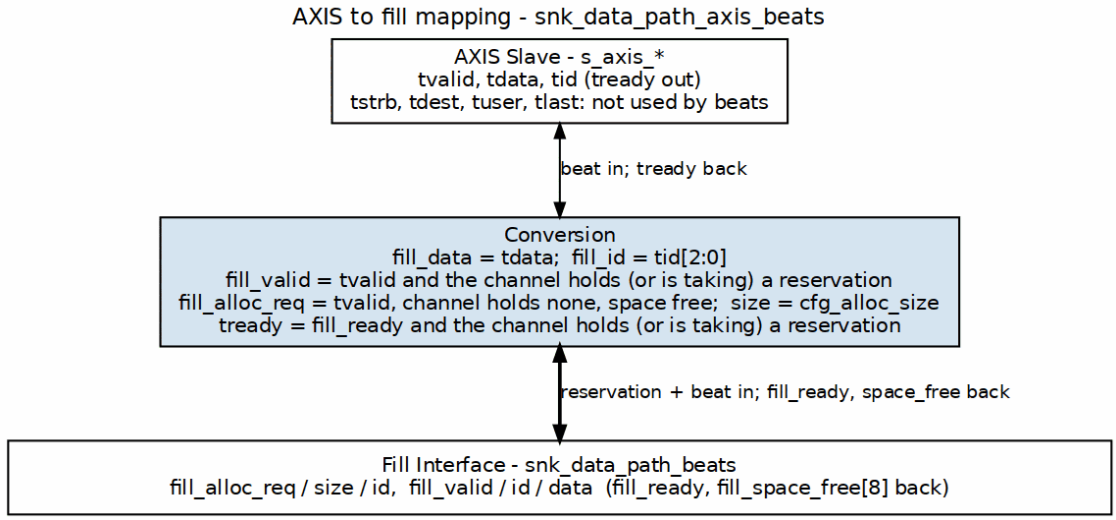
18.3.5 AXI Write Master Interface

Table 3.4.6: AXI Write Master Interface

Signal	Direction	Width	Description
`m_axi_awvalid`	output	1	AW channel valid
`m_axi_awready`	input	1	AW channel ready
`m_axi_awaddr`	output	AW	Write address
`m_axi_awlen`	output	8	Burst length
`m_axi_awsize`	output	3	Burst size
`m_axi_awburst`	output	2	Burst type
`m_axi_awid`	output	ID_W	Transaction ID
`m_axi_wvalid`	output	1	W channel valid
`m_axi_wready`	input	1	W channel ready
`m_axi_wdata`	output	DW	Write data
`m_axi_wstrb`	output	DW/8	Write strobes
`m_axi_wlast`	output	1	Last beat
`m_axi_bvalid`	input	1	B channel valid
`m_axi_bready`	output	1	B channel ready
`m_axi_bresp`	input	2	Write response
`m_axi_bid`	input	ID_W	Response ID

18.4 Signal Mapping

18.4.1 Figure 3.4.2: AXIS to Fill Interface Mapping

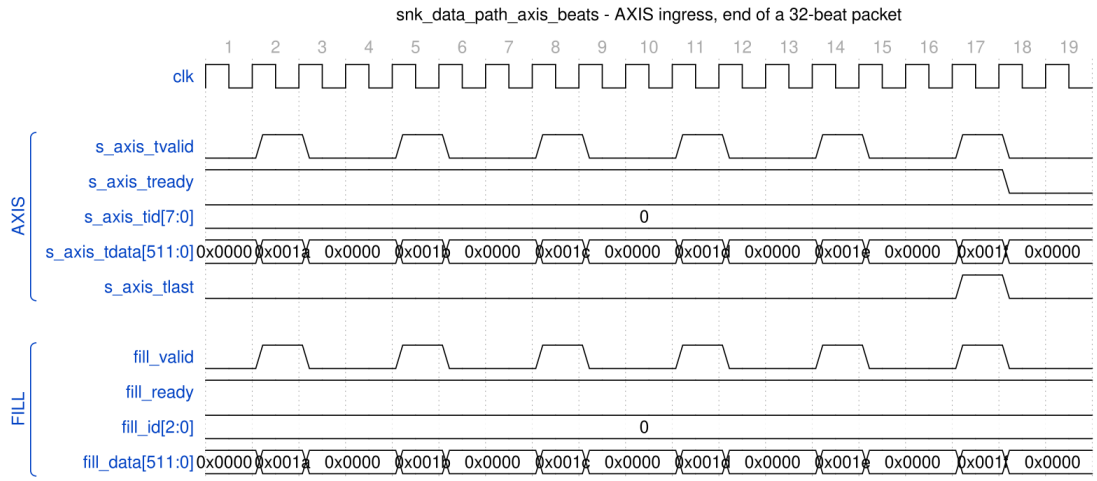


AXIS to fill mapping

Source: 14b_axis_to_fill.dot

18.5 Timing Diagram

18.5.1 Figure 3.4.3: AXIS Ingress Timing



snk_data_path_axis_beats - AXIS ingress, end of a 32-beat packet

Source: snk_data_path_axis_ingress.json, captured from `dv/tests/top_beats/test_rapids_core_beats.py` (sink path, channel 0, 32 beats, 512-bit data, `TEST_LEVEL=full`) with `WAVES=1`.

Reading it: each AXIS beat is passed straight through to the fill interface in the same cycle – `fill_valid` mirrors `s_axis_tvalid`, `fill_id` is `s_axis_tid`, and `fill_data` is `s_axis_tdata` (the low 16 bits are shown; the payload counts up 0x1a..0x1f). `s_axis_tready` is high while channel 0 has allocated space and drops the cycle after `s_axis_tlast` because the packet count is now complete and the next beat would need a fresh allocation (Figure 3.3.3a).

18.6 Usage Notes

1. **TID Usage:** AXIS TID directly maps to channel ID for multi-channel routing
2. **Allocation Required:** Space must be allocated via `snk_alloc_*` before AXIS data arrives
3. **Backpressure:** `s_axis_tready` deasserts when SRAM space exhausted
4. **Packet Boundaries:** TLAST marks end of AXIS packets, mapped to fill_last
5. **Partial segments (rapids BUG-009, 2026-09-29):** allocation is in `cfg_alloc_size` segments, and a packet that ends mid-segment leaves the rest of its segment allocated for the channel's next packet (`r_pending_alloc`). That shifts the allocation phase: with whole-segment-only allocation the last `segment - residue` slots of the buffer could never be allocated again until something drained, and a write burst equal to the buffer depth – which needs every slot – waited forever (Genesys 2: a 4-beat run, then a 128-beat-burst run accepted 124 beats and issued no AW). The ingress now allocates the remainder when less than a segment is free (`fill_alloc_size = min(cfg_alloc_size, space_free)`), so the buffer can always be filled to the top. Regression: `test_residue_full_burst`.
6. **Allocation settle and no dropped beats (same bug, third finding):** `fill_space_free` reflects an allocation three cycles after its handshake. A one- or two-beat partial allocation is consumed before that, and a second allocation against the stale view over-allocated the buffer (the 8-bit space count wrapped to 254 and the ingress ran into a full FIFO). Each channel now holds its next allocation for three cycles after the last (`r_alloc_settle`), and `s_axis_tready` includes `fill_ready` on allocation cycles as well, so a beat is never accepted that the FIFO cannot store.

Last Updated: 2026-09-29

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

19 Sink SRAM Controller Specification

Module: snk_sram_controller_beats.sv **Location:** projects/components/dma-ip/rapids/rtl/macro_beats/ **Status:** Implemented **Last Updated:** 2025-01-10

19.1 Overview

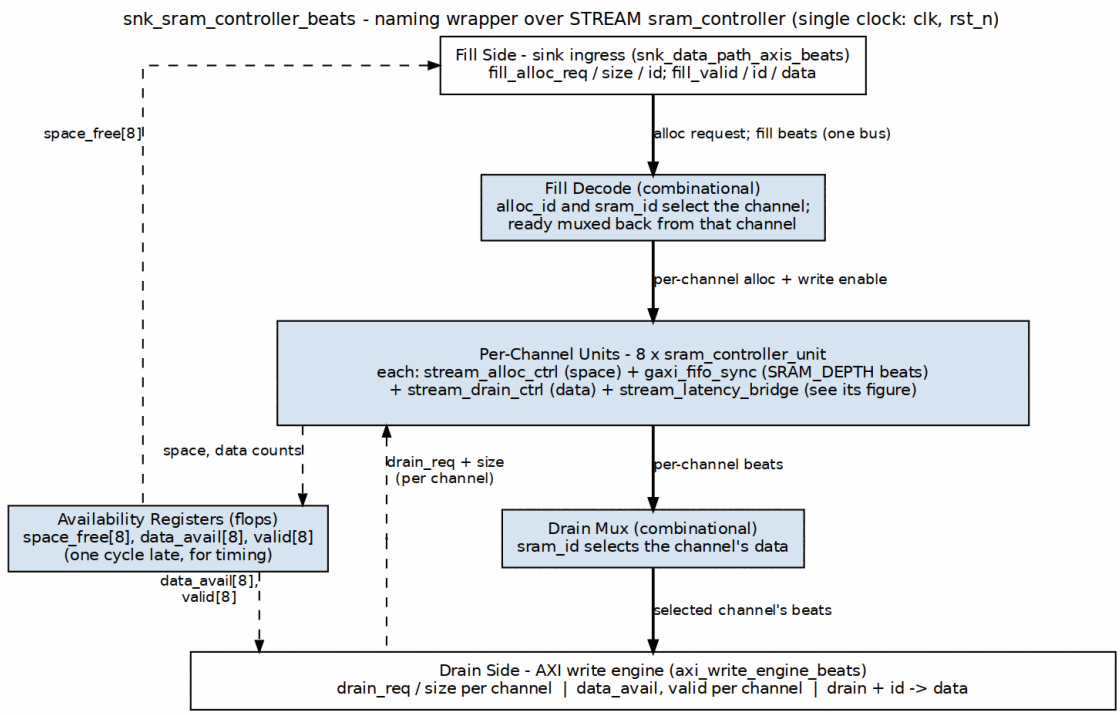
The Sink SRAM Controller is a naming wrapper, with no logic of its own, around STREAM's sram_controller – the one per-channel SRAM implementation both DMAs share since bdf4e0dff. It maps the RAPIDS fill_*/drain_* names onto STREAM's ports; the eight per-channel units (allocation counter, FIFO, drain counter, latency bridge) live inside STREAM's block. It manages 8 channel units, providing per-channel buffering with channel arbitration for the shared AXI write engine.

19.1.1 Key Features

- **One shared implementation:** instantiates STREAM's sram_controller once; it holds 8 sram_controller_units, each a gaxi_fifo_sync FIFO with stream_alloc_ctrl, stream_drain_ctrl and stream_latency_bridge
- **Channel Arbitration:** none inside the controller – the AXI write engine picks a channel and presents it on drain_id
- **Flow Control Integration:** per-channel stream_alloc_ctrl / stream_drain_ctrl virtual FIFOs; the drain counter's virtual depth is 2 x SRAM_DEPTH (stream BUG-011)
- **Space Tracking:** Reports available space per channel

19.1.2 Block Diagram

19.1.3 Figure 3.5.1: Sink SRAM Controller Block Diagram



Sink SRAM controller block diagram

Source: 15_snk_sram_controller.dot

19.2 Parameters

```
parameter int NUM_CHANNELS = 8;
parameter int DATA_WIDTH = 512;
parameter int SRAM_DEPTH = 512;                    // Per-channel depth
parameter int ADDR_WIDTH = $clog2(SRAM_DEPTH);
```

: Table 3.5.1: Sink SRAM Controller Parameters

19.3 Port List

19.3.1 Clock and Reset

Table 3.5.2: Clock and Reset

Signal	Direction	Width	Description
`clk`	input	1	System clock
`rst_n`	input	1	Active-low reset

19.3.2 Fill Interface (ID-Selected)

Table 3.5.3: Fill Interface (ID-Selected)

Signal	Direction	Width	Description
`fill_alloc_req`	input	1	Allocation request (single, ID-selected)
`fill_alloc_size`	input	8	Beats to allocate
`fill_alloc_id`	input	CIW	Transaction ID selects the channel
`fill_space_free`	output	NC x SCW	Available space per channel
`fill_valid`	input	1	Fill data valid
`fill_ready`	output	1	Ready for fill data
`fill_id`	input	CIW	Transaction ID selects the channel
`fill_data`	input	DW	Fill data

19.3.3 Arbitrated Drain Interface (to AXI Write Engine)

Table 3.5.4: Arbitrated Drain Interface

Signal	Direction	Width	Description
`drain_data_avail`	output	NC x SCW	Data available per channel
`drain_req`	input	NC	Drain reservation request per channel
`drain_size`	input	NC x 8	Beats to reserve

Signal	Direction	Width	Description
`drain_valid`	output	NC	Drain valid, registered (for arbitration)
`drain_valid_comb`	output	NC	Drain valid, combinational (gates `m_axi_wvalid`)
`drain_read`	input	1	Consumer read strobe
`drain_id`	input	CIW	Channel ID select for drain
`drain_data`	output	DW	Drain data (muxed from selected channel)

19.3.4 Debug Interface

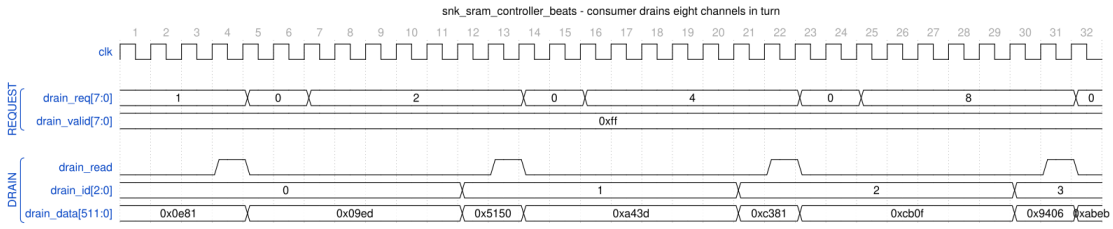
Table 3.5.5: Debug Interface

Signal	Direction	Width	Description
`dbg_bridge_pending`	output	NC	Bridge has a pending beat per channel
`dbg_bridge_out_valid`	output	NC	Bridge output valid per channel

19.4 Drain Selection

There is no arbiter inside the controller. Every channel unit raises its own `drain_valid[ch]` / `drain_size[ch]` when it has data; the AXI write engine picks a channel, drives it on `drain_id`, and the controller steers that channel's beats to the single `drain_*` port. Switching channels costs nothing beyond the consumer's own decision, and a channel keeps presenting `drain_valid` until it is empty.

19.4.1 Figure 3.5.2: Drain Selection by the Consumer



snk_sram_controller_beats - consumer drains eight channels in turn

Source: snk_sram_controller_drain_select.json, captured from `dv/tests/macro_beats/test_snk_sram_controller_beats.py` (`multi_channel`, 8 channels, 512-bit data, 512-entry SRAM, `REG_LEVEL=GATE`) with `WAVES=1`.

Reading it: all eight channels have data (`drain_valid` = 0xff) and the consumer walks them in turn. It raises one bit of `drain_req` at a time; the controller answers with `drain_id` naming that channel and `drain_data` presenting its head word on the same cycle, and the consumer's `drain_read` pulse pops it. Each channel's turn lasts nine cycles here because the test releases the request and re-arms the next one; the controller itself needs no gap between channels.

19.5 Integration with AXI Write Engine

```
snk_sram_controller_beats #(
    .NUM_CHANNELS(8),
    .DATA_WIDTH(512),
    .SRAM_DEPTH(512)
) u_snk_sram_ctrl (
    .clk                    (clk),
    .rst_n                  (rst_n),

    // Per-channel fill interface
    .fill_alloc_req         (fill_alloc_req),
    .fill_alloc_size        (fill_alloc_size),
    .fill_space_free        (fill_space_free),
    .fill_valid             (fill_valid),
    .fill_ready             (fill_ready),
    .fill_data              (fill_data),

    // Arbitrated drain interface
    .drain_req              (sram_drain_req),
    .drain_id               (sram_drain_id),
    .drain_size             (sram_drain_beats),

    // Status
    .drain_data_avail       (channel_data_avail),
    .dbg_bridge_pending     (channel_empty),
    .dbg_bridge_out_valid   (channel_full)
);
```

Last Updated: 2025-01-10

20 Source Data Path Specification

Module: `src_data_path_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

20.1 Overview

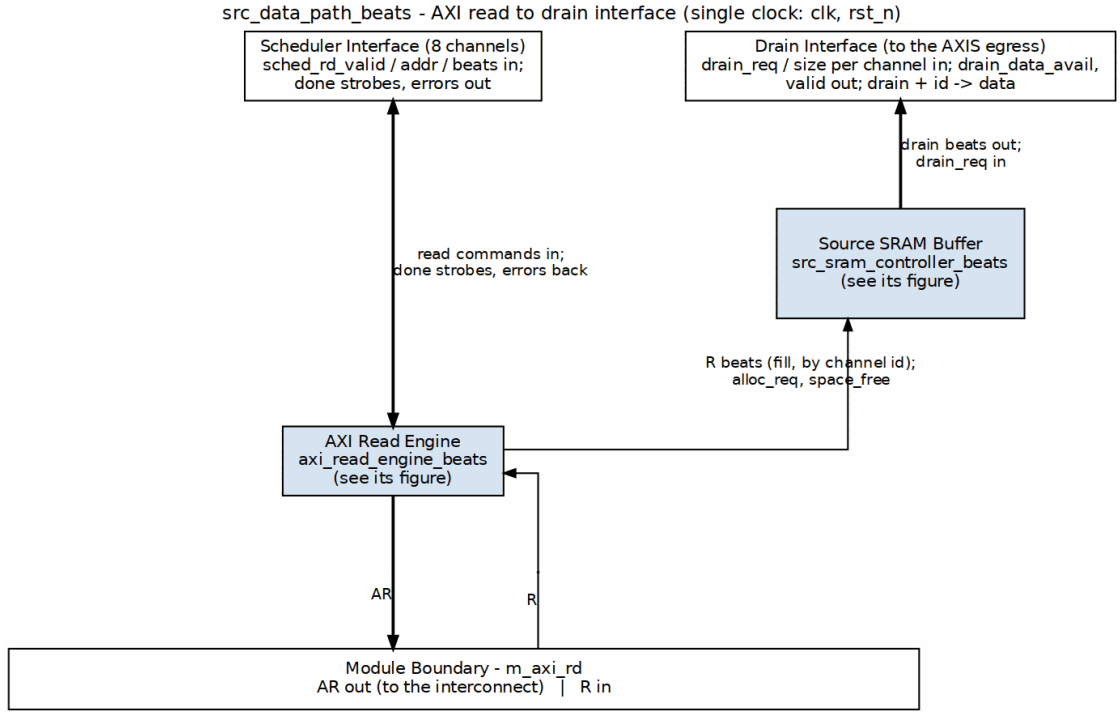
The Source Data Path integrates the AXI read engine and SRAM controller for memory-to-network transfers. Data is read from system memory via AXI4, buffered in SRAM, and delivered through the drain interface to the network.

20.1.1 Key Features

- **AXI Read Engine:** Streaming burst reads from system memory
- **8-Channel SRAM Controller:** Per-channel buffering with flow control
- **Beat-Level Tracking:** Precise flow control at beat granularity
- **Scheduler Integration:** Receives transfer requests from scheduler

20.1.2 Block Diagram

20.1.3 Figure 3.6.1: Source Data Path Block Diagram



Source data path block diagram

Source: 16_source_data_path.dot

20.2 Parameters

```
parameter int NUM_CHANNELS = 8;
parameter int ADDR_WIDTH = 64;
parameter int DATA_WIDTH = 512;
parameter int AXI_ID_WIDTH = 8;
parameter int SRAM_DEPTH = 512;
parameter int AR_MAX_OUTSTANDING = 8;
parameter int R_PHASE_FIFO_DEPTH = 64;
```

: Table 3.6.1: Source Data Path Parameters

20.3 Data Flow

20.3.1 Figure 3.6.2: Source Path Data Flow Sequence

```
1. Scheduler Read Request
   - sched_rd_valid = 1
   - sched_rd_addr = source address
   - sched_rd_beats = N
   - sched_rd_id = channel
        |
        v
2. Space Check (stream_alloc_ctrl inside STREAM's sram_controller)
   - Check src_drain_space_free[channel] >= N
   - Allocate N beats of space
        |
        v
3. AXI Read Transaction
   - axi_read_engine issues AR
   - m_axi_araddr = sched_rd_addr
   - m_axi_arlen = calculated from beats
        |
        v
4. R Channel Data
   - Data arrives on m_axi_rdata
   - Written to channel's SRAM partition
        |
        v
5. SRAM Write Complete
   - stream_drain_ctrl notified via the unit's latency bridge
   - Data available for drain
        |
        v
6. Drain Interface
   - src_drain_valid = 1
   - Data delivered to network master
        |
        v
7. Completion
   - sched_rd_done_strobe = 1
   - Scheduler notified of completion
```

20.4 Port List

20.4.1 Clock and Reset

Table 3.6.2: Clock and Reset

Signal	Direction	Width	Description
`clk`	input	1	System clock
`rst_n`	input	1	Active-low reset

20.4.2 Configuration

Table 3.6.3: Configuration

Signal	Direction	Width	Description
`cfg_axi_rd_xfer_beats`	input	8	AXI read transfer size in beats (all channels)

20.4.3 Scheduler Interface

Table 3.6.4: Scheduler Interface

Signal	Direction	Width	Description
`sched_rd_valid`	input	NC	Channel requests read
`sched_rd_addr`	input	NC x AW	Source addresses
`sched_rd_beats`	input	NC x 32	Beats remaining to read
`sched_rd_done_strobe`	output	NC	Burst completed (pulsed 1 cycle)
`sched_rd_beats_done`	output	NC x 32	Beats completed in burst
`sched_rd_error`	output	NC	Sticky error flag per channel

20.4.4 AXI Read Master Interface

Table 3.6.5: AXI Read Master Interface

Signal	Direction	Width	Description
`m_axi_arvalid`	output	1	AR channel valid

Signal	Direction	Width	Description
m_axi_arready	input	1	AR channel ready
m_axi_araddr	output	AW	Read address
m_axi_arlen	output	8	Burst length
m_axi_arsize	output	3	Burst size
m_axi_arburst	output	2	Burst type
m_axi_arid	output	ID_W	Transaction ID
m_axi_rvalid	input	1	R channel valid
m_axi_rready	output	1	R channel ready
m_axi_rdata	input	DW	Read data
m_axi_rresp	input	2	Read response
m_axi_rid	input	ID_W	Response ID
m_axi_rlast	input	1	Last beat

20.4.5 Drain Interface (Output to Network)

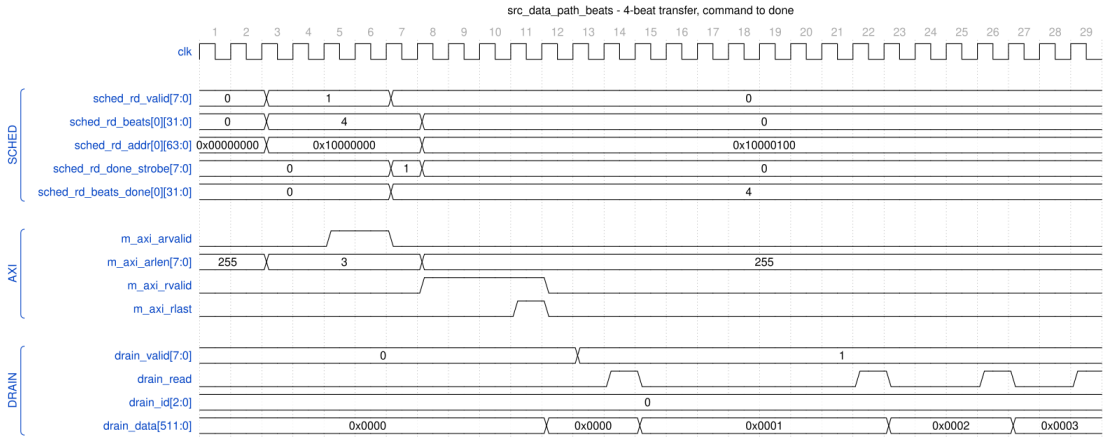
Table 3.6.6: Drain Interface

Signal	Direction	Width	Description
drain_data_avail	output	NC x SCW	Data available per channel
drain_req	input	NC	Drain reservation request per channel
drain_size	input	NC x 8	Beats to reserve
drain_valid	output	NC	Drain data valid per channel
drain_valid_comb	output	NC	Combinational per-channel valid (beat gate)
drain_read	input	1	Consumer read strobe
drain_id	input	CIW	Channel ID select for drain
drain_data	output	DW	Drain data (muxed from

Signal	Direction	Width	Description
			selected channel)

20.5 Timing Diagram

20.5.1 Figure 3.6.3: Source Path Transfer Timing



src_data_path_beats - 4-beat transfer, command to done

Source: src_data_path_transfer.json, captured inside `rapids_core_beats.u_src.u_source_data_path`, captured from `dv/tests/top_beats/test_rapids_core_beats.py` (source path, channel 0, 4 beats, 512-bit data, `REG_LEVEL=GATE`) with `WAVES=1`.

Reading it: the scheduler holds `sched_rd_valid[0]` with 4 beats at 0x10000000. The read engine issues the AR (`arlen = 3`) two cycles later; the four R beats land from cycle 8, `rlast` on the fourth, and the cycle after it the engine pulses `sched_rd_done_strobe[0]` with `beats_done = 4` – the scheduler advances its address to 0x10000100 on that strobe. The data reaches the source SRAM two cycles after the last R beat (`drain_valid[0]` rises) and the AXIS side drains it one word per `drain_read` pulse at the pace the egress consumer allows.

20.6 Integration Example

```
src_data_path_beats #(
    .NUM_CHANNELS(8),
    .ADDR_WIDTH(64),
    .DATA_WIDTH(512),
    .SRAM_DEPTH(512)
) u_source_path (
    .clk                    (clk),
```

```
    .rst_n                  (rst_n),

    // Scheduler interface
    .sched_rd_valid         (sched_rd_valid),
    .sched_rd_addr          (sched_rd_addr),
    .sched_rd_beats         (sched_rd_beats),
    .sched_rd_done_strobe   (sched_rd_done_strobe),
    .sched_rd_beats_done    (sched_rd_beats_done),
    .sched_rd_error         (sched_rd_error),

    // AXI read master
    .m_axi_arvalid          (src_axi_arvalid),
    .m_axi_arready          (src_axi_arready),
    .m_axi_araddr           (src_axi_araddr),
    .m_axi_arlen            (src_axi_arlen),
    .m_axi_rvalid           (src_axi_rvalid),
    .m_axi_rready           (src_axi_rready),
    .m_axi_rdata            (src_axi_rdata),
    .m_axi_rlast            (src_axi_rlast),

    // Drain interface
    .drain_valid            (src_drain_valid),
    .drain_data             (src_drain_data),
    .drain_id               (src_drain_id),
    .drain_data_avail       (src_data_avail)
);
```

Last Updated: 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

21 Source Data Path AXIS Wrapper Specification

Module: `src_data_path_axis_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

21.1 Overview

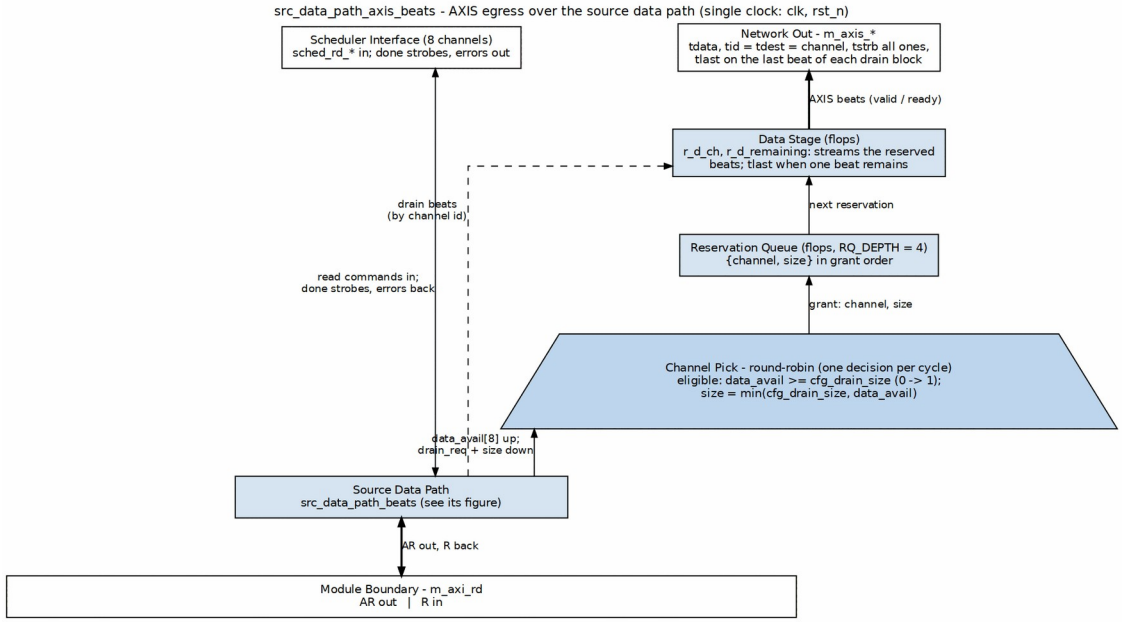
The Source Data Path AXIS wrapper adds an AXI-Stream master interface to the source data path, enabling standard AXIS egress for memory-to-network transfers.

21.1.1 Key Features

- **AXI-Stream Master Interface:** Standard AXIS for data egress
- **Per-Channel TID Mapping:** Drain channel ID maps to AXIS TID
- **TLAST Generation:** Packet boundary marking
- **Core Source Integration:** Wraps source_data_path module

21.1.2 Block Diagram

21.1.3 Figure 3.7.1: Source Data Path AXIS Block Diagram



Source data path AXIS block diagram

Source: 17_source_data_path_axis.dot

21.2 Parameters

```
parameter int NUM_CHANNELS = 8;
parameter int ADDR_WIDTH = 64;
parameter int DATA_WIDTH = 512;
parameter int AXI_ID_WIDTH = 8;
parameter int SRAM_DEPTH = 512;
parameter int TID_WIDTH = 3;                    // log2(NUM_CHANNELS)
parameter int TDEST_WIDTH = 1;
parameter int TUSER_WIDTH = 1;
```

: Table 3.7.1: Source Data Path AXIS Parameters

21.3 Port List

21.3.1 Clock and Reset

Table 3.7.2: Clock and Reset

Signal	Direction	Width	Description
`clk`	input	1	System clock
`rst_n`	input	1	Active-low reset

21.3.2 Configuration

Table 3.7.3: Configuration

Signal	Direction	Width	Description
`cfg_axi_rd_xfer_beats`	input	8	AXI read transfer size in beats (all channels)
`cfg_drain_size`	input	8	Beats to drain per request

21.3.3 Scheduler Interface

Table 3.7.4: Scheduler Interface

Signal	Direction	Width	Description
`sched_rd_valid`	input	NC	Channel requests read

Signal	Direction	Width	Description
sched_rd_addr	input	NC x AW	Source addresses
sched_rd_beats	input	NC x 32	Beats remaining to read
sched_rd_done_strobe	output	NC	Burst completed (pulsed 1 cycle)
sched_rd_beats_done	output	NC x 32	Beats completed in burst
sched_rd_error	output	NC	Sticky error flag per channel

21.3.4 AXI Read Master Interface

Table 3.7.5: AXI Read Master Interface

Signal	Direction	Width	Description
m_axi_arvalid	output	1	AR channel valid
m_axi_arready	input	1	AR channel ready
m_axi_araddr	output	AW	Read address
m_axi_arlen	output	8	Burst length
m_axi_arsize	output	3	Burst size
m_axi_arburst	output	2	Burst type
m_axi_arid	output	ID_W	Transaction ID
m_axi_rvalid	input	1	R channel valid
m_axi_rready	output	1	R channel ready
m_axi_rdata	input	DW	Read data
m_axi_rresp	input	2	Read response
m_axi_rid	input	ID_W	Response ID
m_axi_rlast	input	1	Last beat

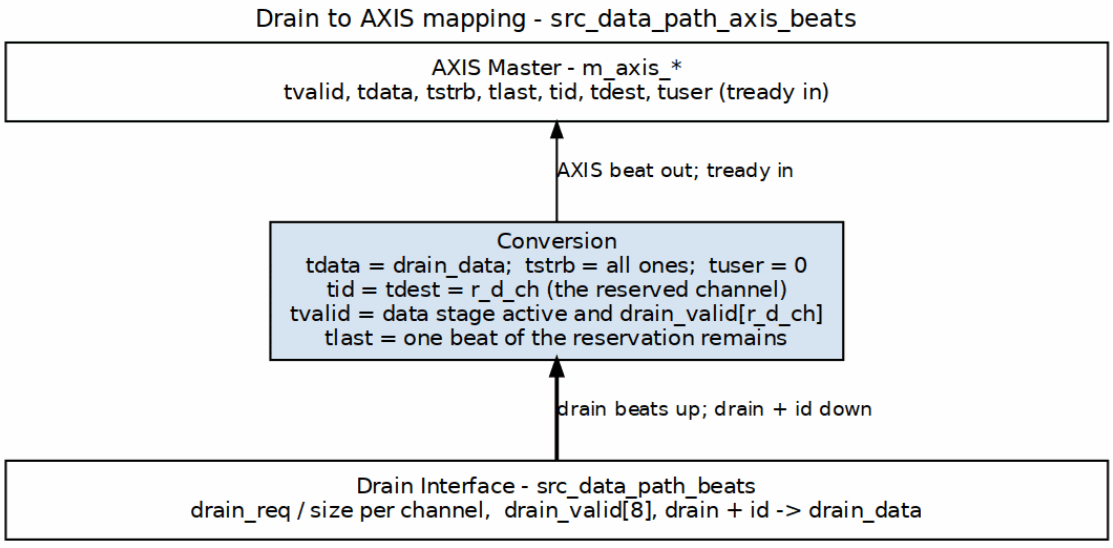
21.3.5 AXI-Stream Master Interface

Table 3.7.6: AXI-Stream Master Interface

Signal	Direction	Width	Description
`m_axis_tvalid`	output	1	Data valid
`m_axis_tready`	input	1	Consumer ready
`m_axis_tdata`	output	DW	Data payload
`m_axis_tstrb`	output	SW	Byte strobes
`m_axis_tlast`	output	1	Last beat of packet
`m_axis_tid`	output	AXIS_ID_WIDTH	Stream ID (channel)
`m_axis_tdest`	output	AXIS_DEST_WIDTH	Destination
`m_axis_tuser`	output	AXIS_USER_WIDTH	User sideband

21.4 Signal Mapping

21.4.1 Figure 3.7.2: Drain to AXIS Interface Mapping

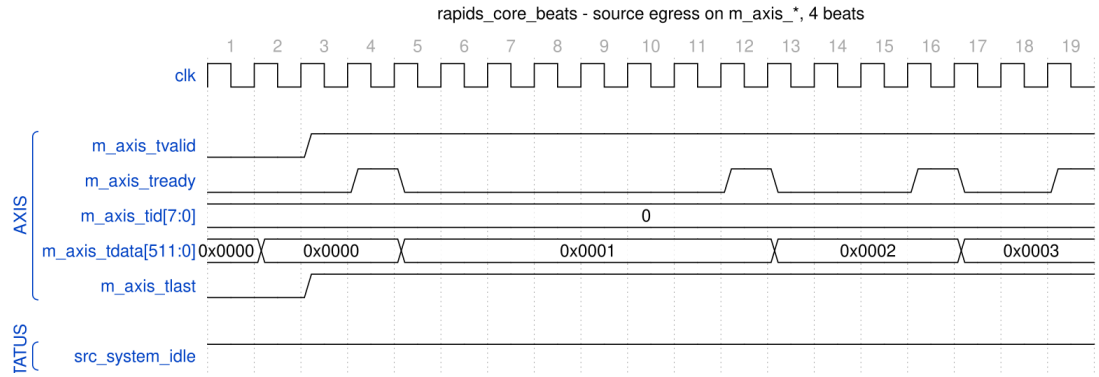


Drain to AXIS mapping

Source: 17b_drain_to_axis.dot

21.5 Timing Diagram

21.5.1 Figure 3.7.3: AXIS Egress Timing



rapids_core_beats - source egress on m_axis_*, 4 beats

Source: src_data_path_axis_egress.json, captured from `dv/tests/top_beats/test_rapids_core_beats.py` (source path, channel 0, 4 beats, 512-bit data, `REG_LEVEL=GATE`) with `WAVES=1`; the consumer is the framework AXIS slave with its default ready pacing.

Reading it: `m_axis_tvalid` rises with the first word and stays high for the whole burst; each beat leaves on a `tready` pulse from the consumer (cycles 4, 12, 16 and 19), `tdata` advancing 0, 1, 2, 3 in the low bits and `tid` naming channel 0. `tlast` closes each drain request's packet (`cfg_drain_size` beats at most); the words arrived from the read engine one at a time here, so every request carried a single word and every beat is marked last. `src_system_idle` is already back high: `system_idle` is the AND of the schedulers' idle flags, and the source scheduler retires a descriptor on the read engine's done strobe (Figure 3.6.3), before the words have drained to the AXIS side. The sink half is different – its scheduler waits for the write commits, so `snk_system_idle` follows the last B response.

21.6 Usage Notes

1. **TID Generation:** Drain channel ID directly maps to AXIS TID
2. **TLAST Generation:** Drain last signal maps to AXIS TLAST
3. **Backpressure:** Network TREADY backpressure propagates to SRAM drain
4. **Packet Boundaries:** TLAST marks end of AXIS packets

21.7 Integration Example

```
src_data_path_axis_beats #(
    .NUM_CHANNELS(8),
    .ADDR_WIDTH(64),
    .DATA_WIDTH(512),
    .AXIS_ID_WIDTH (3)
) u_source_axis (
    .clk                    (clk),
    .rst_n                  (rst_n),

    // Scheduler interface
    .sched_rd_valid         (sched_rd_valid),
    .sched_rd_addr          (sched_rd_addr),
    .sched_rd_beats         (sched_rd_beats),
    .sched_rd_done_strobe   (sched_rd_done_strobe),

    // AXI read master
    .m_axi_arvalid          (src_axi_arvalid),
    .m_axi_arready          (src_axi_arready),
    .m_axi_araddr           (src_axi_araddr),
    .m_axi_rvalid           (src_axi_rvalid),
    .m_axi_rready           (src_axi_rready),
    .m_axi_rdata            (src_axi_rdata),

    // AXIS master
    .m_axis_tvalid          (network_tvalid),
    .m_axis_tready          (network_tready),
    .m_axis_tdata           (network_tdata),
    .m_axis_tlast           (network_tlast),
    .m_axis_tid             (network_tid)
);
```

Last Updated: 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

22 Source SRAM Controller Specification

Module: `src_sram_controller_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

22.1 Overview

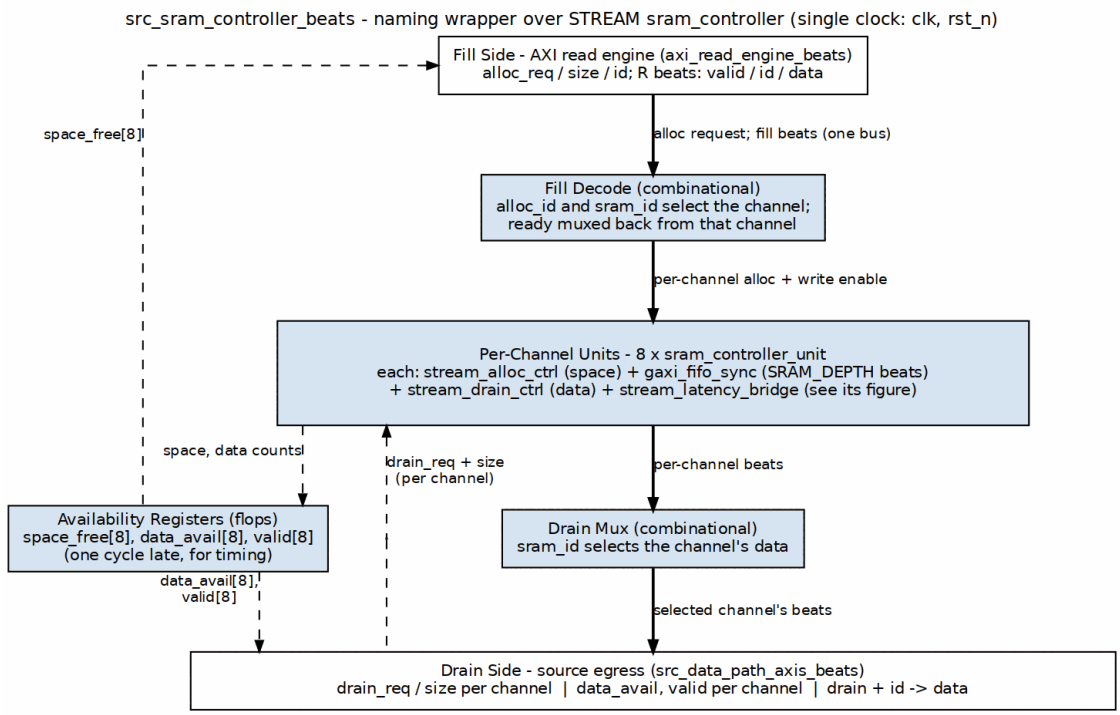
The Source SRAM Controller is a naming wrapper, with no logic of its own, around STREAM's `sram_controller` – the one per-channel SRAM implementation both DMAs share since `bdf4e0dff`. It maps the RAPIDS `fill_*`/`drain_*` names onto STREAM's ports; the eight per-channel units (allocation counter, FIFO, drain counter, latency bridge) live inside STREAM's block. It manages 8 channel units for the source data path, providing per-channel buffering with channel arbitration for data delivery to the network.

22.1.1 Key Features

- **One shared implementation:** instantiates STREAM's `sram_controller` once; it holds 8 `sram_controller_units`, each a `gaxi_fifo_sync` FIFO with `stream_alloc_ctrl`, `stream_drain_ctrl` and `stream_latency_bridge`
- **Channel Arbitration:** none inside the controller – the AXIS egress arbiter picks a channel and presents it on `drain_id`
- **Flow Control Integration:** per-channel `stream_alloc_ctrl` / `stream_drain_ctrl` virtual FIFOs; the drain counter's virtual depth is 2 x SRAM_DEPTH (stream BUG-011)
- **Data Availability Tracking:** Reports available data per channel

22.1.2 Block Diagram

22.1.3 Figure 3.8.1: Source SRAM Controller Block Diagram



Source SRAM controller block diagram

Source: 18_src_sram_controller.dot

22.2 Parameters

```
parameter int NUM_CHANNELS = 8;
parameter int DATA_WIDTH = 512;
parameter int SRAM_DEPTH = 512;                    // Per-channel depth
parameter int ADDR_WIDTH = $clog2(SRAM_DEPTH);
```

: Table 3.8.1: Source SRAM Controller Parameters

22.3 Port List

22.3.1 Clock and Reset

Table 3.8.2: Clock and Reset

Signal	Direction	Width	Description
`clk`	input	1	System clock
`rst_n`	input	1	Active-low reset

22.3.2 Fill Interface (ID-Selected, from AXI Read Engine)

Table 3.8.3: Fill Interface (ID-Selected)

Signal	Direction	Width	Description
`fill_alloc_req`	input	1	Allocation request (single, ID-selected)
`fill_alloc_size`	input	8	Beats to allocate
`fill_alloc_id`	input	CIW	Transaction ID selects the channel
`fill_space_free`	output	NC x SCW	Available space per channel
`fill_valid`	input	1	Fill data valid
`fill_ready`	output	1	Ready for fill data
`fill_id`	input	CIW	Transaction ID selects the channel
`fill_data`	input	DW	Fill data

22.3.3 Arbitrated Drain Interface (to Network)

Table 3.8.4: Arbitrated Drain Interface

Signal	Direction	Width	Description
`drain_data_avail`	output	NC x SCW	Data available per channel
`drain_req`	input	NC	Drain reservation request per channel
`drain_size`	input	NC x 8	Beats to reserve

Signal	Direction	Width	Description
drain_valid	output	NC	Drain data valid per channel
drain_valid_comb	output	NC	Combinational per-channel valid (beat gate)
drain_read	input	1	Consumer read strobe
drain_id	input	CIW	Channel ID select for drain
drain_data	output	DW	Drain data (muxed from selected channel)

22.3.4 Debug Interface

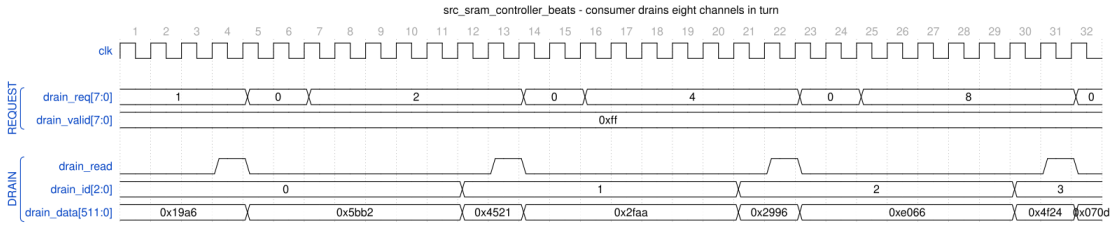
Table 3.8.5: Debug Interface

Signal	Direction	Width	Description
dbg_bridge_pending	output	NC	Bridge has a pending beat per channel
dbg_bridge_out_valid	output	NC	Bridge output valid per channel

22.4 Drain Selection

There is no arbiter inside the controller. Every channel unit raises its own `drain_valid[ch]` / `drain_size[ch]` when it has data; the AXIS egress arbiter picks a channel, drives it on `drain_id`, and the controller steers that channel's beats to the single `drain_*` port. Switching channels costs nothing beyond the consumer's own decision, and a channel keeps presenting `drain_valid` until it is empty.

22.4.1 Figure 3.8.2: Drain Selection by the Consumer



src_sram_controller_beats - consumer drains eight channels in turn

Source: src_sram_controller_drain_select.json, captured from `dv/tests/macro_beats/test_src_sram_controller_beats.py` (`multi_channel`, 8 channels, 512-bit data, 512-entry SRAM, `REG_LEVEL=GATE`) with `WAVES=1`.

Reading it: the same selection contract as the sink controller (Figure 3.5.2). With `drain_valid` = 0xff the consumer asks for one channel at a time through `drain_req`; `drain_id` and `drain_data` follow the request on the same cycle and `drain_read` pops the word. The nine-cycle turn per channel is the test's pacing, not the controller's.

22.5 Comparison: Sink vs Source SRAM Controller

Table 3.8.6: Sink vs Source Controller Comparison

Aspect	Sink Controller	Source Controller
Fill Source	Network ingress	AXI read data
Drain Destination	AXI write engine	Network egress
Fill Interface	Per-channel from network	Per-channel from AXI R
Drain Interface	Arbitrated SRAM read	Arbitrated drain to network
Primary Use	Network -> Memory	Memory -> Network

22.6 Integration Example

```
src_sram_controller_beats #(
    .NUM_CHANNELS(8),
    .DATA_WIDTH(512),
    .SRAM_DEPTH(512)
) u_src_sram_ctrl (
    .clk                    (clk),
    .rst_n                  (rst_n),

    // Per-channel fill interface (from AXI read engine)
    .fill_alloc_req         (fill_alloc_req),
    .fill_alloc_size        (fill_alloc_size),
    .fill_space_free        (fill_space_free),
    .fill_valid             (fill_valid),
    .fill_ready             (fill_ready),
    .fill_data              (fill_data),

    // Arbitrated drain interface (to network)
```

```
    .drain_valid            (src_drain_valid),
    .drain_data             (src_drain_data),
    .drain_id               (src_drain_id),

    // Status
    .drain_data_avail       (channel_data_avail),
    .dbg_bridge_pending     (channel_empty),
    .dbg_bridge_out_valid   (channel_full)
);
```

Last Updated: 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

23 RAPIDS Core Beats Specification

Module: `rapids_core_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

23.1 Overview

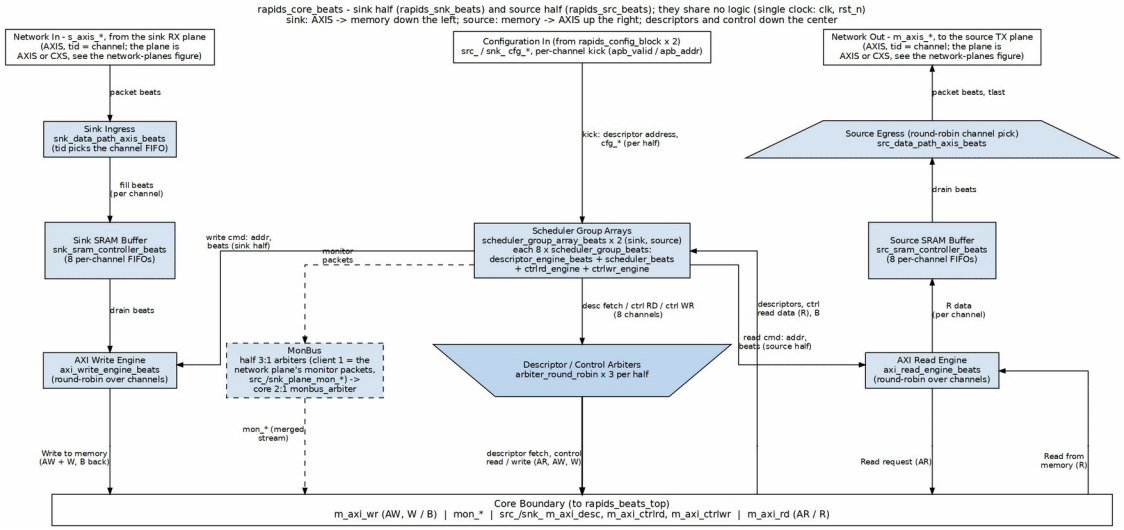
The RAPIDS Core Beats module is the top-level integration of the "beats" architecture, combining the scheduler group array with sink and source data paths.

23.1.1 Key Features

- **Complete RAPIDS Core:** Scheduler array + sink path + source path
- **8-Channel Architecture:** Full multi-channel support
- **Unified MonBus:** Aggregated monitoring from all subsystems
- **Configurable Parameters:** Data width, address width, SRAM depths

23.1.2 Block Diagram

23.1.3 Figure 3.9.1: RAPIDS Core Beats Block Diagram



RAPIDS core beats block diagram

Source: 19_rapids_core.dot

23.2 Parameters

```
parameter int NUM_CHANNELS = 8;
parameter int ADDR_WIDTH = 64;
parameter int DATA_WIDTH = 512;
parameter int DESC_DATA_WIDTH = 256;
parameter int AXI_ID_WIDTH = 8;
parameter int SRAM_DEPTH = 512;

// AXI Parameters
parameter int AR_MAX_OUTSTANDING = 8;
parameter int AW_MAX_OUTSTANDING = 8;
parameter int PIPELINE = 1;              // write-engine pipelining: 1
= up to AW_MAX_OUTSTANDING bursts in flight per channel
parameter int R_PHASE_FIFO_DEPTH = 64;
parameter int W_PHASE_FIFO_DEPTH = 64;
parameter int B_PHASE_FIFO_DEPTH = 16;

// MonBus Parameters
parameter int MON_UNIT_ID = 1;

// Feature Enables
parameter bit ENABLE_AXIS_WRAPPERS = 0;          // Use AXIS
interfaces
```

: Table 3.9.1: RAPIDS Core Beats Parameters

23.3 Port List

23.3.1 Clock and Reset

Table 3.9.2: Clock and Reset

Signal	Direction	Width	Description
`clk`	input	1	System clock
`rst_n`	input	1	Active-low reset

23.3.2 Per-Channel APB Programming

The source and sink halves each carry their own kick-off port; there is no shared `apb_*` port.

Table 3.9.3: APB Programming Interface

Signal	Direction	Width	Description
`src_apb_valid`	input	NC	Source channel kick-off (per-channel)
`src_apb_ready`	output	NC	Source ready for kick-off
`src_apb_addr`	input	NC x AW	Source first descriptor address
`snk_apb_valid`	input	NC	Sink channel kick-off (per-channel)
`snk_apb_ready`	output	NC	Sink ready for kick-off
`snk_apb_addr`	input	NC x AW	Sink first descriptor address

23.3.3 Per-Channel Configuration

Configuration is per half as well: every knob exists as `src_cfg_*` and `snk_cfg_*` (76 configuration ports in total, the bulk of them the descriptor monitor masks). The scheduler-facing ones are:

Table 3.9.4: Per-Channel Configuration

Signal	Direction	Width	Description
`src_cfg_channel_enable` / `snk_cfg_channel_enable`	input	NC	Enable per channel
`src_cfg_channel_reset` / `snk_cfg_channel_reset`	input	NC	Soft reset per channel
`src_cfg_sched_enable` / `snk_cfg_sched_enable`	input	1	Enable the scheduler half

Signal	Direction	Width	Description
src_cfg_sched_timeout_cycles / snk_cfg_sched_timeout_cycles	input	32	Write-progress timeout window (cycles)
src_cfg_sched_timeout_limit / snk_cfg_sched_timeout_limit	input	8	Consecutive-timeout windows before fatal escalation (0 = never)
src_cfg_sched_timeout_enable / snk_cfg_sched_timeout_enable	input	1	Enable timeout detection
cfg_axi_rd_xfer_beats	input	8	Source read burst size (ARLEN encoding)
cfg_axi_wr_xfer_beats	input	8	Sink write burst size (ARLEN encoding)
cfg_alloc_size	input	8	Beats allocated per sink fill request
cfg_drain_size	input	8	Beats drained per source AXIS packet

*_cfg_sched_timeout_limit is passed straight through to the scheduler_group_array_beats instance (recoverable-timeout escalation, see the Scheduler FUB). The write-side COMMIT strobes (sched_wr_commit_strobe / sched_wr_commit_beats) generated by the sink data path's axi_write_engine_beats are routed internally from the sink data path up into the scheduler array so completion is gated on B responses.

23.3.4 Descriptor AXI Master Interfaces

There are TWO descriptor masters, one per half, 18 ports each. Both are read-only, 256-bit.

Table 3.9.5: Descriptor AXI Master Interfaces (src and snk)

Signal	Direction	Width	Description
src_m_axi_desc_arvalid /	output	1	AR channel valid

Signal	Direction	Width	Description
snk_m_axi_desc_arvalid			
src_m_axi_desc_arready / snk_m_axi_desc_arready	input	1	AR channel ready
src_m_axi_desc_araddr / snk_m_axi_desc_araddr	output	AW	AR address
src_m_axi_desc_arid / snk_m_axi_desc_arid	output	IW	AR id
src_m_axi_desc_arlen / snk_m_axi_desc_arlen	output	8	Burst length
src_m_axi_desc_arsize / snk_m_axi_desc_arsize	output	3	Burst size
src_m_axi_desc_arburst / snk_m_axi_desc_arburst	output	2	Burst type
src_m_axi_desc_rvalid / snk_m_axi_desc_rvalid	input	1	R channel valid
src_m_axi_desc_rready / snk_m_axi_desc_rready	output	1	R channel ready
src_m_axi_desc_rdata / snk_m_axi_desc_rdata	input	256	R data (one descriptor)
src_m_axi_desc_rresp / snk_m_axi_de	input	2	R response

Signal	Direction	Width	Description
sc_rresp			
src_m_axi_desc_rlast / snk_m_axi_desc_rlast	input	1	R last
src_m_axi_desc_rid / snk_m_axi_desc_rid	input	IW	R id

The AR sideband (arlock, arcache, arprot, arqos, arregion) is present on both masters and driven to AXI defaults.

23.3.5 Sink AXI Write Master Interface

Table 3.9.6: Sink AXI Write Master Interface

Signal	Direction	Width	Description
m_axi_wr_awvalid	output	1	AW channel valid
m_axi_wr_awready	input	1	AW channel ready
m_axi_wr_awaddr	output	AW	Write address
m_axi_wr_awid	output	IW	Write id (carries the channel index)
m_axi_wr_awlen	output	8	Burst length
m_axi_wr_awsize	output	3	Burst size
m_axi_wr_awburst	output	2	Burst type
m_axi_wr_wvalid	output	1	W channel valid
m_axi_wr_wready	input	1	W channel ready
m_axi_wr_wdata	output	DW	Write data
m_axi_wr_wstr	output	DW/8	Write strobes

Signal	Direction	Width	Description
b			
m_axi_wr_wlast	output	1	Last beat
m_axi_wr_bvalid	input	1	B channel valid
m_axi_wr_bready	output	1	B channel ready
m_axi_wr_bid	input	IW	B id (routes the commit to a channel)
m_axi_wr_bresp	input	2	Write response

The AW sideband (awlock, awcache, awprot, awqos, awregion) is present and driven to AXI defaults. 21 ports in total.

23.3.6 Source AXI Read Master Interface

Table 3.9.7: Source AXI Read Master Interface

Signal	Direction	Width	Description
m_axi_rd_arvalid	output	1	AR channel valid
m_axi_rd_arready	input	1	AR channel ready
m_axi_rd_araddr	output	AW	Read address
m_axi_rd_arid	output	IW	Read id (carries the channel index)
m_axi_rd_arlen	output	8	Burst length
m_axi_rd_arsize	output	3	Burst size
m_axi_rd_arburst	output	2	Burst type
m_axi_rd_rvalid	input	1	R channel valid
m_axi_rd_rrea	output	1	R channel

Signal	Direction	Width	Description
dy			ready
m_axi_rd_rdata	input	DW	Read data
m_axi_rd_rid	input	IW	R id (routes the beat to a channel)
m_axi_rd_rresp	input	2	Read response
m_axi_rd_rlast	input	1	Last beat

23.3.7 Sink Ingress – AXIS Slave

The sink takes network traffic on a standard AXI-Stream slave port. In a top this port is driven by the RX network plane (`fub_axis_*`), whichever protocol the plane speaks on the pins, so the core is the same in the AXIS and CXS tops (see Network Planes). There is no `snk_fill_*` port at core level: the fill handshake is internal to `snk_data_path_axis_beats`, which converts AXIS into it.

Table 3.9.8: Sink Ingress AXIS Slave

Signal	Direction	Width	Description
s_axis_tvalid	input	1	Beat valid
s_axis_tready	output	1	Ready for beat
s_axis_tdata	input	DW	Beat data
s_axis_tstrb	input	SW	Byte strobes
s_axis_tlast	input	1	Last beat of packet
s_axis_tid	input	AXIS_ID_WIDTH	Channel id (low CIW bits select the channel)
s_axis_tdest	input	AXIS_DEST_WIDTH	Destination
s_axis_tuser	input	AXIS_USER_WIDTH	User sideband

23.3.8 Source Egress – AXIS Master

Likewise there is no `src_drain_*` port at core level; the drain handshake is internal to `src_data_path_axis_beats`.

Table 3.9.9: Source Egress AXIS Master

Signal	Direction	Width	Description
`m_axis_tvalid`	output	1	Beat valid
`m_axis_tready`	input	1	Downstream ready
`m_axis_tdata`	output	DW	Beat data
`m_axis_tstrb`	output	SW	Byte strobes
`m_axis_tlast`	output	1	Last beat of packet
`m_axis_tid`	output	AXIS_ID_WIDTH	Channel id
`m_axis_tdest`	output	AXIS_DEST_WIDTH	Destination
`m_axis_tuser`	output	AXIS_USER_WIDTH	User sideband

23.3.9 Unified MonBus Interface

One packet port, carrying a struct rather than a flat 64-bit bus.

Table 3.9.10: Unified MonBus Interface

Signal	Direction	Width	Description
`mon_valid`	output	1	Packet valid
`mon_ready`	input	1	Consumer ready
`mon_packet`	output	`monitor_common_pkg::monitor_packet_t`	Packet payload
`mon_timestamp`	output	`monitor_common_pkg::monbus_timestamp_t`	Packet timestamp

23.3.10 Aggregate Status

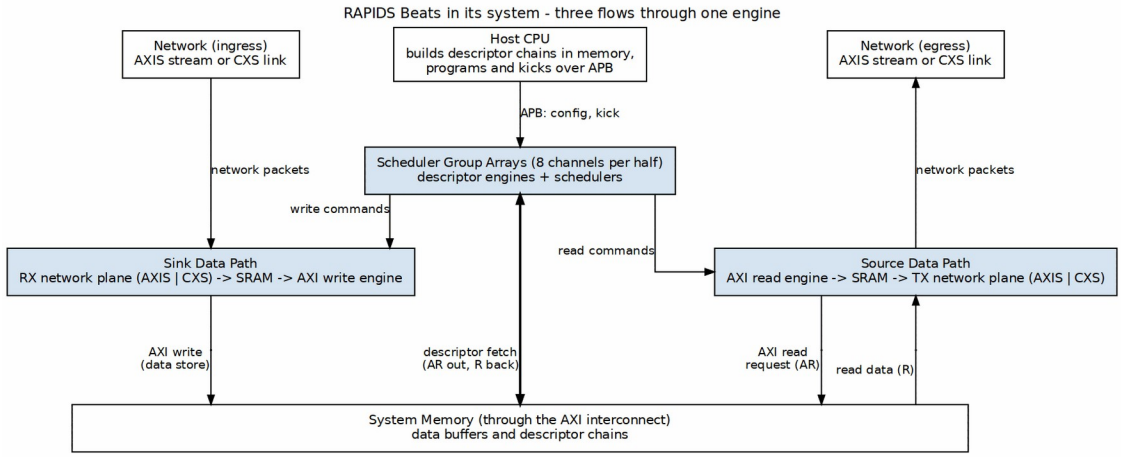
Status is reported per half. There is no `all_channels_idle`, `sink_idle`, `source_idle` or `error_flags` port.

Table 3.9.11: Aggregate Status

Signal	Direction	Width	Description
src_system_idle	output	1	Source half idle (all source schedulers idle)
snk_system_idle	output	1	Sink half idle (all sink schedulers idle)
src_scheduler_idle / snk_scheduler_idle	output	NC	Per-channel scheduler idle
src_scheduler_state / snk_scheduler_state	output	NC x 7	Per-channel FSM state
src_descriptor_engine_idle / snk_descriptor_engine_idle	output	NC	Per-channel descriptor engine idle
src_sched_error / snk_sched_error	output	NC	Per-channel sticky scheduler error

23.4 Data Flow

23.4.1 Figure 3.9.2: RAPIDS Core Complete Data Flow



RAPIDS core data flow

Source: 22_core_data_flow.dot

23.5 MonBus Aggregation

The unified MonBus output aggregates sources from all subsystems:

Table 3.9.12: MonBus Source Assignment

Source Range	Origin	Description
0-15	Scheduler Array	Per-channel scheduler + desc engine
16-23	Sink Data Path	Sink SRAM controllers
24-31	Source Data Path	Source SRAM controllers
32	AXI Write Engine	Write completions
33	AXI Read Engine	Read completions

23.6 Integration Example

```
rapids_core_beats #(
    .NUM_CHANNELS(8),
    .ADDR_WIDTH(64),
    .DATA_WIDTH(512),
    .SRAM_DEPTH(512)
) u_rapids_core (
    .clk                        (clk),
    .rst_n                      (rst_n),

    // APB kick-off (per half)
    .src_apb_valid              (src_kick_valid),
    .src_apb_ready              (src_kick_ready),
    .src_apb_addr               (src_kick_addr),
    .snk_apb_valid              (snk_kick_valid),
    .snk_apb_ready              (snk_kick_ready),
    .snk_apb_addr               (snk_kick_addr),

    // Configuration (per half)
    .src_cfg_channel_enable     (src_cfg_ch_enable),
    .snk_cfg_channel_enable     (snk_cfg_ch_enable),
    .src_cfg_sched_timeout_cycles(src_cfg_timeout),
    .snk_cfg_sched_timeout_cycles(snk_cfg_timeout),
    .cfg_axi_rd_xfer_beats      (cfg_rd_beats),
    .cfg_axi_wr_xfer_beats      (cfg_wr_beats),
    .cfg_alloc_size             (cfg_alloc_size),
    .cfg_drain_size             (cfg_drain_size),

    // Descriptor AXI -- one master per half
    .src_m_axi_desc_arvalid     (src_desc_arvalid),
    .src_m_axi_desc_arready     (src_desc_arready),
    .src_m_axi_desc_araddr      (src_desc_araddr),
    .src_m_axi_desc_rvalid      (src_desc_rvalid),
    .src_m_axi_desc_rready      (src_desc_rready),
    .src_m_axi_desc_rdata       (src_desc_rdata),
    .snk_m_axi_desc_arvalid     (snk_desc_arvalid),
    .snk_m_axi_desc_arready     (snk_desc_arready),
    .snk_m_axi_desc_araddr      (snk_desc_araddr),
    .snk_m_axi_desc_rvalid      (snk_desc_rvalid),
    .snk_m_axi_desc_rready      (snk_desc_rready),
    .snk_m_axi_desc_rdata       (snk_desc_rdata),

    // Sink AXI write master (memory side)
    .m_axi_wr_awvalid           (wr_awvalid),
    .m_axi_wr_awready           (wr_awready),
    .m_axi_wr_awaddr            (wr_awaddr),
```

```
    .m_axi_wr_awlen             (wr_awlen),
    .m_axi_wr_wvalid            (wr_wvalid),
    .m_axi_wr_wready            (wr_wready),
    .m_axi_wr_wdata             (wr_wdata),
    .m_axi_wr_wlast             (wr_wlast),
    .m_axi_wr_bvalid            (wr_bvalid),
    .m_axi_wr_bready            (wr_bready),
    .m_axi_wr_bresp             (wr_bresp),

    // Source AXI read master (memory side)
    .m_axi_rd_arvalid           (rd_arvalid),
    .m_axi_rd_arready           (rd_arready),
    .m_axi_rd_araddr            (rd_araddr),
    .m_axi_rd_arlen             (rd_arlen),
    .m_axi_rd_rvalid            (rd_rvalid),
    .m_axi_rd_rready            (rd_rready),
    .m_axi_rd_rdata             (rd_rdata),
    .m_axi_rd_rlast             (rd_rlast),

    // Sink ingress -- AXIS slave (network in)
    .s_axis_tvalid              (network_rx_tvalid),
    .s_axis_tready              (network_rx_tready),
    .s_axis_tdata               (network_rx_tdata),
    .s_axis_tlast               (network_rx_tlast),
    .s_axis_tid                 (network_rx_tid),

    // Source egress -- AXIS master (network out)
    .m_axis_tvalid              (network_tx_tvalid),
    .m_axis_tready              (network_tx_tready),
    .m_axis_tdata               (network_tx_tdata),
    .m_axis_tlast               (network_tx_tlast),
    .m_axis_tid                 (network_tx_tid),

    // Unified MonBus
    .mon_valid                  (mon_valid),
    .mon_ready                  (mon_ready),
    .mon_packet                 (mon_packet),
    .mon_timestamp              (mon_timestamp),

    // Status (per half)
    .src_system_idle            (src_idle),
    .snk_system_idle            (snk_idle)
);
```

Note: `rapids_core_beats` is a core-level block. The register interface, configuration mapping, and top-level integration (APB slave, descriptor kick-off, AXI monitors, and the MonBus AXI-Lite

group) are provided by the `rapids_regs`, `rapids_config_block`, and `rapids_beats_top` modules – see RAPIDS Registers, RAPIDS Config Block, and RAPIDS Beats Top.

Last Updated: 2026-07-02

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

24 RAPIDS Registers Specification

Module: `rapids_regs.sv` (generated) **Source:** `projects/components/dma-ip/rapids/rtl/macro_beats/rapids_regs.rdl`, `rapids_mon_regs.rdl` **Generated:** `projects/components/dma-ip/rapids/regs/generated/rtl/rapids_regs.sv` **Status:** Implemented (PeakRDL-generated)

24.1 Overview

`rapids_regs` is the PeakRDL-generated register block for the RAPIDS Beats top-level. It exposes configuration and status through a single command/response CPU interface and drives the hardware via a `hwif_out` structure that `rapids_config_block` maps onto the core/monitor `cfg_*` signals.

A single addrmap (one APB slave) contains two regfiles:

- **Base config/status** registers at `0x100-0x3FF`.
- A nested **`rapids_mon_regs`** monitor regfile at base `0x1000`, decoded under `hwif_out.MON.*` (all DAXMON / RDMON / WRMON / MON_FIFO and per-monitor performance registers).

Because the monitor regfile lives at `0x1000`, the register-block address decode requires at least **13 bits** (`cpuif_addr` is a 13-bit compare in the generated RTL). The APB address bus feeding the block (`APB_ADDR_WIDTH`) must therefore be >= 13 to reach the monitor regfile.

24.2 Base Register Map (0x100-0x3FF)

Table 3.10.1: Base Register Map

Offset	Name	Access	Description
0x100	GLOBAL_CTRL	RW	GLOBAL_EN[0], GLOBAL_RST[1] (level; write 0 to release)
0x104	GLOBAL_STATUS	RO	SYSTEM_IDLE[0] (all channels idle)
0x108	VERSION	RO	MINOR[7:0]=0x5A, MAJOR[15:8], NUM_CHANNELS[23:16]=0x08

Offset	Name	Access	Description
0x120	CHANNEL_ENABLE	RW	CH_EN[7:0] per-channel enable
0x124	CHANNEL_RESET	RW	CH_RST[7:0] per-channel reset (level; write 0 to release)
0x140	CHANNEL_IDLE	RO	CH_IDLE[7:0] – not driven at the top level, reads 0
0x144	DESC_ENGINE_IDLE	RO	Per-channel descriptor-engine idle
0x148	SCHEDULER_IDLE	RO	Per-channel scheduler idle (excludes CH_ERROR)
0x150-0x16C	CH_STATE[0..7].STATE	RO	Per-channel FSM state (stride 0x4)
0x170	SCHED_ERROR	RO	Per-channel sticky scheduler error
0x174	AXI_RD_COMPLETE	RO	Per-channel read-complete flags (SRC half only; reads 0 in SNK)
0x178	AXI_WR_COMPLETE	RO	Per-channel write-complete flags – not driven, reads 0
0x200	SCHED_TIMEOUT_CYCLES	RW	TIMEOUT_CYCLES[31:0] write-progress window (reset 1000)
0x204	SCHED_CONFIG	RW	SCHED_EN, TIMEOUT_EN, ERR_EN, COMPL_EN, PERF_EN
0x208	SCHED_TIMEOUT_LIMIT	RW	LIMIT[7:0] consecutive-timeout escalation limit (reset 4, 0 = never)
0x220	DESCENG_CONFIG	RW	DESCENG_EN[0], PREFETCH_EN[1], FIFO_THRESH[5:2] (reset 8)
0x224	DESCENG_ADDR0_BASE	RW	Descriptor address range 0 base [31:0]
0x228	DESCENG_AD	RW	Descriptor address range 0

Offset	Name	Access	Description
	DR0_LIMIT		limit [31:0]
0x22C	DESCENG_ADDR1_BASE	RW	Descriptor address range 1 base [31:0]
0x230	DESCENG_ADDR1_LIMIT	RW	Descriptor address range 1 limit [31:0]
0x240	CTRL_CONFIG	RW	CTRLRD_MAX_TRY[8:0] control-read poll retry budget (0-511, reset 16)
0x2A0	AXI_XFER_CONFIG	RW	AXI transfer sizing configuration
0x2B0	PERF_CONFIG	RW	Performance profiler configuration
0x2C0	OBS_CTRL	RW	Observation mux control (channel/category select)
0x2C4	OBS_FLAGS	RO	Observation flags
0x2C8	OBS_DATA0	RO	Observation data word 0
0x2CC	OBS_DATA1	RO	Observation data word 1
0x35C	PERF_CH_SEL	RW	Performance channel select
0x378	HIST_SEL	RW	Histogram bucket select
0x37C	HIST_DATA	RO	Histogram bucket data
0x380	HIST_TOTAL	RO	Histogram total count

24.2.1 Key Register Fields

SCHED_TIMEOUT_LIMIT (0x208) – `LIMIT[7:0]` sets the number of consecutive write-progress timeout windows a channel tolerates before the recoverable timeout escalates to a fatal, sticky `CH_ERROR`. `0` = never escalate (pure soft timeout). Total time to escalate is approximately `LIMIT x TIMEOUT_CYCLES`. Reset value is 4.

DESCENG_CONFIG (0x220) – `PREFETCH_EN[1]` and `FIFO_THRESH[5:2]` control the now-functional descriptor prefetch: prefetch off = on-demand chaining (~1 ahead), prefetch on = buffer up to `FIFO_THRESH` descriptors ahead.

CTRL_CONFIG (0x240) – `CTRLRD_MAX_TRY[8:0]` bounds the control-read poll retry budget (0-511, reset 16) fed to every channel's `ctrlrd_engine`. A `CTRL_READ` descriptor polls its gate address once per retry; if the masked value never matches within `CTRLRD_MAX_TRY` attempts the engine raises `ctrlrd_error` so a never-satisfied gate cannot hang the channel. See the Control-

Read / Control-Write Engine specs (Sections 2.8 / 2.9) and the Scheduler control interface (Section 2.1).

24.3 Monitor Register Map (rapids_mon_regs @ 0x1000)

The monitor regfile is instantiated at base `0x1000` under `hwif_out.MON.*`. It holds the MonBus FIFO status, the three AXI-monitor configuration groups (DAXMON = descriptor monitor, RDMON = read monitor, WRMON = write monitor), and per-monitor performance counters.

Table 3.10.2: Monitor Register Map (base 0x1000)

Offset	Name	Access	Description
0x1000	`MON.MON_FIFO_STATUS`	RO	MonBus capture/error FIFO status
0x1004	`MON.MON_FIFO_COUNT`	RO	MonBus FIFO occupancy
0x10C0	`MON.DAXMON_ENABLE`	RW	Descriptor-monitor enables (incl. `COMPRESS_EN`)
0x10C4	`MON.DAXMON_TIMEOUT`	RW	Descriptor-monitor timeout cycles
0x10C8	`MON.DAXMON_LATENCY_THRESH`	RW	Descriptor-monitor latency threshold
0x10CC	`MON.DAXMON_PKT_MASK`	RW	Descriptor-monitor packet mask
0x10D0	`MON.DAXMON_ERR_CFG`	RW	Descriptor-monitor error select/mask
0x10D4-0x10DC	`MON.DAXMON_MASK1/2/3`	RW	Descriptor-monitor category masks
0x10E0-0x10FC	`MON.RDMON_*`	RW	Source-egress AXIS monitor-lite config (same layout as DAXMON; SRC instance, rapids TASK-015)
0x1100-0x111C	`MON.WRMON_*`	RW	Sink-ingress AXIS monitor-lite config (same layout as DAXMON; SNK instance, rapids TASK-015)

Offset	Name	Access	Description
0x1150-0x1178	MON.DAXMON_PERF_*	RO/RW	Descriptor-monitor performance counters
0x1180-0x11A8	MON.RDMON_PERF_*	RO/RW	Read-monitor performance counters
0x11B0-0x11D8	MON.WRMON_PERF_*	RO/RW	Write-monitor performance counters
0x11E0-0x11EC	MON.{RD,WR}MON_PERF_CH_*	RO	Per-channel producer/backpressure/starve/idle
0x11F0	MON.RDMON_PERF_CH_OVERFLOW	RO	Read-monitor per-channel overflow
0x11F4	MON.WRMON_PERF_CH_OVERFLOW	RO	Write-monitor per-channel overflow

Each *_ENABLE register carries MON_EN, ERR_EN, COMPL_EN, TIMEOUT_EN (and COMPRESS_EN on the write monitor); each *_PERF_* group provides window, producer, backpressure, starve, idle, beat, byte (lo/hi), and burst counters.

Last Updated: 2026-07-02

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

25 RAPIDS Config Block Specification

Module: rapids_config_block.sv **Location:** projects/components/dma-ip/rapids/rtl/macro_beats/ **Status:** Implemented

---

25.1 Overview

rapids_config_block is a combinational adapter that maps the register block's hwif_out structure onto the flat cfg_* signals consumed by rapids_core_beats and the AXI monitors. It contains no state; every output is a direct assignment (with a small number of width expansions and global-enable gates).

The monitor registers arrive under hwif_out.MON.*; base configuration and status arrive at the top level of hwif_out.

25.2 Mapping Groups

25.2.1 Base Configuration

Routes scheduler, channel, and descriptor-engine configuration to the core:

- cfg_channel_enable = CHANNEL_ENABLE.CH_EN & {8{GLOBAL_CTRL.GLOBAL_EN}} (per-channel enable gated by the global master enable).
- cfg_sched_timeout_cycles = SCHED_TIMEOUT_CYCLES.TIMEOUT_CYCLES (32-bit).
- cfg_sched_timeout_limit = SCHED_TIMEOUT_LIMIT.LIMIT (8-bit, direct).
- cfg_sched_{timeout,err,compl,perf}_enable = SCHED_CONFIG.{TIMEOUT,ERR,COMPL,PERF}_EN. COMPL_EN is the one the scheduler group acts on for its monitor packets: 0 drops the scheduler's and descriptor engine's CORE Completion packets at the group (emitters still acknowledged); ERR_EN and PERF_EN reach the group but gate nothing today (errors always reported, perf not implemented).
- cfg_desceng_* from DESCENG_CONFIG (enable / prefetch / FIFO threshold).
- Descriptor address ranges (cfg_desceng_addr{0,1}_{base,limit}) are zero-extended from the 32-bit register fields to the 64-bit ADDR_WIDTH.

25.2.2 Monitor Configuration (hwif_out.MON.*)

Three parallel groups map to the descriptor-read monitor and, since rapids TASK-015, the two AXIS data-path monitor-lites (one per half):

Table 3.11.1: Monitor Register-to-Config Mapping

Register group	cfg_* target
MON.DAXMON_*	cfg_desc_mon_* (descriptor monitor)
MON.RDMON_*	cfg_rdeng_mon_* (source-egress AXIS monitor-lite; wired in the SRC half only)
MON.WRMON_*	cfg_wreng_mon_* (sink-ingress AXIS monitor-lite; wired in the SNK half only)

The *_ENABLE bits map onto the AXIS monitor-lite's class enables: MON_EN (gated by GLOBAL_EN), ERR_EN (error, channel-change and TSTRB checks), COMPL_EN (one Completion/STREAM_END per packet), PERF_EN (stream start/pause/resume and Credit/BACKPRESSURE stall reports), TIMEOUT_EN. *_TIMEOUT is MICROSECONDS in its low 16 bits (0 = never), *_LATENCY_THRESH the stall CYCLES that raise a backpressure report, and *_PKT_MASK is a MASK: bit[type] = 1 masks (drops) that packet type at the monitor-lite and again at the monbus group. It resets to all-masked (0xFFFF); the host clears the bits of the classes it wants to see (rapids BUG-008 corrected the description, which used to read "1 = enable"; the polarity and the reset are as intended).

Within each group the enable/timeout/latency/mask fields map 1:1, e.g. cfg_desc_mon_enable = MON.DAXMON_ENABLE.MON_EN & GLOBAL_CTRL.GLOBAL_EN, cfg_desc_mon_timeout_cycles = MON.DAXMON_TIMEOUT.TIMEOUT_CYCLES, and the category masks (timeout/compl/thresh/perf/addr/debug) pass through directly. Key enables are gated by GLOBAL_CTRL.GLOBAL_EN; masks pass through ungated.

25.2.3 Performance and Observation

PERF_CONFIG, PERF_CH_SEL, and OBS_CTRL map to the profiler and channel-observation-mux cfg_* signals.

Last Updated: 2026-07-02

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

26 RAPIDS Beats Top Specification

Module: `rapids_beats_top.sv`, `rapids_beats_cxs_top.sv` (generated), `rapids_beats_top_base.sv` **Location:** `projects/components/dma-ip/rapids/rtl/top_beats/` **Status:** Implemented

---

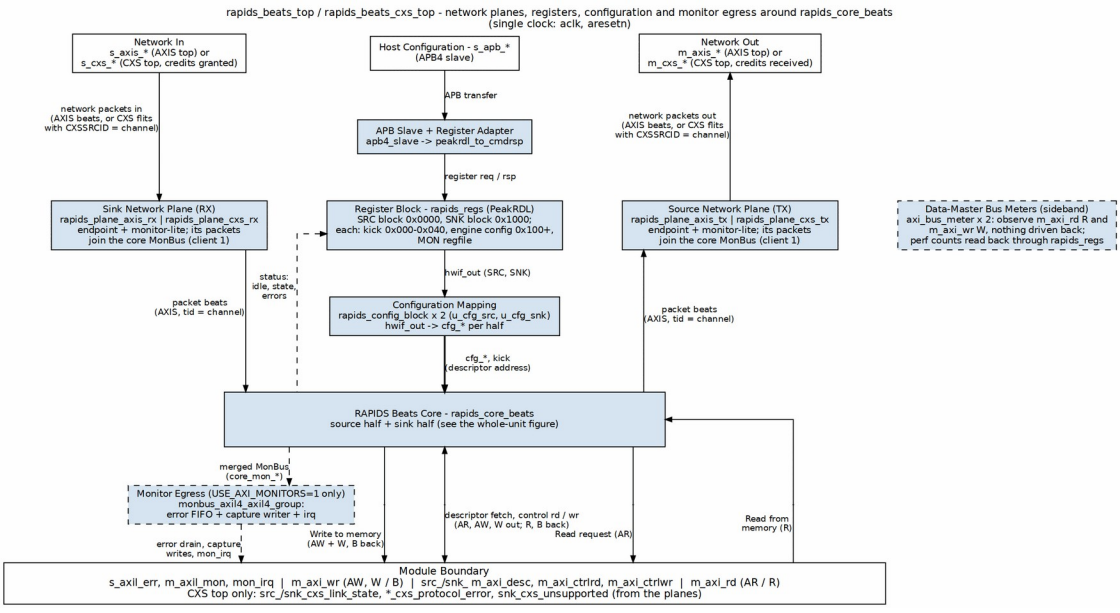
26.1 Overview

`rapids_beats_top` is the synthesizable top-level integration of the RAPIDS Beats accelerator. It wraps the register block, config-block adapter, and core with a single APB slave for all software access, and adds optional AXI transaction monitors whose events are merged with the core's descriptor-monitor packet and delivered through a MonBus AXI-Lite group (error-drain slave, capture master, and interrupt).

The top is generated (rapids TASK-024). `rapids_beats_top_base` holds everything described on this page; `bin/gen_plane_tops.py` wraps it with one RX and one TX network plane to make `rapids_beats_top` (AXIS planes) and `rapids_beats_cxs_top` (CXS planes). The planes are described in the Network Planes chapter; edit the base or the generator, never a generated top.

26.2 Integration Datapath

26.2.1 Figure 3.12.1: RAPIDS Beats Top Integration



RAPIDS beats top integration

Source: 20_rapids_beats_top.dot

26.2.2 Address Decode

Table 3.12.1: Top-Level Address Decode

Range	Target	Purpose
0x000-0x03F	`rapids_regs` (kick)	Per-channel descriptor kick-off registers
0x100-0x3FF	`rapids_regs` (base)	Configuration / status registers
0x1000+	`rapids_regs` (MON regfile)	Monitor configuration / performance

Because the monitor regfile is at `0x1000`, the APB address bus must be at least 13 bits wide (`APB_ADDR_WIDTH >= 13`) to reach it.

26.3 Write-engine pipelining (PIPELINE)

`PIPELINE` reaches `axi_write_engine_beats` through the core and both halves. `0` is the engine's original contract, one burst in flight per channel; `1` allows up to `AW_MAX_OUTSTANDING` per channel. Every module that declares the parameter, from the engines to this top, defaults to `1` since 2026-09-28: once rapids BUG-005 made `PIPELINE = 0` honour one-in-flight, the 8-channel Genesys 2 build measured 49.9 % AXI4-wr engaged utilization at 4096 beats per channel (59.5 % at 1024), against 100 % on the bitstream before the fix – which had reached line rate only because the pre-fix engine ran two bursts in flight by accident. The harness simulation reproduces the same numbers (82.4 % at 1024 beats with `PIPELINE = 0`, 99.9 % with `1`). One burst in flight cannot cover the write-response round trip at 8-beat bursts, so `1` is the design point at every level; `0` is selectable for a design that must bound each channel to a single outstanding write, never the default.

26.4 AXI Monitors (USE_AXI_MONITORS)

When `USE_AXI_MONITORS = 1` the top builds the `monbus_axil4_axil4_group` and feeds it the core's MonBus stream. That stream carries the scheduler groups' completion and error events and two monitor-lite instances per half: `axi4_master_rd_monlite` on the half's descriptor read master, inside `scheduler_group_array_beats` (agent 0x08), and a network monitor-lite in the half's network plane (rapids TASK-015, moved to the planes by TASK-024): the sink plane's lite on `s_axis_*` or `s_cxs_*` (agent 0x09, configured by `SNK.MON.WRMON_*`) and the source plane's lite on `m_axis_*` or `m_cxs_*` (agent 0x0A, configured by `SRC.MON.RDMON_*`). The plane's packets enter the core on `snk_plane_mon_*` / `src_plane_mon_*`. Each half merges its scheduler array (client 0), its plane monitor (client 1) and a tied-off placeholder client in a 3:1 `monbus_arbiter`;

the core merges the two halves 2:1. The AXIS planes also place an `axis4_slave` / `axis4_master` skid stage (`SKID_DEPTH` = 4) on the network port in every build; the CXS planes buffer in the receive FIFO instead. Only the monitor tap is gated by `USE_AXI_MONITORS`. The data masters `m_axi_rd` and `m_axi_wr` carry NO monitors (an earlier revision of this page said rd/wr monitor-lite blocks sat on the data masters; the RTL has never had them). Data-master observation lives in the characterization harness as external instruments (`axi_bus_meter`, `axis_bus_meter`, and the interface observers on `USE_OBSERVERS` builds). The group's AXIS filter slot is fed from `SRC.MON.RDMON_*`, so it filters both halves' network packets (a CXS plane's `PROTOCOL_CXS` packets ride the AXIS slot); `*_PKT_MASK` masks on a set bit and resets to all-masked (rapids BUG-008 corrected its description). The group provides:

- `s_axil_err_*` – AXI-Lite (32-bit) **error-drain slave**: CPU reads captured error events from the error FIFO.
- `m_axil_mon_*` – AXI-Lite (64-bit) **capture master**: bulk-writes the MonBus trace to system memory (base/limit/watermark from `cfg_mon_*`).
- `mon_irq` – interrupt on error/threshold events.

When `USE_AXI_MONITORS = 0` the descriptor and network monitor-lites are omitted (the AXIS skid stages and CXS receive FIFO stay), the core MonBus is dropped (always-ready), and the AXI-Lite group outputs are tied off (`s_axil_err` read-inactive, `m_axil_mon` write-inactive, `mon_irq = 0`).

26.4.1 Monitor registers on a monitors-off build (USE_MON_REGS)

`USE_MON_REGS` defaults to `USE_AXI_MONITORS != 0` and is passed to both config blocks. When it is 0 the blocks strap every `cfg_*mon_*` output off, and the APB path answers the two MON register windows – SRC MON at 0x0800-0x0FFF and SNK MON at 0x1800-0x1FFF, i.e. `paddr[11]` set in either 4 KB half – with `PSLVERR` and zero read data instead of letting the registers accept a write and read it back. The address map does not change, so one host image still addresses both builds; what changes is that "not built" is distinguishable from "built and set to zero". A blocked access is still accepted (holding `cmd_ready` low would hang the bus) and the guard's error beat replaces the adapter's response for that one transfer (rapids ISSUE-003; the shape is STREAM's TASK-002 guard). With `USE_MON_REGS = 1` the guard folds away to the straight-through wiring.

26.5 Parameters

```
parameter int NUM_CHANNELS        = 8;
parameter int DATA_WIDTH          = 512;
parameter int ADDR_WIDTH          = 64;
parameter int AXI_ID_WIDTH        = 8;
parameter int SRAM_DEPTH          = 4096;
parameter int APB_ADDR_WIDTH      = 13;   // bit 12 selects SRC (0) /
SNK (1)
parameter int APB_DATA_WIDTH      = 32;
```

```
parameter int USE_AXI_MONITORS    = 1;    // 1 = descriptor + network
monitor-lites + MonBus group
parameter bit USE_MON_REGS        = (USE_AXI_MONITORS != 0); // 0 =
MON windows answer PSLVERR
parameter int MON_MAX_TRANSACTIONS = 16;
parameter int AR_MAX_OUTSTANDING  = 8;
parameter int AW_MAX_OUTSTANDING  = 8;
parameter int PIPELINE            = 1;   // write-engine pipelining;
see the note below
parameter int AXIS_ID_WIDTH       = 8;   // network stream tid
(carries the channel)
parameter int AXIS_DEST_WIDTH     = 4;
parameter int AXIS_USER_WIDTH     = 1;
// rapids_beats_cxs_top only (see the Network Planes chapter):
parameter int CXS_MAX_CREDIT      = 32;  // one credit = one flit
parameter int CXS_RX_FIFO_DEPTH   = CXS_MAX_CREDIT;
parameter bit CXS_CREDIT_BEYOND_SPEC = (CXS_MAX_CREDIT > 63);
parameter logic [7:0] CXS_IDLE_CYCLES = 8'd32;
```

: Table 3.12.2: RAPIDS Beats Top Parameters

26.6 Top-Level Interfaces

Table 3.12.3: Top-Level Interfaces

Interface	Signals	Notes
Clock / reset	`aclk`, `aresetn`, `cam_clear`	Single clock domain; `cam_clear` for monitor CAMs
APB slave	`s_apb_paddr` (APB_ADDR_WIDTH), `s_apb_psel/penable/pwrite/pwdata/pstrb`, `s_apb_prdata/pready/pslverr`	32-bit data
Descriptor AXI (master)	`m_axi_desc_ar*`, `m_axi_desc_r*`	256-bit read data
Data read AXI (master)	`m_axi_rd_ar*`, `m_axi_rd_r*`	DATA_WIDTH; no monitor
Data write AXI (master)	`m_axi_wr_aw*`, `m_axi_wr_w*`, `m_axi_wr_b*`	DATA_WIDTH; no monitor

Interface	Signals	Notes
Network in (AXIS top)	s_axis_tdata/tstrb/tlast/tid/tdest/tuser/tvalid/tready	Sink ingress, tid = channel
Network out (AXIS top)	m_axis_tdata/tstrb/tlast/tid/tdest/tuser/tvalid/tready	Source egress, tid = channel
Network in (CXS top)	s_cxs_*, snk_cxs_link_state, snk_cxs_protocol_error, snk_cxs_unsupported	CXS receiver, CXSSRCID = channel
Network out (CXS top)	m_cxs_*, src_cxs_link_state, src_cxs_protocol_error	CXS transmitter, CXSSRCID = channel
MonBus error-drain slave	s_axil_err_ar*, s_axil_err_r*	AXI-Lite, 32-bit read
MonBus capture master	m_axil_mon_aw*, m_axil_mon_w* (64-bit wdata), m_axil_mon_b*	AXI-Lite write
MonBus interrupt	mon_irq	Error/threshold interrupt
MonBus config	cfg_mon_base_addr, cfg_mon_limit_addr, cfg_mon_flush_watermark	Capture region + flush watermark
Status	system_idle, sched_error[NC-1:0]	Aggregate status

Last Updated: 2026-10-16

27 Network Planes Specification

Modules: `rapids_plane_axis_rx.sv`, `rapids_plane_axis_tx.sv`, `rapids_plane_cxs_rx.sv`, `rapids_plane_cxs_tx.sv` **Location:** `projects/components/dma-ip/rapids/rtl/plane/` **Generator:** `projects/components/dma-ip/rapids/bin/gen_plane_tops.py` **Status:** Implemented (rapids TASK-024, TASK-025)

27.1 Overview

A network plane is the endpoint between the core's network-side AXIS stream and the pins: the RX plane feeds the sink half's ingress, the TX plane takes the source half's egress. Each plane carries the monitor-lite for its port. Both RAPIDS trees (byte-granular `rtl/top`, beat-granular `rtl/top_beats`) share the same four planes.

Every plane has the same parameters and the same inner interface, so a top selects AXIS or CXS by module name alone:

- `fub_axis_*`: the core-facing AXIS stream (`tdata`, `tstrb`, `tlast`, `tid` = channel, `tdest`, `tuser`)
- `cfg_mon_*` in, `sts_mon_*` out: the `SNK.WRMON_*` / `SRC.RDMON_*` monitor CSRs, from `rapids_config_block`
- `monbus_*` out: client 1 of the half's 3:1 `monbus_arbiter`, entering the core on `snk_plane_mon_*` / `src_plane_mon_*`

Table 3.13.1: RAPIDS Network Planes

Plane	Contents	Outer pins
`rapids_plane_axis_rx`	`axis4_slave_monlite` (skid stage, `SKID_DEPTH` = 4, + `axis_monitor_lite` tap)	`s_axis_*`
`rapids_plane_axis_tx`	`axis4_master_monlite` (skid stage + tap)	`m_axis_*`
`rapids_plane_cxs_rx`	`cxs_slave_monlite` (`u_endpoint`) + `cxs_to_axis` (`u_cxs_to_axis`)	`s_cxs_*`, `snk_cxs_*` status
`rapids_plane_cxs_tx`	`axis_to_cxs` (`u_axis_to_cxs`) + `cxs_master_monlite`	`m_cxs_*`, `src_cxs_*` status

Plane	Contents	Outer pins
	(u_endpoint)	

The monitor counters of every plane are at `u_snk_plane.u_endpoint` and `u_src_plane.u_endpoint` in the top.

27.2 Generated Tops

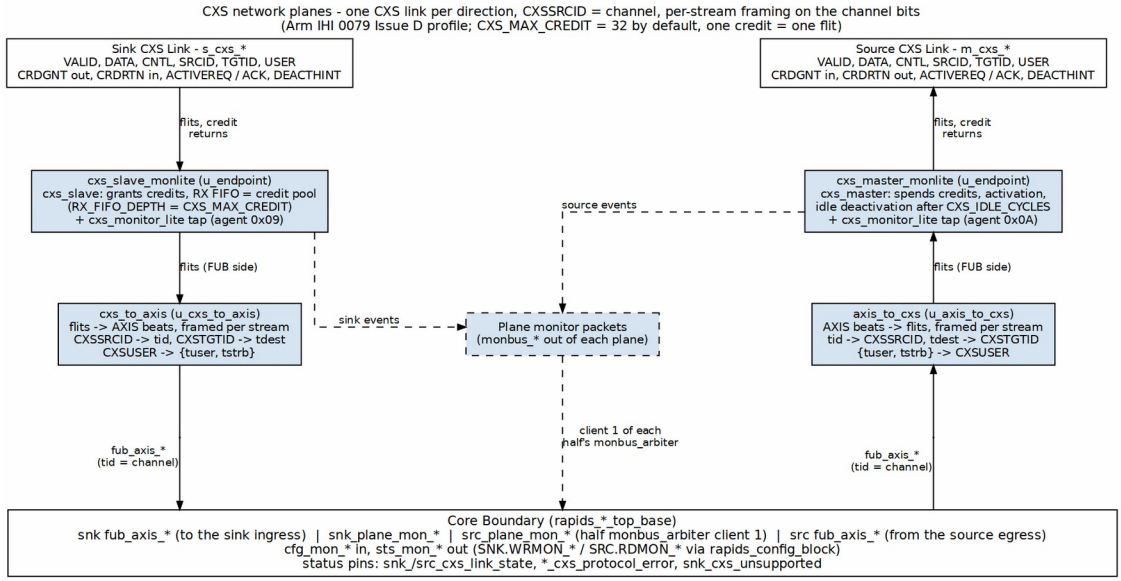
Table 3.13.2: Generated Tops

Top	Tree	Base	Planes
rapids_top	byte	rapids_top_base	AXIS
rapids_cxs_top	byte	rapids_top_base	CXS
rapids_beats_top	beats	rapids_beats_top_base	AXIS
rapids_beats_cxs_top	beats	rapids_beats_top_base	CXS

`gen_plane_tops.py` copies the base's parameter and port text verbatim, then instantiates the base (`u_base`) and the two planes (`u_snk_plane`, `u_src_plane`). The AXIS tops therefore keep exactly the interface they had before the split, and existing instantiations are unchanged. Run the generator with `--check` to fail on a stale top.

27.3 CXS Planes

27.3.1 Figure 3.13.1: CXS Network Planes



CXS network planes

Source: 23_cxs_planes.dot

27.3.2 Receive (sink) plane

`cxs_slave_monlite` is `cxs_slave` with a `cxs_monitor_lite` tap. `cxs_slave` grants credits up to `CXS_MAX_CREDIT`, runs the activation handshake, and holds received flits in a receive FIFO of `RX_FIFO_DEPTH` (default `CXS_MAX_CREDIT`) entries. The FIFO is the credit pool: a credit is only granted for a free entry, so a flit always has space. `cxs_to_axis` turns flits back into AXIS beats: `CXSSRCID` becomes `tid`, `CXSTGTID` becomes `tdest`, and `CXSUSER` is split into `{tuser, tstrb}`. When the core stops taking beats the FIFO fills, and the slave stops granting credits; that is the CXS form of the sink's `tready` backpressure.

`cxs_to_axis` is combinational and stateless, so it can only represent the flits `axis_to_cxs` makes: at most one START and one END per flit, with a packet starting at byte 0. Any other flit (a transmitter that packs two packets into one flit) is still passed on, with `tlast` from END[0], and sets the sticky `snk_cxs_unsupported`.

27.3.3 Transmit (source) plane

`axis_to_cxs` packs AXIS beats into flits (`tid` -> `CXSSRCID`, `tdest` -> `CXSTGTID`, `{tuser, tstrb}` -> `CXSUSER`) and `cxs_master_monlite` sends them, one credit per flit. The master requests activation when it has a flit to send and deactivates the link after `CXS_IDLE_CYCLES` idle cycles.

27.3.4 Per-stream framing

axis_to_cxs and both endpoints' control checkers track an open packet per stream, keyed on the low CXS_STREAM_SRCID_BITS = CIW bits of CXSSRCID (the channel); cxs_to_axis needs no state because each beat carries its own tid. The byte-granular source arbitrates per drain block while a packet spans the whole descriptor, so with two or more channels active the egress switches channel in the middle of a packet. Per-link framing (one open packet per link) rejects that traffic; per-stream framing accepts it, as IHI 0079D 2.5.2 allows.

27.3.5 Parameters

Table 3.13.3: CXS Plane Parameters

Parameter	Default	Description
CXS_MAX_CREDIT	32	Credits granted by the receiver. One credit is one flit; the grant -> flit -> free round trip is about 3 cycles
CXS_RX_FIFO_DEPTH	CXS_MAX_CREDIT	Receive FIFO depth (the credit pool); need not be a power of two
CXS_CREDIT_BEYOND_SPEC	CXS_MAX_CREDIT > 63	Permits more credits than IHI 0079D allows; the far end must match
CXS_IDLE_CYCLES	32	Idle cycles before the transmitter deactivates the link
CXS_IW, CXS_DESTW, CXS_UW, CXS_CNTL_PW	derived	Port widths; do not override

DATA_WIDTH must be at most 1024: CXSUSER is at most 128 bits and carries tstrb (DATA_WIDTH/8 bits). tuser is dropped from CXSUSER when DATA_WIDTH/8 + AXIS_USER_WIDTH > 128.

27.3.6 Monitoring

The CXS planes drive cxs_monitor_lite from the same cfg_mon_* CSRs as the AXIS planes and use the same agent IDs (sink 0x09, source 0x0A). Packets are tagged PROTOCOL_CXS; the top's monitor group filters them through its AXIS mask slot. A PACKET_END completion carries the channel in event_data[55:48]; channel_id holds {PRCLTYPE[2:0], TGTID[5:0]}. The lite's output queue is 4 entries deep; when events outrun it the lite drops and counts them (an Error / EVENT_DROPPED packet), so a check on completions counts delivered plus dropped.

27.4 Planned Changes

The ID mapping above is provisional. The ID fields, and possibly other fields or new signals, are expected to change so that a flit can carry routing and payload type: delta configuration and delta memory traffic as distinct from endpoint or cell configuration and endpoint or cell memory traffic. The encoding is not yet defined, and this chapter will be revised when it is.

27.5 Verification

Table 3.13.4: CXS Top Test Cells

Suite	Cells
`dv/tests/top/ test_rapids_cxs_top.py`	source, sink, source_bytes, sink_bytes, source_concurrent, sink_concurrent, cxs_mon, ext_addressing
`dv/tests/top_beats/ test_rapids_beats_cxs_top.py`	source, sink, source_concurrent, sink_concurrent, cxs_mon, ext_addressing

The concurrent cells start four channels with one `KICK_ENABLE` write, so channels interleave on the link. Every cell also requires a clean link: no BFM or wire-monitor violation and the DUT's protocol-error flags low. The core testbenches drive packets on `plane_mon_*` and require them on the MonBus output.

Last Updated: 2026-10-16

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

28 Interface Overview

Last Updated: 2026-10-16

28.1 External Interfaces

The RAPIDS Beats architecture exposes the following external interfaces:

28.1.1 AXI4 Master Interfaces

Table 4.0.1: AXI4 Master Interfaces

Interface	Purpose	Data Width
Descriptor AXI	Fetch descriptors from memory	256-bit
Sink AXI Write	Write sink data to memory	512-bit (param)
Source AXI Read	Read source data from memory	512-bit (param)

28.1.2 Streaming Interfaces

Table 4.0.2: Streaming Interfaces

Interface	Type	Purpose
Fill Interface	Custom	Network data ingress (to SRAM), inside the core
Drain Interface	Custom	Network data egress (from SRAM), inside the core
AXIS Sink	AXI-Stream Slave	Network ingress on the AXIS tops; the core-facing stream of every plane
AXIS Source	AXI-Stream Master	Network egress on the AXIS tops; the core-facing stream of every plane
CXS Sink	CXS receiver	Network ingress on the CXS tops (one link, `CXSSRCID` = channel)

Interface	Type	Purpose
CXS Source	CXS transmitter	Network egress on the CXS tops (one link, `CXSSRCID` = channel)

28.1.3 Control/Monitor Interfaces

Table 4.0.3: Control and Monitor Interfaces

Interface	Type	Purpose
APB	APB slave	Per-channel kick-off
Configuration	Custom	Global/per-channel config
Status	Custom	Idle, state, error flags
MonBus	Custom 64-bit	Monitor packet output

28.2 Interface Documentation

- AXI4 Interface Specification - Descriptor, sink write, source read
- AXIS Interface Specification - Optional streaming wrappers
- MonBus Interface Specification - Monitor packet format
- CXS Interface Specification - CXS network links (CXS tops)

28.3 Interface Signal Naming Conventions

Table 4.0.4: Signal Naming Conventions

Prefix	Meaning	Example
`m_axi_`	AXI master port	`m_axi_arvalid`
`s_axi_`	AXI slave port	`s_axi_awready`
`m_axis_`	AXIS master port	`m_axis_tvalid`
`s_axis_`	AXIS slave port	`s_axis_tready`
`m_cxs_`	CXS transmitter port	`m_cxs_valid`
`s_cxs_`	CXS receiver port	`s_cxs_crdgnt`
`cfg_`	Configuration input	`cfg_enable`

Prefix	Meaning	Example
`snk_`	Sink data path	`snk_fill_valid`
`src_`	Source data path	`src_drain_ready`
`sched_`	Scheduler interface	`sched_rd_valid`
`monbus_`	Monitor bus	`monbus_pkt_data`

Last Updated: 2026-10-16

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

29 AXI4 Interface Specification

Last Updated: 2025-01-10

29.1 Overview

The RAPIDS Beats architecture uses three AXI4 master interfaces: 1. **Descriptor AXI:** Fetches descriptors from memory (256-bit data) 2. **Sink AXI Write:** Writes sink data to memory (512-bit data, configurable) 3. **Source AXI Read:** Reads source data from memory (512-bit data, configurable)

29.2 Descriptor AXI Master Interface

29.2.1 Purpose

Fetches descriptor packets from system memory. Shared across all 8 channels with round-robin arbitration.

29.2.2 Signal Table

Table 4.1.1: Descriptor AXI Master Interface Signals

Signal	Direction	Width	Description
`m_axi_arvalid`	output	1	AR channel valid
`m_axi_arready`	input	1	AR channel ready
`m_axi_araddr`	output	AW	Read address
`m_axi_arid`	output	ID_W	Transaction ID
`m_axi_arlen`	output	8	Burst length (fixed: 0 = 1 beat)
`m_axi_arsize`	output	3	Burst size (fixed: 5 = 32 bytes)
`m_axi_arburst`	output	2	Burst type (fixed: INCR)

Signal	Direction	Width	Description
m_axi_rvalid	input	1	R channel valid
m_axi_rready	output	1	R channel ready
m_axi_rdata	input	256	Read data (descriptor)
m_axi_rid	input	ID_W	Response ID
m_axi_rresp	input	2	Read response
m_axi_rlast	input	1	Last beat

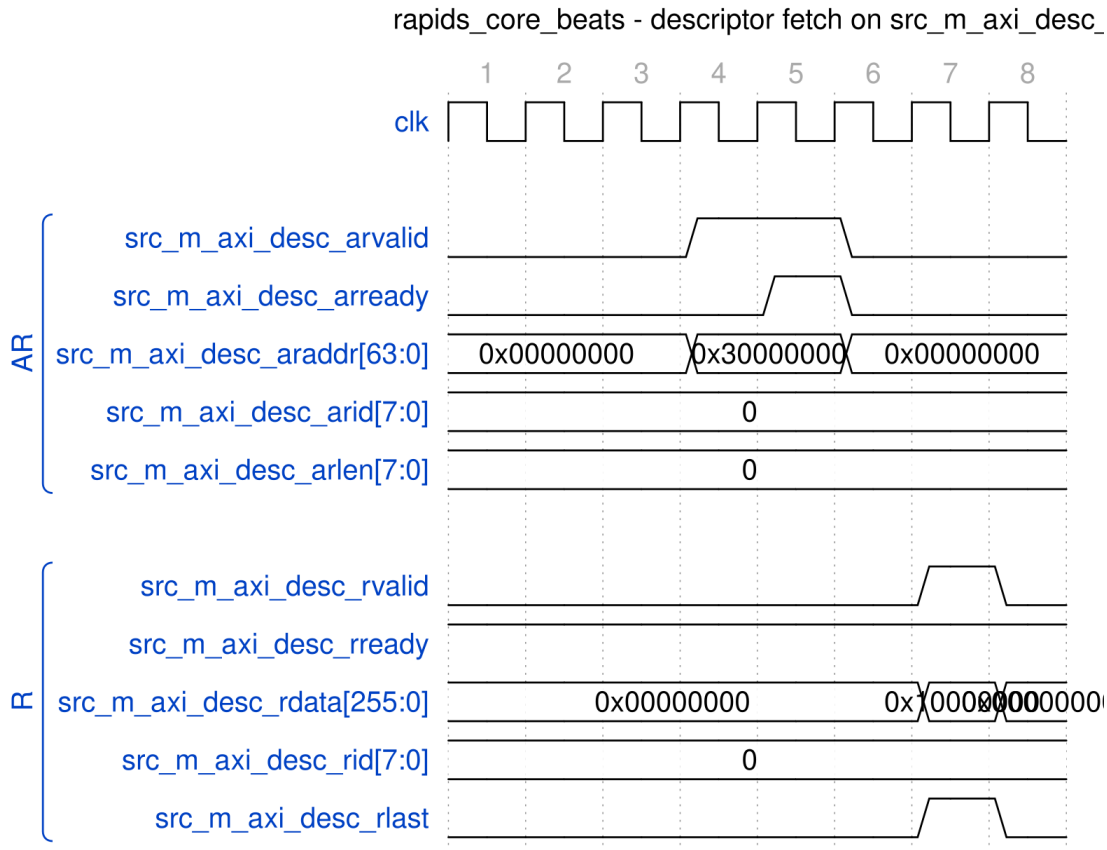
29.2.3 Transaction Characteristics

Table 4.1.2: Descriptor AXI Transaction Characteristics

Parameter	Value	Description
Data Width	256 bits	Fixed descriptor size
Burst Length	1 beat	Single descriptor per transaction
Burst Type	INCR	Incrementing burst
Max Outstanding	8	Configurable via parameter
ID Usage	Per-channel	ID encodes source channel

29.2.4 Timing Diagram

29.2.5 Figure 4.1.1: Descriptor Fetch Timing



*rapids_core_beats - descriptor fetch on src_m_axi_desc_**

Source: axi4_descriptor_fetch.json, captured from `dv/tests/top_beats/test_rapids_core_beats.py` (source path, channel 0, 4 beats, 512-bit data, `REG_LEVEL=GATE`) with `WAVES=1`.

Reading it: one single-beat read (`arlen = 0`) of the 256-bit descriptor at 0x30000000 with `arid` = channel 0. The AR handshakes when the slave raises `arready` two cycles after `arvalid`; the R beat returns two cycles after that with `rlast` set and `rid` echoing the channel. The descriptor itself is the 256-bit `rdata` (its low word, 0x10000000, is the source address).

29.3 Sink AXI Write Master Interface

29.3.1 Purpose

Writes sink data from SRAM to system memory. Supports burst transactions for efficient memory bandwidth.

29.3.2 Signal Table

Table 4.1.3: Sink AXI Write Master Interface Signals

Signal	Direction	Width	Description
m_axi_awvalid	output	1	AW channel valid
m_axi_awready	input	1	AW channel ready
m_axi_awaddr	output	AW	Write address
m_axi_awid	output	ID_W	Transaction ID
m_axi_awlen	output	8	Burst length (0-255)
m_axi_awsize	output	3	Burst size
m_axi_awburst	output	2	Burst type
m_axi_wvalid	output	1	W channel valid
m_axi_wready	input	1	W channel ready
m_axi_wdata	output	DW	Write data
m_axi_wstrb	output	DW/8	Write strobes
m_axi_wlast	output	1	Last beat
m_axi_bvalid	input	1	B channel valid
m_axi_bready	output	1	B channel ready
m_axi_bid	input	ID_W	Response ID
m_axi_bresp	input	2	Write response

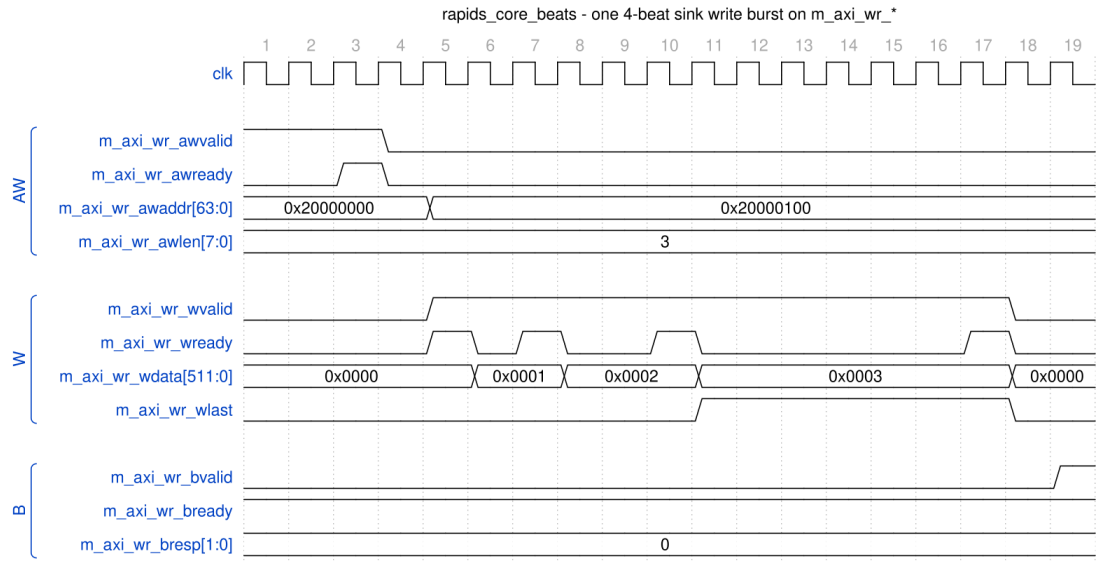
29.3.3 Transaction Characteristics

Table 4.1.4: Sink AXI Write Transaction Characteristics

Parameter	Value	Description
Data Width	512 bits	Configurable
Burst Length	1-256 beats	Based on transfer size
Burst Type	INCR	Incrementing burst
Max Outstanding AW	8	Configurable
W FIFO Depth	64	Configurable
B FIFO Depth	16	Configurable

29.3.4 Timing Diagram

29.3.5 Figure 4.1.2: Sink AXI Write Burst Timing



*rapids_core_beats - one 4-beat sink write burst on m_axi_wr_**

Source: axi4_sink_write_burst.json, captured from `dv/tests/top_beats/test_rapids_core_beats.py` (sink path, channel 0, 4 beats, 512-bit data, `REG_LEVEL=GATE`) with `WAVES=1`; the memory is the framework AXI4 write slave with its default pacing.

Reading it: the AW (`awlen = 3`, address 0x20000000) had been held for the slave and handshakes at cycle 3; `awaddr` steps to the next burst's 0x20000100 immediately. W starts two cycles later and

the four beats go out on the slave's `wready` pulses, the low word of `wdata` counting 0 to 3 and `wlast` on the fourth; `bvalid` (OKAY) comes one cycle after the last W handshake.

29.4 Source AXI Read Master Interface

29.4.1 Purpose

Reads source data from system memory into SRAM. Supports burst transactions for efficient memory bandwidth.

29.4.2 Signal Table

Table 4.1.5: Source AXI Read Master Interface Signals

Signal	Direction	Width	Description
`m_axi_arvalid`	output	1	AR channel valid
`m_axi_arready`	input	1	AR channel ready
`m_axi_araddr`	output	AW	Read address
`m_axi_arid`	output	ID_W	Transaction ID
`m_axi_arlen`	output	8	Burst length (0-255)
`m_axi_arsize`	output	3	Burst size
`m_axi_arburst`	output	2	Burst type
`m_axi_rvalid`	input	1	R channel valid
`m_axi_rready`	output	1	R channel ready
`m_axi_rdata`	input	DW	Read data
`m_axi_rid`	input	ID_W	Response ID
`m_axi_rresp`	input	2	Read response
`m_axi_rlast`	input	1	Last beat

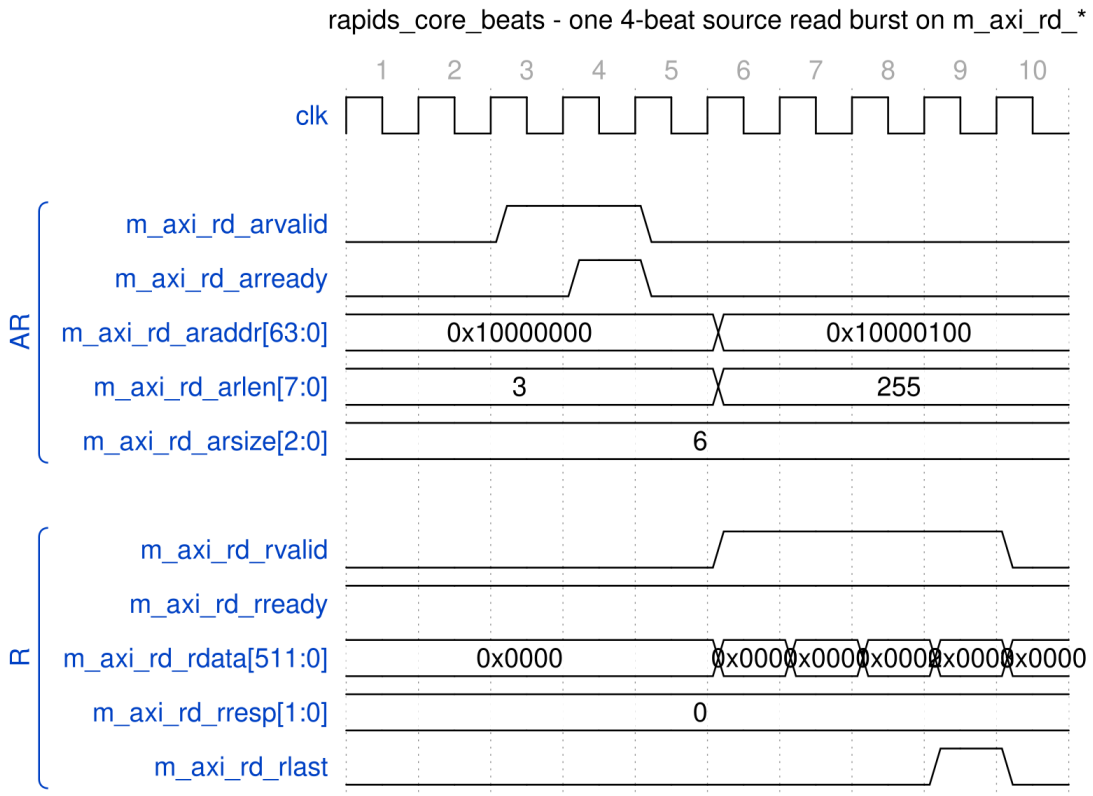
29.4.3 Transaction Characteristics

Table 4.1.6: Source AXI Read Transaction Characteristics

Parameter	Value	Description
Data Width	512 bits	Configurable
Burst Length	1-256 beats	Based on transfer size
Burst Type	INCR	Incrementing burst
Max Outstanding AR	8	Configurable
R FIFO Depth	64	Configurable

29.4.4 Timing Diagram

29.4.5 Figure 4.1.3: Source AXI Read Burst Timing



*rapids_core_beats - one 4-beat source read burst on m_axi_rd_**

Source: axi4_source_read_burst.json, captured from `dv/tests/top_beats/test_rapids_core_beats.py` (source path, channel 0, 4 beats, 512-bit data, `REG_LEVEL=GATE`) with `WAVES=1`.

Reading it: AR for four 64-byte beats (`arlen = 3`, `arsize = 6`) at 0x10000000 handshakes at cycle 4; the four R beats follow back to back from cycle 6 with `rready` held high, `rresp` OKAY and `rlast` on the fourth. `araddr` and `arlen` return to their idle values (the next address, 0x10000100, and 0xff) once the AR is accepted.

29.5 AXI Protocol Compliance

29.5.1 Supported Features

Table 4.1.7: AXI Feature Support

Feature	Descriptor	Sink Write	Source Read
Burst Type INCR	Yes	Yes	Yes
Burst Type FIXED	No	No	No
Burst Type WRAP	No	No	No
Exclusive Access	No	No	No
Locked Access	No	No	No
Unaligned Access	No	Yes	Yes
Narrow Transfers	No	Yes	Yes

29.5.2 Error Handling

- **SLVERR:** Transaction aborted, error reported to scheduler
- **DECERR:** Transaction aborted, error reported to scheduler
- **Timeout:** Configurable watchdog, error reported to scheduler

---

Last Updated: 2025-01-10

30 AXI-Stream Interface Specification

Last Updated: 2025-01-10

30.1 Overview

The RAPIDS Beats architecture provides optional AXI-Stream (AXIS) wrappers for network interfaces: 1. **AXIS Sink Slave:** Receives network data for memory writes 2. **AXIS Source Master:** Transmits network data from memory reads

These wrappers are available when `ENABLE_AXIS_WRAPPERS = 1`.

30.2 AXIS Sink Slave Interface

30.2.1 Purpose

Receives streaming network data and routes to per-channel SRAM buffers based on TID.

30.2.2 Signal Table

Table 4.2.1: AXIS Sink Slave Interface Signals

Signal	Direction	Width	Description
`s_axis_tvalid`	input	1	Data valid
`s_axis_tready`	output	1	Ready to accept data
`s_axis_tdata`	input	DATA_WIDTH	Data payload
`s_axis_tstrb`	input	DATA_WIDTH/8	Byte enables (not used by the beats design)
`s_axis_tlast`	input	1	Last beat of packet
`s_axis_tid`	input	TID_WIDTH	Stream ID (channel select)
`s_axis_tdest`	input	TDEST_WIDTH	Destination routing
`s_axis_tuser`	input	TUSER_WIDTH	User sideband

30.2.3 Signal Details

30.2.3.1 TID (Transaction ID)

TID is used for per-channel routing:

Table 4.2.2: TID Channel Mapping

TID Value	Destination
0	Channel 0 SRAM
1	Channel 1 SRAM
2	Channel 2 SRAM
3	Channel 3 SRAM
4	Channel 4 SRAM
5	Channel 5 SRAM
6	Channel 6 SRAM
7	Channel 7 SRAM

30.2.3.2 TSTRB (Byte Enables)

The ports are `s_axis_tstrb` / `m_axis_tstrb` (there is no `tkeep`). In the beats design they are not load-bearing: the source drives all ones and the sink ignores the ingress strobes, so every beat carries DATA_WIDTH/8 valid bytes. The byte-granular RAPIDS (`rtl/top/rapids_top.sv`, rapids TASK-019) packs bytes from lane 0 and uses a contiguous `tstrb` on the last beat of a packet.

Table 4.2.3: TSTRB Configuration

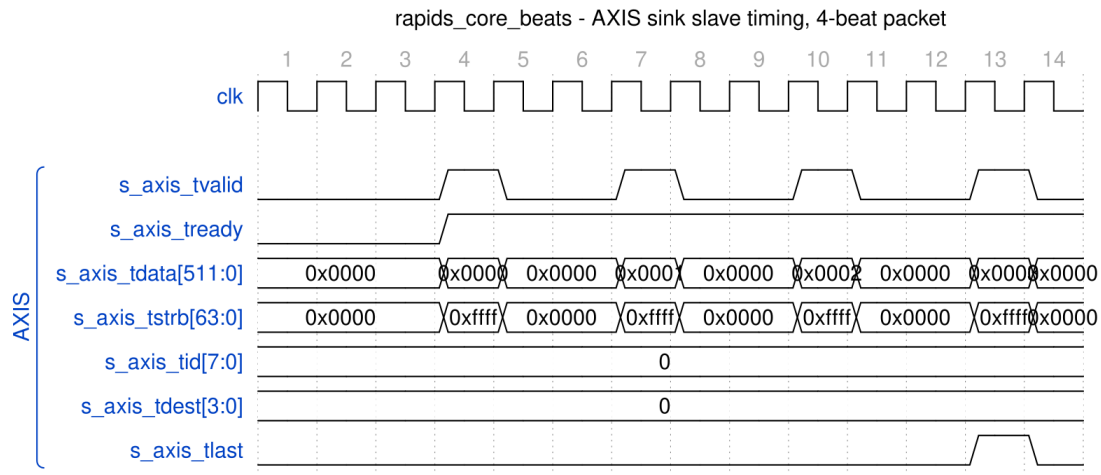
DATA_WIDTH	TSTRB Width	Beats design	Byte-granular RAPIDS
512 bits	64 bits	all ones	contiguous from lane 0, partial on the last beat
256 bits	32 bits	all ones	contiguous from lane 0, partial on the last beat

30.2.3.3 TLAST (Packet Boundary)

TLAST marks the end of an AXIS packet. This is mapped to the internal `fill_last` signal for packet boundary tracking.

30.2.4 Timing Diagram

30.2.5 Figure 4.2.1: AXIS Sink Slave Timing



rapids_core_beats - AXIS sink slave timing, 4-beat packet

Source: axis_sink_slave.json, captured from `dv/tests/top_beats/test_rapids_core_beats.py` (sink path, channel 0, 4 beats, 512-bit data, `REG_LEVEL=GATE`) with `WAVES=1`; the producer is the framework AXIS master.

Reading it: `s_axis_tready` is high from the moment the sink has space, so each beat is accepted the cycle it is offered; the producer's pacing leaves two idle cycles between beats. `tstrb` is all ones on every beat, `tid` selects channel 0 and `tlast` marks the fourth beat as the end of the packet.

30.2.6 Backpressure

The sink interface asserts backpressure (`s_axis_tready = 0`) when: - Target channel SRAM is full - No space allocated for incoming data - Internal pipeline stalls

---

30.3 AXIS Source Master Interface

30.3.1 Purpose

Transmits streaming network data from per-channel SRAM buffers with TID indicating source channel.

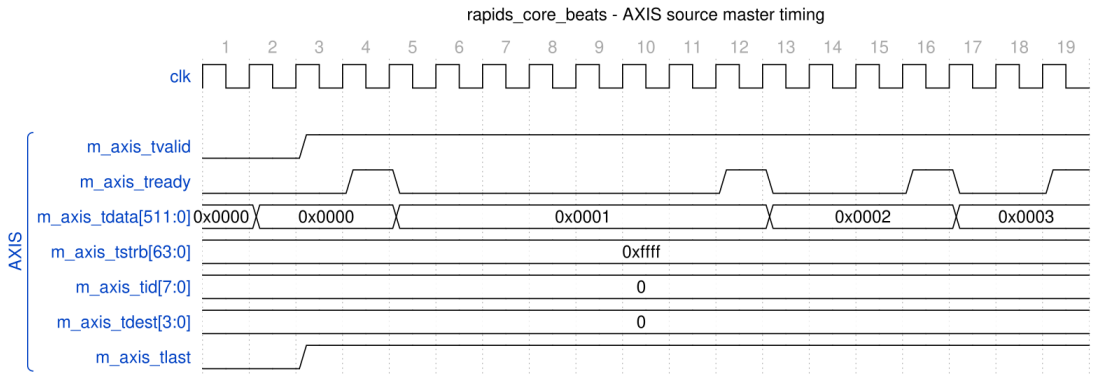
30.3.2 Signal Table

Table 4.2.4: AXIS Source Master Interface Signals

Signal	Direction	Width	Description
m_axis_tvalid	output	1	Data valid
m_axis_tready	input	1	Ready to accept data
m_axis_tdata	output	DATA_WIDTH	Data payload
m_axis_tstrb	output	DATA_WIDTH/8	Byte enables (all ones in the beats design)
m_axis_tlast	output	1	Last beat of packet
m_axis_tid	output	TID_WIDTH	Stream ID (source channel)
m_axis_tdest	output	TDEST_WIDTH	Destination routing
m_axis_tuser	output	TUSER_WIDTH	User sideband

30.3.3 Timing Diagram

30.3.4 Figure 4.2.2: AXIS Source Master Timing



rapids_core_beats - AXIS source master timing

Source: axis_source_master.json, captured from `dv/tests/top_beats/test_rapids_core_beats.py` (source path, channel 0, 4 beats, 512-bit data, `REG_LEVEL=GATE`) with `WAVES=1`; the consumer is the framework AXIS slave with its default ready pacing.

Reading it: the master raises `m_axis_tvalid` and holds it, with `tdata`, `tstrb` (all ones), `tid` and `tdest` stable, until the consumer's `tready` pulse takes the beat; it then presents the next word without dropping `tvalid`. Four beats leave on four `tready` pulses. `tlast` is high on every beat of this capture because the source packetises per drain request and each request carried one word (see Figure 3.7.3); with `cfg_drain_size` larger than one and data queued, a packet spans several beats and `tlast` marks only its final one.

30.3.5 Flow Control

The source interface respects network backpressure: - Data held when `m_axis_tready = 0` - No data loss on backpressure - SRAM drain pauses until ready

---

30.4 AXIS vs Custom Fill/Drain Interfaces

30.4.1 Comparison

Table 4.2.5: Interface Comparison

Aspect	Custom Fill/Drain	AXIS
Standard	Proprietary	AXI-Stream
Channel ID	fill_id / drain_id	TID
Packet Boundary	fill_last / drain_last	TLAST
Byte Enables	fill_strb / drain_strb	TSTRB
Interoperability	RAPIDS only	Industry standard

30.4.2 Selection Guide

Table 4.2.6: Interface Selection Guide

Use Case	Recommended Interface
Internal RAPIDS testing	Custom Fill/Drain
Network IP integration	AXIS
FPGA board-level design	AXIS
Maximum performance	Custom Fill/Drain

30.5 Configuration Parameters

```
// AXIS Interface Parameters
parameter int DATA_WIDTH = 512;        // Data payload width
parameter int TID_WIDTH = 3;           // Stream ID width (log2
```

```
channels)
parameter int TDEST_WIDTH = 1;        // Destination width
parameter int TUSER_WIDTH = 1;        // User sideband width
```

Last Updated: 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

31 MonBus Interface Specification

Last Updated: 2025-01-10

31.1 Overview

The MonBus (Monitor Bus) is a 64-bit packet-based interface for reporting internal events, status, and performance data. All RAPIDS subsystems generate MonBus packets that are aggregated and output through a unified interface.

31.2 MonBus Signal Interface

31.2.1 Signal Table

Table 4.3.1: MonBus Interface Signals

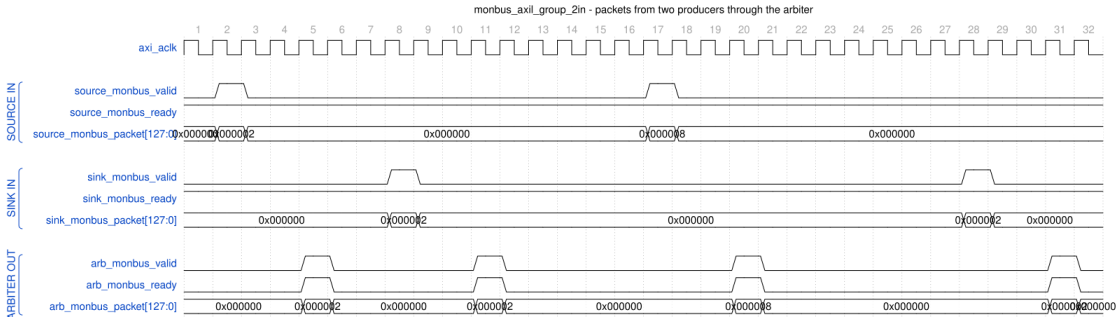
Signal	Direction	Width	Description
`monbus_pkt_valid`	output	1	Packet valid
`monbus_pkt_ready`	input	1	Consumer ready
`monbus_pkt_data`	output	64	Packet data

31.2.2 Handshaking Protocol

Standard valid/ready handshaking: - Data transfers when `valid & ready` both high - Producer holds `valid` until `ready` seen - Consumer asserts `ready` when able to accept

31.2.3 Timing Diagram

31.2.4 Figure 4.3.1: MonBus Packet Transfer Timing



monbus_axil_group_2in - packets from two producers through the arbiter

Source: monbus_group_packet_transfer.json, captured from `dv/tests/macro/test_monbus_axil_group.py` (`basic_flow`, error FIFO 64 x 32, `TEST_LEVEL=full`) with `WAVES=1`. The 128-bit packets are shown as their top 24 bits (`[127:104]`: packet type nibble, then the protocol field at `[108:105]`, so `0x000002` is AXIS and `0x000008` is CORE).

Reading it: a MonBus link is a single-beat valid/ready handshake with the packet presented alongside `valid`. The group holds `ready` high on both inputs, so the source and sink packets are accepted the cycle they are offered, and each one reappears on the arbiter output three cycles later as another one-cycle `valid`/`ready` pair. Between packets the links idle low; nothing is held across transfers.

31.3 MonBus Packet Format

31.3.1 64-bit Packet Structure

```
63      56 55      48 47      40 39      32 31               0
+--------+---------+---------+---------+-------------------+
| Type   | Unit ID | Agent ID| Channel | Payload (32 bits) |
+--------+---------+---------+---------+-------------------+
```

31.3.2 Figure 4.3.2: MonBus Packet Format

Table 4.3.2: MonBus Packet Fields

Field	Bits	Description
Type	[63:56]	Packet type code
Unit ID	[55:48]	Reporting unit identifier

Field	Bits	Description
Agent ID	[47:40]	Source agent within unit
Channel	[39:32]	Channel number (0-7)
Payload	[31:0]	Type-specific payload

31.4 Packet Types

31.4.1 Type Code Definitions

Table 4.3.3: MonBus Packet Types

Type Code	Name	Description
0x01	DESC_FETCH	Descriptor fetch started
0x02	DESC_COMPLETE	Descriptor fetch completed
0x03	SCHED_START	Scheduler started transfer
0x04	SCHED_DONE	Scheduler transfer complete
0x10	AXI_RD_START	AXI read burst started
0x11	AXI_RD_DONE	AXI read burst completed
0x12	AXI_WR_START	AXI write burst started
0x13	AXI_WR_DONE	AXI write burst completed
0x20	SRAM_FILL	SRAM fill event
0x21	SRAM_DRAIN	SRAM drain event
0x30	ERROR	Error event
0x31	TIMEOUT	Timeout event

Type Code	Name	Description
0xFF	HEARTBEAT	Periodic heartbeat

31.4.2 Payload Formats by Type

31.4.2.1 DESC_FETCH / DESC_COMPLETE (0x01, 0x02)

Table 4.3.4: Descriptor Packet Payload

Bits	Description
[31:16]	Descriptor address [47:32]
[15:0]	Descriptor address [31:16]

31.4.2.2 SCHED_START / SCHED_DONE (0x03, 0x04)

Table 4.3.5: Scheduler Packet Payload

Bits	Description
[31:16]	Transfer ID
[15:0]	Beat count

31.4.2.3 AXI_RD/WR_START/DONE (0x10-0x13)

Table 4.3.6: AXI Event Packet Payload

Bits	Description
[31:24]	AXI ID
[23:16]	Burst length
[15:0]	Reserved

31.4.2.4 ERROR (0x30)

Table 4.3.7: Error Packet Payload

Bits	Description
[31:24]	Error code
[23:16]	Error source
[15:0]	Error details

31.5 MonBus Source Aggregation

31.5.1 RAPIDS Core MonBus Sources

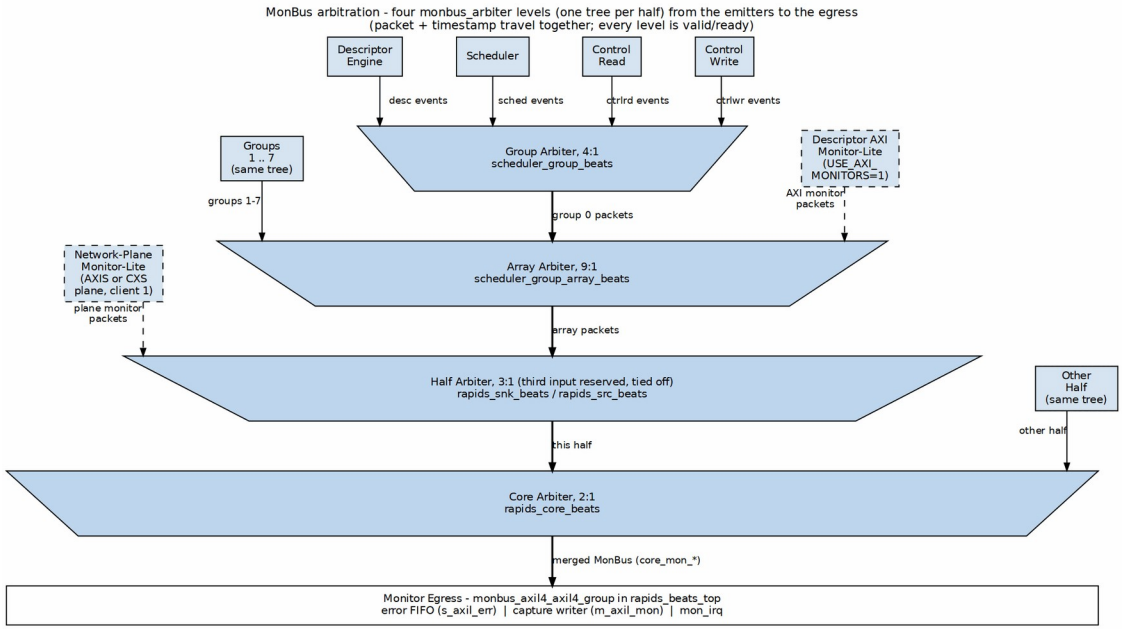
Table 4.3.8: MonBus Source Assignment

Source ID	Agent	Description
0-7	Scheduler[0:7]	Per-channel scheduler events
8-15	DescEngine[0:7]	Per-channel descriptor events
16-23	SnkSRAM[0:7]	Sink SRAM controller events
24-31	SrcSRAM[0:7]	Source SRAM controller events
32	AXI Write Engine	Write transaction events
33	AXI Read Engine	Read transaction events
34	Error Handler	Error events

Scheduler and DescEngine COMPLETION packets are subject to the half's `SCHED_CONFIG.COMPL_EN` (reset 1): the scheduler group drops them ahead of its arbiter when the bit is clear and acknowledges the emitter, so the data path is unaffected. Their ERROR packets are always forwarded (rapids ISSUE-005).

31.5.2 Aggregation Hierarchy

31.5.3 Figure 4.3.3: MonBus Aggregation Tree



MonBus aggregation tree

Source: 21_monbus_aggregation.dot

31.6 Integration Guidelines

31.6.1 Consumer Requirements

MonBus consumers must: 1. Provide sufficient ready cycles to prevent packet loss 2. Handle bursty traffic (multiple sources may have packets simultaneously) 3. Process or buffer packets at wire rate

31.6.2 Recommended Consumer Patterns

```
// Pattern 1: Direct FIFO buffering
gaxi_fifo_sync #(
    .DATA_WIDTH(64),
    .DEPTH(256)
) u_gaxi_fifo_sync (
    .axi_aclk               (axi_aclk),
    .axi_aresetn            (axi_aresetn),
    .wr_valid               (wr_valid),
    .wr_ready               (wr_ready),
```

```
    .wr_data                (wr_data),
    .rd_ready               (rd_ready),
    .count                  (count),
    .rd_valid               (rd_valid),
    .rd_data                (rd_data)
);
```

31.7 Timing Requirements

Table 4.3.9: MonBus Timing Parameters

Parameter	Value	Description
Min Ready	1 cycle	Must assert ready within N cycles
Max Latency	Configurable	Before packet dropped
Packet Rate	Variable	Depends on activity

Last Updated: 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

32 CXS Interface Specification

Applies to: `rapids_beats_cxs_top`, `rapids_cxs_top` **Protocol:** Arm IHI 0079 (CXS) Issue D, one fixed profile **Last Updated:** 2026-10-16

32.1 Overview

The CXS tops replace the two AXIS network ports with one CXS link per direction: `s_cxs_*` (a CXS receiver) into the sink half and `m_cxs_*` (a CXS transmitter) out of the source half. One link carries every channel; the channel rides in `CXSSRCID`. The planes that implement the links are described in the Network Planes chapter (Section 3.13).

32.2 Link Profile

Table 4.4.1: CXS Link Profile

Property	Value
Issue	D (the issue with SRCID, TGTID and USER)
`CXSMAXPKTPERFLIT`	2
Packet start alignment	16 bytes; RAPIDS always starts a packet at byte 0
Packet mapping	One AXIS packet (`tlast`-delimited) is one CXS packet
`CXSSRCID`	The channel (AXIS `tid`)
`CXSTGTID`	AXIS `tdest`
`CXSUSER`	`{tuser, tstrb}`; `tuser` dropped when `DATA_WIDTH/8 + AXIS_USER_WIDTH > 128`
Stream framing	Per stream on the channel bits of `CXSSRCID`
Link control	Explicit credit return, activation, deactivation
Parity	Off

32.3 Signal Table

Table 4.4.2: CXS Interface Signals

Signal	Sink link	Source link	Width	Description
{s,m}_cxs_valid	in	out	1	Flit valid
{s,m}_cxs_data	in	out	DATA_WIDTH	Flit data
{s,m}_cxs_cntl	in	out	CXS_CNTL_PW	START / END / pointers
{s,m}_cxs_srcid	in	out	AXIS_ID_WIDTH	Channel
{s,m}_cxs_tgtid	in	out	AXIS_DEST_WIDTH	tdest
{s,m}_cxs_user	in	out	CXS_UW	{tuser, tstrb}
{s,m}_cxs_crdgnt	out	in	1	Credit grant (receiver to transmitter)
{s,m}_cxs_crdrtn	in	out	1	Credit return (transmitter to receiver)
{s,m}_cxs_activereq	in	out	1	Transmitter requests the link
{s,m}_cxs_activeack	out	in	1	Receiver acknowledges
{s,m}_cxs_deacthint	out	in	1	Receiver asks the transmitter to deactivate
snk_cxs_link_state, src_cxs_link_state	out	out	2	{ACTIVEREQ, ACTIVEACK}
snk_cxs_protocol_error,	out	out	1	Sticky protocol violation

Signal	Sink link	Source link	Width	Description
src_cxs_protocol_error				
snk_cxs_unsupported	out	-	1	Sticky: a flit AXIS framing cannot carry

32.4 Credit Flow

One credit permits one flit (IHI 0079D 2.1). The receiver grants up to CXS_MAX_CREDIT (default 32) credits, one per free entry in its receive FIFO, so every flit sent against a credit has space. The transmitter spends one credit per flit and returns the credits it still holds on deactivation. Credits are a single pool for the link; a grant carries no ID and cannot be reserved for a channel.

The grant -> flit -> free round trip between the RAPIDS endpoints is about 3 cycles, so 32 credits sustains one flit per cycle with margin for pipeline stages or a clock crossing on the link. More than 63 credits is beyond the spec and requires CXS_CREDIT_BEYOND_SPEC on both ends.

32.5 Link Activation

Table 4.4.3: Link Activation Sequence

Step	Transmitter (m_cxs_*)	Receiver (s_cxs_*)
1	Raises ACTIVEREQ when it has a flit to send (and only once the previous activation's ACTIVEACK has fallen: 4-phase)	-
2	-	Raises ACTIVEACK; starts granting credits
3	Sends flits, one credit each	Frees an entry and regrants as the core drains
4	After CXS_IDLE_CYCLES idle cycles in RUN (or on DEACTHINT) drops ACTIVEREQ, then returns every held credit on CRDRTN, one per cycle	-
5	-	Drops ACTIVEACK once its credits are back

32.6 Backpressure

On the sink link, a full receive FIFO means no further grants: the transmitter stalls for lack of credit. This is the CXS form of the AXIS sink's `s_axis_tready = 0`, including the byte-granular sink's hold until a channel's descriptor has been fetched. On the source link, the plane stalls the core's egress (`fub_axis_tready = 0`) while it has no credit.

32.7 Planned Changes

The `CXSSRCID` = channel and `CXSTGTID` = `tdest` mapping is provisional. The ID fields, and possibly other fields or new signals, will change to carry routing and payload type (delta configuration or memory traffic, as distinct from endpoint or cell configuration or memory traffic). The encoding is not yet defined.

33 Product Requirements Document (PRD)

33.1 RAPIDS - Rapid AXI Programmable In-band Descriptor System

Version: 1.0 **Date:** 2025-09-30 **Status:** Active Development - Validation in Progress **Owner:** RTL Design Sherpa Project **Parent Document:** `/PRD.md`

33.2 1. Executive Summary

The Rapid AXI Programmable In-band Descriptor System (RAPIDS) is a custom hardware accelerator designed for efficient memory-to-memory data movement with network interface integration. It demonstrates complex FSM coordination, descriptor-based DMA operations, and comprehensive monitoring capabilities.

> **Status (2026-07-22):** RAPIDS was rearchitected to the "beats" design. Current RTL is in `rtl/fub_beats/`, `rtl/macro_beats/`, `rtl/top_beats/` (control engines in `rtl/fub/`); current specs are `docs/rapids_beats_has/` and `docs/rapids_beats_mas/`. The old `rapids_fub//rapids_macro/` RTL, the `docs/rapids_spec/` tree, and the program engine / network master / network slave blocks described in parts of this PRD were retired (network interfaces are now AXIS; control is APB). Sections describing the pre-beats design are kept as history and marked where practical.
>
> **Status (2026-09-29, rapids TASK-019):** RAPIDS is byte-granular. The product RTL is the un-suffixed tree `rtl/fub/`, `rtl/macro/`, `rtl/top/rapids_top.sv`: descriptor length in BYTES, byte-granular addresses, WSTRB/TSTRB carry partial beats, one AXIS packet per descriptor. The `*_beats` tree is the beat-granular stepping stone, kept runnable and green as the characterized Genesys 2 design. Tests: `dv/tests/fub`, `macro`, `top`.

33.2.1 1.1 Quick Stats

- **Modules:** ~20 SystemVerilog files (beats architecture)
- **Architecture:** 3 major groups (Scheduler group, Sink path, Source path)
- **Interfaces:** AXI4 (memory), AXIS (network), APB (control), MonBus (monitoring)
- **Test Coverage:** see `dv/tests/` beats suites (fub_beats/macro_beats/top_beats)

- **Status:** Active validation, known issues documented

33.2.2 1.2 Subsystem Goals

- **Primary:** Demonstrate complex accelerator design patterns
- **Secondary:** Provide DMA-style memory transfer capability
- **Tertiary:** Educational reference for descriptor-based engines

---

33.3 2. Documentation Structure

This PRD provides a high-level overview. **Detailed specifications are maintained separately:**

33.3.1 Complete RAPIDS Specification

> **Status (2026-07-22):** The old `docs/rapids_spec/` tree was replaced by two beats spec trees.

Byte-granular RAPIDS books (the product design): HAS Index, MAS Index. Chapters that are identical for both designs are linked from the Beats books below.

Architecture Spec (HAS), RAPIDS Beats: `docs/rapids_beats_has/`

- **HAS Index** - Complete HAS structure
- Product Overview
- Block Diagram / Data Flow / Channel Architecture
- Interfaces: Summary, AXI4, AXIS, APB, MonBus
- Use cases: `docs/rapids_beats_has/ch04_use_cases/`
- Programming: Descriptor Format, Register Map, Initialization, Error Handling
- Performance: `docs/rapids_beats_has/ch06_performance/`

Micro-Architecture Spec (MAS): `docs/rapids_beats_mas/`

- **MAS Index** - Complete MAS structure
- Overview: Architecture, Port List, Clocks and Reset

FUB blocks (`docs/rapids_beats_mas/ch02_fub_blocks/`): - Scheduler - Per-channel scheduler FSM - Descriptor Engine - Descriptor fetch/parsing - AXI Read Engine / AXI Write Engine - Alloc Ctrl / Drain Ctrl / Latency Bridge - CtrlRd Engine / CtrlWr Engine

Macro blocks (`docs/rapids_beats_mas/ch03_macro_blocks/`): - Scheduler Group / Scheduler Group Array - Sink Data Path + AXIS wrapper, SRAM controllers - Source Data Path + AXIS wrapper, SRAM controllers - RAPIDS Core / Registers / Top

Interfaces (`docs/rapids_beats_mas/ch04_interfaces/`): - AXI4 Interface - AXIS Interface - MonBus Interface

33.3.2 Known Issues

Location: `projects/components/dma-ip/rapids/known_issues/`

- **Index** - Issue tracking overview (the old scheduler.md credit-counter write-up was retired with the pre-beats RTL)
- **Sink Data Path** - RESOLVED 2026-09-25 (write-error flag wired; timeouts are the monitor's job by design)
- **Sink SRAM Control** - RESOLVED (moved to resolved/ in d587852c2)
- Plus `known_issues/active/` for current beats issues

33.3.3 Other Documentation

- **CLAUDE** - AI assistance guide for this subsystem
- **TASKS** - Work items (largely pre-beats history)
- **Validation Report** - Test results (pre-beats snapshot)

33.4 2.4 Organizational Standards - RAPIDS Code Location

MANDATORY: All RAPIDS-specific code must be in the project area

33.4.1 Code Organization Principle

"All RAPIDS-specific verification code MUST reside in `projects/components/dma-ip/rapids/dv/` for easy discovery."

This subsystem follows the repository-wide organizational standard (see `/PRD.md` Section 2.3) requiring all project-specific code to be located in the project area, NOT the framework area.

33.4.2 RAPIDS Directory Structure

```
projects/components/dma-ip/rapids/
├── rtl/                              # RTL source code
│   ├── includes/                     # RAPIDS packages
│   ├── fub/                          # Control engines (ctrlrd/ctrlwr)
│   ├── fub_beats/                    # Beats functional unit blocks
│   ├── macro_beats/                  # Beats assemblies + registers
```

```
│   ├── macro/                    # MonBus group
│   └── top_beats/                # rapids_beats_top.sv
│
└── dv/                           # Design verification (all RAPIDS-
specific)
    ├── tbclasses/                # RAPIDS TB classes HERE (not
framework!)
    │   ├── scheduler_tb.py       # Scheduler testbench class
    │   ├── descriptor_engine_tb.py
    │   └── rapids_core_beats_tb.py
    │
    ├── components/               # RAPIDS-specific BFMs
    │   └── data_mover_bfm.py
    │
    └── tests/                    # Test runners (import TB classes)
        ├── fub/                  # Control engine tests
        ├── fub_beats/            # Beats FUB tests
(test_scheduler_beats.py, ...)
        ├── macro/                # MonBus group test
        ├── macro_beats/          # Multi-block scenarios
        └── top_beats/            # Full RAPIDS operation
```

33.4.3 What Goes Where?

Code Type	CORRECT Location	WRONG Location
RAPIDS TB Classes	`projects/components/dma-ip/rapids/dv/tbclasses/`	`bin/TBClasses/` (framework area)
RAPIDS-Specific BFMs	`projects/components/dma-ip/rapids/dv/components/`	`bin/TBClasses/` (framework area)
RAPIDS Scoreboards	`projects/components/dma-ip/rapids/dv/` (project area)	`bin/TBClasses/scoreboards/`
Test Runners	`projects/components/dma-ip/rapids/dv/tests/`	Anywhere else
Shared AXI4/APB BFMs	`bin/TBClasses/{protocol}/`	Project area

33.4.4 Import Pattern for RAPIDS Tests

CORRECT - Import from Project Area:

```python
# Import framework utilities (PYTHONPATH includes bin/)
import os, sys
from TBClasses.shared.utilities import get_repo_root
from TBClasses.shared.tbbase import TBBase
```

```
# Add repo root to Python path using robust git-based method
repo_root = get_repo_root()
sys.path.insert(0, repo_root)

# Import RAPIDS TB classes from PROJECT AREA
from projects.components.dma_ip.rapids.dv.tbclasses.scheduler_tb
import SchedulerTB
from
projects.components.dma_ip.rapids.dv.tbclasses.descriptor_engine_tb
import DescriptorEngineTB

# Shared framework components
from CocoTBFramework.components.axi4.axi4_master import AXI4Master
```

WRONG - Don't Import from Framework:

```
# DON'T DO THIS!
from TBClasses.rapids.scheduler_tb import SchedulerTB  # WRONG!
```

33.4.5 Benefits of This Organization

1. **Easy Discovery** - All RAPIDS code in ONE place: `projects/components/dma-ip/rapids/`
2. **Clear Ownership** - RAPIDS team owns their `dv/` area completely
3. **No Confusion** - Never wonder "where does this TB class live?"
4. **Maintainability** - Changes isolated to RAPIDS area don't affect other projects
5. **Framework Stays Clean** - Only truly shared cross-project code in framework

33.4.6 Compliance Status

RAPIDS is now compliant - All TB classes moved to project area as of 2025-10-18

Migration History: - **Before:** TB classes incorrectly in `bin/TBClasses/rapids/` - **After:** TB classes correctly in `projects/components/dma-ip/rapids/dv/tbclasses/` - **Test Imports:** Updated to import from project area

Complete Documentation: See `/PRD.md` Section 2.3 for repository-wide organizational standards.

33.5 3. Architecture Overview

33.5.1 3.1 Top-Level Block Diagram

```
RAPIDS Beats (Rapid AXI Programmable In-band Descriptor System)
├── Scheduler Group (scheduler_group_beats /
scheduler_group_array_beats)
│   ├── Scheduler          (Per-channel FSM, scheduler_beats.sv)
│   ├── Descriptor Engine  (Descriptor fetch/parsing,
descriptor_engine_beats.sv)
│   └── Control Engines    (ctrlrd_engine.sv / ctrlwr_engine.sv)
│
├── Sink Data Path (AXIS Network → SRAM → System Memory)
│   ├── AXIS Slave ingress   (snk_data_path_axis_beats.sv)
│   ├── Sink SRAM Controller (snk_sram_controller_beats.sv +
alloc/drain ctrl)
│   └── AXI Write Engine     (axi_write_engine_beats.sv)
│
├── Source Data Path (System Memory → SRAM → AXIS Network)
│   ├── AXI Read Engine        (axi_read_engine_beats.sv)
│   ├── Source SRAM Controller (src_sram_controller_beats.sv +
alloc/drain ctrl)
│   └── AXIS Master egress     (src_data_path_axis_beats.sv)
│
└── Control and Monitoring
    ├── APB CSRs           (rapids_config_block.sv, PeakRDL
rapids_regs.rdl)
    └── MonBus Group       (macro/monbus_axil_group_2in.sv)
```

See: `docs/rapids_beats_mas/ch01_overview/01_architecture.md` for detailed architecture

33.5.2 3.2 Data Flow

Sink Path (Receive): 1. Network packets arrive via Network Slave 2. Buffered in Sink SRAM 3. DMA'd to system memory via AXI4 Write Engine 4. Completion reported via MonBus

Source Path (Transmit): 1. Descriptor specifies data location in system memory 2. Source AXI Reader fetches data to Source SRAM 3. Network Master transmits to network 4. Completion reported via MonBus

Scheduler Coordination: - Parses descriptors from Descriptor Engine - Manages credit-based flow control - Sequences program engine operations - Coordinates sink/source data paths

---

33.6 4. Key Features

33.6.1 4.1 Descriptor-Based Operation

Feature	Status	Description
Descriptor FIFO	yes	Queued descriptor processing
Multi-field parsing	yes	Address, length, control fields
Chained descriptors	⏳	Future enhancement
Completion reporting	yes	Via MonBus packets

33.6.2 4.2 Data Path Features

Feature	Status	Description
SRAM buffering	yes	Decouple network from memory
AXI4 burst support	yes	Efficient memory transfers
Backpressure handling	yes	Flow control on all interfaces
Data alignment	yes	Handle unaligned transfers

33.6.3 4.3 Scheduler Features

Feature	Status	Description
Task FSM	yes	Multi-state coordination
Credit management	⏳	Pre-beats scheduler had exponential encoding (0→1, 1→2, ..., 15→∞); scheduler_beats.sv has no credit management yet (planned later phase)
Program sequencing	yes	Coordinated operations
Error detection	yes	Timeout, overflow detection

Credit Management Details (pre-beats, historical): - Uses **exponential credit encoding** for compact configuration - 4-bit `cfg_initial_credit` decodes to actual credit counts: - `0` → 1 credit

(2^0), 4 → 16 credits (2^4), 8 → 256 credits (2^8) - 15 → ∞ (unlimited credits, 0xFFFFFFFF) - Encoding applied at initialization; runtime operations are linear (increment/decrement by 1) - Provides wide range (1 to 16384) with minimal configuration overhead

See: `docs/rapids_beats_mas/ch02_fub_blocks/01_scheduler.md` for the current scheduler specification

33.6.4 4.4 Monitoring Integration

Feature	Status	Description
MonBus packets	yes	StandardAMBA 64-bit format
Descriptor events	yes	Start/complete reporting
Error events	yes	Timeout, overflow, underflow
Performance metrics	⏳	Future enhancement

33.7 5. Interfaces

33.7.1 5.1 External Interfaces

Interface	Type	Width	Purpose
APB	Slave	32-bit	Control/status registers
AXI4 (Sink)	Master	Configurable	Write to system memory
AXI4 (Source)	Master	Configurable	Read from system memory
AXIS (Sink)	Slave	Configurable	Network ingress (tid = channel)
AXIS (Source)	Master	Configurable	Network egress (tid = channel)
MonBus	Master	64-bit	Monitor packet output

See: `docs/rapids_beats_mas/ch04_interfaces/` and `docs/rapids_beats_has/ch03_interfaces/` for complete interface specs

33.7.25.2 Configuration Parameters

```
// Example RAPIDS instantiation sketch (see
rtl/top_beats/rapids_beats_top.sv for the
// full parameter/port list)
rapids_beats_top #(
    .ADDR_WIDTH(64),
    .DATA_WIDTH(512),
    .NUM_CHANNELS(8)
) u_rapids (
    .clk                (clk),
    .rst_n              (rst_n),
    // APB control interface
    .s_apb_*            (...),
    // AXI4 memory interfaces (read + write masters)
    .m_axi_*            (...),
    // AXIS network interfaces (tid = channel)
    .s_axis_*           (...),
    .m_axis_*           (...),
    // MonBus / monitor group egress
    .mon_*              (...)
);
```

33.8 6. Use Cases

33.8.16.1 DMA-Style Transfers

Scenario: Move data from network to system memory

Flow: 1. Software writes descriptor to Descriptor Engine 2. Scheduler parses descriptor, activates Sink path 3. Network packets arrive via Network Slave 4. Data buffered in Sink SRAM 5. AXI4 Write Engine DMAs to system memory 6. Completion packet on MonBus

33.8.26.2 Network Packet Processing

Scenario: Read data from memory, transmit to network

Flow: 1. Descriptor specifies source address, length 2. Source AXI Reader fetches data to SRAM 3. Network Master transmits to network 4. Completion/error reporting via MonBus

33.8.36.3 Custom Data Path Acceleration

Educational value: Shows how to build custom accelerators - Descriptor-based control - Multi-block FSM coordination - Buffering strategies - Error handling - Performance monitoring

---

33.9 7. Test Coverage

33.9.1 7.1 Current Status

> **Status (2026-07-22):** The numbers below are a pre-beats snapshot (they include the retired program engine). Current beats regression: fub 232+ and macro 182 tests passing; run the beats suites for live numbers.

Overall (pre-beats snapshot): ~85% functional coverage (basic scenarios validated, descriptor engine complete)

Component	Test Coverage	Status
Scheduler	~95%	Credit encoding fixed and verified (43/43 tests passing)
Descriptor Engine	100%	**All tests passing** (14/14 tests, 100% success rate)
Program Engine	~85%	Alignment tested
Sink Data Path	~75%	Basic flows working
Source Data Path	~70%	Basic flows working
SRAM Controllers	~80%	Buffer management tested
Integration	~60%	More stress testing needed

Test Location: `projects/components/dma-ip/rapids/dv/tests/fub_beats/` and `projects/components/dma-ip/rapids/dv/tests/macro_beats/` (plus `fub/`, `macro/`, `top_beats/`)

Recent Achievements: - **Descriptor Engine (2025-10-13):** Achieved 100% test pass rate using continuous background monitoring pattern - 14/14 tests passing across all test levels (basic, medium, full) - All test classes passing (APB_ONLY, MIXED) - All delay profiles passing (fast_producer, fast_consumer, fixed_delay, minimal_delay) - Applied continuous monitoring methodology for asynchronous output capture

See: `vault/Tasks/projects/components/dma-ip/rapids/RAPIDS_Validation_Status_Report.md` for detailed results

33.9.2 7.2 Test Strategy

FUB (Functional Unit Block) Tests: - Individual block testing - Located in `projects/components/dma-ip/rapids/dv/tests/fub_beats/` (+ `fub/` for control engines) - Focus on module-level functionality

Macro Tests: - Multi-block scenarios - Located in `projects/components/dma-ip/rapids/dv/tests/macro_beats/` (+ `macro/` for the monbus group) - End-to-end data flow validation

Top Tests: - Full RAPIDS operation - Located in `projects/components/dma-ip/rapids/dv/tests/top_beats/` - Realistic traffic patterns

---

33.10 8. Known Issues Summary

33.10.1 8.1 Critical Issues

Scheduler Credit Counter Initialization - FIXED (2025-10-11, pre-beats) - **File:** the retired pre-beats scheduler.sv (replaced by `rtl/fub_beats/scheduler_beats.sv`, which currently has no credit management - planned for a later phase) - **Issue:** Credit counter was initializing to 0 instead of using exponential encoding - **Fix Applied:** Implemented exponential credit encoding - **Status:** Fixed and verified pre-beats; the known_issues/scheduler.md write-up was retired with that RTL (see `known_issues/README.md`)

Fix Details:

```
// Previous (wrong):
r_descriptor_credit_counter <= 32'h0;

// Fixed - Exponential encoding:
// 0→1, 1→2, 2→4, 3→8, ..., 14→16384, 15→∞
r_descriptor_credit_counter <= (cfg_initial_credit == 4'hF) ?
32'hFFFFFFFF :
                               (cfg_initial_credit == 4'h0) ?
32'h00000001 :
                               (32'h1 << cfg_initial_credit);
```

Encoding Rationale: - Compact 4-bit configuration covers 1 to 16384 credits - Fine-grained control for low traffic (1, 2, 4, 8) - High-throughput support (256, 1024, 16384) - Special unlimited mode (15 → ∞) - Exponential encoding applied at initialization only; runtime operations are linear

33.10.2 8.2 Medium Priority Issues

Descriptor Engine Edge Cases: - Some stress test failures under high load - Edge case handling needs improvement - **Priority:** P2

SRAM Control Timing: - Rare timing issues in back-to-back operations - **Priority:** P2

See: `known_issues/` directory for complete issue tracking

33.11 9. Integration Guidelines

33.11.1 9.1 Quick Start

```
rapids_beats_top #(
    .ADDR_WIDTH(64),
    .DATA_WIDTH(512),
    .NUM_CHANNELS(8)
) u_rapids (
    // Clocking & Reset
    .clk                (system_clk),
    .rst_n              (system_rst_n),

    // APB Control (connect to control fabric)
    .s_apb_paddr        (ctrl_paddr),
    .s_apb_psel         (ctrl_psel),
    .s_apb_penable      (ctrl_penable),
    // ... other APB signals

    // AXI4 Memory (connect to memory controller)
    // ... AXI write channels (sink), AXI read channels (source)

    // AXIS Network (connect to network fabric; tid = channel)
    .s_axis_tdata       (net_rx_data),
    .s_axis_tvalid      (net_rx_valid),
    // ... AXIS slave (receive), AXIS master (transmit)

    // Monitor group egress (AXI-Lite drain / IRQ)
    // ... mon_* signals
);
```

See `rtl/top_beats/rapids_beats_top.sv` for the complete, authoritative port list.

33.11.2 9.2 Configuration Steps

1. **Initialize via APB registers** (use the generated regmap, access registers by name):

- Configure channel enables and thresholds
- Set timeout thresholds

2. **Load Descriptors:**
   - Write descriptors to Descriptor Engine FIFO
   - Each descriptor specifies: address, length, control bits
3. **Enable Operation:**
   - Set enable bits via APB registers
   - Monitor MonBus for completion/error packets

See: `docs/rapids_beats_has/ch05_programming/` for register and programming details

33.12 10. Development Status

33.12.1 10.1 Current Phase

Phase: Validation and Bug Fixing (In Progress)

- Core architecture implemented
- Basic functionality verified
- Scheduler credit counter bug fixed (exponential encoding implemented)
- ⏳ Credit management tests need verification (remove workarounds)
- ⏳ Stress testing ongoing
- ⏳ Edge case refinement

See: `TASKS.md` for detailed work items

33.12.2 10.2 Roadmap

Near-Term (Q4 2025): - Fix scheduler credit counter bug (completed 2025-10-11) - ⏳ Verify credit management tests (remove workarounds) - ⏳ Complete descriptor engine stress testing - ⏳ Integration test suite expansion - ⏳ Performance benchmarking

Long-Term (2026+): - Chained descriptor support - Advanced error recovery - Performance optimizations - Multi-channel support

---

33.13 11. Performance Characteristics

33.13.1 11.1 Throughput

Target: Match network/memory interface bandwidth

Bottlenecks: - SRAM buffer size - AXI4 burst efficiency - Scheduler overhead

Optimization: - Increase SRAM depth for larger packets - Tune AXI4 burst parameters - Pipeline scheduler operations

33.13.2 11.2 Latency

Components: - Descriptor parsing: ~10 cycles - SRAM buffering: Configurable depth - AXI4 memory access: System dependent - End-to-end: Typically <100 cycles for small packets

33.13.3 11.3 Resource Utilization

Area: - Scheduler: ~2K LUTs - Each data path: ~3K LUTs - SRAM buffers: Configurable (dominant area) - Total: ~10K LUTs + SRAM

Power: - Clock gating opportunities in idle blocks - SRAM power depends on depth/width

---

33.14 12. Verification Infrastructure

33.14.1 12.1 Test Organization

Location: `projects/components/dma-ip/rapids/dv/tests/`

Structure:

```
projects/components/dma-ip/rapids/dv/tests/
├── fub/                    # Control engine tests (ctrlrd/ctrlwr)
├── fub_beats/              # Beats FUB tests (scheduler, descriptor
engine,
│                           #   alloc/drain ctrl, latency bridge)
├── macro/                  # MonBus group test
├── macro_beats/            # Multi-block scenarios (scheduler group,
data paths,
│                           #   SRAM controllers)
└── top_beats/              # Full RAPIDS operation
```

33.14.2 12.2 CocoTB Framework

Location: `projects/components/dma-ip/rapids/dv/tbclasses/` (RAPIDS-specific) plus shared BFMs in `bin/TBClasses/`

Components: - RAPIDS-specific drivers - Descriptor generators - Traffic patterns - Monitor checkers

See: `../RTLDesignSherpa-DV/docs/components/<family>/` for shared framework docs

33.14.3 12.2.1 MANDATORY: BFM Usage for FUB Tests

CRITICAL DESIGN REQUIREMENT

All RAPIDS FUB (Functional Unit Block) level tests MUST use CocoTB Framework BFMs. Manual handshake driving is NOT allowed.

Required BFM Components:

Interface Type	Framework Location	BFM Component
Custom valid/ready	`bin/TBClasses/gaxi/`	GAXI Master/Slave
AXI4	`bin/TBClasses/axi4/`	AXI4 Master/Slave
AXI4-Lite (AXIL)	`bin/TBClasses/axil4/`	AXIL Master/Slave
APB	`bin/TBClasses/apb/`	APB Master/Slave
AXI-Stream (AXIS)	`bin/TBClasses/axis4/`	AXIS Master/Slave (RAPIDS network interfaces are AXIS)
MonBus	`bin/TBClasses/monbus/`	MonBus drivers

Rationale: 1. **Consistency**: All tests use standardized handshake protocols 2. **Correctness**: BFMs handle complex timing scenarios (backpressure, randomization) 3. **Reusability**: Same BFM across all RAPIDS tests 4. **Maintainability**: Fix once in BFM, all tests benefit 5. **Coverage**: BFMs include comprehensive timing profiles

Example - Program Engine:

```
# WRONG: Manual handshake (violates design requirement)
async def send_request(self, addr, data):
    self.dut.program_valid.value = 1
    self.dut.program_pkt_addr.value = addr
    # ... manual handshaking logic ...

# CORRECT: Use GAXI Master BFM
from CocoTBFramework.components.gaxi.gaxi_master import GAXIMaster

class ProgramEngineTB(TBBase):
    def __init__(self, dut):
        super().__init__(dut)
        self.program_master = GAXIMaster(
            dut=dut,
```

```
            clock=dut.clk,
            valid_signal='program_valid',
            ready_signal='program_ready',
            data_signals=['program_pkt_addr', 'program_pkt_data'],
            data_widths=[64, 32]
        )

    async def send_request(self, addr, data):
        await self.program_master.write({'program_pkt_addr': addr,
'program_pkt_data': data})
```

See: - `projects/components/dma-ip/rapids/CLAUDE.md` - Rule #1 for complete BFM usage guidelines - `../RTLDesignSherpa-DV/docs/components/gaxi/` - GAXI BFM documentation - `bin/TBClasses/axi4/` - AXI4 BFM sources (full framework docs in the RTLDesignSherpa-DV repo)

33.14.4 12.3 Test File Structure (Standard Pattern)

MANDATORY: All RAPIDS tests must follow this structure

RAPIDS tests follow the same pattern as AMBA tests for consistency across the repository:

```
# Example:
projects/components/dma-ip/rapids/dv/tests/fub_beats/test_scheduler_be
ats.py

import os
import pytest
import cocotb
from cocotb_test.simulator import run

# Import REUSABLE testbench class (NOT defined in this file!)
from projects.components.dma_ip.rapids.dv.tbclasses.scheduler_tb
import SchedulerTB
from TBClasses.shared.utilities import get_paths, create_view_cmd
from TBClasses.shared.tbbase import TBBase

#
==========================================================================
=====
# COCOTB TEST FUNCTIONS - prefix with "cocotb_" to prevent pytest
collection
#
==========================================================================
=====

@cocotb.test(timeout_time=100, timeout_unit="ms")
async def cocotb_test_basic_flow(dut):
```

```
    """Test basic descriptor flow."""
    tb = SchedulerTB(dut)
    await tb.setup_clocks_and_reset()  # Mandatory init method
    await tb.initialize_test()
    result = await tb.test_basic_descriptor_flow()
    assert result, "Basic descriptor flow test failed"

@cocotb.test(timeout_time=100, timeout_unit="ms")
async def cocotb_test_credit_encoding(dut):
    """Test exponential credit encoding."""
    tb = SchedulerTB(dut)
    await tb.setup_clocks_and_reset()  # Mandatory init method
    await tb.initialize_test()
    result = await tb.test_exponential_encoding_all_values()
    assert result, "Credit encoding test failed"

#
================================================================================
=====
# PARAMETER GENERATION - at bottom of file
#
================================================================================
=====

def generate_scheduler_test_params():
    """Generate test parameters for scheduler tests."""
    return [
        # (channel_id, num_channels, data_width, credit_width)
        (0, 8, 512, 8),  # Standard configuration
        # Add more parameter sets as needed
    ]

scheduler_params = generate_scheduler_test_params()

#
================================================================================
=====
# PYTEST WRAPPER FUNCTIONS - at bottom of file
#
================================================================================
=====

@pytest.mark.parametrize("channel_id, num_channels, data_width,
credit_width", scheduler_params)
def test_basic_flow(request, channel_id, num_channels, data_width,
credit_width):
    """
```

```
    Scheduler basic flow test.

    Run with: pytest
projects/components/dma-ip/rapids/dv/tests/fub_beats/test_scheduler_be
ats.py::test_basic_flow -v
    """
    module, repo_root, tests_dir, log_dir, rtl_dict = get_paths({
        'rtl_fub_beats': '../../rtl/fub_beats'
    })

    dut_name = "scheduler_beats"
    toplevel = dut_name

    verilog_sources = [
        os.path.join(repo_root, 'rtl', 'amba', 'includes',
'monitor_pkg.sv'),
        # RAPIDS package(s) from the project includes area
        # (prefer get_sources_from_filelist() with rtl/filelists/ in
real tests)
        os.path.join(rtl_dict['rtl_fub_beats'], '..', 'includes',
'rapids_pkg.sv'),
        os.path.join(rtl_dict['rtl_fub_beats'], f'{dut_name}.sv'),
    ]

    # Format parameters for unique test name
    cid_str = TBBase.format_dec(channel_id, 2)
    nc_str = TBBase.format_dec(num_channels, 2)
    dw_str = TBBase.format_dec(data_width, 4)
    cw_str = TBBase.format_dec(credit_width, 2)
    test_name_plus_params =
f"test_{dut_name}_cid{cid_str}_nc{nc_str}_dw{dw_str}_cw{cw_str}"

    # Add worker ID for pytest-xdist parallel execution
    worker_id = os.environ.get('PYTEST_XDIST_WORKER', '')
    if worker_id:
        test_name_plus_params = f"{test_name_plus_params}_{worker_id}"

    log_path = os.path.join(log_dir, f'{test_name_plus_params}.log')
    sim_build = os.path.join(tests_dir, 'local_sim_build',
test_name_plus_params)
    os.makedirs(sim_build, exist_ok=True)
    os.makedirs(log_dir, exist_ok=True)

    rtl_parameters = {
        'CHANNEL_ID': channel_id,
        'NUM_CHANNELS': num_channels,
```

```
        'DATA_WIDTH': data_width,
        'CREDIT_WIDTH': credit_width,
        # Add other RTL parameters as needed
    }

    extra_env = {
        'LOG_PATH': log_path,
        'TEST_CHANNEL_ID': str(channel_id),
        'TEST_NUM_CHANNELS': str(num_channels),
        'TEST_DATA_WIDTH': str(data_width),
    }

    compile_args = ["-Wno-TIMESCALEMOD"]
    sim_args = []
    plusargs = []

    cmd_filename = create_view_cmd(log_dir, log_path, sim_build,
module, test_name_plus_params)

    try:
        run(
            python_search=[tests_dir],
            verilog_sources=verilog_sources,
            includes=[
                os.path.join(repo_root, 'projects', 'components',
'dma-ip', 'rapids', 'rtl', 'includes'),
                os.path.join(repo_root, 'rtl', 'amba', 'includes'),
            ],
            toplevel=toplevel,
            module=module,
            testcase="cocotb_test_basic_flow",  # ← cocotb test
function name
            parameters=rtl_parameters,
            sim_build=sim_build,
            extra_env=extra_env,
            waves=False,
            keep_files=True,
            compile_args=compile_args,
            sim_args=sim_args,
            plusargs=plusargs,
        )

        print(f"Scheduler basic flow test completed!")
        print(f"Logs: {log_path}")

    except Exception as e:
        print(f"Scheduler basic flow test failed: {str(e)}")
```

```
        print(f"Logs preserved at: {log_path}")
        raise
```

Key Structure Requirements:

1. **Testbench Class Location:**
   - ALWAYS in `projects/components/dma-ip/rapids/dv/tbclasses/`
   - NEVER inline in test file
   - Reusable across multiple test files
2. **CocoTB Test Functions:**
   - Prefix with `cocotb_` to prevent pytest collection
   - Located at top of test file
   - Use `@cocotb.test()` decorator
   - Call testbench methods
3. **Parameter Generation:**
   - Function returns list of parameter tuples
   - Located near bottom of file (before pytest wrappers)
   - Stored in variable (e.g., `scheduler_params`)
4. **Pytest Wrapper Functions:**
   - Located at bottom of file
   - Use `@pytest.mark.parametrize()` with parameter variable
   - Build unique test names with `TBBase.format_dec()`
   - Call `run()` with `testcase="cocotb_test_function_name"`
   - Handle parallel execution (`PYTEST_XDIST_WORKER`)
5. **Mandatory TB Methods:**
   - `async def setup_clocks_and_reset(self)` - Complete initialization
   - `async def assert_reset(self)` - Assert reset signal(s)
   - `async def deassert_reset(self)` - Deassert reset signal(s)

See: - `val/amba/test_apb4_slave.py` - Reference example - `projects/components/dma-ip/rapids/CLAUDE.md` - Detailed TB requirements

---

33.15 13. Quick Reference

33.15.1 13.1 Key Files

File	Purpose
`projects/components/dma-ip/rapids/PRD.md`	This document (overview)
`projects/components/dma-ip/rapids/CLAUDE.md`	AI assistance guide
`projects/components/dma-ip/rapids/TASKS.md`	Work items (largely pre-beats history)
`projects/components/dma-ip/rapids/docs/rapids_beats_has/`	**Architecture specification**
`projects/components/dma-ip/rapids/docs/rapids_beats_mas/`	**Micro-architecture specification**
`projects/components/dma-ip/rapids/known_issues/`	Bug tracking
`vault/Tasks/projects/components/dma-ip/rapids/RAPIDS_Validation_Status_Report.md`	Test results (pre-beats snapshot)

33.15.2 13.2 Commands

```
# Run RAPIDS tests
pytest projects/components/dma-ip/rapids/dv/tests/fub_beats/ -v     #
Individual blocks
pytest projects/components/dma-ip/rapids/dv/tests/macro_beats/ -v   #
Multi-block
pytest projects/components/dma-ip/rapids/dv/tests/top_beats/ -v     #
Full system

# Run specific FUB test
pytest
projects/components/dma-ip/rapids/dv/tests/fub_beats/test_scheduler_be
ats.py -v

# Lint RAPIDS RTL
verilator --lint-only
projects/components/dma-ip/rapids/rtl/fub_beats/scheduler_beats.sv

# View specifications
cat
projects/components/dma-ip/rapids/docs/rapids_beats_mas/rapids_beats_m
```

```
as_index.md
cat
projects/components/dma-ip/rapids/docs/rapids_beats_mas/ch02_fub_block
s/01_scheduler.md
```

33.16 14. Success Criteria

33.16.1 14.1 Functional

- All major blocks implemented
- Basic data flows working
- Scheduler credit bug fixed (exponential encoding implemented)
- ⏳ Credit management tests verified (remove workarounds, run full suite)
- ⏳ 100% descriptor test pass rate (currently ~80%)
- ⏳ Stress tests passing

33.16.2 14.2 Quality

- ⏳ >90% functional coverage (currently ~80%)
- ⏳ >85% code coverage
- All FSMs documented
- ⏳ Integration guide complete

33.16.3 14.3 Documentation

- Complete specification in docs/rapids_beats_has/ + docs/rapids_beats_mas/
- Known issues documented
- ⏳ Integration examples
- ⏳ Performance characterization

33.17 15. Educational Value

RAPIDS demonstrates: - Complex FSM coordination (scheduler ↔️ data paths) - Descriptor-based DMA design patterns - Buffer management strategies - Credit-based flow control with exponential encoding - Multi-interface integration - Comprehensive monitoring - Error detection and reporting - Compact configuration encoding strategies

Target Audience: - Advanced RTL designers - Accelerator architects - DMA engine developers - System integration engineers

33.18 16. Documentation Generation

33.18.1 16.1 Generating PDF/DOCX from Specification

Tool: `bin/md_to_docx.py` (driven by the wrapper scripts in `docs/` - preferred)

Use the wrapper scripts to convert the linked HAS/MAS spec indexes into single all-inclusive PDF/DOCX files:

Basic Usage:

```
# Preferred: wrapper scripts (they call md_to_docx.py with the house
style)
cd projects/components/dma-ip/rapids/docs
./generate_has_pdf.sh --rev 0.8     # builds
RAPIDS_Beats_HAS_v0.8.docx/.pdf
./generate_mas_pdf.sh --rev 0.7     # builds
RAPIDS_Beats_MAS_v0.7.docx/.pdf

# Direct tool invocation (from repo root), if you need custom options
python bin/md_to_docx.py \

projects/components/dma-ip/rapids/docs/rapids_beats_mas/rapids_beats_m
as_index.md \
    -o
projects/components/dma-ip/rapids/docs/RAPIDS_Beats_MAS_draft.docx \
    --toc \
    --title-page \
    --pdf
```

Key Features: - **Recursive Collection:** Follows all markdown links in the index file - **Heading Demotion:** Automatically adjusts heading levels for included files - **Table of Contents:** `--toc` flag generates automatic ToC - **Title Page:** `--title-page` flag creates title page from first heading - **PDF Export:** `--pdf` flag generates both DOCX and PDF - **Image Support:** Resolves images relative to source directory - **Template Support:** Optional custom DOCX/DOTX template via `-t` flag

Common Workflow:

```
# 1. Update spec content under rapids_beats_has/ or rapids_beats_mas/
# 2. Generate documentation with a bumped revision
cd projects/components/dma-ip/rapids/docs
./generate_mas_pdf.sh --rev 0.8

# 3. Output files created in docs/:
#    - RAPIDS_Beats_MAS_v0.8.docx
#    - RAPIDS_Beats_MAS_v0.8.pdf
```

Debug Mode:

```
# Generate debug markdown to see combined output
python bin/md_to_docx.py \

projects/components/dma-ip/rapids/docs/rapids_beats_mas/rapids_beats_m
as_index.md \
    -o output.docx \
    --debug-md

# This creates debug.md showing the complete merged content
```

Tool Requirements: - Python 3.6+ - Pandoc installed and in PATH - For PDF generation: LaTeX (e.g., texlive) or use Pandoc's built-in PDF writer

See: `bin/md_to_docx.py` for complete implementation details

33.19 16.2 PDF Generation Location

IMPORTANT: PDF files should be generated in the docs directory:

`projects/components/dma-ip/rapids/docs/`

Quick Command: Use the provided shell scripts:

```
cd projects/components/dma-ip/rapids/docs
./generate_has_pdf.sh    # Architecture spec (HAS)
./generate_mas_pdf.sh    # Micro-architecture spec (MAS)
```

The shell scripts will automatically: 1. Use the md_to_docx.py tool from bin/ 2. Process the rapids_beats_has / rapids_beats_mas index files 3. Generate both DOCX and PDF files in the docs/ directory 4. Create table of contents and title page

See: `bin/md_to_docx.py` for complete implementation details

33.20 17. References

33.20.1 16.1 Internal Documentation

- **Complete Spec:** `docs/rapids_beats_has/` + `docs/rapids_beats_mas/` ← **Primary technical reference**
- **Validation:** `vault/Tasks/projects/components/dma-ip/rapids/RAPIDS_Validation_Status_Report.md` (pre-beats snapshot)

- **Master PRD:** `/PRD.md`
- **Repository Guide:** `/CLAUDE.md`

33.20.2 16.2 Related Subsystems

- **AMBA:** `rtl/amba/` - Monitor infrastructure used in RAPIDS (monitors in `rtl/amba/ocelus/`)
- **Common:** `rtl/common/` - Building blocks (counters, FIFOs, etc.)
- **CocoTB Framework:** `bin/TBClasses/` (shared) + `projects/components/dma-ip/rapids/dv/tbclasses/` (RAPIDS)

33.20.3 16.3 External References

- AXI4 Specification: ARM IHI0022E
- AXIL4 Specification: ARM IHI0022E (subset)
- Network interface specs (custom Network protocol)

Document Version: 1.0 **Last Updated:** 2025-09-30 **Review Cycle:** Monthly during active development **Next Review:** 2025-10-30 **Owner:** RTL Design Sherpa Project

---

33.21 Navigation

- **← Back to Root:** `/PRD.md`
- **Complete Specification:** `docs/rapids_beats_has/rapids_beats_has_index.md` + `docs/rapids_beats_mas/rapids_beats_mas_index.md`
- **AI Guidance:** `CLAUDE.md`
- **Tasks:** `TASKS.md`
- **Issues:** `known_issues/`

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

34 Claude Code Guide: RAPIDS Subsystem

Version: 1.0 **Last Updated:** 2025-10-11 **Purpose:** AI-specific guidance for working with RAPIDS subsystem

---

34.1 Quick Context

What: Rapid AXI Programmable In-band Descriptor System - Custom DMA-style accelerator with AXIS network interfaces **Status:** Active development - "beats" rearchitecture is the current RTL **Your Role:** Help users understand architecture, fix bugs, extend functionality

> **Status (2026-07-22):** RAPIDS was rearchitected to the "beats" design. Current RTL lives in `rtl/fub_beats/`, `rtl/macro_beats/`, and `rtl/top_beats/rapids_beats_top.sv` (plus control engines in `rtl/fub/`). The old `rapids_fub/`/`rapids_macro/` modules (scheduler.sv, program_engine.sv, network_slave.sv, ...) and the old `docs/rapids_spec/` tree are gone. Sections below that describe the pre-beats design are marked accordingly.
>
> **Status (2026-09-29, rapids TASK-019):** RAPIDS is byte-granular. The product tree is the un-suffixed `rtl/fub/`, `rtl/macro/`, `rtl/top/rapids_top.sv` (descriptor length in BYTES, byte addresses, WSTRB/TSTRB carry partial beats, one AXIS packet per descriptor), tested in `dv/tests/fub`, `macro`, `top`. The `*_beats` tree is the beat-granular stepping stone: it stays runnable and green (the characterized Genesys 2 design) and is not maintained forward. The two trees share `rapids_config_block`, the regs, `rapids_pkg` and the control engines. Byte-specific contracts: the sink ingress holds `s_axis_tready` until the channel's descriptor is fetched (issue the descriptor first, or stream concurrently); a `tlast` packet whose byte count differs from the descriptor sets that channel's sticky `sched_wr_error`. TYPE=EXT (row/column striding) descriptors stay beat-aligned by design: aligned addresses, beat-multiple lengths. Linear descriptors may start at any byte on either side.
>
> Silicon (2026-09-30, `projects/fpga-systems/Genesys2/dma-ip/rapids/reports/perf/README.md` v0.2): 117/117 byte-wise points on the standard bitstream; beat-aligned rows on the word-wide checker bitstream reach 3182 MB/s sink and 3199 MB/s source of the 3200 MB/s peak. Traps: the word-wide

checker CRCs only slice 0 of each beat and ignores strobes (beats golden, not byte-wise); the aligned utilization is NOT within 0.5 pp of RAPIDS Beats because of fixed start-up terms from the packet-record gating, accepted as by design (rapids TASK-021); head-of-line blocking on a shared AXIS stream is documented, not fixed; RRESP/BRESP injection is unexercised (rapids TASK-020).

Status (2026-10-15, rapids TASK-024): network planes. The network endpoints moved out of `rapids_src`/`rapids_snk` into swappable planes in `rtl/plane/` (`rapids_plane_{axis,cxs}_{rx,tx}`), shared by both trees. The four tops `rapids_top`, `rapids_cxs_top`, `rapids_beats_top` and `rapids_beats_cxs_top` are GENERATED by `bin/gen_plane_tops.py` from `rtl/top/rapids_top_base.sv` and `rtl/top_beats/rapids_beats_top_base.sv`. Edit the base or the generator, never a top, and run `--check`. The AXIS tops keep their old interface. CXS profile, limits (DATA_WIDTH <= 1024) and tests: `docs/rapids_planes.md`. Traps: the core's plane monitor rides `src_/snk_plane_mon_*` (monbus_arbiter client 1); the monitor counters live at `u_{src,snk}_plane.u_endpoint`.

rapids TASK-025 (2026-10-16): ONE CXS link per direction, CXSSRCID = channel. Packets are framed per stream on the channel bits (the byte source interleaves channels mid-packet). `CXS_MAX_CREDIT` defaults to 32 (one credit = one flit; ~3-cycle credit round trip), owner 2026-10-16; above 63 needs `CXS_CREDIT_BEYOND_SPEC`. It is not tied to SRAM_DEPTH, which is beats per channel. CXS cells must include a concurrent-channel case: single-channel cells hid the framing bug.

Complete Specification: - `projects/components/dma-ip/rapids/docs/rapids_beats_has/` - Architecture spec (HAS); index: `rapids_beats_has_index.md` - `projects/components/dma-ip/rapids/docs/rapids_beats_mas/` - Micro-architecture spec (MAS); index: `rapids_beats_mas_index.md` - `projects/components/dma-ip/rapids/docs/rapids_has/` and `rapids_mas/` - the byte-granular RAPIDS books (index: `rapids_has_index.md`, `rapids_mas_index.md`); chapters identical to Beats are linked from the `rapids_beats_*` books, not copied - Built PDFs: `docs/RAPIDS_HAS_v0.3.pdf`, `docs/RAPIDS_MAS_v0.2.pdf` (byte design); `docs/RAPIDS_Beats_HAS_v0.10.pdf`, `docs/RAPIDS_Beats_MAS_v0.8.pdf` (beats design)

34.2 Global Requirements Reference

IMPORTANT: Check `/GLOBAL_REQUIREMENTS.md` before starting RAPIDS work

All mandatory requirements are consolidated in the global requirements document: - **See:** `/GLOBAL_REQUIREMENTS.md` - Repository-wide mandatory requirements - **RAPIDS-Specific:** Attribution format, BFM usage requirements - **Universal:** TB location, three methods, TBBase inheritance, 100% success

This CLAUDE.md provides RAPIDS-specific guidance. Also review: - Root `/CLAUDE.md` - Repository-wide patterns - `projects/components/CLAUDE.md` - Project area standards (reset macros, FPGA attributes) - `bin/TBClasses/` - Shared TB framework (flat protocol dirs: `gaxi/`, `axi4/`, `apb/`, ...); the full CocoTBFramework (components/, tbclasses/) lives in the separate RTLDesignSherpa-DV repo

---

34.3 Critical Rules for This Subsystem

34.3.1 Rule #0.1: Testbench Location and Test Structure (MANDATORY)

See: `/GLOBAL_REQUIREMENTS.md` Section 2.1. Test structure itself – Pattern B, the `cocotb_test_*` prefix, pytest wrapper naming – is the `test-patterns` skill.

RAPIDS keeps all of its verification in the project area:

```
projects/components/dma-ip/rapids/dv/
├── tbclasses/     # 22 TB classes: scheduler_tb,
descriptor_engine_tb,
│                  # rapids_core_beats_tb, rapids_beats_top_tb, ...
├── components/    # RAPIDS-specific BFMs (data_mover_bfm.py)
└── tests/         # runners by layer: fub/ fub_beats/ macro/
macro_beats/ top_beats/
```

Scoring choice (GR 2.4). The descriptor and program engines are in-order, so they verify by queue access – `monitor._recvQ.popleft()` compared against an expected value. The memory model is for integration tests, where several masters are in flight and order is not guaranteed. Reaching for the memory model on an in-order engine buys nothing and hides ordering bugs.

34.3.2 Rule #0.5: Config Before Reset (RAPIDS-specific)

The three mandatory TB methods are `/GLOBAL_REQUIREMENTS.md` Section 2.2. What is particular to RAPIDS is **when** configuration is applied: several modules latch config during reset, so a value written after `deassert_reset()` is simply never seen.

Set these in `setup_clocks_and_reset()` *before* asserting reset:

Signal	Why it must be early
`cfg_initial_credit`	the credit counter initialises to `(1 << cfg)` during reset
`cfg_use_credit`	selects the flow-control mode the counter comes up in
`cfg_timeout_threshold`	watchdog loads at reset
`cfg_sram_depth`	must be stable before the memory controllers leave reset

(The credit signals belong to the retired pre-beats `scheduler.sv`; `rtl/fub_beats/scheduler_beats.sv` has no credit management yet. The config-before-reset rule still holds for the beats modules.)

34.3.3 Rule #0.75: Audit Signal Naming BEFORE Writing Testbenches

Why factory prefix collisions break BFM discovery, and the audit tool that catches them, are in `vault/handbook/design/naming-and-style.md`. Run `bin/audit_signal_naming_conflicts.py` over `rtl/` before writing a TB. What is RAPIDS-specific is below.

Known RAPIDS Conflicts: See `projects/components/dma-ip/rapids/known_issues/scheduler_group_signal_naming_conflicts.md` for conflicts documented against the pre-beats scheduler_group.sv (the patterns still apply to `scheduler_group_beats.sv`): - `desc` prefix: 4 internal + 4 external (AR/R channels) - `prog` prefix: 4 internal + 6 external (AW/W/B channels)

Recommended Workflow: 1. Write RTL module with signals 2. **Run audit script to detect conflicts** 3. Fix naming conflicts (rename internal signals with `_to_sched` suffix) 4. Write testbench using factory pattern matching

Three Solutions When Conflicts Found: 1. **Rename internal signals** (recommended): `desc_valid` → `desc_to_sched_valid` 2. **Use explicit signal_map**: Bypass pattern matching with manual signal mapping 3. **Test at higher level**: Where internal signals aren't visible (e.g., rapids_top)

Complete Guide: `bin/SIGNAL_NAMING_AUDIT.md`

34.3.4 Rule #1: MANDATORY BFM Usage for FUB-Level Tests

Never hand-drive a valid/ready handshake and never write a custom protocol driver. The interface-to-BFM map, the factory list, the trap list and the extract-vs-embed criteria are in `vault/handbook/dv/bfm-usage.md`.

RAPIDS-specific: the network interfaces are **AXIS**, so they take the `axis4` factories; the custom valid/ready interfaces (program, descriptor) take GAXI. `dv/components/data_mover_bfm.py` is the one extracted RAPIDS BFM.

Manual driving is acceptable for throwaway debug and for clock/reset init – never in a production testbench.

34.3.5 Rule #2: Know the Known Issues

Check `projects/components/dma-ip/rapids/known_issues/` (and `active/`) before diagnosing anything.

Historical: the pre-beats `scheduler.sv` credit counter was hardcoded to 0, then fixed with exponential encoding (0->1, 1->2, ..., 15->inf). That scheduler was replaced by `rtl/fub_beats/scheduler_beats.sv`, which has no credit management yet, and its `known_issues/scheduler.md` write-up was retired with it. Do not chase that bug in beats code.

34.3.6 Rule #3: RAPIDS is Complex - Understand Block Interactions

Key Interaction Patterns:

1. **Descriptor Flow:**

   ```
   Software → AXIL4 → Descriptor Engine → Scheduler → Data Paths
   ```

2. **Sink Data Path (Network → Memory):**

   ```
   Network Slave → Sink SRAM Control → Sink AXI Write Engine →
   System Memory
   ```

3. **Source Data Path (Memory → Network):**

   ```
   System Memory → Source AXI Read Engine → Source SRAM Control →
   Network Master
   ```

4. **Monitoring:**

   ```
   All Blocks → MonBus Reporter → MonBus Output
   ```

Never work on one block in isolation without understanding its upstream/downstream dependencies!

34.3.7 Rule #4: Test Strategy is Multi-Layered

RAPIDS testing follows this hierarchy:

1. **FUB (Functional Unit Block) Tests:**
   `projects/components/dma-ip/rapids/dv/tests/fub_beats/` (+ `fub/` for control engines)
   - Individual block testing
   - Focus: Module-level functionality
   - Example: Scheduler FSM, Descriptor Engine FIFO
2. **Macro Tests:**
   `projects/components/dma-ip/rapids/dv/tests/macro_beats/` (+ `macro/` for the monbus group)
   - Multi-block scenarios
   - Focus: Block-to-block interfaces
   - Example: Scheduler group, sink/source data path end-to-end
3. **Top Tests:** `projects/components/dma-ip/rapids/dv/tests/top_beats/`
   - Full RAPIDS operation
   - Focus: Realistic traffic patterns
   - Example: Complete DMA transfer with monitoring

When creating/modifying tests, ensure appropriate test layer is used!

34.4 Architecture Quick Reference

34.4.1 Block Organization

```
RAPIDS Beats Architecture (rtl/)
├── fub_beats/                          (per-block engines)
│   ├── scheduler_beats.sv              (per-channel scheduler FSM)
│   ├── descriptor_engine_beats.sv      (descriptor fetch, parsing)
│   ├── axi_read_engine_beats.sv        (AXI4 read from system memory)
│   ├── axi_write_engine_beats.sv       (AXI4 write to system memory)
│   ├── alloc_ctrl_beats.sv             (SRAM space allocation)
│   ├── drain_ctrl_beats.sv             (SRAM drain control)
│   └── latency_bridge_beats.sv         (latency-hiding bridge)
├── fub/                                (control engines)
│   ├── ctrlrd_engine.sv                (control read engine)
│   └── ctrlwr_engine.sv                (control write engine)
├── macro_beats/                        (assemblies)
│   ├── scheduler_group_beats.sv / scheduler_group_array_beats.sv
│   ├── snk_data_path_beats.sv / snk_data_path_axis_beats.sv   (AXIS
network → SRAM → memory)
│   ├── src_data_path_beats.sv / src_data_path_axis_beats.sv   (memory
→ SRAM → AXIS network)
```

```
│   ├── snk_sram_controller_beats.sv / src_sram_controller_beats.sv
│   ├── rapids_config_block.sv + rapids_*_regs.rdl             (APB
CSRs via PeakRDL)
│   └── rapids_core_beats.sv         (core integration)
├── macro/
│   └── monbus_axil_group_2in.sv     (MonBus aggregation / AXI-Lite
drain)
└── top_beats/
    └── rapids_beats_top.sv          (top level: APB config, AXI4
masters, AXIS in/out, MonBus)
```

See: `docs/rapids_beats_mas/ch02_fub_blocks/` and `docs/rapids_beats_mas/ch03_macro_blocks/` for detailed block descriptions

34.4.2 Module Quick Reference

Module	Location	Purpose	Documentation (docs/rapids_beats_mas/)
scheduler_beats.sv	`fub_beats/`	Per-channel scheduler FSM	`ch02_fub_blocks/01_scheduler.md`
descriptor_engine_beats.sv	`fub_beats/`	Descriptor fetch, parsing	`ch02_fub_blocks/02_descriptor_engine.md`
axi_read_engine_beats.sv	`fub_beats/`	Memory reads	`ch02_fub_blocks/03_axi_read_engine.md`
axi_write_engine_beats.sv	`fub_beats/`	Memory writes	`ch02_fub_blocks/04_axi_write_engine.md`
alloc_ctrl_beats.sv	`fub_beats/`	SRAM allocation	`ch02_fub_blocks/05_beats_alloc_ctrl.md`
drain_ctrl_beats.sv	`fub_beats/`	SRAM drain control	`ch02_fub_blocks/06_beats_drain_ctrl.md`
latency_bridge_beats.sv	`fub_beats/`	Latency-hiding bridge	`ch02_fub_blocks/07_beats_latency_bridge.md`
snk_data_path_beats.sv	`macro_beats/`	Sink data path	`ch03_macro_blocks/03_sink_data_path.md`
src_data_path_beats.sv	`macro_beats/`	Source data path	`ch03_macro_blocks/06_source_data_path.md`

Module	Location	Purpose	Documentation (docs/rapids_beats_mas/)
rapids_core_beats.sv	macro_beats/	Core integration	ch03_macro_blocks/09_rapids_core_beats.md
rapids_beats_top.sv	top_beats/	Top-level integration	ch03_macro_blocks/12_rapids_beats_top.md

34.4.3 Interface Summary

Interface	Type	Width	Purpose	Specification
APB4	Slave	32-bit	Control/status registers	docs/rapids_beats_has/ch03_interfaces/04_apb_interface.md
AXI4 (Sink)	Master	Configurable	Write to system memory	docs/rapids_beats_mas/ch04_interfaces/01_axi4_interface_spec.md
AXI4 (Source)	Master	Configurable	Read from system memory	docs/rapids_beats_mas/ch04_interfaces/01_axi4_interface_spec.md
AXIS (Sink)	Slave	Configurable	Network ingress (tid = channel)	docs/rapids_beats_mas/ch04_interfaces/02_axis_interface_spec.md
AXIS (Source)	Master	Configurable	Network egress (tid = channel)	docs/rapids_beats_mas/ch04_interfaces/02_axis_interface_spec.md
MonBus	Master	64-bit	Monitor packet output	docs/rapids_beats_mas/ch04_interfaces/03_monbus_interface_spec.md

34.5 Common User Questions and Responses

34.5.1 Q: "How does the scheduler work?"

A: Direct answer:

The scheduler is a complex FSM that coordinates RAPIDS operations:

1. **Idle State:** Waits for descriptors from Descriptor Engine
2. **Parse State:** Extracts descriptor fields (address, length, control)
3. **Credit Check:** Verifies credit availability (if credit mode enabled)
4. **Execute State:** Activates appropriate data path (sink or source)
5. **Monitor State:** Tracks operation progress
6. **Complete State:** Generates completion packets, updates credits

Key FSM States: - `IDLE` → `PARSE` → `CREDIT_CHECK` → `EXECUTE` → `MONITOR` → `COMPLETE` → `IDLE`

Credit Management: - Historical: the pre-beats scheduler.sv implemented exponential credit encoding (0→1, 1→2, 2→4, ..., 15→∞) - see Q&A below for the historical details - Current: `rtl/fub_beats/scheduler_beats.sv` has no credit management yet (planned later phase)

See: - `projects/components/dma-ip/rapids/docs/rapids_beats_mas/ch02_fub_blocks/01_scheduler.md` - Complete FSM specification - `projects/components/dma-ip/rapids/known_issues/README.md` - Known bugs and workarounds

34.5.2 Q: "How do I configure RAPIDS?"

> **Status (2026-07-22):** The beats top (`rapids_beats_top.sv`) exposes an APB4 config slave (`s_apb_*`) with PeakRDL-generated registers (`rtl/macro_beats/rapids_regs.rdl`); access registers by name via the generated regmap, not hardcoded offsets. The AXIL4 write sequence below is from the pre-beats design and is kept for historical flavor only.

A: Configuration via memory-mapped register interface:

```
// 1. Initialize RAPIDS via AXIL4 writes
// Configure SRAM depths
write_axil(ADDR_SINK_SRAM_DEPTH, 1024);
write_axil(ADDR_SOURCE_SRAM_DEPTH, 1024);

// Set timeout thresholds
write_axil(ADDR_TIMEOUT_THRESHOLD, 1000);

// Configure initial credits (exponential encoding)
```

```
// 0→1, 1→2, 2→4, 3→8, 4→16, etc.
write_axil(ADDR_INITIAL_CREDIT, 4);  // 4 = 16 credits (2^4)
write_axil(ADDR_CREDIT_ENABLE, 1);   // Enable credit mode (now
fixed!)

// 2. Load descriptors
write_descriptor(addr, length, control_bits);

// 3. Enable operation
write_axil(ADDR_ENABLE, 1);
```

See: - `projects/components/dma-ip/rapids/docs/rapids_beats_has/ch05_programming/03_initialization.md` - Programming model - `projects/components/dma-ip/rapids/docs/rapids_beats_has/ch05_programming/02_register_map.md` - Register definitions

34.5.3 Q: "What's the data flow for network to memory transfer?"

A: Sink path data flow:

```
1. Network Packet Arrives
   ↓
2. Network Slave receives packet
   - Validates packet format
   - Handshakes with Network interface
   ↓
3. Sink SRAM Control buffers data
   - Writes to SRAM
   - Manages write pointers
   - Handles backpressure
   ↓
4. Sink AXI Write Engine
   - Reads from SRAM
   - Generates AXI4 write transactions
   - Bursts data to system memory
   ↓
5. Completion Reporting
   - Generates MonBus completion packet
   - Updates scheduler state
```

Key Considerations: - SRAM acts as buffer to decouple network from memory timing - AXI4 bursts used for efficient memory access - Backpressure propagates from memory to network

See: `projects/components/dma-ip/rapids/docs/rapids_beats_mas/ch03_macro_blocks/03_sink_data_path.md`

34.5.4 Q: "What's the credit counter bug and how does exponential encoding work?"

> **Status (2026-07-22):** Historical. This bug and fix lived in the retired pre-beats scheduler.sv; `rtl/fub_beats/scheduler_beats.sv` currently has no credit management (planned for a later phase). Kept as background for when credits return.

A: Scheduler credit counter exponential encoding (pre-beats):

The scheduler uses **exponential credit encoding** to provide a wide range of credit values (1 to 16384) with a compact 4-bit configuration:

Encoding Table: | `cfg_initial_credit` | Actual Credits | Use Case | |———————|———————-|———-| | 0 | 1 | Minimum (2^0) | | 1 | 2 | Very low traffic (2^1) | | 2 | 4 | Low traffic (2^2) | | 3 | 8 | (2^3) | | 4 | 16 | Typical (2^4) | | 5 | 32 | (2^5) | | 6 | 64 | Medium traffic (2^6) | | 7 | 128 | (2^7) | | 8 | 256 | High traffic (2^8) | | 10 | 1024 | Very high traffic (2^10) | | 14 | 16384 | Maximum finite (2^14) | | 15 | ∞ (0xFFFFFFFF) | Unlimited credits |

FIXED Implementation (in the retired pre-beats scheduler.sv):

```
// Exponential credit encoding: 0→1, 1→2, 2→4, ..., 14→16384, 15→∞
r_descriptor_credit_counter <= (cfg_initial_credit == 4'hF) ?
32'hFFFFFFFF :
                               (cfg_initial_credit == 4'h0) ?
32'h00000001 :
                               (32'h1 << cfg_initial_credit);
```

Was Broken (Before Fix):

```
r_descriptor_credit_counter <= 32'h0;  // WRONG: Hardcoded to 0
```

Impact (Before Fix): - Credit-based flow control didn't work - All operations blocked if credit mode enabled - Descriptors never processed

Encoding Rationale: - Compact 4-bit config covers wide range: 1 to 16384 credits - Fine-grained control for low traffic (1, 2, 4, 8) - High-throughput support (256, 1024, 16384) - Special unlimited mode (15 → ∞)

Important: Exponential encoding applies **only at initialization**. Once running, the counter operates linearly (increment/decrement by 1).

See: - `projects/components/dma-ip/rapids/docs/rapids_beats_mas/ch02_fub_blocks/01_scheduler.md` - Current scheduler specification - `projects/components/dma-ip/rapids/known_issues/README.md` - Issue tracking (the old scheduler.md write-up was retired with the pre-beats RTL)

34.5.5 Q: "How do I run RAPIDS tests?"

A: Multi-layered test approach:

```
# All of these go through the area Makefile. A bare pytest drops the
level,
# the derived worker count and the reruns, and skips clean-all -- see
# vault/handbook/dv/running-regressions.md
cd projects/components/dma-ip/rapids/dv/tests

# 1. FUB tests - individual blocks. AREAS narrows the sweep;
#    the dispatcher's areas are: fub fub_beats macro macro_beats
top_beats
make run-scheduler_beats-gate AREAS=fub_beats
make run-all-gate AREAS=fub_beats      # every beats FUB test
make run-all-gate AREAS=fub            # control engines

# 2. Macro tests - multi-block scenarios
make run-all-func AREAS=macro_beats

# 3. Top tests - full RAPIDS operation
make run-all-func AREAS=top_beats

# Waves: WAVES=1 via the target, not --vcd. create_view_cmd() writes a
# ready-made gtkwave command beside the log.
make run-scheduler_beats-gate-waves AREAS=fub_beats
```

Test Organization: - **FUB tests:** Focus on individual block functionality - **Macro tests:** Verify block-to-block interfaces - **Top tests:** Validate complete data flows

Current status: regenerate it – `dv/tests/analyze_beats_coverage.py` writes to `dv/tests/coverage_reports/`. The ~80% figure that sat here was a pre-beats snapshot, as is `vault/Tasks/projects/components/dma-ip/rapids/RAPIDS_Validation_Status_Report.md`, which remains the written-up version of that era.

34.6 Integration Patterns

34.6.1 Pattern 1: Basic RAPIDS Instantiation

The byte-granular top (`rtl/top/rapids_top.sv`, rapids TASK-019). The beat-granular stepping stone `rapids_beats_top` has the same ports with the same names; only the descriptor's length unit differs (bytes here, beats there).

```
rapids_top #(
    .NUM_CHANNELS       (8),
```

```
    .DATA_WIDTH         (256),
    .ADDR_WIDTH         (64),
    .AXI_ID_WIDTH       (8),
    .SRAM_DEPTH         (1024),
    .USE_AXI_MONITORS   (1)
) u_rapids (
    .aclk               (system_clk),
    .aresetn            (system_rst_n),

    // APB4 configuration slave (SRC block at 0x0000, SNK block at
0x1000)
    .s_apb_paddr        (cfg_paddr),
    .s_apb_psel         (cfg_psel),
    .s_apb_penable      (cfg_penable),
    .s_apb_pwrite       (cfg_pwrite),
    .s_apb_pwdata       (cfg_pwdata),
    .s_apb_pstrb        (cfg_pstrb),
    .s_apb_prdata       (cfg_prdata),
    .s_apb_pready       (cfg_pready),
    .s_apb_pslverr      (cfg_pslverr),

    // AXI4 read master (source: memory -> m_axis)
    .m_axi_rd_arvalid   (mem_arvalid),
    .m_axi_rd_arready   (mem_arready),
    .m_axi_rd_araddr    (mem_araddr),
    .m_axi_rd_arlen     (mem_arlen),
    .m_axi_rd_rdata     (mem_rdata),
    .m_axi_rd_rvalid    (mem_rvalid),
    .m_axi_rd_rready    (mem_rready),
    // ... remaining m_axi_rd_* AR/R signals

    // AXI4 write master (sink: s_axis -> memory), WSTRB carries the
byte enables
    .m_axi_wr_awvalid   (mem_awvalid),
    .m_axi_wr_awready   (mem_awready),
    .m_axi_wr_awaddr    (mem_awaddr),
    .m_axi_wr_awlen     (mem_awlen),
    .m_axi_wr_wdata     (mem_wdata),
    .m_axi_wr_wstrb     (mem_wstrb),
    .m_axi_wr_wvalid    (mem_wvalid),
    .m_axi_wr_wready    (mem_wready),
    // ... remaining m_axi_wr_* AW/W/B signals

    // AXI-Stream: ingress to the sink, egress from the source (tid =
channel)
    .s_axis_tdata       (net_in_tdata),
    .s_axis_tstrb       (net_in_tstrb),
```

```
    .s_axis_tlast       (net_in_tlast),
    .s_axis_tid         (net_in_tid),
    .s_axis_tvalid      (net_in_tvalid),
    .s_axis_tready      (net_in_tready),
    .m_axis_tdata       (net_out_tdata),
    .m_axis_tstrb       (net_out_tstrb),
    .m_axis_tlast       (net_out_tlast),
    .m_axis_tid         (net_out_tid),
    .m_axis_tvalid      (net_out_tvalid),
    .m_axis_tready      (net_out_tready),
    // ... tdest/tuser, the descriptor and control AXI masters, MonBus
    .mon_irq            (dma_irq)
);
```

34.6.2 Pattern 2: Configuration Sequence

```
// Recommended initialization sequence
initial begin
    // 1. Assert reset
    aresetn = 0;
    repeat(10) @(posedge aclk);
    aresetn = 1;

    // 2. Configure RAPIDS via AXIL4
    axil_write(ADDR_SINK_SRAM_DEPTH, 1024);
    axil_write(ADDR_SOURCE_SRAM_DEPTH, 1024);
    axil_write(ADDR_TIMEOUT_THRESHOLD, 1000);
    axil_write(ADDR_INITIAL_CREDIT, 4);  // 4 = 16 credits (2^4,
exponential encoding)
    axil_write(ADDR_CREDIT_ENABLE, 1);  // Enable credit mode (now
fixed!)

    // 3. Load descriptors
    for (int i = 0; i < num_descriptors; i++) begin
        axil_write(ADDR_DESC_ADDR, descriptor[i].addr);
        axil_write(ADDR_DESC_LEN, descriptor[i].length);
        axil_write(ADDR_DESC_CTRL, descriptor[i].control);
        axil_write(ADDR_DESC_COMMIT, 1);
    end

    // 4. Enable RAPIDS operation
    axil_write(ADDR_ENABLE, 1);
end
```

34.6.3 Pattern 3: MonBus Integration

```
// Always add downstream FIFO for MonBus
gaxi_fifo_sync #(
    .DATA_WIDTH(64),
```

```
    .DEPTH(256)
) u_rapids_mon_fifo (
    .axi_aclk   (aclk),
    .axi_aresetn(aresetn),
    // producer side: wr_ready is "not full", so it is what
backpressures MonBus
    .wr_valid   (monbus_pkt_valid),
    .wr_data    (monbus_pkt_data),
    .wr_ready   (monbus_pkt_ready),
    // consumer side
    .rd_valid   (fifo_mon_valid),
    .rd_data    (fifo_mon_data),
    .rd_ready   (consumer_ready)
);
```

34.7 Anti-Patterns to Catch

34.7.1 Anti-Pattern 1: Not Understanding Exponential Credit Encoding

```
WRONG:
.cfg_initial_credit(4'd16),  // Thinks this gives 16 credits - it
doesn't!

CORRECTED:
"Credits use exponential encoding:
- cfg_initial_credit = 4 → 16 credits (2^4)
- cfg_initial_credit = 8 → 256 credits (2^8)
- cfg_initial_credit = 15 → ∞ credits (unlimited)
(Pre-beats behavior; scheduler_beats.sv has no credit management yet.
See the encoding
table in the credit-counter Q&A above.)"
```

34.7.2 Anti-Pattern 2: Insufficient SRAM Depth

```
WRONG:
.SRAM_DEPTH(16)  // Too small for realistic packets

CORRECTED:
"SRAM depth should match typical packet sizes.
Recommended: 1024-4096 entries depending on data width and packet
sizes."
```

34.7.3 Anti-Pattern 3: No MonBus Downstream Handling

```
WRONG:
assign monbus_pkt_ready = 1'b1;  // Always ready = potential packet
loss
```

```
CORRECTED:
"Connect to FIFO or proper consumer:
gaxi_fifo_sync #(.DATA_WIDTH(64), .DEPTH(256)) u_mon_fifo (
    .wr_valid(monbus_pkt_valid),
    .wr_data (monbus_pkt_data),
    .wr_ready(monbus_pkt_ready),
    ...
);"
```

34.7.4 Anti-Pattern 4: Testing Individual Blocks in Isolation

```
WRONG:
"Only test scheduler without descriptor engine"

CORRECTED:
"RAPIDS blocks are tightly coupled. Always test:
1. FUB tests for basic block functionality
2. Integration tests for block interactions
3. System tests for complete flows"
```

34.8 Debugging Workflow

34.8.1 Issue: Scheduler Not Processing Descriptors

Check in order: 1. Credit configuration correct? (Remember exponential encoding!) 2. Descriptors loaded via AXIL4? 3. RAPIDS enabled? (`ADDR_ENABLE = 1`) 4. Reset properly deasserted? 5. Descriptor engine FIFO not empty?

Debug commands:

```
cd projects/components/dma-ip/rapids/dv/tests
make run-scheduler_beats-gate AREAS=fub_beats          # -v --tb=short
by default
make run-scheduler_beats-gate-waves AREAS=fub_beats    # WAVES=1, not
--vcd
# create_view_cmd() writes the gtkwave command beside the log
```

34.8.2 Issue: Data Path Stalls

Check in order: 1. SRAM depth sufficient? 2. Downstream interfaces ready? 3. AXI4 backpressure handling? 4. Network flow control? 5. Buffer overflow/underflow detection?

Waveform Analysis: - Check SRAM read/write pointers - Verify AXI4 handshakes - Inspect Network TREADY signals

34.8.3 Issue: MonBus Packets Not Generated

Check in order: 1. Operations completing successfully? 2. MonBus ready signal asserted? 3. MonBus reporter enabled? 4. Downstream FIFO not full?

34.9 Testing Guidance

34.9.1 Running Tests

Tests are organised by layer under `dv/tests/`: `fub/` and `fub_beats/` for single blocks, `macro/` and `macro_beats/` for multi-block scenarios, `top_beats/` for full RAPIDS operation. `make list` enumerates the roots.

This area's Makefile takes an `AREAS` variable (default `fub fub_beats macro macro_beats top_beats`) to scope a run to some layers:

```
cd projects/components/dma-ip/rapids/dv/tests

make run-all-gate AREAS=fub_beats          # one layer
make run-scheduler_beats-gate AREAS=fub_beats
make clean-all && make run-all-full-parallel   # everything; clean-all first
```

Target grammar, the levels and why `clean-all` is not optional: `vault/handbook/dv/running-regressions.md`.

34.9.2 Test Coverage Status

Generate it, do not read it from here – a pasted table is stale the day after it is written. `dv/tests/analyze_beats_coverage.py` writes to `dv/tests/coverage_reports/`.

34.10 Key Documentation Links

Specification (`projects/components/dma-ip/rapids/docs/`): - `rapids_beats_has/rapids_beats_has_index.md` - architecture spec (HAS) - `rapids_beats_mas/rapids_beats_mas_index.md` - micro-architecture spec (MAS) - MAS `ch02_fub_blocks/`, `ch03_macro_blocks/`, `ch04_interfaces/` - per-block and interface detail; HAS `ch05_programming/` - the programming model

This component: `PRD.md`, `TASKS.md`, `known_issues/`.

Validation: `vault/Tasks/projects/components/dma-ip/rapids/RAPIDS_Validation_Status_Report.md` – test results, but a **pre-beats snapshot** (last updated 2026-07-22; it still discusses

program_engine, network_slave and the retired `scheduler.sv`). `PRD.md` labels it the same way. For current numbers run `dv/tests/analyze_beats_coverage.py`.

Framework BFM docs: `../RTLDesignSherpa-DV/docs/components/<family>/` (published at sean-galloway.github.io/RTLDesignSherpa-DV).

34.11 Verification Patterns

These were ~270 lines of general DV method sitting in a component file. They now live in the handbook, where every area can find them:

- **Asynchronous outputs need a background monitor** – `vault/handbook/dv/async-output-capture.md`. This is RAPIDS' own lesson: the descriptor-engine test found 5 of 12 descriptors and filed it as a 42% engine defect, when the engine was correct and the test was not watching.
- **Delay profiles** – `vault/handbook/dv/randomization.md`. RAPIDS' `DelayProfile` enum re-implements `DEFAULT_PROFILES`; name a catalogue profile instead.
- **Extract a BFM or embed it** – `vault/handbook/dv/bfm-usage.md`, which uses RAPIDS' own `data_mover_bfm.py` (extracted) and the embedded AXI responder in `descriptor_engine_tb.py` as the worked pair.
- **100% success is required** – `/GLOBAL_REQUIREMENTS.md` 3.3, which names this file as its source. A 70% threshold hides a defect rather than tolerating it.

34.12 Documentation Generation

Both specs build from their index file into DOCX and PDF, written into `docs/`. Use the wrapper scripts – they apply the house style:

```
cd projects/components/dma-ip/rapids/docs
./generate_has_pdf.sh --rev 0.8     # RAPIDS_Beats_HAS_v0.8.docx/.pdf
./generate_mas_pdf.sh --rev 0.7     # RAPIDS_Beats_MAS_v0.7.docx/.pdf
```

They call `bin/md_to_docx.py`, which follows the markdown links out of the index, demotes headings and builds the ToC. Pipeline details, the LaTeX/PDF engine choice and the caption traps: `vault/handbook/authoring/doc-pipeline.md`.

Version: 1.2 **Last Updated:** 2025-10-14 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

35 RAPIDS Beats Architecture HAS

Version: 0.8 **Date:** 2026-07-13 **Purpose:** Hardware Architecture Specification for RAPIDS Phase 1 "Beats" Architecture **Classification:** External Interface Specification

---

35.1 Document Organization

Note: All chapters linked below for automated document generation.

35.1.1 Front Matter

- Document Information
- Revision History
- Acronyms and Definitions

35.1.2 Chapter 1: Overview

- Product Overview
- Key Features
- System Context

35.1.3 Chapter 2: Architecture

- Block Diagram
- Data Flow
- Channel Architecture

35.1.4 Chapter 3: Interfaces

- Interface Summary
- AXI4 Master Interface
- AXIS Interface
- APB Configuration Interface
- Monitor Bus Interface
- CXS Network Interface

35.1.5 Chapter 4: Use Cases

- Network-to-Memory Transfer
- Memory-to-Network Transfer
- Descriptor Chaining

- Multi-Channel Operation

35.1.6 Chapter 5: Programming Model

- Descriptor Format
- Register Map
- Initialization Sequence
- Error Handling

35.1.7 Chapter 6: Performance

- Throughput Targets
- Latency Characteristics
- Resource Estimates

35.2 Quick Reference

35.2.1 RAPIDS Beats at a Glance

RAPIDS Beats Feature Summary

Feature	Specification
Architecture	Network-to-Memory Accelerator
Channels	8 independent DMA channels
Data Width	512-bit (configurable)
Address Width	64-bit
Descriptor Size	256-bit
AXI Protocol	AXI4 (Read/Write Masters)
Network Protocol	AXI-Stream (Sink/Source)
Monitoring	64-bit MonBus packets

35.2.2 Interface Summary

External Interface Summary

Interface	Type	Direction	Width	Purpose
Descriptor AXI	AXI4 Master	Bidirectional	256-bit	Descriptor fetch
Sink AXI	AXI4	Write	512-bit	Memory write

Interface	Type	Direction	Width	Purpose
	Master			
Source AXI	AXI4 Master	Read	512-bit	Memory read
Sink AXIS	AXIS Slave	Input	512-bit	Network ingress
Source AXIS	AXIS Master	Output	512-bit	Network egress
MonBus	Custom	Output	64-bit	Monitoring

35.3 Related Documentation

- **RAPIDS Beats MAS** - Module Architecture Specification
- **PRD.md** - Product requirements
- **ADDRESS_INCREMENT_PATTERNS.md** - DMA addressing patterns

35.4 Specification Conventions

35.4.1 Signal Naming

- **Clock:** `aclk` (AXI clock domain)
- **Reset:** `aresetn` (active-low, async assert, sync deassert)
- **AXI Signals:** Standard ARM AMBA naming
- **AXIS Signals:** Standard AXI-Stream naming

35.4.2 Diagram Conventions

- **Mermaid:** Block diagrams and flowcharts
- **WaveDrom:** Timing diagrams
- **Tables:** Interface specifications and register maps

Last Updated: 2026-07-02 **Maintained By:** RAPIDS Architecture Team